目　录

第一章　基础算法 ……（1）
§1.1　递推法 ……（1）
一、倒推法 ……（2）
二、顺推法 ……（4）
§1.2　贪心法 ……（8）
§1.3　递归法 ……（18）
§1.4　分治法 ……（22）
§1.5　枚举法 ……（31）
§1.6　模拟法 ……（40）
第二章　顺序统计算法和中位数 ……（50）
§2.1　顺序统计的算法 ……（50）
一、划分方法 ……（51）
二、二分法求解 ……（52）
§2.2　中位数的应用 ……（53）
第三章　有关数论的算法 ……（60）
§3.1　求最大公约数 ……（60）
§3.2　求解模线性方程 ……（63）
§3.3　求解模线性方程组 ……（68）
§3.4　模取幂运算 ……（71）
§3.5　素数的测试 ……（72）
§3.6　整数的因子分解 ……（74）
第四章　计算几何学 ……（78）
§4.1　线段的性质 ……（78）
§4.2　确定任意一对线段是否相交 ……（83）
§4.3　寻找凸包 ……（91）
一、graham 扫描法 ……（92）
二、Jarris 步进法 ……（101）
§4.4　寻找最近点对 ……（106）
第五章　显式图的基本算法 ……（113）
§5.1　显式图的表示 ……（113）
一、邻接表 ……（113）
二、邻接矩阵 ……（114）
§5.2　宽度优先搜索 ……（115）
§5.3　深度优先搜索 ……（128）
§5.4　有向图的最短路问题 ……（143）
一、单源最短路径问题 ……（144）
二、每对顶点间的最短路径问题 ……（148）
第六章　隐式图的基本算法 ……（163）
§6.1　回溯法的讨论 ……（163）
一、如何求 n 皇后问题 ……（163）

二、回溯法的算法分析和程序框架 ……(167)
三、应用算法框架解题 ……(170)
四、回溯法的深入 ……(171)
§6.2 广度优先搜索 ……(177)
§6.3 双向广度优先搜索 ……(193)
§6.4 分支定界法 ……(203)
一、分支定界法算法思想 ……(203)
二、分支定界法的算法框架 ……(206)
三、应用框架解题 ……(209)
§6.5 A*算法 ……(213)
一、计算估价函数F(N) ……(214)
二、按 F 值递增顺序排列待扩展结点 ……(214)
三、检查调整重合状态 ……(216)
四、A*算法框架 ……(217)
五、使用 A* 算法求8数码问题 ……(224)
六、分阶段 A* 算法 ……(228)
§6.6 博奕树 ……(232)
一、博奕树的数据结构和算法思想 ……(232)
二、博奕树的算法框架 ……(237)
三、应用框架解题的一个实例 ……(242)
第七章 网络流的算法 ……(250)
§7.1 基本概念和基本定理 ……(251)
一、网络与流 ……(251)
二、可行流与最大流 ……(251)
三、可改进路P ……(252)
四、截集与截量 ……(253)
五、多个源和多个汇的网络 ……(254)
§7.2 寻求最大流的标号法 ……(255)
§7.3 最小费用最大流问题 ……(267)
§7.4 网络流算法的应用 ……(273)
第八章 动态程序设计 ……(303)
§8.1 矩阵链乘法 ……(303)
一、最优括号化的结构 ……(305)
二、递归定义最优解的值 ……(305)
三、按自上而下记忆化方式或自底向上的方式求最优解 ……(307)
四、构造最优解的方法 ……(309)
§8.2 最长公共子序列 ……(312)
一、刻划LCS问题的最优解的结构 ……(312)
二、递归定义LCS的长度值 ……(313)
三、按自底向上方式计算LCS的长度 ……(313)
四、构造一个最长公共子序列 ……(314)
§8.3 应用举例 ……(315)
第九章 题库 ……(330)
一、神秘的大陆 ……(330)

二、迷宫车间 ……(331)
三、Hamilton 机器人 ……(332)
四、魔方工具包 ……(333)
五、Tom、Jerry 和奶酪 ……(334)
六、债务 ……(335)
七、Sinistra 的城市 ……(336)
八、游戏 ……(337)
九、加法链 ……(337)
十、黑白棋 ……(338)
十一、化妆品 ……(339)
十二、花园 ……(340)
十三、晚会 ……(341)
十四、最短路径 ……(342)
十五、车用地图 ……(343)
十六、运输地图 ……(344)
十七、覆盖框 ……(345)
十八、错链 ……(346)
十九、高科技计划 ……(350)
二十、排序序列 ……(350)

第一章　基础算法

算法是一组（有限个）规则，它为某个特定问题提供了解决问题的运算序列。通俗点说，就是计算机解题的过程。在这个过程中，无论是形成解题思路还是编写程序，都是在实施某种算法。前者是推理实现的算法，后者是操作实现的算法。

计算机解题的核心是算法设计。一个算法应该具有以下五个重要特征:

(1) 有穷性　一个算法必须保证执行有限步之后结束;

(2) 确切性　算法的每一步骤必须确切定义;

(3) 输入　一个算法有0个或多个输入，以刻划运算对象的初始情况。所谓0个输入是指算法本身定出了初始条件;

(4) 输出　一个算法有一个或多个输出，以反映对输入数据加工后的结果。没有输出的算法是毫无意义的;

(5) 能行性　算法原则上能够精确地运行，而且人们用笔和纸做有限次即可完成。

下面，我们对构成算法所依据的一些基本方法（公式、方案、原则），即解题思路展开讨论，对于读者来说，为了获得一个既有效又优美的算法，必须了解一些基本的常用的算法设计思路。

§1.1　递推法

有一类试题，每相邻两项数之间的变化有一定的规律性，我们可将这种规律归纳成如下简捷的递推关系式:

$$F_n = g(F_{n-1})$$

这就在数的序列中，建立起后项和前项之间的关系。然后从初始条件（或最终结果）入手，一步步地按递推关系式递推，直至求出最终结果（或初始值）。很多程序就是按这样的方法逐步求解的。如果对一个试题，我们要是能找到后一项数与前一项数的关系并清楚其起始条件（最终结果），问题就好解决，让计算机一步步算就是了。让高速的计算机从事这种重复运算，可真正起到“物尽其用”的效果。

递推分倒推法和顺推法两种形式。一般分析思路:

if 求解初始条件 F_1

then begin {倒推}

由题意（或递推关系）确定最终结果 F_n;

求出倒推关系式 $F_{i-1}=g'(F_i)$;

i=n; {从最终结果 F_n 出发进行倒推}

while 当前结果 F_i 非初始值 F_1 do 由 $F_{i-1}=g(F_i)$ 倒推前项;

输出倒推结果 F_1 和倒推过程;

```
    end {then}
  else begin {顺推}
    由题意(或递推关系)确定初始值F1(边界条件);
    求出顺推关系式 F_i=g(F_{i-1});
    i=1;{由边界条件 F_1 出发进行顺推}
    while 当前结果 F_i 非最终结果 F_n do 由 F_i=g(F_{i-1}) 顺推后项;
    输出顺推结果 F_n 和顺推过程;
    end; {else}
```

一、倒推法

所谓倒推法，就是在不知初始值的情况下，经某种递推关系而获知问题的解或目标，再倒过来，推知它的初始条件。因为这类问题的运算过程是一一映射的，故可分析得其递推公式。然后再从这个解或目标出发，采用倒推手段，一步步地倒推到这个问题的初始陈述。

下面举例说明。

【例 1】贮油点

一辆重型卡车欲穿过 1000 公里的沙漠，卡车耗油为 1 升/公里，卡车总载油能力为 500 公升。显然卡车装一次油是过不了沙漠的。因此司机必须设法在沿途建立几个储油点，使卡车能顺利穿越沙漠，试问司机如何建立这些贮油点？每一贮油点应存多少汽油，才能使卡车以消耗最少汽油的代价通过沙漠？

算法分析:

编程计算及打印建立的贮油点序号，各贮油点距沙漠边沿出发的距离以及存油量。

No.	distance(k.m.)	oil(litre)
1	× ×	× ×
2	× ×	× ×
3	× ×	× ×
…	……	……

设 dis[i]－第 i 个贮油点至终点(i = 0)的距离;

oil [i]－第 i 个贮油点的存贮油量;

我们可以用倒推法来解决这个问题。从终点向始点倒推，逐一求出每个贮油点的位置及存油量。图 1.1 － 1 表示倒推时的返回点。

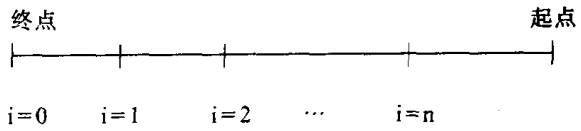


图 1.1 - 1

从贮油点 i 向贮油点 i+1 倒推的策略是，卡车在点 i 和点 i+1 间往返若干次。卡车每次返回 i+1 处时正好耗尽 500 公升汽油，而每次从 i+1 处出发时又必须装足 500 公升汽油。两点之间的距离必须满足在耗油最少的条件下使 i 点贮足 i*500 公升汽油的要求 $(0 \leqslant i \leqslant n-1)$。具体地讲，第一个贮油点 i=1 应距终点 i=0 处 500km，且在该处贮藏 500

公升汽油，这样才能保证卡车能由 i＝1 处到达终点 i＝0 处，这就是说

$$dis[1]=500 \qquad oil[1]=500;$$

为了在 i=1 处贮藏 500 公升汽油，卡车至少从 i=2 处开两趟满载油的车至 i=1 处。所以 i=2 处至少贮有 2*500 公升汽油，即 oil[2]＝500*2＝1000。另外，再加上从 i=1 返回至 i=2 处的一趟空载，合计往返 3 次。三次往返路程的耗油量按最省要求只能为 500 公升，即 $d_{12}=500/3$km，

$$dis[2]=dis[1]+d_{12}=dis[1]+500/3$$

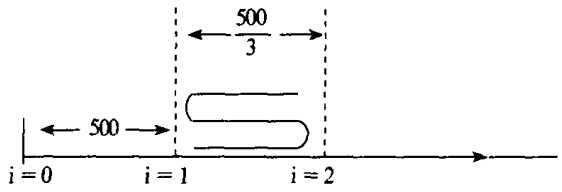


图 1.1-2

为了在 i=2 处贮存 1000 公升汽油，卡车至少从 i=3 处开三趟满载油的车至 i=2 处。所以 i=3 处至少贮有 3*500 公升汽油，即 oil[3]＝500＊3＝1500。加上 i＝2 至 i＝3 处的二趟返程空车，合计 5 次。路途耗油量亦应 500 公升，即 $d_{23}=500/5$，

$$dis[3]=dis[2]+d_{23}=dis[2]+500/5;$$

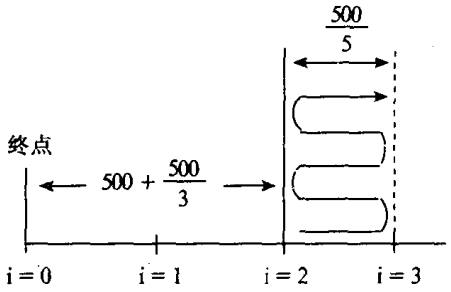


图 1.1-3

依次类推，为了在 i=k 处贮藏 k*500 公升汽油，卡车至少从 i=k+1 处开 k 趟满载车至 i＝k 处，即 oil[k＋1]＝[k＋1]＊ 500＝oil[k]＋500，加上从 i＝k返回 i＝k＋1 的 k－1 趟返程空间，合计 2k－1 次。这 2k－1 次总耗油量按最省要求为 500 公升，即 $d_{k,k+1}=500/(2k-1)$，

$$dis[k+1]=dis[k]+d_{k,k+1}$$

$$=dis[k]+500/(2k-1);$$

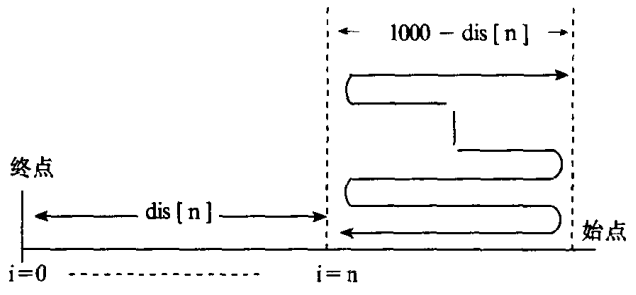


图 1.1-4

最后， i=n 至始点的距离为 1000−dis[n], oil[n]=500*n。为了在 i=n 处取得 n*500 公升汽油，卡车至少从始点开 n+1 次满载车至 i=n，加上从 i=n 返回始点的 n 趟返程空车，合计 2n+1 次，2n+1 趟的总耗油量应正好为 (1000−dis[n])*(2n+1)，即始点藏油为 oil[n]+(1000−dis[n])*(2n+1)。

下面为程序题解:

```
program oil_lib;
var
  k: integer; {贮油点位置序号}
  d,          {累计终点至当前贮油点的距离}
  d1: real;   {i=n 至始点的距离}
  oil, dis: array [1.. 10]  of real;
  i: integer; {辅助变量}
begin
  writeln ('NO.',' distance (k. m)': 30,' oil(1.)': 80);
  k: = 1;
  d:=500;  {从 i=1 处开始向始点倒推}
  dis[1]:=500;
  oil[1]:=500;
  repeat
    k: =k+1;
    d: =d+500/(2*k-1);
    dis[k]: = d;
    oil[k]: = oil[k-1] + 500;
    until d>=1000;
  dis[k]:=1000; {置始点至终点的距离值}
  d1:=1000-dis[k-1]; {求 i=n 处至始点的距离}
  oil[k]: =d1*(2*k+1) + oil[k-1]; {求始点藏油量}
  for i:=0 to k do   {由始点开始，逐一打印始点至当前贮油点的距离和藏油量}
     writeln (i, 1000-dis[k-i]: 30, oil[k-i]: 80);
end. {main}
```

二、顺推法

倒推法的逆过程是顺推法，即由边界条件出发，通过递推关系式推出后项值，再由后项值按递推关系式推出再后项值……，依次递推，直至从问题初始陈述向前推进到这个问题的解为止。

下面举例说明。

【例 2】实数数列

一个实数数列共有 N 项，已知

$$a_i = (a_{i-1} - a_{i+1})/2 + d, (1 < i < N)(N < 60)$$

键盘输入 N，d，a_1，a_n，m，输出 a_m。

输入数据均不需判错。

算法分析:

该题的数学味颇浓。解题前需耐下心来，对公式

$$A_i = (A_{i-1} - A_{i+1})/2 + d \qquad (1 < i < N) \quad (n < 60)$$

作一番推敲，探讨其数值变换规律。不然的话，会无从入手。

令 $X = A_2$ $S_2[i] = (P_i, Q_i, R_i)$ 表示 $A_i = P_iX + Q_iD + R_iA_1$;

我们可以根据

$$A_i = A_{i-2} - 2A_{i-1} + 2D$$
$$= P_iX + Q_iD + R_iA_1$$

推出公式

$$P_iX + Q_iD + R_iA_1 = (P_{i-2} - 2P_{i-1})X + (Q_{i-2} - 2Q_{i-1} + 2)D + (R_{i-2} - 2R_{i-1})A_1$$

比较等号两端 X、D 和 A_1 的系数项，可得

$$P_i = P_{i-2} - 2P_{i-1}$$
$$Q_i = Q_{i-2} - 2Q_{i-1} + 2$$
$$R_i = R_{i-2} - 2R_{i-1}$$

加上两个边界条件

$P_1 = 0$ $Q_1 = 0$ $R_1 = 1$ $(A_1 = A_1)$

$P_2 = 1$ $Q_2 = 0$ $R_2 = 0$ $(A_2 = A_2)$

根据 P_i、Q_i、R_i 的递推式，可以计算出

$S_2[1] = (0, 0, 1)$;

$S_2[2] = (1, 0, 0)$;

$S_2[3] = (-2, 2, 1)$;

$S_2[4] = (5, -2, -2)$;

$S_2[5] = (-12, 8, 5)$;

…… …… …… …… …… …… …… …… …… ……

$S_2[i] = (P_i, Q_i, R_i)$;

…… …… …… …… …… …… …… …… …… ……

$S_2[N] = (P_N, Q_N, R_N)$;

有了上述基础，A_M 便不难求得。有两种方法:

1．由于 A_N、A_1 和 P_N、Q_N、R_N 已知，因此可以先根据公式

$$A_2 = A_N - Q_ND - R_NA_1/P_N$$

求出 A_2。然后将 A_2 代入公式

$$A_3 = A_1 - 2A_2 + 2D$$

求出 A_3。然后将 A_3 代入公式

$$A_4 = A_2 - 2A_3 + 2D$$

求出 A_4。然后将 A_4 代入公式

…… …… …… …… …… …… …… …… …… …… …… …… …… …… …… ……

求出 A_{i-1}。然后将 A_{i-1} 代入公式

$A_i=A_{i-2}-2A_{i-1}+2D$

求出 A_i。依次类推，直至递推至 A_M 为止。

上述算法的缺陷是，由于 A_2 是两数相除的结果，而除数 P_N 递增，因此精度误差在所难免，以后的递推过程又不断地将误差扩大，以至当 M 超过 40 时，求出的 A_M 明显偏离正确值。显然，这种算法虽简单但不可靠。

2．我们令 $A_2=A_2$，$A_3=X$，由 $S_3[i]=(P_i, Q_i, R_i)$ 表示 $A_i=P_iX+Q_iD+R_iA_2$（$i\geqslant 2$）可计算得出

$S_3[2]=(0, 0, 1)=S_2[1]$;

$S_3[3]=(1, 0, 0)=S_2[2]$;

$S_3[4]=(-2, 2, 1)=S_2[3]$;

$S_3[5]=(5, -2, -2)=S_2[4]$;

…… …… …… …… …… …… …… …… …… …… …… …… …… …… …… ……

$S_3[i]=(\cdots\cdots\cdots)=S_2[i-1]$;

…… …… …… …… …… …… …… …… …… ……

$S_3[N]=(\cdots\cdots\cdots)=S_2[N-1]$;

再令 $A_3=A_3$，$A_4=X$，由 $S_4[i]=(P_i, Q_i, R_i)$ 表示 $A_i=P_iX+Q_iD+R_iA_3$（$i\geqslant 3$）可计算得出

$S_4[3]=(0, 0, 1)=S_3[2]=S_2[1]$;

$S_4[4]=(1, 0, 0)=S_3[3]=S_2[2]$;

$S_4[5]=(-2, 2, 1)=S_3[4]=S_2[3]$;

…… …… …… …… …… …… …… …… …… ……

$S_4[i]=(\cdots\cdots\cdots)=S_3[i-1]=S_2[i-2]$;

…… …… …… …… …… …… …… …… …… ……

$S_4[N]=(\cdots\cdots\cdots)=S_3[N-1]=S_2[N-2]$;

依次类推，我们可以发现一个有趣的式子：

$A_N=P_{N-i+2}*A_i+Q_{N-i+2}*D+R_{N-i+2}*A_{i-1}$，即

$$A_i=(A_N-Q_{N-i+2}*D-R_{N-i+2}*A_{i-1})/P_{N-i+2}$$

我们从已知量 A_1 和 A_N 出发，依据上述公式顺序递推 A_2、A_3、…、A_M。由于 P_{N-i+2} 递减，因此最后得出的 A_M 要比第一种算法趋于精确。

下面给出程序题解：

```
program ND1P4;
const
  maxn     =60;
var
```

```
  n, m, i    :integer;
  d          :real;
  list       :array [1..maxn] of real;          {list[i]——对应 a_i}
  s          :array [1..maxn, 1..3] of real;    {S[i, 1]——对应 P_i;
                                                  S[i, 2]——对应 Q_i;
                                                  S[i, 3]——对应 R_i}

procedure init;
  begin
    write('n m d =');
    readln (n, m, d);              {输入项数、输出项序号和常数}
    write('a1 a', n ,' =');
    readln (list[1], list[n]);     {输入 a1 和 an}
  end; {init}

procedure solve;
  begin
    s[1, 1] :=0; s[1, 2] :=0; s[1, 3] :=1;
 {求递推边界 (P1, Q1, R1)和 (P2, Q2, R2)}
    s[2, 1] :=1; s[2, 2] :=0; s[2, 3] :=0;
    {根据公式 Pi ← P(i-2) -2*P(i-1)}
    {Qi ← Q(i-2) -2*Q(i-1)}
    {Ri ← R(i-2) -2*R(i-1)}
    {递推 (P3, Q3, R3)…… Pn, Qn, Rn)}
    for i:=3 to n do
      begin
        s[i, 1] :=s[i-2, 1]-2*s[i-1, 1];
        s[i, 2] :=s[i-2, 2]-2*s[i-1, 2]+2;
        s[i, 3] :=s[i-2, 3]-2*s[i-1, 3];
        end; {for}
  end; {solve}

procedure main;
  begin
    solve;                  {求 (P1, Q1, R1)..(Pn, Qn, Rn)}
    {根据公式 Ai=(An-Q(n-i+2)*d-R(n-i+2)*A(i-1))/P(n-i+2)}
    {递推 A2..Am}
    for i:= 2 to m do
```

```
    list[i] :=(list[n]-s[n-i+2, 2]*d -s[n-i+2, 3] * list[i-1])/s[n-i+2, 1];
  writeln('a', m, '  =', list[m]: 20: 10);    {输出 A_m}
  end; {main}

begin
  init;    {输入数据}
  main;    {递推和输出 A_m}
  readln;
end. {main}
```

§1.2 贪心法

和§1.1节中所讲的顺推法相仿，贪心法也是从问题的某一个初始解出发，向给定的目标递推。但不同的是，推进的每一步不是依据某一固定的递推式，而是做一个当时看似最佳的贪心选择，不断地将问题实例归纳为更小的相似的子问题，并期望通过所做的局部最优选择产生出一个全局最优解。

问题是，这种局部贪心的选择是否可以得出全局最优解呢？在某些情况下是可以的。例如：

【例1】删数问题

键盘输入一个高精度的正整数N，去掉其中任意S个数字后剩下的数字按原左右次序组成一个新的正整数。编程对给定的N和S，寻找一种方案使得剩下的数字组成的新数最小。

输出应包括所去掉的数字的位置和组成的新的正整数。（N不超过240位）

输入数据均不需判错。

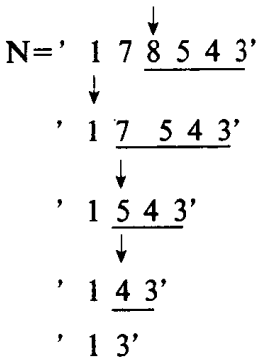
算法分析：

首先我们必须注意，试题中正整数N的有效位数为240位，而计算机中整数有效位（包括符号位）充其量也不过11位，无论如何也达不到试题的数位要求。因此，必须采用可含256个字符的字串来替代整数。

以字串形式输入N，使用尽可能逼近目标的贪心法来逐一删去其中S个数符，每一步总是选择一个使剩下的数最小的数符删去。之所以作出这样贪心的选择，是因为删S个数符的全局最优解，包含了删一个数符的子问题的最优解。

为了保证删1个数符后的数最小，我们按高位→低位的方向搜索递减区间。若不存在递减区间，则删尾数符；否则删递减区间的首字符，这样形成了一个新数串。然后回到串首，重复上述规则，删下一数符……依此类推，直至删除S个数符为止。

例如：N='178543'，S=4，删数过程如下：



显然，按上述规则删去S个数符后，剩余N−S个数符组成的新数必定最小。

下面是程序题解：

```
PROGRAM Find__Min__Integer;
  USES
    Crt;
  VAR
    N                    : string; {数串}
    S, i                 : integer; {S——被删的数字个数}
  BEGIN
    clrscr;
    write('N='); readln(N); {输入数串}
    write('S='); readln(S); {输入应删的数字个数}
    while S>0 do begin {逐一删去S个数符}
      i:=1;
      while (i<length(N)) and (N[i]<=N[i+1]) do inc(i); {搜索递减区间}
      delete(N, i, 1); {删去该区间的首数符}
      dec(S);
    end; {while}
    while (length(N)>1) and (N[1]='0') do delete(N, 1, 1);
     {删去串头的无用零}
    writeln(N); {输出剩下的数码}
  END. {main}
```

读者在看了上述题解后，自然会提出这样一个问题：对一个最优化问题，我们怎样才能知道它是否适用于贪心算法求解？没有一个通用的方法。但适用于贪心策略求解的大多数问题都有两个特点：

1. 贪心选择性质——可通过做局部最优（贪心）选择来达到全局最优解

贪心策略通常是自顶向下做的。第一步为一个贪心选择，将原问题变成一个相似的、但规模更小的问题，而后的每一步都是当前看似最佳的选择。这种选择可能依赖于已作出的所有选择，但不依赖有待于做的选择或子问题的解。从求解的全过程来看，每

一次贪心选择都将当前问题归纳为更小的相似子问题，而每一个选择都仅做一次，无重复回溯过程，因此贪心法有较高的时间效率。

例如【例 1】中，设正整数 N 有 p 位。第一步选择一个使剩下的 p－1 位数最小的数符删去，把问题归纳为在 p－1 位正整数中去掉 S－1 个数字后的新数最小的子问题；第二步再选择一个使得剩下的新数最小的数符删去，又把问题归纳为在 p－2 位正整数中去掉 S－2 个数字后的新数最小的子问题…… 每次选择中，已删去的数不允许再删，而当前选择又不受将来删的数的影响。依照上述方法不断将问题归纳为更小的互为独立的子问题，直至删除 s 个数符为止。显然删数问题符合贪心选择性质。

2. 最优子结构——问题的最优解包含了子问题的最优解

例如【例 1】中，N＝'178543'，S＝4。第一步的贪心选择为删'8'，即第一个子问题的最优解为'8'；第二步的贪心选择为删'7'，即第二个子问题（第一个子问题的子问题）的最优解为'7'；继后的第三个，第四个子问题的最优解分别为'5' 和'4'。很显然，问题的最优解包含了四个子问题的最优解，因此删数问题具有最优子结构性质。

但问题是，对所有具备最优子结构的问题是否都可采用贪心策略呢？不一定。下面，我们来考察一个经典优化问题的两种变形。

【例 2】背包问题

有一个贼在偷窃一家商店时发现有 N 件物品：第 i 件物品值 V_i 元，重 W_i 磅，（$1 \leqslant i \leqslant n$），此处 V_i 和 W_i 都是整数。他希望带走的东西越值钱越好，但他的背包中最多只能装下 W 磅的东西（W 为整数）。有两种偷窃方式：

1) 0-1 背包问题

如果每件物品或被带走或被留下，小偷应该带走哪几件东西？

2) 部分背包问题

如果允许小偷可带走某个物品的一部分，小偷应该带走哪几件东西，每件东西的重量是多少？

算法分析：

两种背包问题都具有最优化结构性质：

对于 0－1 背包问题，考虑重量至多为 W 磅的最值钱的一包东西。如果我们从中去掉物品 J，余下的必须是窃贼从其余的 n－1 件物品中可带走的重量至多为 $W-W_J$ 的最值钱的一件东西；如果我们再从中去掉物品 i，余下的也必须是窃贼从其余 n－2 件可带走的、重量至多为 $W-W_j-W_i$ 的最值钱的一件东西…… ，依次类推。

对于部分背包问题，如果我们考虑从最优货物中去掉某物品 J 的重量 W_P（$W_J \geqslant W_P$），则余下的物品必须是窃贼可以从 N－1 件原有物品和物品 J 的 W_J-W_P 中可带走的、重量至多为 $W-W_P$ 的最值钱的一件东西 …… ，依次类推。

我们可采用贪心策略来解决部分背包问题：

先对每件物品计算其每镑价值 V_i/W_i，然后按每磅价值单调递减的顺序对所有物品排序。例如，总共有三件物品和一个背包。

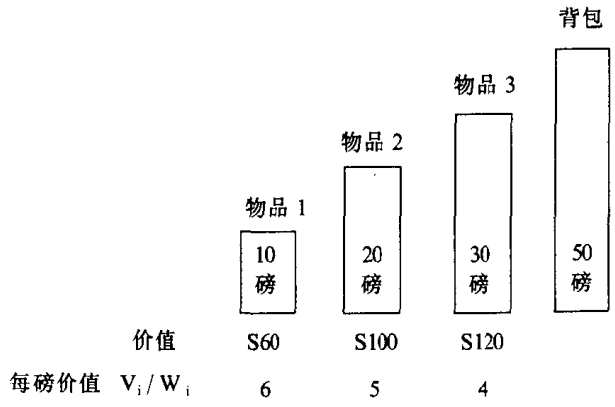


图 1.2-1

按照一种贪心策略，窃贼开始时对具有最大的每磅价值的物品尽量多拿一些。如果他拿完了该物品后仍可以取一些其它物品时，他就再取具有次大的每磅价值的物品，一直继续下去，直到不能取为止。

下面是程序题解:

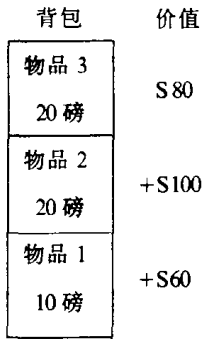


图 1.2-2

```
Program Partial_Bag_Problem;

Const
  Maxn          =  1000;{最多物件数}

Type
  Node          =  Record{物件的数据类型}
                     k            :  Integer;{编号}
                     w, v, vper   :  Real{重量、价值和每
                     磅价值}
                   End;

Var
  N, i          :  Integer;{实际物件数，辅助变量}
  F             :  Text;{文件名串}
  W, V          :  Real;{窃走的物品重量(背包容量)和价值}
  List, Lt      :  Array [1..Maxn] of Node;
                   {物件序列，辅助的物件序列}

Prcedure Merge (p, q, r :  Integer);{将两个按每磅价值递减顺序排列的物件子序列
list [p..q]和 list [q+1..r]合并成一个排好序的物件序列 list [p..r]}
```

```
Var
  i, j, t : Integer;
Begin
  t := p; i := p; j := q + 1;
  While t <= r Do Begin
    If (i <= q) And ((j > r) Or (List[i]. vper >= List[j]. vper)) Then Begin
      Lt[t] := List[i]; Inc(i)
    End {then}
    Else Begin
      Lt[t] := List[j]; Inc(j)
    End; {else}
    Inc(t)
  End; {while}
  For i := p to r Do List[i] := Lt[i]
End; {Merge}

Procedure Merge_Sort(p, r : Integer);
{对物件序列 list[p. .r]按每磅价值递减的顺序进行合并排序}
Var
  q : Integer;
Begin
  If p <> r Then Begin
    q := (p + r - 1) div 2; {计算中间下标 q}
    Merge_Sort(p, q); {对 list[p. .q]进行合并排序}
    Merge_Sort(q+1, r); {对 list[q+1. .r]进行合并排序}
    Merge(p, q, r) {合并两个子序列}
  End {then}
End; {Merge_sort}

Begin
  Assign(F, 'INPUT. DAT'); {输入文件名与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N, W); {读入物件数和背包容量}
  For i := 1 to N Do Begin
  {读入每个物件的重量和价值，计算其每磅价值并记下编号}
    Readln(F, List[i]. w, List[i]. v);
    List[i]. k := i;
    List[i]. vper := List[i]. v / List[i]. w
```

```
End; {for}
  Merge_Sort (1, N); {对物件按每磅价值递减的顺序进行合并排序}
  V := 0; i := 1;
  Writeln ('Num': 5, 'Weight': 9, 'Value': 8); {打印表目}
  While W > List[i]. w Do Begin
  {依次取走当前最值钱的物件，并累计窃走物件的价值和}
    W := W - List[i]. w;
    V := v + List[i]. v;
    Writeln (List[i]. k: 5, ':', List[i]. w: 8: 2, List[i]. v: 8: 2);
    {输出当前窃走的物件的序号、重量和价值}
    Inc (i)
  End; {while}
  V := V + W * List[i]. vper; {小偷带走最后一件物品的一部分，累计输出总价值}
  Writeln (List[i]. k: 5, ':', W:8:2, W * List[i]. vper: 8: 2);
  {输出最后一件物品的序号、被窃部分的重量和价值}
  Writeln ('Total Value = ', V:8:2)
End. {main}
```

由此可见，在部分背包问题中，当我们在考虑是否把一件物品加到背包中时，不需要把加入该物品的子问题解与不取该物品的子问题解加以比较。由这种贪心方式所形成的所有子问题是互相独立的。

若对0-1背包问题采取贪心策略的话，顺序取物品1和2。

但是从下图可以看出，最优解取的是物品2和3，没有取物品1。两种包含物品1的可能解都不是最优解。

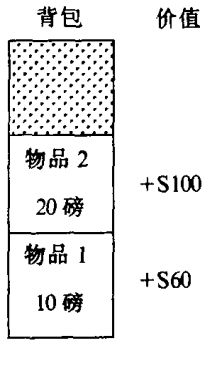


图 1.2-3

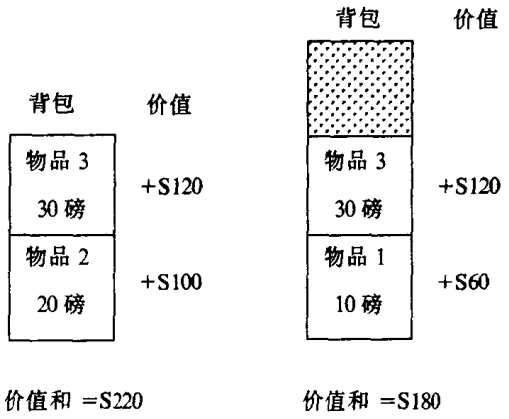


图 1.2-4

在0-1背包问题中不应取物品1的原因在于这样无法将背包填满，空余的空间降低了它的每磅价值，因此当我们考虑是否要把一件物品加到背包中时，必须对把加进该物品的子问题的解与不取该物品的子问题的解进行比较。由这种方式形成的子问题导致了许

多重迭的子问题。对于这一类虽具有最优化结构性质、但产生的子问题互为重迭的试题，一般采用动态程序设计的方法求解。我们将在第八章介绍这一算法。

下面我们介绍一种能确定贪心法何时产生最优解的理论。虽然它还未覆盖贪心法所适用的各种情况，但它确实能应用于许多重要的场合，并且这个理论发展很快，正在覆盖更多的应用。

首先我们来介绍一种称为“矩阵胚”的组合结构。所谓矩阵胚是指一个序对 $M=〔S, L〕$，其中 S 是一个有序非空集合，I 是 S 的一个非空子集族，称为 S 的独立子集。

如果 M 是某个 N*M 的矩阵的话，即

$$M=\left|\begin{array}{l} a_{11},\cdots\cdots a_{1m} \\ \cdots\cdots\cdots\cdots\cdots\cdots \\ a_{i1},\cdots\cdots a_{im} \\ \cdots\cdots\cdots\cdots\cdots\cdots \\ a_{n1},\cdots\cdots a_{nm} \end{array}\right| \quad \begin{array}{l} a_1=(a_{11},\cdots\cdots a_{1m}) \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \\ \text{行向量}\quad a_i=(a_{i1},\cdots\cdots a_{im}) \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \\ a_n=(a_{n1},\cdots\cdots a_{nm}) \end{array}$$

S 是 M 的各个行 $S=(a_1,\cdots\cdots a_n)$，I 是线性无关的若干行 $a_i, a_j, \cdots\cdots a_p$（即当且仅当 $K_i=K_j=\cdots\cdots=K_P=0$ 时，$K_i*a_i+K_j*a_j+\cdots\cdots+K_p*a_p=(0,\cdots\cdots,0)$）。

如果 M 是无向图 G 的矩阵胚的话，则 S 为图的边集，I 是所有构成森林的一组边的子集。

如果我们对 S 的每一个元素 X(X∈S) 赋于一个正的权值 W(X)，则称矩阵胚 M=(S, I) 是加权的。

适宜于用贪心法来获得最优解的许多问题可以归结为在加权矩阵胚中找出一个具有最大权值的独立子集的问题。亦即给定一个加权矩阵胚 M=(S, I)，我们希望找出一个独立的且具有最大可能权值的子集 A 为该矩阵胚的最优子集。由于任一元素 X∈S 的权 W(X) 是正的，所以最优子集 A 也是一个最大独立子集，即 A 不被 M 中比它更大的独立子集所包含。

更一般地说，如果我们遇到的是一个最小化问题，则必须分析该问题是否具备矩阵胚的结构，是否可以对应寻找最优子集的问题。

例如在最小生成树问题中，我们已知的是一个连通无向图 G=(V, E) 及其每条边的边长W(E)，要求找一个包含着连接所有顶点的且具有最小总长度的边的子集。为了将这个问题看作在一矩阵胚中找一个最优子集的问题，我们设加权矩阵胚 M 中的权函数 W。其中 $W'(E)=W_0-W(E)$ 且 W_0 大于各边的最大长度，使得加权矩阵胚中的所有权值都是正的。由于每个最大独立子集 A 对应于一棵生成树，且

$$W'(A)=(|V|-1)W_0-W(A)$$

（|V|—— 顶点个数）

对于任意最大独立子集 A 成立，故使 W'(A) 最大的独立子集必使 W(A) 最小。这样，任意一个可以在某一矩阵胚中寻找最优子集的算法都可用来解决最小生成树问题。

基于上述思想，我们构造了一个解决最小生成树问题的贪心算法：

一开始，先将 G 图中的边都去掉，只留下孤立的顶点。然后逐步地将当前最小边加上去，每次加的时刻，要保持住“没有环”这一性质。在加了 N−1 条边（N 为顶点个数）后，便生成了所要求的最小生成树。

下面，我们给出一个适用于任何加权矩阵胚的贪心算法流程。该算法输入一个加权矩阵胚 M=(S, I) 和一个正的权函数 W，返回的是一个最优（即具有最大权值）独立子集，其中S(M) 和 I(M) 表示 M 的组成。该算法是贪心的，因为它按权值单调递减的顺序来依次考虑每个元素 X ∈ S，如果X 能进入当前独立子集 A（即 A ∪ {X} 仍是独立的），它就立即把该元素加到 A 中

```
procedure greedy(M, W);
begin
 A← φ; {独立子集初始化为空}
 根据权 W 按单调递减顺序对 S(M) 排序;
 for  每个 X∈S〔M〕，根据 W(X) 按递减顺序 do
  if A ∪{X}∈I〔M〕{若 X 进入子集 A 还能保持 A 集合独立的话}
      then A← A ∪ {X}; {则 X 进入 A 集合}
返回最优子集 A;
end ; {greedy}
```

下面例举一个可用矩阵胚来解决的有趣例题。

【例 3】任务调度问题

一个单位时间任务是个作业，如要在计算机上运行一个程序，它恰覆盖一个单位的运行时间。给定一个单位时间任务的集合 S, 对 S 的一个调度即 S 的一个排列，其中规定了这些任务的执行顺序。该调度中的第一个任务开始于时间 0, 结束于时间1; 第二个任务开始于时间 1, 结束于时间 2; …… 。

单处理器上具有期限和罚款的单位时间任务调度问题的输入如下:

1. 包含 n 个单位时间任务的集合 S={1, 2, …… , n};

2. n 个取整的期限 d_1, …… , d_n, ($1 \leqslant d_i \leqslant n$)，任务 i 要求在 d_i 前完成;

3. n 个非负的权（或罚款）w_1, …… , w_n。如果任务 i 没在时间 d_i 之前结束，则导致罚款 w_i;

要求找出 S 的一个调度，使之最小化总的罚款。

算法分析:

这个问题首先我们定义两个概念:

迟任务——调度中在规定期限后完成的任务，试题要求解出最小化迟任务的罚款总和;

早任务——调度中在规定期限前完成的任务，最大化早任务的罚款总和正好对应问题的解;

任意一个调度可通过下述方式安排成规范形式:

1. 按期限递增的顺序对早任务进行调度。即只要在调度中有两个分别完成于时间 K 和 K+1 的早任务 i、j 且 $d_j < d_i$, 我们就互换 i 和 j 的位置，使得任务 i 在交换后仍然是早的，而任务 j 被移到更前的位置;

2. 如果某个早任务 X 跟在迟任务 Y 之后，我们可以交换 X 和 Y 的位置，使得早任务

先于迟任务;某一调度中早任务的集合构成了一个独立的任务集 A，因为 A 中任务按期限单调递增的顺序进行调度后，没有一个任务是迟的，且 A 中期限为 t 或更早的任务个数小于等于 t。

如果 S 是一个带期限的单位时间任务的集合，且 I 为所有独立的任务集构成的集合，则对应的系统（S, I）是一个矩阵胚。因此，我们可以采用贪心法找出具有最大总罚款的独立任务集 A。这个任务集正好对应最小化迟任务的罚款总和。

例如

任务 i	1	2	3	4	5	6	7
期限 d_i	4	2	4	3	1	4	6
罚款 w_i	70	60	50	40	30	20	10

最初，我们设从 1 到 n 的所有 n 个时间空位都是空的。然后按罚款的单调递减顺序（任务 1，任务 2，…，任务 7）来考虑各个子任务。在考虑任务 j 时，如果有一个尚未填入任务号的时间空位，其所代表的时刻等于或小于任务 j 的期限 d_j，则将任务 j 填入最近的这样的空位；如果不存在这样的空位，则将任务 j 赋与一个还未被占的、最近的空位。

按上述贪心策略选择了任务 1, 2, 3, 4, 7，放弃任务 5, 6。最终的最优调度为〈2, 3, 4, 1, 7, 5, 6〉，其总的罚款为 $W_5 + W_6 = 50$

下面给出程序题解:

```
Program Tasks_with_limit_and_fine;

Const
  Maxn                  =  500; {最多任务数}

Type
  Node                  =  Record {任务的数据类型}
                             k, d, w     :  Integer {任务编号，期限，罚款}
                           End;

Var
  N, i, j, num, t       :  Integer;
{N—任务个数; i,j—辅助变量; num—目前早任务个数; t—罚款总和}
  F                     :  Text; {文件变量}
  List, Lt              :  Array [1..Maxn] of Node;
{list—任务序列.list[i]. k<0，则任务 i 为早任务; list[i]. k>0，则任务 i 为迟任务;
lt—辅助数组}
  Pck                   :  Array [1. .Maxn] of Integer; {早任务的编号序列}
```

```
Procedure Merge(p, q, r : Integer);
{将两个按罚款单调递减顺序排列的任务子序列 list[p..q]和 list[q+1..r]合并成一个排好序
的任务序列 list[p..r]}
Var
  i, j, t : Integer;
Begin
  t := p; i := p; j := q + 1;
  While t <= r Do Begin
    If (i <= q) And ((j > r) Or (List[i].w > List[j].w)) Then Begin
      Lt[t] := List [i]; Inc(i)
    End {then}
    Else Begin
      Lt[t] := List[j]; Inc(j)
    End; {else}
    Inc(t)
  End; {while}
  For i := p to r Do List[i] := Lt[i]
End; {Merge}

Procedure Merge_Sort(p, r : Integer);
{对任务序列 list[p..r]按罚款单调递减的顺序进行合并排序}
Var
  q : Integer;
Begin
  If p <> r Then Begin
    q := (p + r - 1) div 2; {计算中间下标 q}
    Merge_Sort(p, q); {对任务子序列 list[p..q]继续递归}
    Merge_Sort(q + 1, r); {对任务子序列 list[q+1..r]继续递归}
    Merge(p, q, r) {合并两个已排好序的子序列}
  End {then}
End; {Merge_sort}

Begin
  Assign (F, 'INPUT. DAT'); {输入文件名与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N); {读入任务数}
  For i := 1 to N Do Begin{读入每个任务的期限和罚款，并记下其序号}
    Readln (F, List[i]. d, List[i]. w);
```

```
    List[i].k := i
  End; {for}
  Merge_Sort(1, N); {对n个任务按罚款单调递减的顺序排序}
  num := 0;
  For i := 1 to N Do Begin {依次处理每一个任务}
    t := 0; {统计目前num个早任务中期限小于等于num的任务个数t}
    For j := 1 to num Do
      If List[Pck[j]].d <= num Then Inc(t);
    If t < List[i].d Then Begin
    {若有一个恰处于或前于第j个任务期限的时间空位}
      Inc(num); {则早任务个数+1; 任务i进入早任务集合}
      Pck[num] := i;
      List[i].k := -List[i].k; {设任务i为早任务标志}
      j := num; {将早任务按期限递增顺序排列}
      While (j > 1) And (List[Pck[j]].d < List[Pck[j - 1]].d) Do Begin
        t := Pck[j]; Pck[j] := Pck[j - 1]; Pck[j - 1] := t;
        Dec(j)
      End {while}
   End {then}
  End; {for}
  For i := 1 to num Do Write(- List[Pck[i]].k: 4); {打印所有的早任务}
  t := 0;
  For i := 1 to N Do {打印所有迟任务并累计罚款总和}
    If List[i].k > 0 Then Begin
      Write(List[i].k: 4);
      t := t + List[i].w
    End;
  Writeln;
  Writeln('Total fine = ', t) {输出总罚款数}
End. {main}
```

§1.3 递归法

设一个未知函数f，用其自身构成的已知函数g来定义：

$$f(n)=g(n, f(n-1)) \qquad n>0$$
$$f(0)=a \qquad n=0$$

为了定义f(n)，必须先定义f(n-1)，为了定义f(n-1)，又必须先定义f(n-2)……，上述这种用自身的简单情况来定义自己的方式称为递归定义。

一个递归定义必须是有确切含义的，也就是说，必须一步比一步简单，最后是有终结的，决不能无限循环下去。在f(n)的定义中，当n为0时定义一个已知数a，是最简单的情况，称为递归边界，它本身不再使用递归的定义。

与递推一样，每一个递归定义都有其边界条件。但不同的是，递推是由边界条件出发，通过递推式求f(n)的值，从边界到求解的全过程十分清楚；而递归则是从函数自身出发来达到边界条件。在通往边界条件的递归调用过程中，系统用堆栈把每次调用的中间结果（局部变量和返回地址值）保存起来，直至求出递归边界值f(0)=a。然后返回调用函数。返回过程中，中间结果相继出栈恢复，f(1)=g(1, a)→f(2)=g(2, f(1))→…→直至求出f(n) =g(n, f(n−1))。

由此可见，递归算法的效率往往很低，费时和费内存空间。但是递归也有其长处，它能使一个蕴含递归关系且结构复杂的程序简洁精炼，增加可读性。 特别是在难于找到从边界到解的全过程的情况下，如果把问题推进一步，其结果仍维持原问题的关系，则采用递归算法编程比较合适。

递归按其调用方式分
- 直接递归——递归过程P直接自己调用自己；
- 间接递归——即P包含另一过程D，而D又调用P；

递归算法适用的一般场合为：

1．数据的定义形式按递归定义。

如裴波那契数列的定义： $f_n=f_{n-1}+f_{n-2}$； $f_0=1$；$f_1=2$。

对应的递归程序为：

```
function fib(n: integer): integer;
begin
  if n=0 then fib:=1    {递归边界}
         else if n=1 then fib:=2
                     else fib:=fib(n-2)+fib(n-1); {递归}
end; {fib}
```

这类递归问题可转化为递推算法，递归边界作为递推的边界条件。例如上例

```
f[0]:=1; f[1]:=2;  {递推边界}
for i:=2 to n do f[i]=f[i-1]+f[i-2];
fib:=f(n);
```

2．数据之间的关系（即数据结构）按递归定义。如树的遍历，图的搜索等。

3．问题解法按递归算法实现。例如回溯法等。

对于2，3，可利用堆栈结构将其转换为非递归算法。

下面例举2个可递归定义问题解法的例题。第一个例题属于计数类型，即计算具有某种特性的对象有多少；第二个例题的类型是枚举，即把所有具备某种特性的对象完全枚举

出来。对这两个试题，我们采用递归方法编写程序，其结构非常简洁而清晰。

【例 1】划分问题

设 s 是一个具有 n 个元素的集合 $s=\{a_1, a_2, \cdots, a_n\}$，现将 s 集合划分成 K 个满足下列条件的子集合 $s_1, s_2, \cdots, s_k$：

1. $s_i \neq \varphi$
2. $s_i \cap s_j = \varphi$
3. $s_1 \cup s_2 \cup s_3 \cup \cdots \cup s_n = s$

$(1 \leqslant i, j \leqslant k, i \neq j)$

则称 $s_1, s_2, \cdots, s_n$ 是 s 的一个划分。它相当于把 s 集合中的 n 个元素 $a_1 \cdots a_n$ 放入 k 个无标号的盒子中，使得没有一个盒子为空。试确定 n 个元素 $a_1 \cdots a_n$ 放入 k 个无标号盒的划分数 s(n, k)。

算法分析：

例如 $s=\{1, 2, 3, 4\}$，$k=3$。细心的读者稍加分析后，不难得出 s 有 6 种不同的划分方案，即划分数为 6。其方案为

$\{1, 2\} \cup \{3\} \cup \{4\}$　　$\{1, 3\} \cup \{2\} \cup \{4\}$

$\{1, 4\} \cup \{2\} \cup \{3\}$　　$\{2, 3\} \cup \{1\} \cup \{4\}$

$\{2, 4\} \cup \{1\} \cup \{3\}$　　$\{3, 4\} \cup \{1\} \cup \{2\}$

如果对于任意的 s 集合和 k 值，就不能凭借直觉和经验计算划分数和枚举划分方案了。必须总结出一个数学规律：

设 n 个元素 $a_1 \cdots a_n$ 放入 k 个无标号盒的划分数为 s(n, k)。在配置过程中，有两种互不相容的情况：

1. 设 $\{a_n\}$ 是 k 个子集中的一个子集，于是把 $\{a_1 \cdots a_{n-1}\}$ 划分为 k−1 子集有 s(n − 1, k−1) 个划分数；

2. 如果 $\{a_n\}$ 不是 k 个子集中的一个，即 a_n 必与其它的元素构成一个子集。首先把 $\{a_1, \cdots, a_{n-1}\}$ 划分成 k 个子集，这共有 s(n−1, k) 种划分方式。然后再把 a_n 加入到 k 个子集中的一个子集中去，这有 k 种加入方式。对于每一种加入方式，都使集合划分为 k 个子集，因此由乘法原理知，划分数共有

$$k * s(n-1, k)。$$

应用加法原理于上述两种情况，得出 $\{a_1, \cdots, a_n\}$ 划分为 k 个子集的划分数：

$$s(n, k)=s(n-1, k-1)+k*s(n-1, k)$$

$$(n > 1, k \geqslant 1)$$

下面，我们来确定 s(n, k) 的边界条件：

1. 我们不可能把 n 个元素不放进任何一个集合中去，即 s(n, 0)=0；也不可能在不允许空盒的情况下把 n 个元素放进多于 n 的 k 个集合中去，即 k > n 时 s(n, k)=0。

2. 把 n 个元素放进一个集合或把 n 个元素放进 n 个集合，方式数显然是 1。即

$$s(n, 1)=1$$

$$s(n, n)=1$$

显然，通过上述分析可得出划分数 s(n, k) 的递归关系式：

$s(n, k)=s(n-1, k-1)+k*s(n-1, k)$　　$(n>k, k\geqslant 1)$

$s(n, k)=0$　　$(n<k)$ 或 $(k=0<n)$

$s(n, k)=1$　　$(k=1)$ 或 $(k=n)$

按照划分数 $s(n, k)$ 的递归定义，可以直接写出它的递归函数 $S(n, k)$

```
function s(n, k: integer) of word;
begin
  if(k=0) or(k>n) then s:=0
                  else if( k=1) or(k=n) then s:=1
                                        else s:=s(n-1, k-1)+k*s(n-1, k);
end; {s}
```

【例 2】背包问题

设有一个背包，可以放入的重量为 s。现有 n 件物品，重量分别为 $w_1, w_2, \cdots, w_n$，并假设 $w_i(1\leqslant i\leqslant n)$ 均为正整数，且顺序存放在数组 w 中（w: array [1..n] of integer）。现要求设计一个布尔函数 knap(s_0, n_0)，如果从这 n 件物品中选择 n_0 件放入此背包，使得放入的重量之和正好为 s_0，函数返回 true，并输出一组被选中的各物品的重量。否则函数返回 false。

算法分析：

背包问题可以递归定义如下：

knap(s,n)=
- true　当 s=0;　{只要不选任何物品放入背包，可使 s=0 的背包问题总有解}　边界条件
- false　当 s<0;　{无论怎样选择总不能使重量之和为负}
- false　当 s>0 且 n<1;　{不取任何东西不可能使重量为正值}
- knap(s, n-1)　{当选择的一组物品中不包含 w_n}
- knap(s-w_n, n-1)　{当选择的一组物品中包含 w_n，即 knap(s-w[n], n-1)=true}

上述递归定义是确定的，因为每递归一次，n 都减 1，s 也可能减少 w_n，所以递归若干次后，一定会出现 $s\leqslant 0$ 或者 $n=0$，无论那种情况都可由递归边界明确定值。

按照背包问题的递归定义，可以直接写出它的递归函数 knap(s, n)说明。任给一对正整数 s_0 和 n_0，调用 knap(s_0, n_0)便能求出背包问题的解。说明一下，函数中输出一组被选中的各物品的重量可能不是唯一的，算法中给出的只是其中一种。

```
program kic;
const M=10; {物品的件数}
var
 w: array [1..M] of integer; {W[i]—第 i 件物品的重量}
 x, y, i: integer; {x, y—选中的物品的重量和及其件数}
 f: boolean; {方案产生标志}
function knap(s, n:integer): boolean;
```

```
begin
 if s=0 then knap:=true {方案产生}
        else if (s<0) or ((s>0) and (n<1))
       {产生的不合理结果说明方案不可能存在}
            then knap:=false
            else begin
              if knap(s-w[n], n-1)=true {选中物品 n}
                then begin
                  writeln('number:', n: 4,'      weight:', w[n]: 4);
                  knap:=true;
                  end  {then}
                else knap:=knap(s, n-1);
                {在除物品 n 外的 n-1 件物品中递归选择}
              end;  {else}

end;  {knap}
begin
 fillchar(w, sizeof(w), 0); {初始化}
 write('object number='); {输入选中的物品的件数}
 repeat readln(y); until y<=M;
 write('total weight='); {输入选中物品的重量和}
 readln(x);
 for i:=1 to y do read(w[i]); {输入每件物品的重量}
 f:=knap(x, y); {递归求解背包问题}
 if not(f) then writeln('not find');
 {若背包中放不下重量和为 X 的 Y 件物品，则输出无解信息}
end.  {main}
```

§ 1.4 分治法

有许多算法在结构上是递归的：为了解决一个给定问题，算法要一次或多次地调用其自身来解决相关的子问题。这些算法通常采用分治策略：将原问题分成 n 个规模较小而结构与原问题相似的子问题。递归地解这些子问题，然后合并其结果就得到原问题的解。n=2 时的分治法又称二分法。

用分治法求解一个问题，所需的时间是由子问题的个数，大小以及把这个问题分解为子问题所需的工作总量来确定的。一般来说，二分法（即把任意大小的问题尽可能地等分为两个子问题）较为有效。

分治法在每一层递归上都有三个步骤：

分解：将原问题分解成一系列子问题；

解决：递归地解各子问题。若子问题足够小，则直接解；

合并：将子问题的结果合并成原问题的解；

【例 1】合并排序

对序列 A〔1〕，A〔2〕，……，A〔n〕进行合并排序。

算法分析：

合并排序的算法就是二分法。

分解：将 n 个元素各含⌈n/2⌉(或⌊n/2⌋)个元素的子序列；

解决：用合并排序法对两个子序列递归地排序；

合并：合并两个已排序的子序列以得到排序结果。

在对子序列排序时，当其长度为 1 时递归结束。单个元素被视为是已排好序的。合并排序的关键步骤在于合并步骤中产生的两个已排好序的子序列

A〔P..Q〕和 A〔Q+1..R〕

将它们合并成一个已排好序的子序列〔P..R〕。我们引入一个辅助过程 merge(A, P, Q, R) 来完成这一合并工作，其中 A 是数组，P, Q, R 是下标。这个过程如果用玩扑克来比喻，就可以看作桌上有两堆牌，每一堆都是排好序的，最小的牌在最上面，我们希望把这两堆牌合并成排好序的一堆加以输出。基本步骤包括：先取面朝上的两堆牌的顶上两张中较小的一张，将它取出面朝下地放到输出堆中。重复这个步骤直到某一输入堆为空。这时把另一个输入堆中余下的牌面朝下放入即可。

```
Procedure Merge(Var A: ListType; p, q, r : Integer);
{将两个已排好序的子序列 A[p..q] 和 A[q+1..r] 合并成一个已排好序的序列 A[p..r]}
Var
  i, j, t : Integer; {左子序列指针; 右子序列指针; 合并后的序列的指针}
  Lt      : ListType; {暂存合并的序列}
Begin
  t := p; i := p; j := q + 1;
  While t <= r Do Begin {若合并未完成，则循环}
    If (i <= q) And ((j > r) Or (A[i] <= A[j])) Then Begin
{若左序列剩有元素并且右序列元素全部合并或者左序列的首元素小于等于右序列的首元素，则左序列的首元素进入合并序列，左序列的首指针+1}
      Lt[t] := A[i]; Inc(i)
    End {then}
    Else Begin {否则左序列的首元素进入合并序列，右序列的首指针+1}
      Lt[t] := A[j]; Inc(j)
    End; {else}
    Inc(t) {合并序列的指针+1}
  End; {while}
```

```
 For i := p to r Do A[i] := Lt[i] {合并后的序列赋给 A}
End; {Merge)
```

现在就可以把 merge 过程作为合并排序的一个子过程来用了。下面是 merge _sort (A, P, R) 对数组 A〔P..R〕进行排序。若 P ≥ R, 该子数组至多只有一个元素，当然就是已排序的。否则，分解步骤就计算出一个中间下标 Q，而将 A〔P..R〕分成 A〔P..Q〕和 A〔Q+1..R 〕。若数组 A〔P..R〕的元素个数 K=R−P+1 为偶数，则两个数组各含 K/2 个元素；否则 A〔P..Q〕含[k/2]+1(⌈k/2⌉)元素、A〔Q+1, R〕含 [k/2] (⌊k /2⌋)个元素。

```
Procedure Merge_Sort (Var A : ListType; p , r : Integer);
Var
  q : Integer;
Begin
  If p <> r Then Begin{若子序列 A 中不止一个元素 }
    q := (p + r − 1) div 2; {计算中间下标 q}
    Merge_Sort(A, p, q); {继续对左子序列 A[p..q]递归排序 }
    Merge_Sort(A, q + 1, r); {继续对右子序列 A[q+1..r]递归排序 }
    Merge(A, p, q , r) {对左子序列和右子序列合并排序 }
  End {then}
End; {Merge_ sort}
```

我们只要调用 merge _sort(A, 1, N)便可对整个序列 A〔1〕…… A〔n〕进行合并排序。如果我们自底向上（此处“底”为 n 是 2 的幂时）来看这个过程的操作时，算法将两个长度为 1 的序列合并成排好序的长度为 2 的序列，继而合并成长度为 4 的序列…… ，依次类推。随着算法自底向上执行，被合并的排序序列长度逐渐增加，一直进行到将两个长度为 n/2 的序列合并成最终排好序的长度为 n 的序列。下图列出了对序列 (5, 2, 4, 6, 1, 3, 2, 6) 进行合并排序的过程。

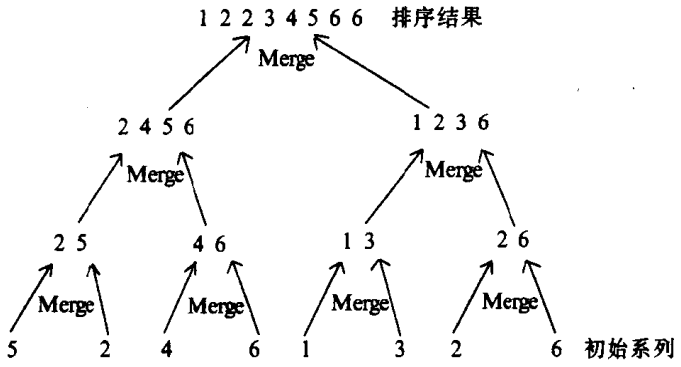


图 1.4 - 1

当然，排序算法很多， 但合并排序法也有其本身的特点—— 当输入的规模足够大

时，合并排序的运行时间要比最坏情况下的插入排序好。分治算法不仅可应用于排序，而且还可应用于许多重要的场合，甚至有些问题看起来非得采用分治法求解不可。

【例 2】导线和开关

如（图 1.4 - 2）所示，具有 3 根导线的电缆把 A 区和 B 区连接起来。在 A 区 3 根导线标以 1, 2, 3; 在 B 区导线 1 和 3 被连到开关 3，导线 2 连到开关 1。

一般说来，电缆含 m（1 ≤ m ≤ 90）根导线，在 A 区标以 1, 2, … m。在 B 区有 m 个开关，标为 1, 2, … m。每一根导线都被严格地连到这些开关中的某一个上；每一个开关上可以连有 0 根或多根导线。

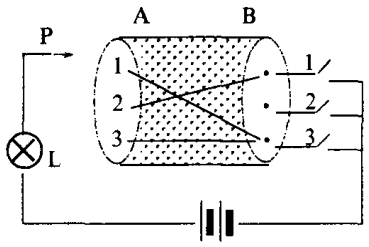


图 1.4 - 2

测量:

你的程序应作某些测量来确定导线和开关怎样连。每个开关或处于接通或处于断开状态，开关的初始状态为断开。我们可用一个探头（probe）P 在 A 区进行测试：如果探头点到某根导线上，当且仅当该导线连到处于接通状态的开关时，灯 L 才会点亮。

你的程序从标准输入（standard input）读入一行以得到数字 m；然后可以通过向标准输出（standard output）写入一行以发出命令（共 3 种命令）。每种命令的开头是一个大写字母:

○ 测试导线命令 T: T 后面跟一个导线标号;

○ 改变开关状态命令 C: C 后面跟一个开关标号;

○ 完成命令 D: D 后面跟的是一个表列（LIST），该表列中的第 i 个元素代表与导线 i 相连的开关号。

在命令 T 和 C 之后，你的程序应从标准输入（standard input）读入一行。若开关状态能使灯亮，则命令 T 的回答应是 Y；反之，回答应是 N。命令 C 的作用是改变开关的状态（若原来是接通则变为断开；若原来是断开则变为接通）。对 C 命令的回答是作为一种反馈信号。

你的程序可以给出一系列命令，将 T 命令与 C 命令以任意顺序混合使用。最后给出命令 D，并结束。你的程序给出的命令总数应不大于 900。

注意:

为了在此任务中能正确使用标准输入（standard input）和标准输出（standard output）。若你使用 pascal，请不要使用其中的 CRT 单元（unit CRT）。

举例:

图 1.4 - 3 给出了一个实例，对应于图 1.4 - 2，这是一个有 8 条命令的对话。

Standard Output	Standard Input
C 3 T 1 T 2 T 3 C 3 C 2 T 2 D 3 1 3	3 Y Y N Y N Y N

图 1.4 - 3

算法分析:

这是一道通过人机对话解答的程序题。程序执行时，由计算机发出 T 或 C 命令。 用户通过键盘输入应答信号。人与计算机间几次交互信息后，最终由计算机给出命令 D 并显示导线和开关是怎样连接的。对于编程序者来说，要解决的问题是如何设计命令顺序，使得用户在依次应答后，程序能自动给出问题的解。

1. T命令和C命令的作用

初始时，B 区 M 个开关断开。此时无论用探头 P 在 A 区测哪根导线，灯 L 也不会亮。因此，程序首先必须使用 C 命令和用户应答将 B 区的部分开关闭合。然后发出测试某些导线的 T 命令，通过用户应答确定这些导线在 B 区的准确位置或范围。若尚有导线未连接，则再通过 C 命令和用户应答来改变某些开关的状态，随后发出 T 命令 ……，C 命令和 T 命令交替使用，直到完成导线和开关的连接 。例如 M=3。初始时

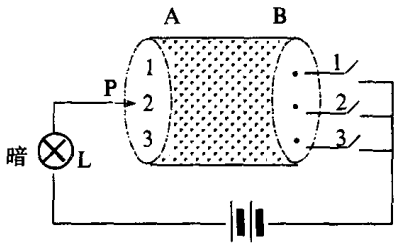


图 1.4 - 4

对话序号	计算机命令	用户应答	结果
1	C3	Y	开关 3 闭合
2	T1	Y	L 灯亮，导线 1 与开关 3 连接
3	T2	N	导线 2 不与开关 3 连接
4	T3	Y	导线 3 与开关 3 连接

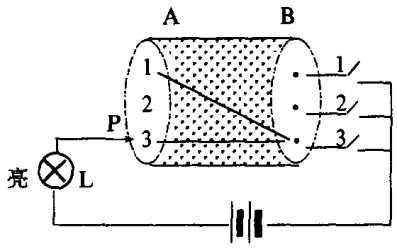


图 1.4 - 5

对话序号	计算机命令	用户应答	结果
5	C3	N	开关 3 断开
6	C2	Y	开关 2 闭合
7	T2	N	导线 2 不与开关 2 连结

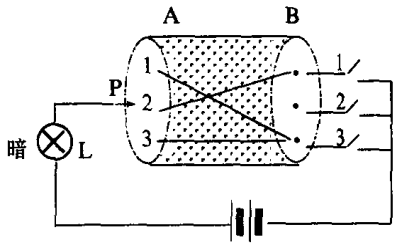


图 1.4 - 6

由此可见，改变开关状态的 C 命令在人机对话中最先使用，然后发出 T 命令测试部分导线。若不能确定每根导线的连接位置，则还得对一些开关使用 C 命令，改变其开关状态，直到最后一个 T 命令完成导线与开关的连接为止。注意，用户对 C 命令的回答仅是作为开关状态改变的一种反馈信号。即无论用户怎样应答， 开关状态总是由应答前的断开变为应答后的闭合，或者由应答前的闭合变为应答后的断开。

继 C 命令后使用 T 命令测试导线。测试导线 i 的过程中:

若仅一个开关闭合且灯亮 (即 Ti 的应答为 Y)，则被测导线 i 与闭合的开关连接;

若仅一个开关断开且灯暗 (即 Ti 的应答为 N)，则被测导线 i 与断开的开关连接;

若多个开关闭合时灯亮 (Ti 的应答为 Y)，或者多个开关断开时灯暗 (Ti 的应答为 N)， 而先前的测试已确定导线 i 只能与这些开关中的某一个相连，则导线与之相连。

例如: 第 7 次对话，用户应答 T2 的命令为 N，而此时开关 1、3 断开，看来导线 2 既可能与开关 1 相连，也可能与开关 3 相连。但是先前第 3 次对话，用户应答 T2 命令为 N，确认导线 2 不与开关 2 连接。无庸置疑，导线 2 的连接对象仅一个 —— 开关 1。

2. 用二分法连接

为了使导线和开关间的连接工作有规律地进行，我们不妨采用二分法。

设当前待连接的开关为 HEAD..TAIL，初始时为 1..M。将这些开关一分为二:

左区间 HEAD..[(HEAD+TAIL−1)/2];

右区间 [(HEAD+TAIL−1)/2]+1..TAIL;

连接到左区间开关的导线集合为 P_1。初始时 P_1 = 〔1..M〕;

连接到右区间开关的导线集合为 P_2。显然初始时 P_2 = 〔 〕;

左区间或右区间开关状态是同一的，设为 STATE。

STATE = 1　　区间的所有开关闭合;

　　0　　区间所有开关断开;

显然初始时 STATE=0;

首先，P_2 集合设空并发出 C 命令，将左区间的所有开关取反。然后对 P_1 集合中的每根导线发出 T 命令。若用户应答 N（灯暗）而左区间开关闭合，或者用户应答 Y（灯亮）而左区间开关断开，则说明相连的导线不在 P_1 中，应从 P_1 集合移入 P_2 集合。

接下去，设左区间开关为待接开关，与之相连的导线集合为 P_1，开关状态已由先前的 C 命令取反。继续上述过程，直至出现下述两种情况之一为止:

① P_1 的集合为空;

② 区间内的开关仅剩一个，将 P_1 集合中所有导线连接到这个开关；最后再将右区间开关设为待连接开关，与之相连的导线集合为 P_2，开关状态不变。仍按上述方法进行连接。

显然，可以用一个二分法的递归过程来描述导线与开关间的连接:

```
procedure check (P1, 开关区间, state);
begin
if P1 〈 〉〔 〕 then
if  区间内剩一个开关
then P1 集合中的所有导线与之相连
else begin
通过 C 命令和用户应答将左区间取反;
P2 集合设空;
对 P1 集合中的每根导线发出 T 命令:
if  ((用户应答 N) and (state=0)) or ((用户应答 Y)  and (state=1))
then  导线从 P1 集合移出，移入 P2 集合;
check (P1, 左区间, 1-state);
check (P2, 右区间, state);
end;  {else}
end;  {check}
```

例如 设 M=3

初始时，设待连接开关区间为 1..3，P_1 = 〔1，2，3〕，STATE = 0。

执行 CHECK (P_1, 〔1..3〕, 0)

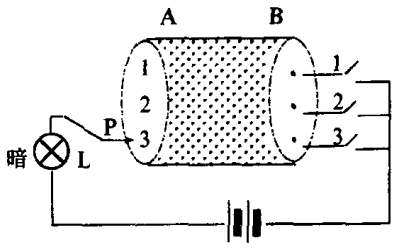


图 1.4 - 7

计算机命令	用户应答
C_1	Y
T_1	N
T_2	Y
T_3	N

使得 P_1 = 〔2〕，P_2 = 〔1，3〕

执行 CHECK (P_1，〔1〕，1〕后，导线 2 连至开关 1。

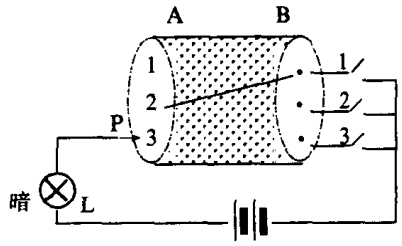


图 1.4 - 8

然后通过 CHECK (P_2，〔2，3〕，0)连接 P_2 集合中的导线与右区间的开关

计算机命令	用户应答
C_2	Y
T_1	N
T_3	N

使得 P_1=〔 〕，P_2=〔1, 3〕，待连接的开关仅剩 3。

执行 CHECK (P_2，〔3〕，0)后，导线 1 和导线 3 分别连至开关 3。

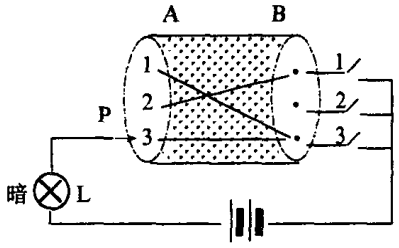


图 1.4 - 9

下面给出程序题解:

```
program wires;

type
  ptype =set of byte;

var
  p1: ptype; {连接左区间开关的导线集合}
  ans: array[1..50] of byte; {ans[i] ——连接导线 i 的开关序号}
  m, i : integer; {导线数，辅助变量}


procedure check(p1: ptype; head, tail, state:integer);
{输入区间左右指针 head 和 tail 和该区间内开关的状态 state; 连接左区间开关的导线集合
p1。运用二分法计算和输出连接方案}
  var
    p2 : ptype; { 连接右区间开关的导线集合 }
    i, j, n : integer;
    c : char;
  begin
    if p1<> [ ] then {若有导线连接左区间的开关}
    if head=tail {若左区间仅剩一个开关，则 p1 中的所有导线与之连接}
      then begin for i:=1 to m do if i in p1 then ans[i]:=head end
      else begin
        i:=(head+tail-1) div 2;
        for j:=head to i do begin
       {通过发出 c 命令和用户应答,使得左区间开关状态取反}
          writeln ('C ', j);
          readln (c);
        end;
        p2:=[ ]; {连接右区间开关的导线集合设空}
        for j:=1 to m do
          if j in p1 then begin {对连接左区间开关的导线 j 发出 t 命令}
            writeln ('T ', j);
            readln (c);
            if (state=0) and (c='N') or (state=1) and (c='Y') then begin
{若灯暗而左区间开关闭合或者灯亮而左区间开关断开，则导线 j 从 p1 集合移出，移入 p2
集合}
```

```
            p1 :=p1-[j];
             p2 :=p2+[j];
           end;
         end;
       check (p1, head, i, 1-state); {完成 p1 中的导线与左区间开关的连接}
       check (p2, i+1, tail, state); {完成 p2 中的导线与右区间开关的连接}
     end; {else}
  end; {check}

procedure show; {输出连接方案}
  begin
    write ('D');
    for i:=1 to m do
      write (' ', ans [i]);
    writeln;
  end; {show}

begin
  readln (m); {输入导线数}
  for i:=1 to m do p1:=p1+[i]; {所有导线进入 p1 集合}
  check (p1, 1, m, 0); {递归完成导线与开关间的连接}
  show; {输出连接方案}
end. {main}
```

§1.5 枚举法

所谓枚举法，指的是从可能的解的集合中一一枚举各元素，用题目给定的检验条件判定哪些是无用的，哪些是有用的。能使命题成立，即为其解。

从枚举法的定义可以看出，枚举法本质上属于搜索算法。但是它与隐式图的搜索（第六章）有所不同，因为适用于枚举法求解的问题必须满足两个条件：

1. 可预先确定解的个数 n;

2. 解变量 A_1, A_2, ……A_n 的值的可能变化范围预先确定

$$A_1 \in \{X_{11}, \cdots\cdots, X_{1p}\}$$
$$\cdots\cdots$$
$$A_i \in \{X_{i1}, \cdots\cdots, X_{iq}\}$$
$$\cdots\cdots$$
$$A_n \in \{X_{n1}, \cdots\cdots, X_{nk}\}$$

例如某问题的解变量有 3 个：A_1, A_2, A_3。其中

A_1 解变量值的可能范围 $A_1 \in \{X_{11}, X_{12}, X_{13}\}$

A_2 解变量值的可能范围 $A_2 \in \{X_{21}, X_{22}, X_{23}\}$

A_3 解变量值的可能范围 $A_3 \in \{X_{31}, X_{32}, X_{33}\}$

则问题的可能解有 27 个

$$(A_1, A_2, A_3) \in \{(X_{11}, X_{21}, X_{31}), (X_{11}, X_{21}, X_{32}), (X_{11}, X_{21}, X_{33})$$
$$(X_{11}, X_{22}, X_{31}), (X_{11}, X_{22}, X_{32}), (X_{11}, X_{22}, X_{33})$$
$$\cdots\cdots$$
$$(X_{13}, X_{23}, X_{31}), (X_{13}, X_{23}, X_{32}), (X_{13}, X_{23}, X_{33})\}$$

在上述可能解的集合中，满足题目给定的检验条件的解元素，即为问题的解。

一般来说，如果预先对问题确定了解的个数以及各个解的值域，我们则可以利用 for 语句和条件判断语句逐步求解或证明：

```
for A1 :=X11 to X11 do
 ..............................
for Ai :=Xi1 to Xiq do
 ..............................
for An :=Xn1 to Xnk do
  if (A1, …, Ai, …, An) 满足检验条件
     then 输出问题的解 (A1, …, Ai, …, An);
```

如果各值域中的值呈等差数列，即

数列 $X_{11}, \cdots, X_{1p}$ 的公差为 d_1；

……

数列 $X_{i1}, \cdots, X_{iq}$ 的公差为 d_i；

……

数列 $X_{n1}, \cdots, X_{nk}$ 的公差为 d_n；

则利用 while 语句和条件判断语句求解或证明，其效率要好得多。

```
A1 ←X11;
while A1 ≤ X1p do begin
  ..................
  Ai ←Xi1;
  while Ai ≤ Xiq do begin
  ..............................
    An ←Xn1;
    while  An ≤ Xnk do begin
   ..................................
     if (A1, ……, An) 满足检验条件 then 输出问题的解 (A1, ……, An);
   ..................................
```

$A_n \longleftarrow A_n + d_n$;
end; {while A_n}
.....
$A_i \longleftarrow A_i + d_i$;
end; {while A_i}
.....
$A_1 \longleftarrow A_1 + d_1$;
end; {while A_1}

如果我们无法预先确定解的个数或各解的值域，我们只能采用隐式图搜索的算法求解，例如回溯法等。由于回溯搜索，每个可能解的枚举一般不止一次，因此在相同的检验条件下，回溯法要比枚举法费时一些。

【例 1】填写运算符

输入任意 5 个数

x1, x2, x3, x4, x5

每相邻两个数间填上一个运算符。在填入四个运算符后，使得表达式值为一个指定值 y(y 由键盘输入)。求出所有满足条件的表达式。

算法分析:

看了题目之后，我们一时难于找到解法，就模仿人去试填运算符。为了不致遗漏，则按枚举的方法填，先镇四个‘+’。表达式自成为检验条件。如果不成立，则将第四个运运算符改‘-’，以后改‘*’、改‘/’。轮完一遍，把第三个运算符改‘-’，第四个运算符按‘+、-、*、/’顺序再轮一遍，……，如此下去，直至第一个运算符，由‘+’至‘/’轮完为止。

要各个运算符均按‘+、-、*、/’轮一遍，则要组织四重循环。若当前运算符轮到‘/’则运算符右端的数必须非零，因为零不能当除数。现在接着考虑‘+、-、*、/’应如何表示，才能方便程序对表达式的求值？再想想如何处理“先乘除/后加减”的优先顺序?

我们在计算时，把‘+、-、*、/’当作 1、2、3、4 放在运算符数组 i[1] … i[4] 中。例如 i[3]=4 表示第三个运算符为‘/’。为了解决运算的优先级问题，我们设置如下变量:

f——减法标志。减法运算时，置 f=-1，否则 f=1;

q——若当前运算符为+(-)时，q 存贮运算符的左项值；若当前运算符为*(/)时，q 存贮两数乘(除)后的结果;

p——累加器。每填一个算符 p=p+f*q。

每填入四个算符后，我们设 f=1、q=第 1 项的值，p=0，然后由左至右计算表达式的值。当分析至第四个算符时，f=是第四次运算符的减标志，q 是第五项值，p 是前三次运算的累加值。此时只要检验 p+f*q=y 是否成立。若成立，则为一个方案；否则重新选择算符。

下面给出程序题解:

```
program fu_hao;
 const
  l=5;
  c: array [1..4] of char=('+','-','*','/');
 var
  n:array [0..l] of integer; {n[0]— 表达式值; n[1] … n[5]— 表达式各项的值}
 procedure init;
  var
   i: integer;
  begin
   writeln;
   for i:=1 to l do{输入表达式各项的值}
    begin
     write ('input number', i, ':'); readln(n[i])
    end; {for}
   write ('input result:'); readln(n[0]); {输入表达式的值}
   writeln
  end; {init}

 procedure make;
  var
   i: array [1..l-1] of 1..4; {运算符数组}
   k, f: integer; {辅助变量, 减标志}
   p, q: real; {累加器, 当前项(或*、/的中间结果)}
   total: integer{方案数}
  begin
   total:=0; {方案数初始化}
   for i[1]:=1 to 4 do{按+、-、*、/顺序试填第一个算符}
    if (i[1]<4) or (n[2]<>0) then {不允许第一个算符为/且右式为0的情况}
     for i[2]:=1 to 4 do{按+、-、*、/顺序试填第二个算符}
      if (i[2]<4) or (n[3]<>0) then {不允许第二个算符为/且右式为0的情况}
       for i[3]:=1 to 4 do{按+、-、*、/ 顺序试填第三个算符}
        if (i[3]<4) or (n[4]<>0) then {不允许第三个算符为/且右式为0的情况}
         for i[4]:=1 to 4 do{按+、-、*、/顺序试填第四个算符}
          if (i[4]<4) or (n[5]<>0) then {不允许第四个算符为/且右式为0的情况}
           begin
            p:=0; q:=n[1]; f:=1;
```

```
            for k:=1 to l-1 do
              case i[k] of
               1: begin {+}
                   p:=p+f*q; f:=1; q:=n[k+1]
                  end; {1}
               2: begin{-}
                   p:=p+f*q; f:=-1; q:=n[k+1]
                  end; {2}
               3: q:=q*n[k+1]; {*}
               4: q:=q/n[k+1] {/}
              end; {case}
             if p+f*q=n[0] then {若表达式成立，则打印方案}
              begin
               inc(total); write (total: 5, ':');
               for k:=1 to l-1 do
                write (n[k], c[i[k]]);
               writeln (n[1], '=', n[0]);
               if total mod 20=0 then {每20个方案为一屏，逐屏显示}
                begin
                 writeln ('Press Enter to continue …');
                 readln
                end {then}
              end {then}
            end {then}
 end; {make}

begin
 init; {输入初始值}
 make; {枚举所有方案}
 writeln ('The End');
 readln
end. {main}
```

枚举法的特点是算法简单，对于可确定解的值域又一时找不到其它好的算法时就可用它。但枚举法又是一种比较笨拙、原始的方法，运算量大是它的弱点。例如本例替换算符要执行 $4^4=256$ 次，这对每秒执行上百万次运算的计算机来说，只是瞬间的事。但是若数字的个数超过 20，枚举法恐怕就无能为力了，因为替换的算符愈多，则循环的阶数愈大、执行的速度愈慢。

对于枚举法也不要过于悲观，从全局观点使用枚举法，计算量容易过大。但是，如果能够排除那些明显不属于解集的元素，在局部地方使用枚举法，其效果会十分显著。

【例 2】时针问题

在图 1.5 - 1所示的 3 × 3 阵列中有 9 个时钟，我们的目标是旋转时钟指针，使所有时钟的指针都指向 12 点。允许旋转时钟指针的方法有 9 种，每一种移动用一个数字号(1, 2, …, 9)表示。图 1.5 - 2 示出了 9 个数字号与相应的受控制的时钟，这些时钟在图中以灰色标出，其指针将顺时针旋转 90°。

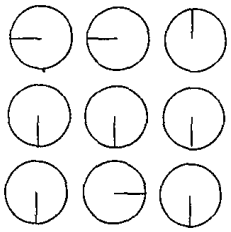


图 1.5 - 1

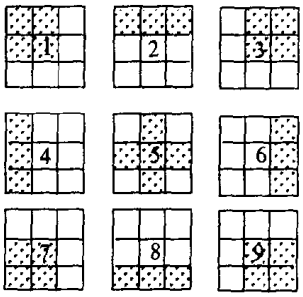


图 1.5 - 2

输入数据:

由输入文件 INPUT. TXT 读 9 个数码，这些数码给出了 9 个时钟时针的初始位置。数码与时刻的对应关系为:

0－12 点;

1－3 点;

2－6 点;

3－9 点。

(图 1.5-1) 中的例子对应下列输入数据:

3 3 0

2 2 2

2 1 2

输出数据:

将一个最短的移动序列 (数字序列) 写入输出文件 OUTPUT. TXT 中，该序列要使所有的时钟指针指向 12 点，若有等价的多个解，仅需给出其中的一个。 在我们的例子中，相应的 OUTPUT. TXT 的内容为:

5849

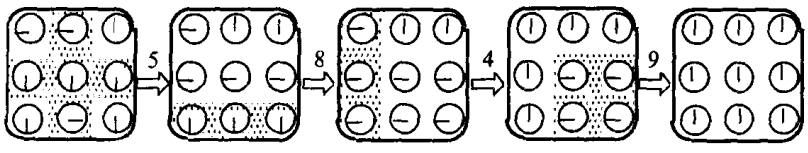


图 1.5 - 3

算法分析:

首先，我们分析一下表示时钟时针初始位置的数码 j (0 ≤ j ≤ 3) 与时刻的对应关系:

0——12 点

1——3 点

2——6 点

3——9 点

移动一次，其时针将顺时针旋转 90 度。由此我们可以得出:

对于任意一个时钟 i (1 ≤ i ≤ 9) 来说，从初始位置 j 出发至少需要 $C_i=(4-j) \bmod 4$ 次操作，才能使得时针指向 12 点。

但试题不允许单个时钟动作，它给出了 9 种旋转时钟时针的方法，每一种方法都使其中若干个时钟同时顺时针旋转 90 度。

从 (图 1.5 -2) 给出的 9 种移动方法所分别控制的时钟范围可以看出:

时钟 1 受第 1、2、4 种移动方法所控制;

时钟 2 受第 1、2、3、5 种移动方法所控制;

时钟 3 受第 2、3、6 种移动方法所控制;

时钟 4 受第 1、4、5、7 种移动方法所控制;

时钟 5 受第 1、3、5、7、9 种移动方法所控制;

时钟 6 受第 3、5、6、9 种移动方法所控制;

时钟 7 受第 4、7、8 种移动方法所控制;

时钟 8 受第 5、7、8、9 种移动方法所控制;

时钟 9 受第 6、8、9 种移动方法所控制;

每一种移动方法用一个 1, 2, …, 9的数字表示。试题要求解出使所有时钟时针都指向12点的一个最短的移动序列 (数字序列)。

我们通过仔细分析题意，可以得出一些规律:

1. 在搜索一个最短序列的过程中，每种移动方法要么不取，要么取 1 次或 2 次或 3 次。(因为每一次操作使得时针顺时针旋转 90 度，操作四次后又恢复原来状态)。9 种旋转方法则可能产生 4^9 个状态;

2．最短移动序列中，数字的排列顺序可以任意。例如，对于如下 9 个时钟时针的初始位置

3 3 0

2 2 2

2 1 2

移动序列 5849 可使所有时钟的指向 12 点。但按序列 4589 的顺序操作， 也可得出同样的结果。因此，对任一个最短移动序列进行从小至大的排序，然后从初始时刻出发，按排序后的序列操作，同样可以得到一个到达目标状态的最佳方案。

设 p_i —— 第 i 种旋转方法的使用次数。($0 \leq p_i \leq 3$, $1 \leq i \leq 9$)

我们采用枚举法，将每种旋转方法可能取的四种状态 (0 表示不取，1、2、3 分别表示使用该种旋转方法 1 次或 2 次或 3 次) 组成可能的解的集合。

$\{P_1, P_2, \cdots\cdots, P_9 \mid 0 \leqslant P_i \leqslant 3, 1 \leqslant i \leqslant 9\}$

该集合共含 4^9 个状态。然后用下述检验条件

$C_1 = (P_1 + P_2 + P_4) \bmod 4$

$C_2 = (P_1 + P_2 + P_3 + P_5) \bmod 4$

$C_3 = (P_2 + P_3 + P_6) \bmod 4$

$C_4 = (P_1 + P_4 + P_5 + P_7) \bmod 4$

$C_5 = (P_1 + P_3 + P_5 + P_7 + P_9) \bmod 4$

$C_6 = (P_3 + P_5 + P_6 + P_9) \bmod 4$

$C_7 = (P_4 + P_7 + P_8) \bmod 4$

$C_8 = (P_5 + P_7 + P_8 + P_9) \bmod 4$

$C_9 = (P_6 + P_8 + P_9) \bmod 4$

判定哪些是无用的，哪些是有用的。由满足上述 9 个条件的 $P_1 \cdots P_9$ 值得出最短移动序列含哪几种移动方法，每种移动方法使用了几次。

上述枚举算法枚举了所有 4^9 个状态，运算量和运行时间颇大。我们可以采取缩小可能解范围的局部枚举法，仅枚举第 1、2、3 种旋转方法可能取的 4^3 个状态，根据这三种旋转方法的当前状态值，由下述公式

$P_4 = \text{order}\ (C_1 - P_1 - P_2);$

$P_6 = \text{order}\ (C_3 - P_2 - P_3);$

$P_5 = \text{order}\ (C_2 - P_1 - P_2 - P_3);$

$P_7 = \text{order}\ (C_4 - P_1 - P_4 - P_5);$

$P_9 = \text{order}\ (C_6 - P_3 - P_5 - P_6);$

$P_8 = \text{order}\ (C_8 - P_7 - P_9 - P_5);$

其中 order (C)=

$$\begin{cases} C \bmod 4 & C > 0 \\ (C + N*4) \bmod 4 & C < 0 \\ (0 \leqslant (N*4 + C) \leqslant 3) & \end{cases}$$

得出其余 $P_4 \cdots\cdots P_9$ 的相应状态值。然后将 $P_1 \cdots\cdots P_9$ 代入下述三个检验条件

$C_7 = (P_4 + P_7 + P_8) \mod 4$

$C_5 = (P_5 + P_1 + P_3 + P_7 + P_9) \mod 4$

$C_9 = (P_6 + P_8 + P_9) \mod 4$

一一枚举，以求得确切解。

由此可见，上述局部枚举的方法（枚举状态数为 4^3 个）比较全部枚举的方法（枚举状态数为 4^9 个）来说，由于大幅度削减了枚举量，减少运算次数，因此其时效显著改善，是一种优化了的算法。

下面给出程序题解:

```
PROGRAM Clocks;
USES
  Crt;{调用屏幕单元}
VAR
  Clock, Map          : array [1..3, 1..3] of integer;
  {CLOCK[i, J]——第((i+2) MOD 3)*3+J个时钟从初始时间到12点需经过的最少移动次数;
  {MAP [i, J]——最短移动序列中，数字((i+2) MOD 3)*3+J的次数}
  f                   : text;{文件变量}
PROCEDURE Init;
  var
    i, j : integer;
    fn   : string;{文件名串}
  begin
    writeln ('Input File name : ');{读入文件名}
    readln (fn);
    assign (f, fn);{文件名与文件变量连接}
    reset (f);{读准备}
    for i:=1 to 3 do
  {从文件中读入9个时钟指针的初始位置，并求出每个时钟从初始时间到12点的最少移动次数}
      for j:=1 to 3 do begin
        read(f, Clock[i, j]);
        Clock[i, j]:=(4-Clock[i, j]) mod 4;
      end;{for}
    close(f);{关闭文件}
  end;{init}
FUNCTION Order (k: integer): integer;
{输入整数K。若K>0,返回K MOD 4;否则返回(K+N*4)MOD 4(0≤(N*4+K)≤3)}
  var
    c : integer;
  begin
    c:=k;
    while c<0 do inc(c, 4);
    while c>4 do dec(c, 4);
    Order:=c;
  end; {order}
PROCEDURE Main;{计算和输出最短移动序列}
```

```
var i, j : integer;
  begin
  {枚举最短移动序列中数字 1、2、3 的个数可能取的 4^3 种状态}
    for Map[1, 1]:=0 to 3 do
      for Map[1, 2]:=0 to 3 do
        for Map[1, 3]:=0 to 3 do begin
          Map[2, 1]:=order(Clock[1, 1]-Map[1, 1]-Map[1, 2]);
          Map[2, 3]:=order(Clock[1, 3]-Map[1, 2]-Map[1, 3]);
          Map[2, 2]:=order(Clock[1, 2]-Map[1, 1]-Map[1, 2]-Map[1, 3]);
          Map[3, 1]:=order(Clock[2, 1]-Map[1, 1]-Map[2, 1]-Map[2, 2]);
          Map[3, 3]:=order(Clock[2, 3]-Map[1, 3]-Map[2, 2]-Map[2, 3]);
          Map[3, 2]:=order(Clock[3, 2]-Map[3, 1]-Map[3, 3]-Map[2, 2]);
          {根据数字 1、2、3 个数的当前值，得出数字 4..9 的个数}
          if ((Map[2, 1]+Map[3, 1]+Map[3, 2]) mod 4 = Clock[3, 1]) and
             ((Map[2, 3]+Map[3, 2]+Map[3, 3]) mod 4 = Clock[3, 3]) and
             ((Map[2, 2]+Map[1, 1]+Map[1, 3]+Map[3, 1]+Map[3, 3])
                  mod 4 = Clock[2, 2]))
         {若数字 4..9 的个数满足检验条件，则输出方案}
          then begin
                 for i:=1 to 3 do
                   for j:=1 to 3 do writeln((i+2) mod 3*3+j, ':' Map[i, j])
                exit; {给出一个解后退出}
               end; {then}
        end; {for}
    writeln('No answer!!!');
  end; {main}
BEGIN
  Init; {输入每个时钟的初始时间并计算从初始时间到 12 点的最少移动次数}
  Main; {计算和输出最短移动序列}
END. {main}
```

§1.6 模拟法

有些自然界和日常生活中的事件，若用计算机很难建立枚举、递归等算法，甚至于建立不了数学模型。例如下题：

【例 1】猜数游戏

人和计算机作猜数游戏。人默想一个四位数，由计算机来猜。计算机将所猜的数显示到屏幕上，并问两个问题：

其一，有几个数字猜对了？

其二，猜对的数字中有几个位置也对？

人通过键盘来回答这两个问题。计算机一次又一次地猜，直到猜对为止。

比如人默想的一个数是 5122，假定计算机第一次猜 1166，然后问你：

1. 有几个数字猜对了？ 1

2. 猜对的数字中有几个位置也对？ 1

假定计算机第二次猜 1287，然后问你：

1. 有几个数字猜对了？ 2

2. 猜对的数字中有几个位置也对？ 0

如果计算机最后一次猜 5122，然后问你：

1. 有几个数字猜对了？ 4

2. 猜对的数字中有几个位置也对？ 4

则表示猜完了。计算机显示最后猜中的数，并报告共猜了多少次。

问题 1　编程实现这样一个猜数的游戏程序。屏幕显示格式为：

第二行显示计算机所猜的四位数

第三行提问猜对的数字个数，用″ Number:″

第四行提问位置对的数字个数，用″ Position:″

第五行显示当前已猜对的步数，用″ Step xx″

注：其中末尾数字由键盘输入。最后给出结束信息，其它由编程者自定。

问题 2　仍是这样一个游戏，但要求计算机既是猜数者，又要模拟默想这个数的人（要猜的数由键盘输入）。屏幕显示格式为：

第一行显示人所默想的数，用″　　″ xxxx

第二行至第五行同问题 1，只不过末尾数字不再由键盘输入，而是计算机判断后自动显示。

问题 3　从文本文件 GUESS.DAT 中读入 20 个四位数，一个接一个地让计算机猜，统计猜中所需的总步数。

算法分析：

对于这样一道试题，人们只能随机的产生猜想，不可能找到一个下一次猜数一定猜中的计算公式，计算内容仅是猜对数字的个数和猜对位置的数字个数。而试题要求非常明确——猜数步数最少。像这类试题，往往弄得人无所适从。

用程序解决这类问题可用模拟法。所谓模拟法就是模拟某个过程，通过改变数学模型的各种参数，进而观察变更这些参数所引起的过程状态的变化。一般题目给出某一概率（例如猜对数字的个数和猜对位置的数字个数），设计者利用随机函数和取整函数去设定在某一范围的随机值，将符合概率的随机值作为参数。然后根据这一模拟的数学模型展开算法设计。模拟法的关键是在概率已知的条件下，如何确定随机值产生的范围。这个随

机值设计得好，模拟效果就好。

下面我们就来分析一下，如何通过模拟法，以最少步数完成猜数游戏。

1. 运用随机函数产生计算机的猜数

试题要求由计算机来猜对方默想的四位数。既然是猜，计算机产生的数理应随机。为此，在程序中必须调用二个库函数：

randomize —— 将随机数发生器初始化；

random(range)—— 产生一个范围在 0 ≤ x < range 中的 word 类型的随机数 x，表示计算机的一次猜数；

2. 最大限度地缩小猜数范围

设 M_i —— 第 i 次猜数的四位数集合。显而易见，$M_0=\{1000,1001,\cdots\cdots,9999\}$；

[number]i ——第 i 次猜对的数字个数；

[position]i ——第 i 次猜对位置的数字个数；

显然 $[number]_i=[position]i=4$，表示经 i 次猜数后，猜中默想的四位数。

计算机第 1 次从 M_0 范围内随机产生一个猜数后，默想一方将猜数和默想数相比较，求出猜对的数字个数 $[number]_1$ 和位置对的数字个数 $[position]_1$。若 $[number]_1<>4$ 或者 $[position]_1<>4$，计算机则一一比较猜数和 M_0 的每个四位数，将 M_0 中相同数字个数为 $[number]_1$ 且相同数字中位置正确的数字个数为 $[position]_1$ 的四位数保留下来，剔除其余四位数，形成 M_1。显然 $M_1 \subseteq M_0$，$M_1 \neq M_0$，默想数 $\in M_1$。

接下去第 2 次猜数。计算机从 M_1 的范围里随机产生一个猜数。默想一方根据猜数求 $[number]_2$ 和 $[position]_2$。若 $[number]_2<>4$ 或者 $[position]_2<>4$，计算机再将猜数与 M_1 比较，保留据有 $[number]_2$ 和 $[position]_2$ 特征的四位数，剔除其余四位数，形成 M_2。显然 $M_2 \subseteq M_1$，$M_2 \neq M_1$，默想数 $\in M_2$。

.....

第 i 次猜数，依上述方法缩小猜数范围 M_{i-1}，建立新的猜数范围 M_i（$M_i \subseteq M_{i-1}$，$M_i \neq M_{i-1}$，默想数 $\in M_2$）。

直至 n 次猜数后，出现下列两种情况之一：

1. 若 M_n 空，则默想的四位数不存在，猜数失败。
2. 若 $[number]_n=[position]_n=4$，则计算机成功地猜中了默想的四位数，猜数的步数为 n。

通过上述算法，可使计算机在每次猜数后根据默想方给出的信息，最大限度地缩小默想数存在的范围 —— 即猜数随机产生的范围。若每次如实的输入猜对的数字个数和位置对的数字个数，则计算机一定以最小步数 n 猜中这个默想数。

下面，我们运用逐步求精的方法给出程序题解。

第一层

1．初始化

```
uses crt;
```

1.1 定义和说明；

```
begin
  1.2 init;   {初始化}
  repeat      {正确输入选择项}
    在屏幕中央打印' while problem do you want to do? (1-3):';
    choice ← readkey;
  until (choice>='1')  and (choice<='3');
  clrscr;          {清屏}
2．求解问题
  case choice of
    2.1 '1': do1;        {解答问题 1}
    2.2 '2': do2;        {解答问题 2}
    2.3 '3': do3;        {解答问题 3}
  end;  {case}
  在屏幕中央打印' Thanks for using welcome back again!'
  halt (0);              {停止程序执行}
  end.                   {main}
```

第二层

1．求精 1.1——定义与说明

```
type integers=array[0 … 3] of integer;        {将四位整数定义为数组类型}
var
    wait   :boolean;          {等待按键标志  (wait=true)}
    choice:char;              {选择子题的参数 ('1'≤choice≤'3')}

    t1     :integer ;         {猜对的数字个数}
    t2     :integer ;         {猜对位置的数字个数}
    steps  :integer ;         {猜数的次数}
    code   :integer ;
```

{val(s, v, code)函数的参数．val 把字串 s 转换成它的数字表示存入整数 v 中。若转换成功，code=0; 否则 code 为无效字符的序数。}

```
    i      :integer ;         {辅助变量}
    to_guess :integer ;       {numbers 序列的尾指针}
    numbers  :array[0..9000] of integer ;
```

{计算机猜数的集合．每猜一次数，将该集合中的每个四位数与当前所猜数比较，剔除相同数字个数 <>t1 或者相同数字中位置正确的数字个数 <>t2 的四位数，余下的数移至表首一端 (numbers[1]—— numbers[to_guess])，下一次的猜数则从该范围随机产生。}

```
    key      : integers;
```

{函数 test1 和 test2 的形参，存储从 numbers[1..toguess]中随机产生的一个四位正数}

```
now      :integers;   {函数 delete 的形参，存储当前所猜的数}
```

至于函数和过程说明，则在下面逐一给出。

2．求精 1.2 —— init 的过程说明

```
procedure  init;
var
  i : integer;
begin
 to_guess ← 9000;   {初始化 numbers 的尾指针}
 for i:=1 to 9000 do numbers[i]← 999+i;
 {numbers 数组初始化，即 numbers[1..9000]← 1000..9999}
 steps ← 0;          {步数初始化}
 wait ← true;        {置等待标志}
end;                 {init}
```

3．求精 2.1 —— do1 的过程说明

```
procedure  do1;
var
   number : integer;
begin
 randomize;          {初始化随机数发生器}
 repeat              {反复猜数}
2.1.1 number ← get_next;
   {在 numbers[1]—numbers[to_guess]范围内随机产生猜数 number}
    if number<1000 then                    {若 numbers 表空，则}
      begin
       clrscr;                             {清屏}
       打印'Do not make fun of me!'        {打印猜数失败信息}
       halt(1)                             {停止程序执行}
      end;
2.1.2 put (number, now);          {将计算机猜数 number 顺序按位存入 now 数组}
      t1 ← -1; t2 ← t1;           {默想一方的应答信息要求输入，不作自动显示}
2.1.3 put_answer;                 {按屏幕显示格式打印提示信息}
2.1.4 get_reply;                  {输入默想方的应答信息}
2.1.5 delete(now);                {缩小下一次猜数范围}
 until (t1=4) and (t2=4);
 在屏幕中央打印猜数步数 'steps' 和猜中的默想数 'number';
end; {do1}
```

4. 求精 2.2 —— do2 的过程说明

```
    procedure  do2
  var
     buffer  :string;        {以字串形式存储要猜的四位数}
     number  :integer;       {以整数形式存储要猜的数和计算机的猜数}
     mark    :integer;       {非法输入标志 (mark<>0)}
     i       :integer;       {辅助变量}
  begin
   mark ← 0;
   init;                     {初始化}
   repeat                    {正确输入要猜的四位数→ number}
     if mark<>0 then  在屏幕底部打印非法输入，要求重新输入信息;
     write ('Your number:');
     readln (buffer);
     val (buffer, number, code); {将字串 buffer 转换为整数 number}
     mark ← 1;
   until (number>=1000) and (number<=9999);
   clrscr;          {清屏}
   在第一行打印要猜的数' number' :;
   put (number, key);  {将要猜的四位数顺序按位存入 key 数组}
   repeat                    {反复猜数}
     number ← get_next;      {随机产生 numbers[1..to_guess] 范围的猜数 number}
     put(number, now);       {将猜数顺序按位存入 now 数组}
2.2.1  t1 ← test1(key, now);  {求猜对的数字个数 t1}
2.2.2  t2 ← test2(key, now);  {求猜对位置的数字个数 t2}
     put_answer;             {按屏幕显示格式打印应答信息 t1, t2}
     delete(now);            {缩小猜数范围}
2.2.3  if wait then pause;   {等待，直至按键}
   until (t1=4) and (t2=4);
   在第七行打印猜数步数 'steps'，猜中的默想数 'number';
  end;{do2}
```

5．求精 2.3 —— do3 的过程说明

```
  procedure  do3
  var
     data    :file of integer;   {文件变量}        A1∈{X_I1, ……, X_IP}
     number  :integer            {要猜的数}
     i, k    :integer;           {辅助变量}
```

```
    total   :integer;                {猜中所需的总步数}
  begin
   total ← 0;
   assign (data, 'guess. dat'); {将外部文件名 guess. dat 赋文件变量 data}
   reset (data);                    {打开 data 文件}
   for k:=0 to 19 do
{从 data 文件中顺序读入 20 个四位数，一个接一个地让计算机猜}
   begin
    init;                           {初始化}
    read (data, number);            {从 data 文件中读一个四位数赋 number}
    在第一行打印第 'k+1' 次要猜的数 number;
    put (number, key);
    ..............                                  → 同 do2
    ..............
    在第七行打印猜数步数 'steps'，猜中的默想数 'number';
    total ← total+steps;            {统计数中所需的总步数}
   end;              {for}
   close (data);                    {关闭 data 文件}
   在屏幕底部打印猜中 20 个数所需的总步数 'total',
   end;   {do3}
```

第三层

1．求精 2.1.1——get_next 的函数说明

```
function get_next: integer;
begin
if (to_guess<>0)
 {若 numbers 表非空，则返回 numbers[1]..numbers[to_guess]之间的一个随机的四位整
数，表示计算机的一个猜数；否则返回 -1}
  then get_nexe ← numbers[random(to_guess)+1]
  else get_nexe ← -1;
end; {get_nexe}
```

2．求精 2.1.2——put(number, now) 的过程说明

```
procedure put (number: integer; var place: integers);
  var
    i: integer;          {循环变量}
  begin
    for i:=0 to 3 do
```

```
        begin
         place[3_i]← number mod 10;
         mumber← trunc(number /10);
         end;   {for}
      end;   {put}
```

将四位整数 number 按低位→高位的顺序存入 place[3]～ place[0]。

3. 求精 2.1.3——put_answer 的过程说明

```
  procedure put_answer;
  var
    i: integer;    {循环变量}
  begin
   在第2行打印计算机猜的数 'now[0] … now[3]';
   在第3行打印 'number:' 字样;
   在第4行打印 'position:' 字样;
   steps← steps+1;              {猜数的步数+1}
   在第5行打印步数位('steps');
   if (t1≥0) then          {需模拟显示默想过程(问题1除外)}
     begin
      在'number:' 后打印 t1;
      在'position:' 后打印 t2;
     end;    {if}
  end;    {put_answer}
```

4. 求精 2.1.4——get_reply;

```
  procedure get_reply;
  var
    buffer: string;
  begin
{以字串格式读入数字对的数字个数至 buffer, 并将 buffer 转换成整数 t1}
   repeat
     光标置 'number:' 后;
     readln(buffer);
     val(buffer, t1, code);
   until (t1≥0) and (t1≤4);
{以字串格式读入位置对的数字个数至 buffer, 并将 buffer 转换成整数 t2}
   repeat
     光标量 'position:' 后;
```

```
  readln ( buffer );
   val ( buffer, t2, code );
until ( t2 ≥ 0) and ( t2 ≤ 4 );
end;   { get _ reply }
```

5. 求精 2.1.5 —— delete (now) 的过程说明

```
procedure delete ( var now : integers );
var
  i: integer;                { numbers 序列的指针 }
  buffer: integers;          { 存储 numbers 序列的元素 }
  begin
   i ← 0;      { 从 numbers 表首开始 }
   repeat     { 将当前所猜的数 now 依顺序与 numbers 序列的每一个四位数进行比较 }
     i ← i+1;     { 指向 numbers 序列的下一个四位数 }
     put ( numbers [ i ], buffer );      { 将四位数顺序按位存入 buffer 数组 }
     if ( test2 ( buffer, now )<>t2 ) or ( test1 ( buffer, now )<>t1) then
{ 若 buffer 与 now 之间相同数字的个数 <>t1 或者相同数字中位置正确的数字个数
<>t2}
       begin
        numbers [ i ]← numbers [ to _ guess ];   { 则将 numbers 表尾的 4 位数移入该位置 }
        to _ guess ← to _ guess − 1;              { 表尾指针 −1}
        i ← i _ 1;                { 继续从 numbers 的当前位置比较下去 }
       end;   {for}
    until i>to _ guess;
{ 直至 number 序列剩下的数中与 now 相同数字的个数为 t1、相同数字中正确位置的个数
为 t2}
   end;       {delete}
```

6. 求精 2.2.1 —— test1 (key, now) 的函数说明

```
      function test1( var key, answer : integers) : integer;
      var
         h :integer;     { 两个四位数 key 和 answer 中相同数字的个数 }
         i :integer;     {key 的指针 }
         j :integer;     {answer 的指针 }
         mark [ i ] : array [ 0 .. 3 ] of boolean;
           { mark [ j ]=  ┌─  talse   answer 的第 j 位数未曾在 key 中出现 }
           {              └─  true    answer 的第 j 位数曾在 key 中出现 }
      begin
```

```
for i:=0 to 3 do mark[i]← flase;          {mark 数组初始化}
 h ← 0;                                   {相同数字的个数初始化}
 for i:=0 to 3 do                         {顺序搜索 key 的每位数字}
  begin
  j ← 0;
  while (j<=3) and ((key[i]<>answer[j]) or (mark[j]<>flase)) do j ← j+1;
  if j<=3 then
```

{若 answer 的第 j 位数字与 key 的第 i 位数字相同且在 key 中未曾出现，则置 answer 的该位数字相同标志，相同数字的个数+1}

```
    begin
     mark[j] ← true;
     h ← h+1;
    end; {for}
   test1 ← h;          {返回相同数字个数 h}
  end;  {test1}
```

7. 求精 2.2.2 ——test2(key, now)的函数说明

```
function test2 (var key, answer :integers) :integer;
var
   h :integer; {相同数字中位置正确的数字个数}
   i :integer; {key 和 answer 的指针}
begin
 h ← 0;
 for i:=0 to 3 do
  if key[i]=answer[i] then h ← h+1
   {统计 key 和 answer 中对应位数字相同的个数}
 test2 ← h;   {返回相同数字中位置正确的数字个数}
end;  {test2}
```

8. 求精 2.2.3 ——pause 的过程说明

```
procedure pause;
begin
 在屏幕下部打印' press any key to continue…');
 while not keypressed do randomize;
 {若不按键，则将随机器发生器初始化为某值}
 if readkey='n'  then wait ← flase;  {若按下的键为'n'，则取消等待标志}
end; {pause}
```

第二章　顺序统计算法和中位数

在一个由 n 个元素组成的集合中，第 i 个顺序统计就是该集合中第 i 小的元素。例如，一个集合元素中的最小值即为第 1 个顺序统计（i=1）；最大值即为第 n 个顺序统计（i=n）。一个中位数非正式地说，即该集合的“中间点”上的元素。当 n 为奇数时，中位数是唯一的，即处于 i=(n+1)/2 处；当 n 为偶数时，则存在两个中位数，分别在 i=n/2 和 i=n/2+1 处。人们通常使用排序的方法对 n 个输入数排序，再在输出数组中标出第 i 个元素为第 i 个顺序统计。

本章中，我们将介绍一种更快的算法。另外介绍两个应用中位数的例题，以便使读者对中位数的用途有所了解。

§2.1　顺序统计的算法

我们可以将由 n 个不同数构成的集合中选择其第 i 个顺序统计的问题，形式地定义如下：

输入：一个含 n 个（不同的）数的集合 A 和一个整数 i(1 ≤ i ≤ n)；

输出：恰大于 A 中其它 i−1 个元素的 X∈A；

首先，我们来考虑 i=1 和 i=n 时的两个特例。究竟要作多少次比较才能确定一个有 n 个元素的集合中的最小元素和最大元素？可以很容易地给出 n−1 次比较这个上界：顺序查看集合中的每个元素，并始终记录到目前为止的最小元素和最大元素。

设 A——n 个元素的集合：

```
TYPE
  ATYPE=array[1..n] of integer;
 max——目前为止的最大元素;
 min——目前为止的最小元素;
```

下面过程可以从集合 A 中找出最大元素 max 和最小元素 min:

```
procedure max_min(A: ATYPE; var max, min: integer);
var
  i:=integer;
begin
  min:=A[1]; max:=A[1];
  for i:=2 to n do begin
     if min>A[i] then min:=A[i]
                 else if max<A[i] then max:=A[i];
end; {max_min}
```

问题是如果顺序数 i ≠1、n 时，又如何解决 A 中第 i 个顺序统计的问题？下面我们要介绍一种二分法策略，它仿照了快速排序，也是对输入数组进行递归划分。和快速排序不同的是，它只是对划分的某一边进行处理。

一、划分方法

设在区间 A〔P..R〕中划分。首先，它从 A〔P..R〕中选出首元素 X=A〔P〕作为“支点”元素来划为 A〔P..R〕：即对 A〔P..R〕进行适当组合后，分成两个子区间 A〔P..J〕和 A〔J+1..R〕。A〔P..J〕中每个元素小于等于 X，A〔J+1..R〕中每个元素大于等于 X。

```
function  partition (var A: ATYPE; p, r: integer): integer;
var
  x                                    : integer; {首元素（支点元素）值}
begin
  x:=A[p] {取首元素为支点元素}
  repeat
    while A[r]>x do dec(r); {向左扩展高区}
    while A[p]<x do inc(p); {向右扩展低区}
    if p<r then
{若低区和高区间未相接，则对低区的右邻元素和高区的左邻元素进行处理}
      begin
        if A[r]<>A[p] then   交换 A[r]和 A[p]
                      else begin
                             inc(p); dec(r);
                           end; {else}
      end; {then}
  until p>=r; {重复扩展低区和高区，直至两个子区间相接}
partition :=r; {返回低区 A[P..Q]和高区 A[Q+1..R]的相接下标 Q}
end; {partition}
```

上述算法的效率取决于原 A[P]值。为了减少运行时间，使其不再受制于一个 A[P]，我们修改算法：先随机地从 A[P..R]中选出一个元素作为支点元素，使得支点元素取 A[P.. R]中任一个的机会均等。

```
Function Randomized_Partition (Var A : ATYPE; p, r : Integer): Integer;
Var
  x              : Integer; {支点元素值}
Begin
  x := A[Random(r-p+1) + p]; {在 A[p..r]中随机地选取一个元素作为支点元素}
  Repeat
    While A[r] > x Do Dec(r); {向左扩展高区}
    While A[p] < x Do Inc(p); {向右扩展低区}
```

```
    If p < r Then Begin
{若低区和高区间未相接，则对低区的右邻元素和高区的左邻元素进行处理}
      If A[r] <> A[p] Then   交换 A[r]和 A[p]
        Else Begin
          Inc(p); Dec(r)
        End {else}
    End    {then}
  Until p >= r; [重复扩展低区和高区，直至两个子区间相接]
  Randomized_Partition := r {返回低区 A[p..q]和高区 A[q+1..r]的相接下标 q}
End; {Randomized_partiton}
```

执行了 randomized_partition 后，数组 A[P..R] 划分成两个非空子数组

低区子数组：A[P..Q]，元素个数 K=P−Q+1;

高区子数组：A[Q+1..R];

使得 A[P..Q]中的每个元素都小于等于 A[Q+1..R]中的每个元素。

二、二分法求解

下面就要确定第 i 个小的元素落在两个子数组 A[P..Q]和 A[Q+1..R]中的哪一个。显然，如果 i ≤ k, 则要找的元素是划分的低区中第 i 小的元素，于是对低区的子数组进一步递归选择；如果 i>k ，则要找的元素落在划分的高区子数组中。因为我们已经知道 A[P..Q]中的 k 个值小于等于 A[P..R]中的第 i 个元素——亦即所求元素必是 A[Q+1..R]中的第 (i−k) 小的元素，于是对高区的子数组递归选择。

按上述方式不断递归划分，直至划分后的区间仅剩一个元素。显然该元素为 A[P..R]中第 i 小的元素。

```
Function Randomized_Select(Var A : ATYPE ; p, r, i : Integer) : Integer;
Var
  q, k          : Integer;
Begin
  If p = r{返回 A[p..r]中第 i 小的元素}
    Then Randomized_Select := A[p]
    Else Begin
      q := Randomized_Partition(A, p, r);
{将 A[p..r]分成两个子序列 A[p..q]和 A[q+1..r]，使得 A[p..q]中的每个元素都小于或等于 A[q+1..r]中的每个元素}
      k := q - p + 1; {计算 A[p..q]中的元素个数 k}
      If i <= k{若要找的元素落在划分的低区中，则在低区中递归选择第 i 个小的元素}
        Then Randomized_Select := Randomized_Select( A, p, q, i)
        Else Randomized_Select := Randomized_Select(A, q + 1, r, i - k)
{若要找的元素落在划分的高区中，则在高区中递归选择第 (i−k) 个小的元素}
```

End {else}
End; {Randomized_Select}

我们只要在程序中调用 randomized_select(A, 1, N, i)，便会得出数组 A 中第 i 个小的元素。

§2.2 中位数的应用

在顺序统计问题中，中位数的应用范围最广泛。例如 X 轴上有 n 个点，由左至右依次排列为

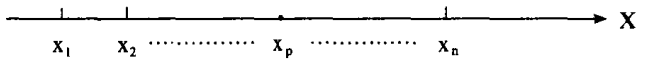


图 2.2 - 1

我们希望在 X 轴上找到一个点，X_p（不一定是 n 个点之一），使得 X_p 与各点的距离和 $\sum_{i=1}^{n} d(X_p - X_i)$ 最小。

显然，这个问题可以归结为中位数问题，即当 n 为奇数时，X_p 则为 $X_{(n+1)/2}$；否则 X_p 为 $(X_{n/2}+X_{n/2+1})/2$。

从这个典型问题出发，我们可以引出中位数的许多应用。

【例 1】主油管的最优位置

Olay 教授正在为一家石油公司咨询。该公司正在计划建造一条由东向西的管道，该管道要穿过一个有 n 口井的油田。从每口井中都有一条喷油管沿最短路径与主管道直接相连（或南或北）。

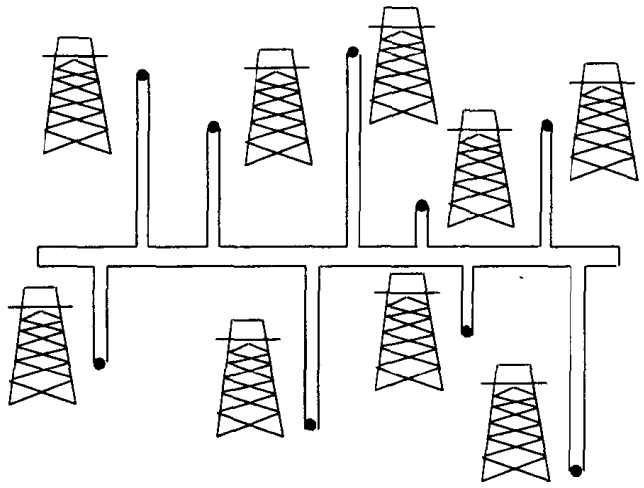


图 2.2 - 2

给定各个井的 X 坐标和 Y 坐标，Olay 教授如何才能选择主管道的最优位置（即使得各喷管长度总和最小的位置?）

算法分析:

由于主管道由东向西，因此要使相连油井与主管道的喷油管道最短，喷油管方向必须南北相连，与主管道垂直，即主管道的最优位置应为一条 $Y=Y_k$ 的水平线。问题是 Y_k 如何确定。

为了使 Y_k 与各油井的 Y 座标 Y_1，……，Y_n 间的距离和最短，我们将 Y_1，……，Y_n 由小到大排序，选择最中间的那个点作为 Y_k。(若油井数为奇数，则第 (n+1) /2 小的 Y 座标作为 Y_k；若油井数为偶数，则第 n/2 小的 Y 座标值与第 (n/2+1) 小的 Y 坐标值的平均数作为 Y_k 的值)

显然，确定主管道最优位置，实际上就是求 n 个油井的 Y 座标的中位数。

下面给出程序题解：

```
Program Oil_Pipes_by_Means_of_MidNumber_And_Statistics;

Const
  Maxn                  =  10000; {最多油井数}

Type
  ListType              =  Array [1..Maxn] of Real; {Y座标序列类型}

Var
  N, i                  :  Integer; {油井数，辅助变量}
  x, y                  :  Real; {辅助变量，主管道的最优位置}
  A                     :  ListType; {油井的Y座标序列}
  F                     :  Text; {文件变量}

Procedure Swap(Var a, b : Real); {交换实数a和b}
Var
  tmp : Real;
Begin
  tmp := a; a := b; b := tmp
End; {swap}

Function Randomized_Partition (Var A : ListType; p, r : Integer) : Integer;
Var
  x             : Real;
Begin
  x := A[Random(r - p + 1) + p];
{随机抽取A[p..r]中的某一值为X的元素，然后将A[p..r]划分成A[p..q] 和A[q+1..r],
使得前者小于等于X，后者大于等于X，返回下标q}
```

```
    Repeat
    While A[r] > x Do Dec(r);
    While A[p] < x Do Inc(p);
    If p < r Then Begin
      If A[r] <> A[p] Then Swap(A[r], A[p])
        Else Begin
          Inc(p); Dec(r)
        End {then}
    End {then}
  Until p >= r;
  Randomized_Partition := r
End; {Randomized_partition}

Function Randomized_Select (Var A : ListType; p, r, i : Integer): Real;
Var
  q, k          : Integer;
Begin
  If p = r {返回A[p..r]中第i小的元素}
    Then Randomized_Select := A[p]
    Else Begin
      q := Randomized_Partition (A, p, r);
{将A[p..r]分成两个子序列A[p..q]和A[q+1..r],使得A[p..q]的每个元素值都小于等
于A[q+1..r]}
      k := q - p + 1;{计算A[p..q]的元素个数k}
      If i <= k {若第i小的元素落在低区中，则在低区中递归选择第i小的元素；否则
在高区中递归选择第i-k小的元素}
        Then Randomized_Select := Randomized_Select (A, p, q, i)
        Else Randomized_Select := Randomized_Select (A, q+1, r, i-k)
    End {else}
End; {Randomized_select}

Begin
  Assign (F, 'INPUT. DAT'); {文件名与文件变量连接}
  Reset (F); {文件读准备}
  Readln (F, N); {从文件中读油井个数}
  For i := 1 to N Do Begin {从文件中读入各油井的X和Y座标}
    Readln (F, x, A[i]);
   Writeln (x: 8: 2, A[i]: 8: 2)
```

```
End; {for}
If Odd(N) {若油井是奇数个，主管道位于N个油井的Y座标中第⌈n/2⌉小的位置，否则位于⌊n/2⌋..⌈n/2⌉之间}
   Then Writeln ('y = ', Randomized_Select (A, 1, N, N div 2 + 1): 8: 2)
   Else Writeln ('y = ', (Randomized_Select (A, 1, N, N div 2): 8: 2 +
                          Randomized_Select (A, 1, N, N div 2 + 1)) / 2: 8: 2);
Close(F) {关闭文件}
End. {main}
```

下面，我们再把问题拓延开来：序列中的元素是平面上的点(X_i, Y_i)，且每个元素带有一个正的权$W_i(1 \leqslant i \leqslant n)$，如何求带权中位数。

【例2】邮局位置

已知n个点$P_1, \cdots\cdots, P_n$及与它们相联系的权$W_1, \cdots\cdots, W_n$。我们希望能找到一个点p(不一定是n个输入点之一)，使和式$\sum_{i=1}^{n} W_i * d(P_1, P_i)$最小。此处$d(a, b)$是指点a与b之间的距离，$d(a, b) = |x_a - x_b| + |y_a - y_b|$。

算法分析：

首先，我们必须明确一个问题：如果找到一个满足和式$\sum_{i=1}^{n} W_i * d(P, P_i)$最小的点P(P为$(X_p, Y_p)$)，则它必须满足如下两个条件：

1. 在X轴上，$\sum_{i=1}^{n} W_i * d(Xp, X_i)$最小；

2. 在Y轴上，$\sum_{i=1}^{n} W_i * d(Yp, Y_i)$最小；

因此问题又分成了两个子问题：在X轴上找满足条件1的Xp和在Y轴上找满足条件2的Yp。这两个子问题又具有一个共同的特征：

一条直线上有若干带权的点，在该直线上寻找一个点A_k，满足$\sum_{i=1}^{n} W_i * d(A_k, A_i)$最小。

我们称这个点为带权中位数。必须明确的是，由于各点被赋了权，因此A_k未必是按递增排序后的$A_1 A_2 \cdots\cdots A_n$中处中间位置的那个点，而是满足下述条件的一个点：

在递增序列$A_1 \cdots\cdots A_{k-1} A_k A_{k+1} \cdots\cdots A_n$中，子序列$A_1 \cdots\cdots A_{k-1}$的权和小于等于W/2，并且子序列$A_{k+1} \cdots\cdots A_n$的权和也小于等于w/2(其中$W = \sum_{i=1}^{n} W_i$)，则$A_K$为带权中位数。

我们也采用二分法求这个A_K(与randomized_select过程大致相同)：将递增序列A(P..R)划分成两个子序列A(P..Q)和A(Q+1..R)，使得A(P..Q)中的每个元素小于A(Q+1..R)中的每个元素，即A(P)..A(Q)点集中在直线的左端，计算出A(P..Q)的权和W′。如果W′≥W/2，说明要找的带权中位数在划分的低区A(P..Q)中，于是在低区中进一步递归选择权和小于等于W/2的子区间。若W′ < W/2，则说明要找的带权中位数在划区的高区A(Q+1..R)中，于是在高区中进一步递归选择权和小于等于(W/2−W′)的子区间。显然，当递归至当前子区间仅剩一个元素时，该元素即为A_K。

我们分两次使用上述算法：第一次求X轴上的n个座标序列$X_1, \cdots\cdots, X_n$中的带权

中位数 X_K;第二次求 Y 轴上的 Y_1, ……, Y_n 中的带权中位数 Y_K。点 (X_K, Y_K) 便是问题的正确解。

依据上述思路，我们不仅可以求二维空间 (X_i, Y_i) 的带权中位数，而且还可以求三维空间 (X_i, Y_i, Z_i) 的带权中位数，无非是增加对 Z 轴的座标序列求带权中位数而已。

下面给出程序题解。

```
Program Post_offices_locations_by_Means_of_MidNumber_And_Statistics;

Const
  Maxn                  =  3000; {最多点数}

Type
  Node                  =  Record {点的数据类型}
                             x          : Array [1..2] of Real;
                                          {X[1]-X座标; X[2]-Y座标}
                             w          : Real; {权}
                           End;
  ListType              =  Array [1..Maxn] of Node; {点的序列类型}

Var
  N, i                  :  Integer; {输入点的个数，辅助变量，N点的权和}
  A                     :  ListType; {输入点序列}
  F                     :  Text; {文件变量}
  w_tot                 :  Real;

Procedure Swap (Var a, b : Node); {输入点a与输入点b交换}
Var
  tmp : Node;
Begin
  tmp := a; a := b; b := tmp
End; {swap}

Function Randomized_Partition (Var A : ListType; p, r, k : Integer;
                               Var w : Real) : Integer;
Var
  i                 : Integer;
{随机抽取点区间[P_p..P_r]中某个点K轴向的座标X(k=1横座标;k=2竖座标)将[P_p..P_r]
分成两个子序列[P_p..P_q]、[P_q+1..P_r],使得[P_p..P_q]每个点K轴向座标小于等于X,;[P_q+1..P_r]
每个点K轴向座标大于等于X，返回[P_p..P_q]的权和W及其第q点的K轴向座标}
```

```
  x                : Real;
Begin
  i := Random(r-p+1) +p;
  x := A[i]. x[k];
  w := A[i]. w;
  Repeat
    While A[r]. x[k] > x Do Dec(r);
    While A[p]. x[k] < x Do Begin
      Inc(p);
      w := w + A[p]. w
    End; {while}
    If p < r Then Swap(A[r], A[p])
  Until p >= r;
  Randomized_Partition := r
End; {randomized_partitiion}

Function Randomized_Select (Var A : ListType; p, r, t : Integer;
                                    s : Real) : Real;
Var
  q              : Integer;
  w              : Real;
begin
If p = r{t轴向座标中的中位数为P点，返回P点的t轴向座标}
    Then Randomized_Select := A[p]. x[t]
    Else Begin
      q := Randomized_Partition (A, p, r, t, w);
{将[Pp..Pr]分成两个序列[Pp..Pq]、[P_{q+1}..Pr],使得[Pp..Pq]每个点t轴向座标小于等于
[P_{q+1}..Pr]每个点的t轴向座标}
      If s <= w
{若低区间的权和W大于等于s,则中位数所在的点处于低区间，P_p点至中位数所在点的
权和仍为s；否则中位数所在的点处高区间，P_q至中位数所在点之间的权和为s减去低区
间的权和W，递归搜索带权中位数所在的子区间}
        Then Randomized_select := Randomized_Select (A, p, q, t, s)
        Else Randomized_select := Randomized_Select (A, q+1, r, t, s-w)
    End {else}
End; {Randomized_Select}

Begin
```

```
  Assign (F, 'INPUT. DAT'); {文件名与文件变量连接}
   Reset (F); {文件读准备}
   Readln (F, N); {从文件中读入点数和各点的座标权重}
   w_tot := 0;
   For i := 1 to N Do Begin
     Readln (F, A[i]. x[1], A[i]. x[2], A[i]. w);
     w_tot := w_tot + A[i]. w
   End; {for}
   Writeln ('x = ', Randomized_Select (A, 1, N, 1, w_tot / 2): 8: 2);
    {递归搜索和输出满足条件的横座标}
   Writeln ('y = ', Randomized_Select (A, 1, N, 2, w_tot / 2): 8: 2);
    {递归搜索和输出满足条件的竖座标}
   Close (F)
End. {main}
```

第三章　有关数论的算法

数论曾一度被认为是纯数学学科，与计算机无关。今天有关数论的算法正在程序设计中被广泛使用。例如有些试题看起来似乎属于盲目搜索类型，但一旦使用了数论算法后，便可使求解过程即简便又高效。

本章介绍一些数论的基本概念和相关算法，它们都是我们应用数论的基础。其中一些内容涉及的知识超过中学生所能接受的范围。限于篇幅和从实用的角度出发，我们略去了推导过程，仅介绍一些问题的由来以及算法步骤。

§3.1　求最大公约数

首先，我们来回顾一下公约数和最大公约数的概念：

如果 d 是 a 的约数并且也是 b 的约数，则 d 是 a 与 b 的公约数。例如

30 的约数为 1, 2, 3, 5, 6, 10, 15, 30;

24 的约数为 1, 2, 3, 4, 6, 12, 24;

因此 24 与 30 的公约数 1, 2, 3, 6。1 是任意两个整数的公约数。

两个不同时为 0 的整数 a 与 b 的最大公约是其值为最大的公约数，表示为 gcd(a, b)。显然，gcd(24, 30)=6。在这节中，我们仅限于对非负整数的情况进行讨论。这一限制是有道理的，因为 gcd(a, b)=gcd(|a|, |b|)。

在数学分析中，人们通常根据 a 和 b 的素数因子分解

$$a=P_1^{e1}P_2^{e2}\cdots\cdots P_r^{er},$$

$$b=P_1^{f1}P_2^{f2}\cdots\cdots P_r^{fr},$$

来求整数 a 和 b 的最大公约数：

$$\gcd(a, b)=P_1^{\min(e1, f1)}P_2^{\min(e2, f2)}\cdots\cdots P_r^{\min(er, fr)}$$

但用这种方法来计算最大公约数，不大可能获得一种有效的算法。在引出新算法之前，我们先来证明一个被称为 gcd（欧几里德）的递归定理：

$$\gcd(a, b)=\gcd(b, a \bmod b)$$

〔证明〕

我们只要证明出 gcd(a, b) 与 gcd(b, a mod b) 可互相整除，上式一定成立。设 d=gcd(a, b)。将 a mod b 化成 a 与 b 的线性组合

$$a \bmod b=a-\lfloor a/b\rfloor \quad b$$

由于 d 能整除 a 和 b, 因此 d 一定能整除 a 与 b 的线性组合，即 d 能整除 (a mod b)。又因为 d 能整除 b 和 (a mod b)，显然 d(=gcd (a, b)) 能整数 gcd(b, a mod b)。

同理可证：gcd(b, a mod b) 能整除 gcd(a, b)。

下述算法是一个直接基于 gcd 定理之上的递归程序。该函数输入非负整数 a、b，返回

a 和 b 的最大公约数。

```
function euclid (a, b: longint): longint;
 begin
  if b=0 then
   euclid:=a
  else
   euclid:=euclid (b, a mod b)
 end; {euclid}
```

由于递归边界 gcd(a, 0)=a 正确且递归调用过程中第二个值参严格递减，所以算法不可能无限递归下去，在运行终止时总能求出正确的答案。例如 gcd(30, 21)的计算过程

EUCLID (30, 21)=EUCLID(21, 9)
=EUCLID (9, 3)
=EUCLID (3, 0)
=3

在这个计算过程中三次递归调用了 EUCLID。

生活中许多有趣的问题，可以运用 gcd 定理来解答。

【例 1】狼找兔子

一座山周围有 n 个洞，顺时针编号为 0, 1, 2, …n－1。

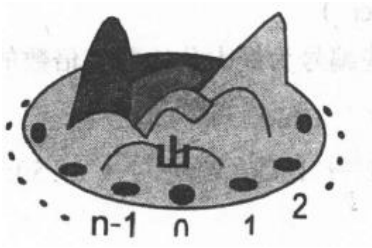


图 3.1 - 1

而一只狼从 0 号洞开始，顺时针方向计数，每遇到 m 个洞就进洞找兔子。例如 n=5, m=3, 狼经过的洞依次为 0, 3, 1, 4, 2, 0。

输入 n、m。试问兔子有没有幸免的机会？如果有，该藏在哪儿？

算法分析:

不妨让兔子躲在一号洞。因为若狼能从 0 号洞到达 1 号洞，则它必能从 1 号洞到达 2, …n－1 号洞，兔子难逃厄运。反过来说，若有安全洞，则 1 号洞就是其中的一个。

再来看狼的运动。狼的第 i 次运动后的洞址应是 (m*i) mod n (i 为正整数)。若 (m*i) mod n=1, 即狼第 i 次运动后到达 1 号洞，则 gcd(m, n)=1。因为若 gcd(m, n)=k>1, 则由 (m′*k*i) mod (n′*k) =1，(设 m′*k=m, n′*k=n)，m′*k*i－[(m′*k*i) / (n′*k)]*(n′*k) =1。得出 k 整除 1，与 k>1 矛盾。而若 gcd(m, n)=1, 则必存在 i 使

(m*i)mod n=1，因而兔子幸免的条件为 gcd(m, n)>1．在此情况下，它应选择除 {i*gcd(m, n)|i=0…(n/gcd(m, n)-1)} 外的洞躲藏。

下面，我们给出程序题解

```
Program Hare_And_Wolf;
Var
  m, n, d, i                : Longint;

function euclid (a, b: longint): longint; {计算和返回 a, b 的最大公约数}
 begin
  if b=0 then
   euclid:=a
  else
   euclid:=euclid(b, a mod b)
 end; {eucliad}

Begin
  Write('N M = '); {输入洞数和狼的步长值}
  Readln(N, M);
  d := euclid(m, n); {计算 m 和 n 的最大公约数}
  If d = 1{若最大公约数为 1，则输出无解信息}
     Then Writeln('No Answer')
     Else Begin {否则输出那些编号为最大公约数整倍数的洞为"危险"的洞}
       Writeln('The hare can'' t hide in ');
       For i := 0 to n div d - 1 Do Write(i * d: 5);
       Writeln
     End {else}
End. {main}
```

下面，我们对欧几里德算法进行推广，使得该算法不仅能得出任意两个正整数 a 和 b 的最大公约数 d, 而且还能计算出满足下式的整系数 X 和 Y

$$d=\gcd(a, b)=ax+by$$ （X 与 Y 可能为 0 或负数）

函数 extenden_euclid 输入整数 a、b, 返回 gcd(a, b) 和对应等式 ax+by 中的 x, y。

```
function extended_euclid (a, b: longint; var x, y: longint): longint;
 var
  t: longint;
 begin
```

```
if b=0 then
  begin
   extended_euclid:=a; x:=1; y:=0
  end {then}
 else
  begin
   extended_euclid:=extended_euclid(b, a mod b, x, y);
   t:=x; x:=y; y:=t-(a div b)*y
  end {else}
end; {extend_euclid}
```

上述函数是从欧几里德算法衍生出来的。算法一开始，它首先测试"b=0?"若 b=0，返回当前最大公约数为 a，x=1 y=0，以使 a=gcd(a, 0)=ax+by；若 b<>0，则欧几里德算法首先计算出 d′=gcd(b, a mod b)和满足下式

$$d'=bx'+(a \bmod b)y'$$

的 x′y′。在这种情况下，我们有 d=gcd(a, b)=d′=gcd(b, a mod b)。为了得到满足 d=ax+by 的 xy，我们利用等式

$$d=d'=bx'+(a \bmod b)y'$$

得出

$$\begin{aligned} d&=bx'+(a-\lfloor a/b\rfloor b)y' \\ &=ay'+b(x'-\lfloor a/b\rfloor y') \end{aligned}$$

因此，当我们选择

$$x=y';\ y=x'-\lfloor a/b\rfloor *y'$$

就可以满足等式 d=ax+by。显然这个算法是正确的。

例如计算 extenden_euclid(99, 78, 0, 0)的过程如下：

a	b	⌊a/b⌋	函数返回值	变量返回值	
				x	y
99	78	1	3	−11	14
78	21	3	3	3	−11
21	15	1	3	−2	3
15	6	2	3	1	−2
6	3	2	3	0	1
3	0		3	1	0

图 3.1 - 2

§3.2 求解模线性方程

当我们定义了一个整数除以另一个整数的余数的概念后，很方便地就可以给出表示同

余的特殊记法:

$a \equiv b(\bmod n)$

表示 (a mod n) =(b mod n)。换句话说，当a和b对模n是相等的(即a和b除以n有相同的余数) 时，有 $a \equiv b(\bmod n)$。等价地有，$a \equiv b(\bmod n)$, 当且仅当n能被(b−a) 整除。例如 $61 \equiv 6(11)$。

我们现在来考虑求解下列方程的问题

$ax \equiv b(\bmod n) \quad (n>0)$

其中a.b和n是已知，要求解出满足上式的对模n的x值。可能没有解，也可能有多个x解。

求解过程如下:

首先，我们通过欧几里德推广算法求出

$d=\gcd(a, n)$和两个满足 $d=ax'+ny'$ 的值 x' 和 y'

表明 X' 是方程 $ax' \equiv b(\bmod n)$的一个解。然后分析

若d不能被b整除，则方程 $ax \equiv b(\bmod n)$没有解；否则有d个解。其中第1个解的值为

$$x_0=x'*(b/d) \bmod n$$

其余d−1个解可以通过对模n加上(n/d)的倍数来得到。即

$$x_i=(x_0+i*(n/d)) \bmod n \qquad (1 \leqslant i \leqslant d-1)$$

例如考察方程 $14x \equiv 30(\bmod 100)$ (这里a=14, b=30, n=100)。 调用欧几里德推广算法后得到d=2, x=−7, y=1。显然

$$x_0=(-7)*15 \bmod 100 = 95$$

$$x_1=95+(1*100/2) \bmod 100 = 45$$

程序分析如下。其中过程 modular_equation_soluer 在输入a, b 和n后，调用子函数 extended_euclid (a, b, x, y)计算出 gcd(a, b)和对应等式 ax+by 中的 x, y。若 b mod (gcd(a, b)) > 0 输出无解信息；否则输出所有的方程解 $x_i\{1 \leqslant i \leqslant \gcd(a, b)\}$。

```
function extended_euclid (a, b: longint; var x, y: longint): longint;
 var
  t: longint;
 begin
  if b=0 then
   begin
    extended_euclid:=a; x:=1; y:=0
   end {then}
  else
  begin
    extended_euclid:=extended_euclid(b, a mod b, x, y);
```

```
    t:=x; x:=y; y:=t-(a div b)*y
   end {else}
 end; {exetend_euclid}

procedure modular_linear_equation_solver (a, b, n: longint);
 var
  d, x, y, e, i: longint;
 begin
  d:=extended_euclid(a, n, x, y);
  if b mod d>0 then
   writeln ('no answer')
  else
   begin
    e:=x*(b div d) mod n;
    for i:=0 to d-1 do
     write ((e+i*(n div d)) mod n: 10)
   end {else}
 end; {modular_linear_equation_solver}
```

有些例题通过求解模线性方程的方法，可以使问题变得十分简洁。

【例 1】量水

有三个分别装有 a 升水，b 升水和 c 升水的量筒{gcd(a, b)=1, c>b>a>0}。现 c 筒装满水，问能否在 c 筒中量出 d 升水(c>d>0)。若能，请列出一种方案。

算法分析:

量水过程实际上就是倒来倒去，每次倒的时候总有如下几个特点:

1. 总有一个筒中的水没有变动;

2. 不是一个筒被倒满就是另一个筒被倒光;

3. c 筒仅起中转作用，而本身容积除了必须足够装下 a 筒和 b 筒的全部水外，别无其它限制。

上述分析使问题骤然变得十分简洁，即实质上整个倒法过程只是对 a 筒倒满 x 次，对 b 筒倒满 y 次，使得总结果有

$$ax+by=d$$

上式中的 x, y 为整数，即可以是正整数(表示该筒(a 或 b)被 c 筒水倒满的次数)亦可以是负整数(表示该筒(a 或 b)被倒满后自 c 筒倒出的次数)。

一般采用猜数的方法求这个不定方程的解 x, y。这种方法局限性很大，很容易疏漏方案。我们不妨把方程化为 $ax=-by+d$, 看成 $ax \equiv d(\text{mod } b)$, 则问题变成了求解模线性方程的问题，解 x 的个数 d 即为方案总数。其中 x_i ($1 \leqslant i \leqslant d$) 表示第 i 种方案中 a 筒倒满的次数。$x_i$ 代入 $ax_i+by_i=d$ 后求出的 y_i 表示 b 筒倒满的次数。

例如，有三量筒: a=3, b=7, c=10。求 c 筒中量出 5 升水的所有方案

解：由题意，d=5 将 a, b, d 代入 $ax \equiv d \pmod b$ 后的模线性方程为

$$3x \equiv 5 \bmod 7$$

得出 $x_1=4$, $y_1=-1$, 表示只要把 a 筒装满 4 次，又倒给 b 筒，把它倒满倒光 1 次，即可在 c 筒中量出 5 升水。倒法过程如图 3.2 - 1(a)。

次数	a	b	c
1	0	0	10
2	*3	0	7
3	0	3	7
4	*3	3	4
5	0	6	4
6	*3	6	1
7	2	*7	1
8	2	0	8
9	0	2	8
10	*3	2	5
11	0	5	5

(a)

次数	a	b	c
1	0	0	10
2	0	*7	3
3	*3	4	3
4	0	4	6
5	*3	1	6
6	0	1	9
7	1	0	9
8	1	*7	2
9	*3	5	2
10	0	5	5

(b)

图 3.2 - 1

另一组解：$x_2=-3$, $y_2=2$, 表明 b 倒满 2 次，然后倒给 a 筒，把它倒满倒光 3 次后即可在筒中量出 5 升水 (见图 3.2 - 1(b))。

下面，我们给出程序题解

```
Program Measure_Water_volume;

Type
  Node                    =   Array [0..1] of Longint;

Var
  a, b, c                 :   Node;
  {a[0], b[0], c[0]—三个筒的容量; a[1], b[1], c[1]—三个筒的当前状态}
  d, Step, {需量的水，倒的次数}
  x                       :   Longint; {a 筒和 b 筒被倒筒的次数}

function extended_euclid (a, b: longint; var x, y: longint): longint;
{计算和返回 d=gcd(a, b) 和两个满足 ax+by=d 的值 x 和 y}
 var
  t: longint;
 begin
  if b=0 then
```

```
  begin
    extended_euclid:=a; x:=1; y:=0
   end {then}
  else
   begin
    extended_euclid:=extended_euclid(b, a mod b, x, y);
    t:=x; x:=y; y:=t-(a div b)*y
   end {else}
 end; {extended_euclid}

procedure modular_linear_equation_solver(a, b, n: longint; Var x0, y0: longint);
 var
  d, x, y, e, i: longint;
 begin
  d:=extended_euclid(a, n, x, y); {求出 a, n 的最大公约数 d 和满足 ax+ny 的值 x 和 y}
  if b mod d>0 then Begin {若 d 不能被 b 整除，则方程 ax ≡ b(mod n)无解}
   writeln ('no answer');
   Halt
  End {then}
  else {否则求出 ax ≡ b(mod n)的一个解 x0=x(b/d) mod n}
   begin
    e:=x*(b div d) mod n;
    x0:= e;
    y0:= (b-a*x0) div n{x0 代入 ax+ny=b 后得出 y0}
   end {else}
 end; {modular_Linear_euation_solver}

Procedure Fill (Var a, b : Node);
{输入倒来倒去的两个筒 a′ 和 b′. 若 a′ 和 b′ 目前的水量和小于 b′ 的容量，则 a′ 向 b ′倒
空，否则 a′ 将 b′ 倒满 }
Var
  t              : Longint;
Begin
  If a[1] < b[0] - b[1] Then t := a[1]
                        Else t := b[0] - b[1];
  a[1] := a[1] - t;
  b[1] := b[1] + t
End; {fill}
```

```
Begin
  Write ('a b c d = '); {输入三个量筒的容量和需量的d升水}
  Readln(a[0], b[0], c[0], d);
  modular_linear_equation_solver (a[0], d, b[0], x, y);
  {求出a筒倒满的次数x和b筒倒满的次数y}
  Step := 0; {倒的次数初始化为0}
  a[1] := 0; b[1] := 0; c[1]:= c[0]; {最初为a、b筒空，c筒满}
  Writeln (Step: 5, ')', a[1]: 5, b[1]: 5, c[1]: 5); {打印初始状态}
  If x > 0 {若a筒被c筒水倒满x次}
    Then Repeat
          If a[1] = 0 {若a筒目前空}
            Then Fill (c, a) {a筒被c筒水倒满}
            Else If b[1] = b[0] Then Fill(b, c)
            {若a筒有水且b筒满，则b筒向c筒倒空}
                                Else Fill(a, b);
            {若a筒有水且b筒未满，则a筒将b筒倒满}
          Inc (Step); {倒的次数+1}
          Writeln (Step: 5, ')', a[1]: 5, b[1]: 5, c[1]: 5)
         {打印a, b, c筒的当前状态}
        Until c[1] = d {直至c筒中剩有d升水}
    Else Repeat {若a筒水被倒满后向c筒倒x次}
          If b[1] = 0{若b筒空}
            Then Fill(c, b) {则c筒将b筒倒满}
            Else If a[1] = a[0] Then Fill(a, c)
            {若b筒非空且a筒满，则a筒向c筒倒空}
                                Else Fill(b, a);
            {若b筒非空且a筒未满，则b筒向a筒倒空}
          Inc(Step); {倒的次数+1}
          Writeln(Step: 5, ')', a[1]: 5, b[1]: 5, c[1]: 5)
          {打印a, b, c筒当前状态}
        Until c[1] = d {直至c筒剩d升水}
End. {main}
```

§ 3.3 求解模线性方程组

我们现在再来考虑一下求解下列方程组的问题:

$$a \equiv a_1 (\mathrm{mod}\ n_1)$$

$$a \equiv a_2 (\mathrm{mod}\ n_2)$$

……

……

……

$a \equiv a_k (\mathrm{mod}\ n_k)$

其中每个模 $n_i > 0$，每对模 n_i 和 n_j 互为质数（$1 \leqslant i, j \leqslant k, i \neq j$），$a_1 \cdots a_k$ 和 $n_1, \cdots n_k$ 已知，要求解出 a 值。

由于求解模线性方程组的算法原理涉及离散数学中的群论知识，因此我们不准备展开算法的理论分析，有兴趣的读者可参阅数论中中国余数定理的内容。下面，我们仅给算法步骤。

设 $n = n_1 * n_2 * \cdots * n_k$；

$m_1 = n/n_1 = n_2 * n_3 \cdots * n_k$；

$m_2 = n/n_2 = n_1 * n_3 * n_4 \cdots * n_k$；

$m_i = n/n_i = n_1 * .. * n_i - 1 * n_i + 1 \cdots * n_k$；

……

$m_k = n/n_k = n_1 * \cdots * n_k - 2 * n_{k-1}$；

由于 m_i 与 n_i 互为质数（即最大公约数为 1），因此我们可以通过求解下列模线性方程

$m_1 * m_1^{-1} \equiv 1 (\mathrm{mod}\ n_1)$

……

$m_i * m_i^{-1} \equiv 1 (\mathrm{mod}\ n_i)$

……

$m_k * m_k^{-1} \equiv 1 (\mathrm{mod}\ n_k)$

得出 $m_1^{-1} \cdots m_i^{-1} \cdots m_k^{-1}$。方程 $m_i * m_i^{-1} \equiv 1 (\mathrm{mod}\ n_i)$ 对模 n_i 有唯一解 $m_i^{-1} (1 \leqslant i \leqslant k)$。同样，我们亦可通过欧几里德的推广形式

$\gcd(n_i, m_i) = n_i * x + m_i * y = 1$ 求出 x 和 y，此时的 y 即为 $m_i^{-1} (1 \leqslant i \leqslant k)$

然后进行 k 次模运算得出

$c_i = m_i * (m_i^{-1} \mathrm{mod}\ n_i) \qquad i = 1 \cdots k$

最后将 $c_1, \cdots c_k$ 代入下述方程

$a = (a_1 * c_1 + \cdots a_i * c_i + \cdots + a_k * c_k) \bmod n$

便可求出 a 值。

【例 1】解模线性方程组

某数除以 5 的余数为 2，除以 13 的余数为 3，求该数的值。

算法分析：

按题意列出模线性方程组

$a \equiv 2 (\mathrm{mod}\ 5)$

$a \equiv 3 (\mathrm{mod}\ 13)$

其中 $n_1 = 5$，$n_2 = 13$，$n = 5 * 13 = 65$；$a_1 = 2$，$m_1 = n/n_1 = 13$；a2=3，$m_2 = n/n_2 = 5$

解下述模线性方程

$13 * m_1^{-1} \equiv 1 (\mathrm{mod}\ 5)$

$$5*m_2^{-1} \equiv 1 \pmod{13}$$

得出 $m_1^{-1}=2$, $m_2^{-1}=8$

$$c_1=m_1(m_1^{-1} \bmod n_1)=13*(2 \bmod 5)=26;$$

$$c_2=m_2(m_2^{-1} \bmod n_2)=5*(8 \bmod 13)=40;$$

$$a \equiv (a_1*c_1+a_2*c_2) \bmod n=(2*26+3*40) \bmod 65$$

$$=(52+120) \bmod 65$$

$$=42$$

下面为程序题解。调用函数 china 之前，已将 k 个模线性方程组

$$a \equiv a_i \pmod{n_i}$$

存入内存。其中 $a_1..a_k$、$n_1..n_k$ 分别存贮在数组 aa[1..k] 和 nn[1..k] 中．调用 china 后将返回模线性方程组的正整数解。

为求得 $y=m_i^{-1}$, china 函数在执行过程中曾 k 次调用子函数 extended_euclid(n_i, m_i, x, y)。

```
const
  maxl=100;
 var
  l: longint;
  aa, nn: array [1. .maxl] of longint;

 function extended_euclid (a, b: longint; var x, y: longint): longint;
  var
   t: longint;
  begin
   if b=0 then
    begin
     extended_euclid:=a; x:=1; y:=0
    end {then}
   else
    begin
     extended_euclid:=extended_euclid (b, a mod b, x, y);
     t:=x; x:=y; y:=t-(a div b)*y
    end {else}
  end; {extended_euclid}

 function china: longint;
  var
   d, x, y, n, m, i, a: longint;
```

```
begin
 n:=1;
 for i:=1 to k do {求 n=n1*...*nk}
  n:=n*ann[i];
 a:=0;
 for i:=1 to 1 do
  begin
   m:=n div nn[k]; {求 mi}
   d:=extended_euclid (nn[i], m, x, y); {通过欧几里德推广形式求 y=mi^-1}
   a:=(a+y*m*aa[i]) mod n {累加 a 值}
  end; {for}
 if a>0 then
  china:=a
 else
  china:=a+n
end; {china}
```

§3.4 模取幂运算

数论计算中经常出现的一种运算就是求一个数的幂 a^b 对另外一个数 n 的模的运算，即计算

$a^b \bmod n$ (a, b 和 n 是正整数)

由于计算机只能表示有限位的整数，所以编程时模取幂的运算要注意值的大小范围. 当 a^b 的值超过 turbo. pascal 定义的整数最大值 (32767)时，mod 运算便无法进行.

如何解决这个问题，我们引出一个能计算 $a^b \bmod n$ 的值的有用算法——反复平方法首先我们必须明确

$$d=a^b \bmod n =(\cdots((((a \bmod n)*a) \bmod n)*a) \bmod n \cdots *a) \bmod n$$

(括号下方：b 个 a)

由此引出一个迭代式

```
d:=a;
for i:=2 to b do d:=d mod n * a;
d:=d mod n;
```

问题是当 b 很大时，运行的时间将受之影响。为了提高时效，我们不妨将 b 转换为二进制数

$$<b_k, b_{k-1}, \cdots, b_1, b_0>$$

然后从最低位 b_0 开始，由右至左逐位扫描。每次迭代时，用到下面两个恒等式中的一个

$a^{2c} \bmod n = (a^c)^2 \bmod n \qquad b_i=0$

$a^{2c+1} \bmod n = a*(a^c)^2 \bmod n \quad b_i=1 \quad (0 \leqslant c \leqslant b)$

其中c为b的二进制数的后缀（$b_{i-1}\cdots b_0$）对应的十进制数。当c成倍增加时，算法保持条件 $d=a^c \bmod n$ 不变，直至c=b。

例如当a=7, b=560, n=561，迭代式计算出的对模561相应值的序列如下表，其中用到的幂的序列见表中c行

i	9	8	7	6	5	4	3	2	1	0
b_i	1	0	0	0	1	1	0	0	0	0
c	560	384	192	96	48	16	8	4	2	0
d	1	256	511	103	460	103	526	157	49	1

下面为程序分析。函数 modular_exponentiation (a, b, n) 输入底数a、次幂b和模n后，通过反复平方法计算和返回 $a^b \bmod n$ 的值。

```
function modular_exponentiation (a, b, n: longint): longint;
{函数输入a, b, n, 返回a^b mod n 的值。}
 var
  d, t: longint;
 begin
  d:=1; t:=a;
  while b>0 do {从右到左逐位分析b的二进制值，并迭代计算a^c mod n}
   begin
    if b mod 2=1 then
     d:=d*t mod n;
    b:=b div 2; t:=t*t mod n
   end; {while}
  modular_exponentiation:=d {返回a^b mod n值}
 end; {modular_exponentiation}
```

§3.5 素数的测试

解决素数测试问题的一种简便方法是试用除法，即试着用2, 3, $\cdots[\sqrt{n}]$ 去除n(这里大于n的偶数可以跳过)。很容易看出n是素数当且仅当没有一个试用除数能被n整除。这种算法的优点是它不仅能确定n是素数还是合数，而且当n是合数时，它实际上确定出了n的素数因子分解。但缺点是该算法的时效取决于n值。若n很小或者n恰好有一个较小的素数因子时，试用除法才能在短时间内出解。

如果我们把问题局限在仅确定 n 是否为素数的范围而不涉及 n 为合数时的素数因子分解，就不必采用试用除法了。因为确定一个数是否素数要比确定一个合数的素数因子分解容易得多。下面，我们来考察一种简便的素数测试方法。在讲述算法之前，我们先引出一个定义：

如果 n 是一个正整数

$$a^{n-1} \equiv 1 \pmod{n}$$

则我们说 n 是一个满足基于 a 的伪素数。即对于 1..n-1 间的任意一个整数 a 来说，$a^{n-1} \equiv \backslash 1 \pmod{n}$，则 n 一定是合数；若 $a^{n-1} \equiv 1 \pmod{n}$，则几乎可以肯定地确认 n 是素数。因为它出错的机会非常少。

例如当 a=2 时，在小于 10^4 的 n 值中，只会产生 22 个伪素数；当 a=3 时，在小于 10^8 的 n 值中，也只有 255 个伪素数。可见 n 愈大，发生测试错误的概率愈小。一个随机选取的 50 位数是基于 2 的伪素数的概率不到一百万分之一。

为了使素数测试中出错的可能性不依赖于 n 而使结果更精确，我们作了如下改进

(1) 试验 s(s>1)个随机选取的基值 a 而不是仅试验一个基值，增加选取基值 a 时的“抽签运气”。

数学家们证明 S 值愈大，出错概率愈小，至多为 2^{-s}。因此只要 S 足够大，结果几乎可行。

(2) 当确定一个基值 a 后，采用反复平方的算法计算 $a^{n-1} \bmod n$。当计算当前模取幂的值时，

若发现

$$a^{2c} \bmod n = 1$$

$$a^{c} \bmod n \neq 1$$

$$a^{c} \bmod n \neq n-1 \qquad (c \text{ 为 } n-1 \text{ 的二进制数的某个后缀值 } 0 \leqslant c \leqslant n-1)$$

时，终止执行并确定 n 为合数。若反复平方的结果 $a^{n-1} \bmod n \neq 1$，返回 n 合数的结果。

告诫读者的是，反复平方过程中若某个模取幂 $a^c \bmod n$ 中的 a^c 值超过计算机允许的最大整数范围，程序可能会给出错误结论。为了避免这种意外，我们特意在程序前部加了编译指令 {$R+}，使得当 a^c 值超过整数的值域时，程序自动终止并给出错误信息。

(3) 试验了 S 个随机选取的基值 a 后还未得出 n 是合数的结论，则确认 n 是素数。

下面是程序题解。其中 modular_exponentiation (a, b, n)是主函数 miller_rabir (n,s) 在计算 $a^{n-1} \bmod n$ 时调用的子函数。在确定了被测试数 n 和选取基数 a 的次数后，可通过调用miller_rabin(n, s) 返回一个布尔标志。若返回 true，说明 n 是素数，若返回 false，说明 n 是合数。

```
{$R+}
function modular_exponentiation ( a, b, n: longint ): longint ;
{输入 a、b 和 n, 返回 a^b mod n 的值}
 var
```

```
d, t: longint;
begin
 d:=1; t:=a;
 while b>0 do {从右到左逐位分析 b 的二进制位，并迭代计算 a^c mod n}
  begin
   if t=1 then {若发现 1 的非平方根，直接返回迭代结果}
    begin
     modular_exponentiation:=d; exit
    end; {then}
   if b mod 2=1 then
    d:=d*t mod n;
   b:=b div 2;
   t:=t*t mod n
  end; {while}
 modular_exponentiation:=d {返回 ab mod n}
end; {modular_exponentiation}

function miller_rabin (n, s: longint): boolean;
{输入被测试的数 n 和试验基数的个数。若 n 是合数返回 false；若 n 是素数返回 true}
var
 i, a: longint;
begin
 randomize;
 for i:=1 to s do {顺序试验 S 个基值}
  begin
   a:=random(n-1)+1; {随机产生当前基值 a, (1≤a≤n-1)}
   if modular_exponentiation(a, n-1, n)<>1 then
   {a^(n-1)≠1(mod n)，则返回 n 是合数的信息}
    begin
     miller_rabin:=false; exit
    end {then}
  end; {for}
 miller_rabin:=true {试验了 S 个基值后未发现 n 是合数，则确定 n 是素数}
end; {miller_rabin}
```

§3.6 整数的因子分解

下面我们来讨论将整数 n 分解为素数的积的问题。§3.5 节讨论的素数测试过程只对 n

进行合数判定，并未列出 n 的素数因子。对一个大整数进行因子分解似乎要比仅确定 n 是素数还是合数困难得多，即便用当今超级的计算机和现行的最佳算法，要对任意一个 200 位的十进制整数进行因子分解也还是不可行的。

这里，我们给出一种启发性方法，虽然它既不能保证其运行时间也不能确保其运行成功，不过在实际应用中，该算法还是非常有效的，因为它极可能找出大整数的素数因子。

该算法计算出下述序列

$$x_1, x_2, x_3, x_4, \cdots x_i, x_{i+1}, \cdots$$

其中 x_1 为 0..n−1 之间的一个随机整数，其它数按下述递归式得出:

$$x_i \leftarrow (x_{i-1}^2 - c) \bmod n \qquad (2 \leqslant i)$$

c 是一个在 1..(n−1) 间随机选取的整数

设 Y 为距离当前项 x_i 最近的 2 的次幂项。我们从第 1 项开始，逐项地延长序列。每生成一项 x_i 后检查若 $(y-x_i)$ 与 n 的最大公约数 d 在 2..n−1 之间，即 $2 \leqslant \gcd(y-x_i, n) \leqslant n-1$，则 d 为素数因子。上述过程一直进行至 $y=x_i$ 为止。(因为 0 与 n 的最大公约数为 0, n 无法分解出因子)

我们用一个 pollard_rho(c, n) 的函数来描述这一算法。该函数输入序列递归式 $x_i \leftarrow (x_{i-1}^2 - c) \bmod n$ 中的 C 值和待因子分解的 n，返回 n 的一个因子。但问题是这个因子可能是还可继续分解的合数。为了真正分解出 n 的素数因子，我们采用了分治策略:

用 miller_rabin(n, 2)函数测试当前因子 n，若为素数，则打印素数因子 n；否则重复下述过程:

调用 pollard_rho(c, n)分解出 n 的又一个因子 t_1; $(n=t_1 * t_2)$

递归分解 n 的因子 t_1;

递归分解 n 的因子 t_2;

直至无法分解下去为止。

我们用一个 rho(n)的递归程序描述这一分治过程，下面是程序题解。由于执行子函数 miller_radin 时，需要通过反复平方法计算 $a^{n-1} \bmod n$ 的值。在迭代过程中，如果当前模取幂 $a^c \bmod n$ 中的 a^c 值超过计算机允许的整数最大值，程序可能会引出错误结论。为此，我们在程序前加了{$+R} 编译指令，限制变量值超过所在类型值域的情况。

```
{$R+}
function euclid (a, b: longint): longint;
 begin
  if b=0 then
   euclid:=a
  else
   euclid:=euclid(b, a mod b)
 end; {euclid}

function pollard_rho (c, n: longint): longint;
 {输入被因式分解的 n 和 x_i+1=(x_i^2 -c)mod n 中的 c，返回 n 的一个因子}
```

```
var
  i, x, y, k, d: longint;
 begin
  randomize;
  i:=1; x:=random(n); y:=x; k:=2; {初始化。产生 0..n-1 的一个随机数作为 x1}
  repeat
   inc(i);
   d:=euclid(n+n+y-x, n); {求 gcd(y-xi, n)>0}
   if (d>1) and (d<n) then {若最大公约数在 2..n-1 之间，则作为一个因子输出}
    begin
     pollard_rho:=d; exit
    end; {then}
   if i=k then {将下标为 2 的幂的变量 xi 存入 y}
    begin
     y:=x; k:=k*2
    end; {then}
   x:=(x*x+n-c) mod n; {求 xi ← x(i-1)^2 -c) mod n c xi >0)}
  until y=x; {循环至 xi = y，说明 n 无法分解出因子}
  pollard_rho:=n
 end; {pollard_rho}

function modular_exponentiation(a, b, n: longint): longint;
{输入 a, b 和 n，返回 a^b mod n 的值}
 var
  d, t: longint;
 begin
  d:=1; t:=a;
  while b>0 do {从右到左逐位分析 b 的二进制值，并迭代计算 a^c mod n}
   begin
    if t=1 then {若发现 1 的非平方根，直接返回当前迭代结果}
     begin
      modular_exponentiation:=d; exit
     end; {then}
    if b mod 2=1 then
     d:=d*t mod n;
    b:=b div 2; t:=t*t mod n
   end; {while}
  modular_exponentiation:=d {返回 a^b mod n}
```

```
    end; {modular_exponentiation}

function miller_rabin(n, s: longint): boolean;
 var
 {输入被测试的数 n 和试验基数的个数。若 n 是素数返回 true；否则返回 false}
  i, a: longint;
 begin
  randomize;
  for i:=1 to s do {试验 S 个随机选取的基值 a}
   begin
    a:=random(n-1)+1; {随机产生一个基值 a(1≤a≤n-1)}
    if modular_exponentiation(a, n-1, n)<>1 then
    {若 a^(n-1)≠1(mod n)，则返回 n 是合数的信息}
     begin
      miller_rabin:=false; exit
     end {then}
   end; {for}
  miller_rabin:=true {试验了 S 个基值后未发现 n 是合数，则确定 n 是素数}
 end; {miller_rabin}

procedure rho(n: longint);
 var
  t: longint;
 begin
  if miller_rabin (n, 2) then {若分解出的因子是素数，则打印}
   writeln(n)
  else
   begin
    repeat
     t:=pollard_rho (random (n-1)+1, n); {设法分解出 n 的一个因子 t}
     if t<n then {若 t 确实是 n 的一个因子}
      begin
       rho(t); {继续分解因子 t}
       rho(n div t) {继续分解另一个因子⌊n/t⌋}
      end {then}
    until t<n {直至无法再分解为止}
   end {else}
 end; {rho}
```

第四章　计算几何学

计算几何学是研究几何问题的算法，在现代工程与数学，诸如计算机图形学、计算机辅助设计、机器人学都要应用计算几何学。青少年程序设计竞赛中不时也出现一些简单的几何类型的试题。

在本章中，我们将学习一些二维的（即平面上的）计算几何学问题。每个输入物体都用一个点的集合 $\{P_i\}$ 表示 $(P_i=(X_i,Y_i);0<i<n-1)$ 这个点的集合描述了几何物体的特征：如一组点或一组线段，或者一个多边形按逆时针方向排列的顶点序列。输出常常是有关这些物体的问题的回答：如是否有直线相交，或者可能出现一个新的几何物体，如顶点集合的凸包（包含这些顶点的最小凸多边形）。

§4.1　线段的性质

从平面上取一固定点 P_1 $(P_1=(X_1,Y_1))$，从 P_i 出发向某点 P_2 $(P_2=(X_2,Y_2))$ 引一条线段，有方向且有长度，这样的线段叫做有向线段，记作 $\overrightarrow{P_1P_2}$，它的长度记作 $\overrightarrow{P_1P_2}=\sqrt{(X_1-X_2)^2+(Y_1-Y_2)^2}$，点 P_1 叫做它的始点，点 P_2 叫做它的终点；如果 P_1 是原点 $(0,0)$，则我们把有向线段 $\overrightarrow{P_1P_2}$ 简记为向量 P_2；如果不规定方向，则记线段为 $\overline{P1\ P2}$。

下面，我们来讨论三个问题。人们习惯使用平面解析几何的方法求解，但由于计算过程中使用了三角函数和除法，使得运算代价比较昂贵并且容易产生舍入误差。例如确定两条线段是否相交，通常是计算出形如 y = kx + d 的两个直线方程，运用除法找出直线的交点，并检查该交点是否同时在两条直线上。当线段几乎平行时，该算法对计算机中除法运算的精确度非常敏感。而我们对这些几何问题的计算，仅限于加、减、乘和比较运算，不可能产生计算误差，而且算法简洁高效。

在算法的程序描述中，我们对每个二维点的数据类型统一定义：

```
TYPE
  POSTYPE=RECORD
            X,Y:REAL;{X座标值和Y座标值}
            END;
```

1. 已知两条有向线段 $\overrightarrow{P_0P_1}$ 和 $\overrightarrow{P_0P_2}$，对他们的公共端点 P_0 来说，$\overrightarrow{P_0P_1}$ 是否在 $\overrightarrow{P_1P_2}$ 的顺时针方向。

首先，我们介绍一种叉积的概念，它是线段算法的中心。

考察下图的两个向量 P_1 和 P_2

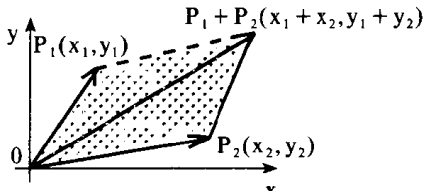


图 4.1-1

我们把叉积 P_1*P_2 看作是由点 (0,0)、P_1、P_2 和 P_1+P_2 四个点围成的平行四边形的阴影面积。亦可把叉积定义为一个矩阵行列式

$$\begin{aligned} P_1*P_2 &= \det \begin{bmatrix} X_1, Y_1 \\ X_2, Y_2 \end{bmatrix} \\ &= X_1*Y_2 - X_2*Y_1 \\ &= -P_2*P_1 \end{aligned}$$

对原点 (0,0) 来说:

若向量 P_1 在向量 P_2 的顺时针方向，则叉积 $P_1*P_2>0$;

若向量 P_1 在向量 P_2 的逆时针方向，则叉积 $P_1*P_2<0$;

若 P_2,P_2 两个向量共线 (方向可以相同或相反)，则叉积 $P_2*P_1=0$。

为了确定对公共端点 P_0，有向线段 $\overrightarrow{P_0\ P_1}$ 是否在有向线段 $\overrightarrow{P_0\ P_2}$ 的顺时针方向，我们仅需把原点平移至 P_0 即可:

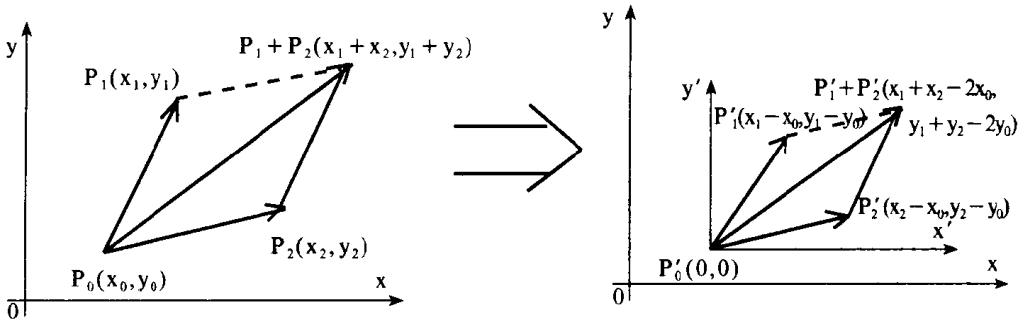


图 4.1-2

即设向量 $P_1' = P_1-P_0$, $P_2' = P_2-P_0$, 其中

$$P_1' = (X_1'、Y_1') = (X_1-X_0, Y_1-Y_0);$$

$$P_2' = (X_2'、Y_2') = (X_2-X_0, Y_2-Y_0);$$

$$P_1'*P_2' = (P_1-P_0)*(P_2-P_0) = (X_1-X_0)(Y_2-Y_0)-(X_2-X_0)(Y_1-Y_0)$$

如果该叉积为正，则 $\overrightarrow{P_0\ P_1}$ 在 $\overrightarrow{P_0\ P_2}$ 的顺时针方向;

如果为负，则 $\overrightarrow{P_0\ P_1}$ 在 $\overrightarrow{P_0\ P_2}$ 的逆时针方向。

我们用函数 multi (P_1, P_2, P_3) 描述上述叉积的计算过程:

```
Function Multi(Var p1, p2, p0 :PosType):Real;
```

{返回 $(P_1-P_0)*(P_2-P_0)$ 的叉积，若该叉积为正，则 $\overrightarrow{P_0P_1}$ 在 $\overrightarrow{P_0P_2}$ 的顺时针方向；若叉积为 0，则 $\overrightarrow{P_0P_1}$ 与 $\overrightarrow{P_0P_2}$ 共线；若该叉积为负，则 $\overrightarrow{P_0P_1}$ 在 $\overrightarrow{P_0P_2}$ 的逆时针方向}

```
Begin
  Multi: = (p1.x - p0.x)*(p2.y - p0.y) - (p2.x - p0.x)*(p1.y - p0.y)
End;{multi}
```

2. 确定两条连续的有向线段 $\overrightarrow{P_0P_1}$ 和 $\overrightarrow{P_1P_2}$ 在 P_1 点是向左转还是向右转

有了叉积的基础，我们毋需通过对角∠ $P_0P_1P_2$ 的计算来确定两条有向线段 $\overrightarrow{P_0P_1}$ 和 $\overrightarrow{P_1P_2}$ 在 P_1 点的转向，而仅需要添加一条辅助的有向线段 $\overrightarrow{P_0P_2}$，并检查 $\overrightarrow{P_0P_2}$ 是在有向线段 $\overrightarrow{P_0P_1}$ 的顺时针方向还是逆时针方向。为了做到这一点，我们计算出叉积 $(P_2-P_0)*(P_1-P_0)=(X_2-X_0)(Y_1-Y_0)-(X_1-X_0)(Y_2-Y_0)$

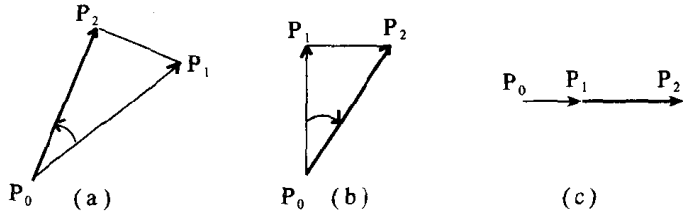


图 4.1-3

若该叉积为正，则 $\overrightarrow{P_0P_2}$ 在 $\overrightarrow{P_0P_1}$ 的逆时针方向，即 P_1 点向左转，(见图 4.1-3(a)。

若该叉积为负，则 $\overrightarrow{P_0P_2}$ 在 $\overrightarrow{P_0P_1}$ 的顺时针方向，即 P_1 点向右转，(见图 4.1-3(b)。

若叉积为 0，说明 P_0 P_1 和 P_2 共线 (见图 4.1 − 3(c))。

显然，我们可以通过调用函数 multi (P_2,P_1,P_0) 计算出叉积 $(P_2-P_0)*(P_1-P_0)$，以确定两条连续的有向线段 $\overrightarrow{P_0P_1}$ 和 $\overrightarrow{P_1P_2}$ 在 P_1 点的转向。

3. 确定两条线段 $\overline{P_1P_2}$ 和 $\overline{P_3P_4}$ 是否相交

我们分两步确定两条线段是否相交

第一步：确定两条线段是否不相交——快速排斥试验

先通过下述方法求出两条线段的界限框 (包含某线段且四边分别平行于 X 轴和 Y 轴的最小矩形)。

线段 $\overline{P_1P_2}$ 的界限框用矩形 $(\hat{P}_1,\hat{P}_2)$ 表示，其左下角的点为 $\hat{P}_1=(\min(X_1,X_2),\min(Y_1,Y_2))$，右上角的点为 $\hat{P}_2=(\max(X_1,X_2),\max(Y_1,Y2))$;

线段 $\overline{P_3P_4}$ 的界限框用矩形 $(\hat{P}_3,\hat{P}_4)$ 表示，其左下角的点为 $\hat{P}_3=(\min(X_3,X_4),\min(Y_3,Y_4))$，右上角的点为 $\hat{P}_4=(\max(X_3,X_4),\max(Y_3,Y_4))$;

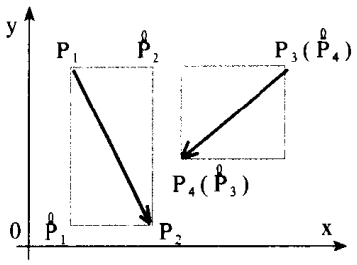


图 4.1-4

很显然．当$\overline{P_1P_2}$界限框右上角$\hat{P}_2$的X座标($\max(X_1,X_2)$)$\geqslant$$\overline{P_3P_4}$界限框左下角$\hat{P}_3$的X座标($\min(X_3,X_4)$),并且$\overline{P_3P_4}$界限框右上角$\hat{P}_4$的X座标($\max(X_3,X_4)$)$\geqslant$$\overline{P_1P_2}$界限框左下角$\hat{P}_1$的X座标($\min(X_1,X_2)$),则两个矩形在X方向相交；同样，当$\overline{P_1P_2}$界限框右上角$\hat{P}_2$的Y座标($\max(Y_1,Y_2)$)$\geqslant$$\overline{P_3P_4}$界限框左下角$\hat{P}_3$的Y座标($\min(Y_3,Y_4)$),并且$\overline{P_3P_4}$界限框右上角$\hat{P}_4$的Y座标($\max(Y_3,Y_4)$)$\geqslant$$\overline{P_1P_2}$界限框左下角$\hat{P}_1$的Y座标($\min(Y_1,Y_2)$),则两个矩形在Y方向相交。

如果两条线段的界限框不能同时在X方向和Y方向相交，则这两条线段不可能相交。因此通过快速排斥试验，可以确定两条线段是否有相交的可能。

在两个矩形同时在两个方向相交的前提下，我们做第二步：确定一条线段$\overline{P_1P_2}$(或$\overline{P_3P_4}$)是否跨立另一条线段$\overline{P_3P_4}$或($\overline{P_1P_2}$)所在的直线——跨立实验。

所谓跨立是指某线段的两个端点是否分别处于另一线段所在直线的两旁，或者是否有其中一个端点处于另一线段所在的直线上。我们可以运用叉积的方法来确定。其设计思想是，从P_1出发向另一线段的两个端点P_3和P_4引出两条辅助线段$\overline{P_1P_3}$、$\overline{P_1P_4}$。然后计算两个叉积:

由$(P_3-P_1)*(P_2-P_1)$确定$\overline{P_1P_3}$在$\overline{P_1P_2}$的哪个方向；由$(P_4-P_1)*(P_2-P_1)$确定$\overline{P_1P_4}$在$\overline{P_1P_2}$的哪个方向。

若两个叉积的正负号相同，说明$\overline{P_1P_3}$和$\overline{P_1P_4}$同在$\overline{P_1P_2}$的一边，即$\overline{P_3P_4}$不能跨立$\overline{P_1P_2}$所在的直线。例如（图4.1－5(b)）;

若两个叉积的正负号相反，说明$\overline{P_1P_3}$和$\overline{P_1P_4}$分别在$\overline{P_1P_2}$两边，即$\overline{P_3P_4}$跨立$\overline{P_1P_2}$所在的直线。例如（图4.1－5(a)）;

若任何一个叉积为0，由于$\overline{P_1P_2}$和$\overline{P_3P_4}$已通过了快速排斥试验，因此P_3和P_4两点中必有一点处于线段$\overline{P_1P_2}$所在的直线上。例如图4.1－5(c)。

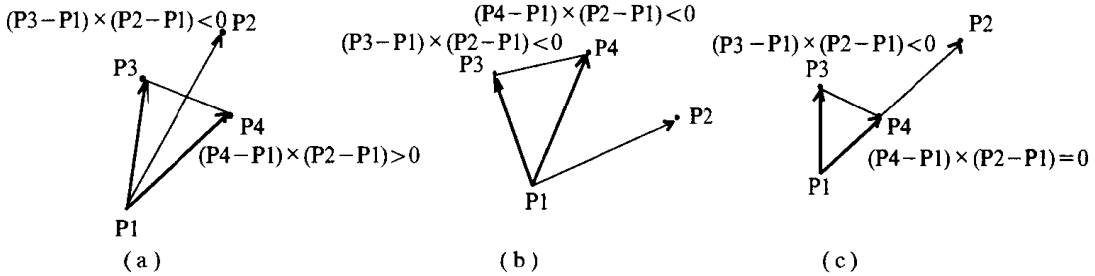


图 4.1-5

显然两条线段相交当且仅当它们能够通过快速排斥试验并且每一条线段能够跨立另一条线段所在的直线。

下面是程序题解:

```
Program Intersect_Example;

Type
  PosType          =  Record{点的数据类型}
                       x, y        :  Real
                      End;
  IntType          =  Record{线段的数据类型}
                       a, b        :  PosType{线段的两个端点}
                      End;
Var
  u,v              :  IntType;{两条线段}

Function Multi(Var p1, p2, p0 :PosType):Real;
{计算和返回叉积值(P1 - P0)*(P2 - P0)}
Begin
  Multi := (p1.x - p0.x)*(p2.y - p0.y) - (p2.x - p0.x)*(p1.y - p0.y)
End;{multi}

Function Across(Var u, v: IntType):Boolean;

Function Max(a, b : Real):Real;{返回两实数a、b的最大值}
Begin
  If a > b Then Max := a
          Else Max := b
End;{max}

Function Min(a, b : Real):Real;{返回两实数a、b的最小值}
Begin
  if a < b Then Min := a
          Else Min := b
End;{min}
Begin
  If (Max(u.a.x, u.b.x) >= Min(v.a.x, v.b.x)) And
    (Max(v.a.x, v.b.x) >= Min(u.a.x, u.b.x)) And
```

```
    (Max(u.a.y, u.b.y) >= Min(v.a.y, v.b.y)) And
    (Max(v.a.y, v.b.y) >= Min(u.a.y, u.b.y)) And
    (Multi(v.a, u.b, u.a) * Multi(u.b, v.b, u.a) > 0) And
    (Multi(u.a, v.b, v.a) * Multi(v.b, u.b, v.a) > 0)
{若线段 u 和线段 v 的界限框同时在 X 方向和 Y 方向相交且 u(或 v)跨立 v(或 u)所在的直线}
    Then Across := True {则返回线段 U 和 V 相交的信息}
    Else Across := False {否则返回不相交信息}
End;{across}

Begin
  Write('(x1, y1)--(x2, y2):');  {输入线段 u 的两个端点的信息}
  Readln(u.a.x, u.a.y, u.b.x, u.b.y);
  Write('(x3, y3)--(x4, y4):');  {输入线段 v 的两个端点的信息}
  Readln(v.a.x, v.a.y, v.b.x, v.b.y);
  Writeln(Across(u, v)){输出两线段是否相交的信息}
End.{min}
```

§4.2 确定任意一对线段是否相交

下面，我们将讨论关于在一组线段中确定其中任意两条线段是否相交的问题。该算法输入由若干条线段组成的集合。若这组线段中任意两条线段相交，则返回布尔标志 true；否则返回 false。

算法使用了一种称为“扫除”的技术，即假设一条垂直扫除线沿 X 轴方向从左到右顺序移动，穿过已知的若干线段。移动过程中，每遇到一个线段端点，它就将穿过扫除线的所有线段放入一个动态数据结构中，并利用它们之间的关系进行排序，核查是否有相交点存在。

为了使问题简化，算法做了两点假设：

1. 没有输入线段是垂直的；

2. 未有三条输入线段相交于同一点的情形。

由于不存在垂直的输入线段，所以任何与给定的垂直扫除线相交的输入线段与其只能有一个交点。我们可以将此时与垂直扫除线相交的线段按交点的 Y 座标值单调递增的顺序放在一个序列 T 中。当线段的左端点遇到扫除线时，线段就进入序列 T，当其右端点遇到扫除线时便随之离开序列 T。

具体点讲，考察两条不相交线段 S_1 和 S_2。如果垂直扫除线平移至 X 位置时与这两条线段相交，则我们说这些线段在 X 是可比的。如果 S_1 和 S_2 在X可比并且 S_1 与在 X 的扫除线的交点比 S_2 在同一条扫除线的交点高，则我们说在 X 处 S_1 位于 S_2 之上，写作 $S_1 >_x S_2$，反映在此时的 T 序列中，S_1 位于 S_2 前。

例如在图 4.2 - 1中，我们有下述关系：

$a >_r c$，即在 r 处的 T 序列为 {a, c};

$a >_t b, b >_t c, a >_t c$，即在 t 处的 T 序列为 {a, b, c};

$b >_u c$，即在 u 处的 T 序列为 {b, c};

线段 d 与其它任何线段都不可比。

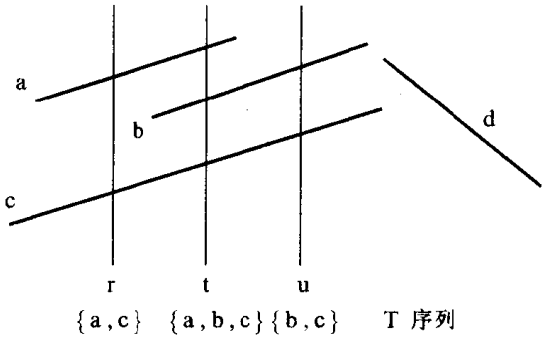


图 4.2 - 1

随着垂直扫除由左至右平移，T 序列中线段的数量和相对位置将由于下述原因而发生变化。

1. 扫除线遇到某线段的左端点时，该线段进入 T 序列；遇到某线段的右端点时，该线段离开 T 序列;

2. 扫除线穿过两线段交点前后的某区域（即在该区域内的 T 序列中，这两条线段是相邻的），它们在 T 序列的位置将对换。

例如图 4.2 - 2 中，扫除线 V 和 W 分别位于线段 e 和 f 的交点的左边和右边。在交点的左边 r 时，$e >_r f$, T 序列为 {e, f}; 而在交点右边的 W 时，$f >_w e$, T 序列则变为 {f, e}。由于没有第三条线段相交于同一点，所以在交点附近必有一个连续区域（如图 4.2 - 2 中的阴影区域），使得垂直扫除线在该区域活动时，相交线段 e 和 f 在序列 T 中是连续的。

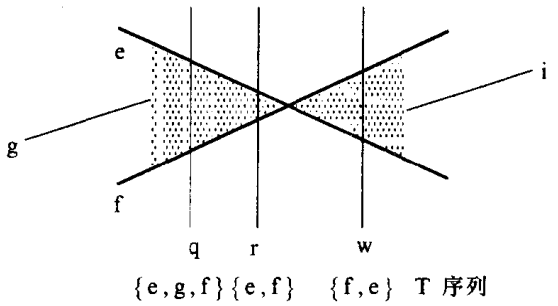


图 4.2 - 2

由此可见，典型的扫除算法，要安排下列两个数据集合:

1. 由 T 序列给出与扫除线相交的线段之间的关系，即线段按相交点 Y 座标递增的顺序排列;

2. 由左至右定义扫除线的一系列暂停位置，我们称每个这样的暂停位置为一个事件点。一般来说，事件点为每条线段的端点。当遇到线段左端点时，就把线段按排序要求插入 T 序列中；当遇到线段右端点时，就把它从 T 中删去；每当两条线段在 T 序列中第一次变为相邻时，我们就检查它们是否相交。

我们用一个称为 any_segments_intersect 的函数来描述算法流程。函数输入由 n 条线段组成的集合 S。如果 S 中任何一对线段相交，算法就返回布尔值 true；否则返回布尔值 false。由于在“扫除”过程中，T 序列需按排序要求频繁地进行插入或删除，因此我们将它放入一个动态的指针链表中。

```
function any_segments_intersect(s): boolean;
begin
T ← Φ;
  对集合 S 中的线段端点从左向右进行排序。对 X 座标相同的端点，把 Y 座标较小的放在前面;
for 对端点排序表中的每一个事件点 p do
    begin
    if p 是线段 S 的左端点 then
     begin
     按排序要求将 S 线段插入序列 T;
     if (s 在 T 中有左邻元素且该线段与 S 相交) or (S 在 T 中有右邻元素且该线段与 S 相交)
    then begin any_segments_intersect ← true; exit; end;
    end;{then}
 if p 是线段 s 的右端点 then
    begin
     if s 在 T 中有左邻元素和右邻元素且两线段相交
       then begin any_segments_intersect ← true; exit; end;
        将线段 s 从序列 T 中删除;
    end;{then}
 end;{for}
any_segments_intersect ← false;
end;{any_segments_intersect}
```

下面是程序题解:

```
Program Any_Segments_Intersect_Programme;

Const
  Maxn                  =   1000;

Type
  PosType               =   Record{点的数据类型}
                              x, y        :  Real{坐标}
                            End;
```

```
  Link                = ^Node;
  Node                = Record{链结点类型}
                          k          : Integer;{线段序号}
                          p, x       : Link{左，右指针}
                        End;
  LstType             = Record{端点的数据类型}
                          n          : Integer;{端点所在的线段序号}
                          w          : 1..2;{左，右端点的标志}
                        End;

Var
  N                   : Integer;{线段数}
  Head                : Link;{存储T序列的链表}
  Sct                 : Array  [1..Maxn, 1..2] of PosType;
{sct[i, 1]——第i条线段的左端点坐标，sct[i, 2]——第i条线段的右端点的坐标}
  Plst                : Array [1..2*Maxn] of LstType;
{线段端点的排序表}
  F                   : Text;{文件变量}

Function Comp(Var p1, p2 : PosType):Shortint;
{若P1的X坐标值小于P2的X坐标值或者X坐标相同而P1的Y坐标较小，则返回-1;}
{若P1的X坐标值大于P2的X坐标值或者X坐标相同而P1的Y坐标较大，则返回1}
{若P1和P2重合则返回0}
Begin
  If (p1.x < p2.x) Or (p1.x = p2.x) And (p1.y < p2.y)
     Then Comp := -1
     Else If (p1.x = p2.x) And (p1.y = p2.y)
             Then Comp := 0
             Else Comp := 1

End;{Comp}

Procedure Swap(Var a, b : LstType);{交换a、b两点}
Var
  l                 : LstType;
Begin
  l := a; a := b; b := l
End;{swap}
```

```
Function Random_Partition (p, r : Integer): Integer;
{从端点序列 PLST[P..R]随机选取一个支点元素 X, 将 PLST 分成两个子序列 PLST[P..J]
和 PLST[J+1..R], 使得 PLST[P..J]中每个端点的 X 坐标小于 X 或者等于 X 但 Y 坐标
较小; 而 PLST[J+1..R]中每个端点的 X 坐标大于 X 或者等于 X 但 Y 坐标较大}
Var
  i, j          : Integer;
  x             : PosType;
  l             : LstType;
Begin
  i := p + Random(r - p + 1);
  x := Sct[Plst[i].n, Plst[i].w];
  i := p; j := r;
  Repeat
    While Comp(Sct[Plst[i].n, Plst[i].w], x) = -1 Do Inc(i);
    While Comp(Sct[Plst[j].n, Plst[j].w], x) =  1 Do Dec(j);
    If i < j Then Begin
      If Comp(Sct[Plst[i].n, Plst[i].w], Sct[Plst[j].n, Plst[j].w]) = 0
        Then Begin
          Inc(i); Dec(j)
        End {then}
        Else Swap(Plst[i], Plst[j])
    End {then}
  Until i >= j;
  Random_Partition := j
End;{Random_partition}

Procedure Quick_Sort(p, r : Integer);
{对 PLST 集合中的线段端点从左向右排序。对 X 坐标相同的端点, 把 Y 坐标较小的放在
前面}
Var
  q, i          : Integer;
Begin
  If r - p + 1 <= 5 Then Begin
    For q := p + 1 to r Do Begin
      i := q;
      While (i > 1) And (Comp(Sct[Plst[i].n, Plst[i].w],
                Sct[Plst[i-1].n, Plst[i-1].w]) = -1) Do Begin
        Swap(Plst[i], Plst[i-1]);
```

```
        Dec(i)
      End {while}
    End {for}
  End {then}
  Else Begin
    q := Random_Partition(p, r);
    Quick_Sort(p, q);
    Quick_Sort(q + 1, r)
  End {else}
End;{quick_sort}
Procedure Init;
Var
  i              : Integer;
  pt             : PosType;
Begin
  Assign(F, {'INPUT.DAT'}'OUTPUT.DAT'); {输入文件名串与文件变量连接}
  Reset(F);{文件读准备}
  Readln(F, N);{读入线段数}
  For i := 1 to N Do Begin{顺序输入N条线段}
    Readln(F, Sct[i, 1].x, Sct[i, 1].y,{读入第i条线段的两个端点坐标}
              Sct[i, 2].x, Sct[i, 2].y);
    If Sct[i, 1].x > Sct[i, 2].x Then Begin
    {确定sct[i,1]存第i条线段的左端点; sct[i]存第i条线段的右端点}
      pt := Sct[i, 1]; Sct[i, 1] := Sct[i, 2]; Sct[i, 2] := pt
    End;{then}
    Plst[i * 2 - 1].n := i; Plst[i * 2 - 1].w := 1;
    {将该线段的两个端点放入端点集合PLST}
    Plst[i * 2].n := i; Plst[i * 2].w := 2
  End;{for}
  Close(F);
  Quick_Sort(1, 2 * N);
  {对端点集合中的所有端点以X坐标值为主关键字进行合并排序}
  Head := nil{T链设空}
End;{init}

Function Multi(Var p1, p2, p0 :PosType):Real;
{计算和返回叉积(P1 - P0)*(P2 - P0)的值}
Begin
```

```
  Multi := (p1.x - p0.x)*(p2.y - p0.y) - (p2.x - p0.x)*(p1.y - p0.y)
End ;{ multi }

Function Across (a, b : Integer): Boolean ;
{若编号为 a 和 b 的两条线段相交，则返回 true；否则返回 false}

Function Max (a, b : Real): Real ;{返回实数 a 和实数 b 中的最大值}
 Begin
  If a > b Then Max := a
          Else Max := b
End ;{ max }

Function Min (a, b : Real): Real ;{返回实数 a 和实数 b 中的最小值}
 Begin
  if a < b Then Min := a
          Else Min := b
End ;{ min }

Begin
  If (Sct[a, 2].x >= Sct[b, 1].x) And (Sct[b, 2].x >= Sct[a, 1].x) And
     (Max(Sct[a, 1].y, Sct[a, 2].y) >= Min(Sct[b, 1].y, Sct[b, 2].y)) And
     (Max(Sct[b, 1].y, Sct[b, 2].y) >= Min(Sct[a, 1].y, Sct[a, 2].y)) And
     (Multi(Sct[b, 1], Sct[a, 2], Sct[a, 1])*
           Multi(Sct[a, 2], Sct[b, 2], Sct[a, 1]) > 0) And
     (Multi(Sct[a, 1], Sct[b, 2], Sct[b, 1])*
           Multi(Sct[b, 2], Sct[a, 2], Sct[b, 1]) > 0)
{若线段 a 和线段 b 的两个界限框同时在 x 方向和 y 方向相交且互相“跨立”，则返回相交标志 true；否则返回 false}
     Then Across := True
     Else Across := False
End ;{ across }

Function Comp_Sect (u, v : Integer): Boolean ;
{计算 (V1 - U1)*(V2 - U1) 的叉积值。若为负返回 false (线段 u 位于 v 下方)；否则返回 true (线段 u 位于 v 上方)}
Begin
  If Multi(Sct[v, 1], Sct[v, 2], Sct[u, 1]) < 0
     Then Comp_Sect := False
```

```
    Else Comp_Sect := True
End;{comp}

Function Insert(k : Integer): Link;
{将编号为k的线段按排序要求插入T链表，并返回插入位置的地址指针}
Var
  p, z          : Link;
Begin
  New(z); z^.k := k;{申请结点z的内存，该结点的线段序号为k,左指针为空}
  p := Head; z^.p := nil;{从链首开始，顺链搜索插入位置P}
  While (p <> nil) And Comp_Sect(k, p^.k) Do Begin
    z^.p := p;
    p := p^.x
  End;{while}
  z^.x := p;{将z插入P结点前，设定z的右指针}
  If p <> nil Then p^.p := z;{设定P的左右指针}
  If z^.p = nil
     Then Head := z
     Else z^.p^.x := z;
  Insert := z{返回z的地址指针}
End;{insert}

Function Any_Segments_Intersect: Boolean;
Var
  i             : Integer;
  Av, Bl, s     : Link;
Begin
  For i := 1 to 2 * N Do{搜索端点排序表中的每一个事件点}
    If Plst[i].w = 1 Then Begin{若当前端点是左端点}
      s := Insert(Plst[i].n);{按排序要求将端点所在的线段插入T链}
      Av := s^.x; Bl := s^.p;{求该线段在T中的左邻结点和右邻结点}
      If (Av <> nil) And Across(s^.k, Av^.k) Or
{若左邻结点存在且与该线段相交或者右邻结点存在且与该线段相交，则返回true}
         (Bl <> nil) And Across(s^.k, Bl^.k) Then Begin
        Any_Segments_Intersect := True;
        Exit
      End{then}
    End{then}
```

```
    Else Begin {否则当前端点是右端点，则从 T 链首出发搜索端点所在的线段}
      s := Head;
      While s^.k <> Plst[i].n Do s := s^.x;
      Av := s^.x; Bl := s^.p;{求该线段的左邻元素和右邻元素}
      If (Av <> nil) And (Bl <> nil) And Across(Av^.k, Bl^.k) Then Begin
      {若左邻结点和右邻结点存在且两线段相交，则返回 true}
        Any_Segments_Intersect := True;
        Exit
      End;{then}
      s^.x^.p := s^.p;{将该线段从 T 链中删除}
      If s^.p = nil Then Head := s^.x
                    Else s^.p^.x := s^.x;
      Dispose(s)
    End;{else}
  Any_Segments_Intersect := False{返回线段不相交信息}
End;{Any_Segments_Intersect}

Begin
  Init;{输入所有线段的端点坐标，建立端点排序表}
  Writeln(Any_Segments_Intersect);{计算和输出线段是否相交的信息}
End.{main}
```

§4.3 寻找凸包

有一个点集 $Q=\{P_0,..,P_{n-1}\}$，它的凸包 ch(q)是一个最小的凸多边形 P，且满足 Q 中的每个点或者在 P 的边界上，或者在 P 的内部。在直观上，我们可以把 Q 中的每个点看作露在板外的铁钉，那么凸包就是包含所有铁钉的一个拉紧的橡皮绳所构成的形状，例如图 4.3 − 1。

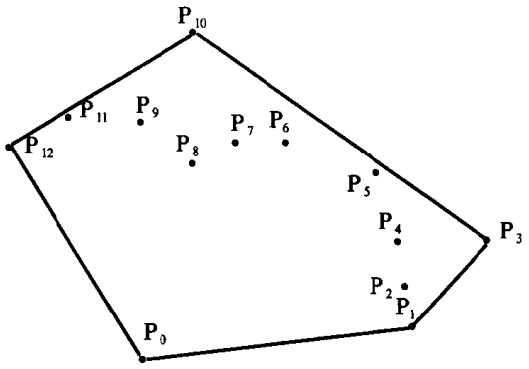


图 4.3-1

在计算一组点的凸包的基础上，可以引出一些计算几何学问题的算法。例如考虑二维的最远点对问题:

已知平面上一组 n 个点，并希望找出彼此间最远的两个点。很显然，这两个点必定是凸包的顶点。运用计算凸包的算法求最远点对问题，可使算法效率提高不少。

下面我们将阐述两种计算 CH(Q) 的算法。这两种算法都按逆时针方向输出凸包的顶点。

一、graham 扫描法

首先我们必须明确，在点集 Q 中处于最低位置（Y 坐标值最小）的一个点 P_0 是凸包 ch(q) 的一个顶点。如果这样的顶点有多个，则选取最左边的点为 P_0。我们从 P_0 出发，按逆时针方向寻找凸包的顶点。寻找过程中，除 P_0 外的每个点都要被扫描一次，其顺序是依据各点在逆时针方向上相对 P_0 的极角的递增次序。极角的大小，可以通过计算叉积

$$(P_i - P_0) * (P_j - P_0)$$

来确定:

若叉积 >0, 则说明相对 P_0 来说，P_j 的极角大于 P_i 的极角，P_i 先于 P_j 被扫描;

若叉积 <0, 则说明 P_j 的极角小于 P_i 的极角，P_j 先于 P_i 被扫描;

若叉积 $=0$, 则说明两个极角相等。在这种情况下，由于 P_i 和 P_j 中距离 P_0 较近的点不可能是凸包的顶点，因此我们只需扫描其中一个与 P_0 距离较远的点。

例如图 4.3 − 2 说明了图 4.3 − 1 中的点按相对于 P_0 的极角进行排序后得到的扫描序列 $< P_1, P_2 .. P_n >$。

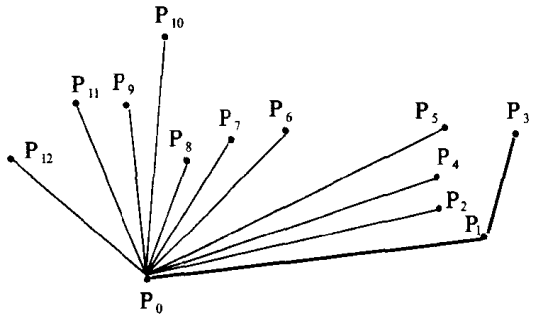


图 4.3 - 2

接下来的问题是如何确定当前被扫描的点 $P_i(1 < i < n-1)$ 是凸包上的点呢？我们设置一个关于候选点的堆栈 S 来解决凸包问题。初始时 P_0 和排序后的 P_1, P_2 作为初始凸包相继入栈。扫描过程中，Q 集合中的其它点都被推入堆栈一次，而不是凸包 ch(q) 顶点的点最终将弹出堆栈。当算法结束时，堆栈 S 中仅包含 ch(q) 的顶点，其顺序为各点在边界上出现的逆时针方向排列的顺序。

由于我们是沿逆时针方向通过凸包的，因此如果栈顶元素是凸包的顶点，则它应该向左转指向当前被扫描的点 P_i。如果它不是向左转，则它不属于凸包中的顶点，应从堆栈 S 中移出。在弹出了所有非左转的顶点后，我们就把 P_i 推入堆栈 S，继续扫描序列中的下一个点 P_{i+1}。

判断当前栈顶元素是否向左转指向扫描点 P_i，只需计算下述叉积的值。

$(P_i - P_{top-1}) * (P_{top} - P_{top-1})$（其中 top 为栈顶指针，$P_{top-1}$ 为次栈顶元素）

若叉积大于等于0，说明线段$\overline{P_{top-1}P_i}$在$\overline{P_{top-1}P_{top}}$的顺时针方向或共线，$P_{top}$未向左转。在依次扫描了$P_3..P_{n-1}$后，堆栈S从底P到顶部依次是按逆时针方向排列的ch(q)中的顶点。

算法流程如下：

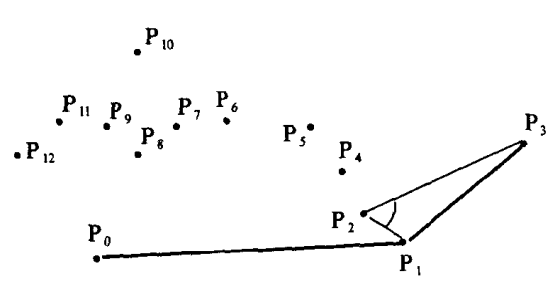
```
procedure graham_scan (q);
begin
```

求q中Y坐标值最小的点P_0。若具有Y坐标最小值的顶点有多个，则取最左边的点为P_0；

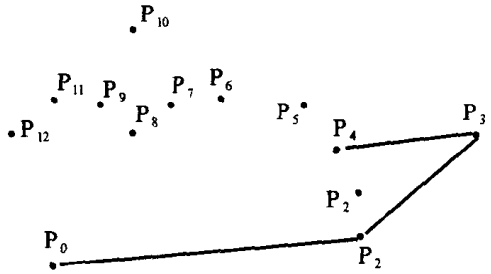
对q的剩余点按逆时针相对P_0的极角进行排序。若有数个极角相同的点，则保留其中一个与P_0距离最远的点，得出序列$<P_1,P_2,\cdots\cdots,P_{n-1}>$；

```
  P0,P1,P2相继入栈S;
  for i:=3 to n-1 do
    begin
     while 由次栈顶元素、栈顶元素和Pi所形成角不是向左转 do 栈顶元素出栈S;
    Pi入栈S;
  end ;{for}
 打印按逆时针方向排列的ch(q)中的顶点s[1..top];
end ;{graham _ scan}
```

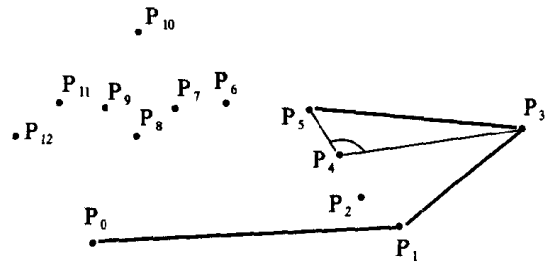
例如graham_scan(q)输入图4.3－3所示的点集q后，其执行过程如下：



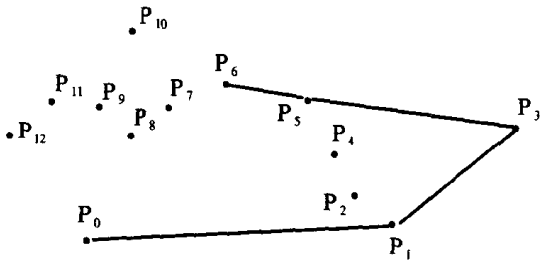
(a)



(b)



(c)



(d)

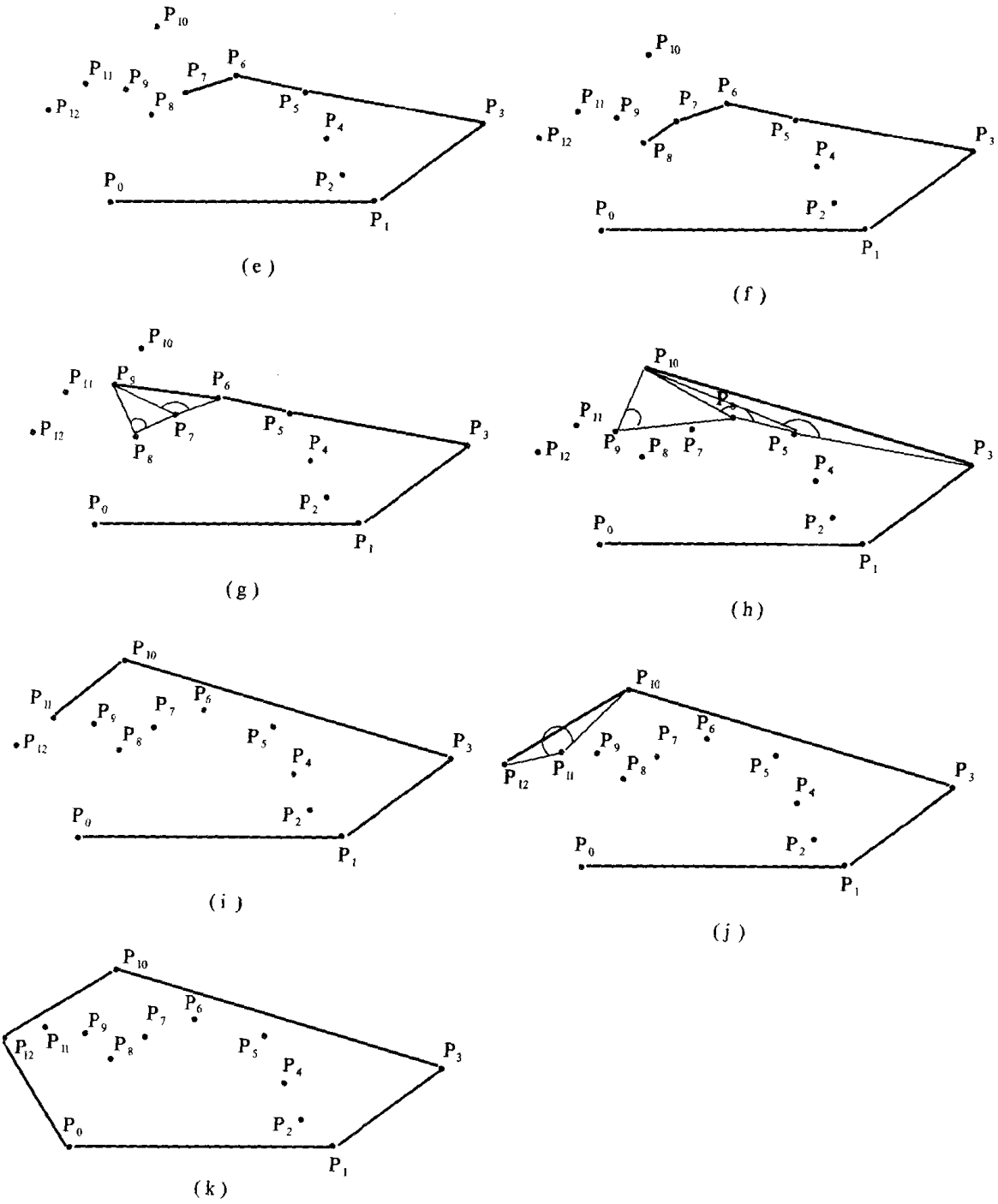


图 4.3 - 3

程序题解如下:

```
Program Graham_Scan;
Const
  Maxn                    =   500;{最多点数}
Type
```

```
  PosType              =  Record {二维点的类型}
                            x, y        :  Real
                          End ;

Var
  N, top               : Integer ;{实际点数，栈顶指针}
  List                 : Array [0.. Maxn ] of PosType ;{点序列}
  Stk                  : Array [1.. Maxn ] of Integer ;{栈}
  F                    : Text ;{文件变量}

Procedure Swap ( Var a, b : PosType );{顶点 a 和顶点 b 交换}
Var
  t                 :  PosType ;
Begin
  t := a; a := b; b := t
End ;{swap }

Function Multi ( Var p1, p2, p0 : PosType ): Real ;
{返回 ( P1 - P0 ) * ( P2 - P0 ) 的叉积，若该叉积为正，则 P0P1 在 P0P2 的顺时针方向；
若叉积为 0，则 P0P1 与 P0P2 共线；若该叉积为负，则 P0P1 在 P0P2 的逆时针方向}
Begin
  Multi := ( p1.x - p0.x ) * ( p2.y - p0.y ) - ( p2.x - p0.x ) * ( p1.y - p0.y )
End ;{ multi }

Function Comp ( Var p1, p2 : PosType ): Boolean ;
Var
  t                 :  Real ;
Begin
  t := Multi ( p1, p2, List [0] );
  {通过计算 ( P1 - P0 ) * ( P2 - P0 ) 的叉积确定 P2 和 P1 的两个极角}
  If ( t > 0 ) Or ( t = 0 ) And
     ( Sqr ( p1.x - List [ 0 ].x ) + Sqr ( p1.y - List [ 0 ].y ) <
       Sqr ( p2.x - List [ 0 ].x ) + Sqr ( p2.y - List [ 0 ].y ))
{若 P2 的极角大于 P1 的极角或者两极角相同但 P2 距离 P0 较远，则返回 true，否则返回
false }
     Then Comp := True
     Else Comp := False
```

```
End;{comp}

Procedure Sort(p, r : Integer);
{对list[p..r]的点按逆时针相对P0的极角进行合并排序。如果有数个点的极角相同，则
留下其中一个与P0距离最远的点}
Var
  i, j          : Integer;
  x             : PosType;
Begin
  if r - p + 1 <= 5 Then Begin{若list[p..r]间点数少于或等于5}
    For j := p + 1 to r Do Begin
{将子序列list[1..p]和list[p+1..r]合并排序成一个list[1..r]}
      i := j;
      While (i > 1) And Comp(List[i], List[i-1]) Do Begin
        Swap(List[i], List[i-1]);
        Dec(i)
      End{then}
    End{for}
  End{then}
  Else Begin
    x := List[p + Random(r - p + 1)];{在list[p..r]中随机选取一个支点X}
    i := p; j := r;
    Repeat
{将list[p..r]分成两个子序列list[p..j]和list[j+1..r], 使得list[p..j]的每个点的极角小
于等于X, list[j+1..r]的每个点的极角大于等于X}
      While Comp(List[i], x) Do Inc(i);
      While Comp(x, List[j]) Do Dec(j);
      If i < j Then Swap(List[i], List[j])
    Until i>= j;
    Sort(p, j);{对list[p.j]继续递归排序}
    Sort(j + 1, r){对list[j+1..r]继续递归排序}
  End{else}
End;{sort}

Procedure Init;
Var
  i             : Integer;
Begin
```

```
  Assign(F, ' INPUT.DAT '); {将文件名与文件变量}
  Reset(F);{文件读准备}
  Readln(F, N);{从文件读入点的个数}
  For i := 0 to N - 1 Do Begin
{依次读入每个点，并求出Y坐标值最小的点、或具有Y最小值的数个点中最左边的点P0}
    Readln(F, List[i].x, List[i].y);
    If (List[i].y < List[0].y) Or (List[i].y = List[0].y)
       And (List[i].x < List[0].y) Then Swap(List[0], List[i])
  End;{for}
  Sort(1, N - 1){对除P0外的其余n个点按逆时针相对P0的极角进行合并排序}
End;{init}

Procedure Graham;
Var
  i              : Integer;
Begin
  For i := 1 to 3 Do Stk[i] := i - 1;{P0,P1,P2依次入栈}
  Top := 3;
  For i := 3 to N - 1 Do Begin{依次对P3..Pn-1的每个点进行一次迭代}
    While Multi(List[i], List[Stk[Top]], List[Stk[Top - 1]]) >= 0 Do
      Dec(Top);
    {所有未向左转指向Pi的栈顶元素不是凸包的顶点，因此从栈中去掉}
    Inc(Top);
    Stk[Top] := i{Pi入栈}
  End;{for}
  For i := 1 to Top Do{依次打印堆栈中的凸包顶点}
    Writeln('(', List[Stk[i]].x:7:2, ',', List[Stk[i]].y:7:2,')');
  Writeln
End;{graham}

Begin
  Init;{输入点集q; 计算P0; 对除P0外的n-1个点按逆时针相对P0的极角进行合并排序}
  Graham{通过扫描法计算和输出凸包ch(q)}
End.{main}
```

一些试题的算法就是按上述求解凸包的思路构思的。

【例 1】巨人和鬼

一组 n 个巨人正与 n 个鬼进行战斗，每个巨人的武器是一个质子包，它可以把一串质子流射中鬼而把鬼消灭。质子流沿直线行进，在击中鬼时就终止。巨人决定采取下述策略。他们寻找鬼配对，以形成 n 个巨人——鬼对，然后每个巨人同时向他选取的鬼射出一串质子流。我们知道，让质子流互相交叉是很危险的。因此巨人选择的配对方式应该使质子流都不会交叉。假定每个巨人和每个鬼的位置都是平面上的一个固定点，并且没有三个位置共线，求一种配对方案。

算法分析:

我们设 $P_1..P_n$ 为巨人的固定点；$P_{n+1}..P_{2n}$ 为鬼的固定点，我们采取分治策略寻找序列 $[P_p..P_r]$ 中的配对方案（初始时 $[P_p..P_r]$ 为 $[P_1..P_{2n}]$):

在 $[P_p..P_r]$ 中找出一个最低位置（Y 坐标值最小）的一个点 P_0，如果这样的点有多个，则选取最左边的点为 P_0，P_0 与 P_p 交换。然后将其余点 $[P_{P+1}..P_r]$ 按相对 P_p 的极角递增的顺序排列。显然 P_p 与其余点 $P_{P+1}..P_r$ 之间的任何线段是不会交叉的。我们从 P_p 开始寻找一个巨人和鬼成对的最小子区间 $[P_p..P_i]$($p \leqslant i \leqslant r$)。若该子区间仅剩一个元素，配对结束；否则巨人（鬼）P_p 与鬼（巨人）P_i 配对。这样使得尚未配对的巨人和鬼分布在两个子区间 $[P_{p+1}..P_{i-1}]$，$[P_{i+1}..P_r]$。继续按上述分治策略分别递归求解 $[P_{p+1}..P_{i-1}]$ 和 $[P_{i+1}..P_r]$。

下面是程序题解:

```
Program Giants__And__Monsters;

Const
  Maxn                  =   100;

Type
  Node                  =   Record
                              k          :  Integer;{k=1:巨人; k=-1:魔鬼}
                              x, y       :  Real{坐标}
                            End;
Var
  N, i                  :  Integer;{魔鬼和巨人的对数，辅助变量}
  F                     :  Text;{文件变量}
  List, Lt              :  Array [1..2 * Maxn] of Node;{点集，辅助点集}
  p0                    :  Node;{最低位置点}

Function Comp(Var p1, p2 : Node):Boolean;
{计算(P1 - P0)*(P2 - P0)的叉积值。若值为正（相对于 P0 来说，P2 的极角大于 P0 的极
```

```
角）返回 true；否则返回 false｝
Begin
  If (p1.x - p0.x)*(p2.y - p0.y) - (p1.y - p0.y)*(p2.x - p0.x) > 0
     Then Comp := True
     Else Comp := False
End;

Procedure Merge(p, q, r : Integer);
｛将两个已按极角递增顺序排好序的子序列 list[p..q]和 list[q+1..r]合并排序成一个序列
list[p..r]｝
Var
  i, j, t        : Integer;
Begin
  t := p; i := p; j := q + 1;
  While t <= r Do Begin
    If (i <= q) And ((j > r) Or Comp(List[i], List[j])) Then Begin
      Lt[t] := List[i]; Inc(i)
    End
    Else Begin
      Lt[t] := List[j]; Inc(j)
    End;｛else｝
    Inc(t)
  End;｛while｝
  For i := p to r Do List[i] := Lt[i]
End;｛merge｝

Procedure Merge_Sort(p, r : Integer);
Var
  q              : Integer;
Begin
  If p <> r Then Begin
    q := (p + r - 1) div 2;｛计算中间下标 q｝
    Merge_Sort(p, q);｛对子序列 list[p..q]递归排序｝
    Merge_Sort(q + 1, r);｛对子序列 list[q+1..r]递归排序｝
    Merge(p, q, r)｛合并两个排好序的子序列｝
  End｛then｝
End;｛merge_sort｝
```

```
Procedure Swap (Var a, b : Node);{交换 a 和 b 两点}
Var
  t              :  Node;
Begin
  t := a; a := b; b := t
End;

Procedure Out_pos (Var p : Node);{输出 p 点的 X 和 Y 坐标}
Begin
  Write(p.x:8:2, p.y:8:2)
End;

Procedure Pick (p, r : Integer);
Var
  i, m           :  Integer;
Begin
  If p < r Then Begin
    m := p;{求出 Y 坐标值最小的点或具有 Y 最小值的数个点中最左边的点 m}
     For i := p to r Do
      If (List[i].y < List[m].y) Or (List[i].y = List[m].y)
          And (List[i].x < List[m].x) Then m := i;
    Swap(List[p], List[m]);{m 点与 p 点交换并设为 P0}
    p0 := List[p];
    Merge_Sort(p + 1, r);{对 list[p+1..r]按极角递增的顺序排序}
    m := List[p].k; i := p;{求巨人和魔鬼成对的最小子区间 list[p..i]}
   Repeat
      Inc(i);
      m := m + List[i].k
    Until m = 0;
    Out_pos(List[p]);{list[p]和 list[i]配对}
    Out_Pos(List[i]);
    Writeln;
    Pick(p+1, i - 1);{递归搜索子序列 list[p+1..i-1]中配对情况}
    Pick(i + 1, r){递归搜索子序列 list[i+1..r]中配对情况}
  End{then}
End;{pick}

Begin
```

```
  Assign(F,' INPUT.DAT');{输入文件名串与文件变量连接}
  Reset(F);{文件读准备}
  Readln(F, N);{读入巨人和鬼的对数}
  For i := 1 to N Do Begin{读入N个巨人的位置}
    Readln(F, List[i].x, List[i].y);
    List[i].k := 1
  End;
  For i := N + 1 to 2 * N Do Begin{读入N个魔鬼的位置}
    Readln(F, List[i].x, List[i].y);
    List[i].k := -1
  End;
  Pick(1, 2 * N)
End.{main}
```

二、Jarris 步进法

Jarris步进法运用了一种称为打包的技术来计算一个点的集合Q的凸包ch(q)。形像地看，该算法模拟在集合Q外紧紧地包了一层纸。开始时，把纸的末端粘在集合中最低的点上，即粘在与Graham扫描法开始相交的点P_0上。该点为凸包的一个顶点。然后把纸拉向右边使其绷紧，再将纸拉高一些直至碰到一个点。该点也必定是凸包的一个顶点。我们使纸保持绷紧状态，用这种方法继续围绕顶点集合直至回到原始点P_0。

显而易见，除了P_0外Q集合中最高的顶点P_k亦是ch(q)中的一个顶点。若有数个这样的点，我们则选取距离凸包上顺时针方向的相邻点最远的一个顶点作P_k。

像Graham扫描法一样，Jarris步进法也是构造ch(q)的一个逆时针方向排列的顶点序列$<_kP_0, P_1, .., P_k, .., P_{n-1}>$。其中$<P_0, P_1, .., P_k>$为逆时针排列的右链，而$<P_{n-1}, ...P>$显然为顺时针排列的左链。

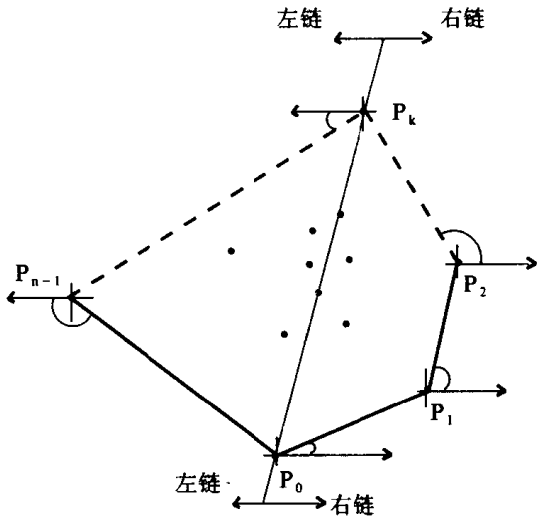


图4.3-4

我们分别构造 ch(q)的左链和右链，用堆栈S存储当前链的顶点。左链构造后，S中的顺序为左链各点在边界上出现的逆时针方向排列的顺序；右链构造后，S的顺序为右链各点在边界上出现的顺时间方向排列的顺序。

我们从 P_0 开始，下一个凸包顶点 P_1 具有相对于 P_0 的最小极角（如有数个这样的点，则选取距离 P_0 最远的点作为 P_1）。类似地，P_2 具有相对于 P_1 的最小极角，依次类推。当我们到达最高顶点 P_k 时，我们已经构造好 ch(q)的右链。为了构造其左链，我们再从 P_0 出发，选取相对于 P_0 具有最小极角的点作为 P_{n-1}，但这时的 X 轴是原 X 轴的反方向。如此继续下去，根据负 X 轴的极角逐渐形成左链，直至回到最高点 P_k。

由此可见，从S的栈顶元素 P_{top} 出发，延伸当前链的下一个凸包顶点 P_m 时，只要选取具有相对于 P_{top} 最小的一个顶点。如果有数个这样的点，则应选择距离 P_{top} 最远的一个点作 P_m，P_m 入栈。然后从 P_m 出发，依上述方法继续延伸当前链 ....，直至到达最高点 P_k 为止。所有入栈的点都是 ch(q)的顶点，因此毋需出栈操作。问题是如何找出相对于当前 P_{top} 具有最小极角的顶点 P_m 呢？ 为了回避极角的计算，我们采取下述方法：

$P_m \leftarrow P_1$ {注 P_1 为最高点}

for i:=2 to n−1 do

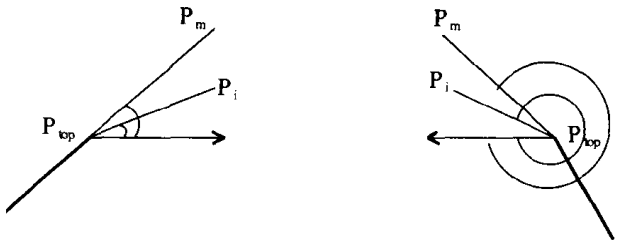
if (构造右链时 $\overline{P_iP_{top}}$ 在 $\overline{P_mP_{top}}$ 的逆时针方向) or (构造左链时 $\overline{P_iP_{top}}$ 在 $\overline{P_mP_{top}}$ 的顺时针方向) or ($\overline{P_iP_{top}}$ 与 $\overline{P_mP_{top}}$ 共线且 $|\overline{P_mP_{top}}| < |\overline{P_iP_{top}}|$)

then $P_m \leftarrow P_i$;

判断线段 $\overline{P_iP_{top}}$ 在线段 $\overline{P_mP_{top}}$ 的哪个时针方向，可通过计算叉积

$$(P_i - P_{top}) * (P_m - P_{top})$$

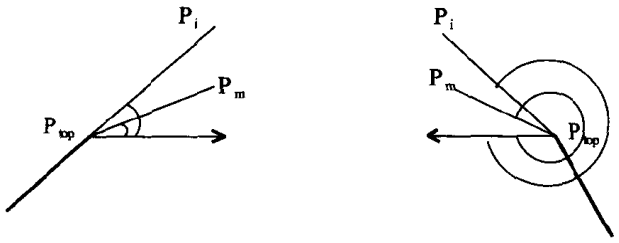
若叉积 >0，$\overline{P_iP_{top}}$ 在 $\overline{P_mP_{top}}$ 的顺时针方向。在构造右链时，说明 P_i 相对 P_{top} 的极角比 P_m 相对 P_{top} 的极角小（图 4.3 − 5(a)）；在构造左链时，说明 P_i 的极角比 P_m 大（图 4.3 − 5(b)）。



构造左链时 (a)　　构造右链时 (b)

图 4.3-5

若叉积 <0，$\overline{P_iP_{top}}$ 在 $\overline{P_mP_{top}}$ 的逆时针方向。在构造右链时，说明 P_i 相对 P_{top} 的极角比 P_m 相对 P_{top} 的极角大（图 4.3 − 6(a)）；在构造左链时，说明 P_i 的极角比 P_m 小（图 4.3 − 6(b)）。



构造左链时 (a)　　构造右链时 (b)

图 4.3 - 6

若叉积 $=0$, $\overline{P_iP_{top}}$ 与 $\overline{P_mP_{top}}$ 共线。在 P_i 和 P_m 中，哪个点距离 P_{top} 愈远，则成为凸包顶点的希望愈大。

显然，经过 n－2 次比较后得出的 P_m 即为相对当前 P_{top} 具有最小极角的顶点，即当前链中连接 p_{top} 的下一个凸包顶点。

下面是程序题解:

```
Program Jarvis_Scan;

Const
  Maxn                  =   500;{最多点数}

Type
  PosType               =   Record{二维点的数据类型}
                              x, y        :  Real
                            End;

Var
  N                     :   Integer;{实际点数}
  List                  :   Array [0..Maxn] of PosType;{点集}
  Stk                   :   Array [1..Maxn] of Integer;{堆栈}
  F                     :   Text;{文件变量}

Procedure Swap(Var a, b : PosType);{a 点与 b 点交换}
Var
  t                 :  PosType;
Begin
  t := a; a := b; b := t
```

```
End;{Swap}

Procedure Init;
Var
  i                : Integer;
Begin
  Assign(F,' INPUT.DAT');{文件变量与文件名连接}
  Reset(F);{文件读准备}
  Readln(F, N);{读入点数}
  For i := 0 to N - 1 Do Begin
  {依次读入每个点，求出Y坐标值最小（或具有Y最小值的数个点中最左边的点）的点
P0和最高点P1}
    Readln(F, List[i].x, List[i].y);
    If (List[i].y < List[0].y) Or (List[i].y = List[0].y)
       And (List[i].x < List[0].y) Then Swap(List[0], List[i]);
    If (i > 0) And (List[i].y > List[1].y) Then Swap(List[1], List[i])
  End;{for}
  Close(F)
End;{init}

Function Multi(Var p1, p2, p0 :PosType):Real;
{返回 (P1 - P0)*(P2 - P0)的叉积，若该叉积为正，P0P1⃗ 在 P0P2⃗ 的顺时针方向，若
叉积为0，则P0P1⃗ 与P0P2⃗ 共线；若叉积为负，P0P1⃗ 在P0P2⃗ 的逆时针方向。}
Begin
  Multi := (p1.x - p0.x)*(p2.y - p0.y) - (p2.x - p0.x)*(p1.y - p0.y)
End;{Multi}

Function Dis(Var p1, p2 : PosType):Real;
{计算线段P1P2‾的距离的平方}
Begin
  Dis := Sqr(p1.x - p2.x) + Sqr(p1.y - p2.y)
End;{Dis}

Procedure Left_And_Right(k : Shortint; Var p : Integer);
{输入左，右链标志．若k=-1,构造左链。stk[1..p]依次为逆时针排列的左链上的凸包顶
点；k=1,构造右链，stack[2..p]依次为顺时针排列的右链上的凸包顶点。返回栈顶指针p}
Var
```

```
  i, m            :  Integer;
  mul             :  Real;
Begin
  Stk[1] := 0;{p0 点入栈}
  p := 1; { right chain }
  Repeat{循环，依次选取当前链的每一个凸顶点}
    m := 1;{选取具有相对于 P 栈顶的最小极角的一个点作为下一个凸包顶点 Pm，若有数个这样的点，则选取距离 P 栈顶最远的点作 Pm}
   For i := 2 to N - 1 Do Begin
       mul := Multi(List[i], List[m], List[Stk[p]]);
       If (k * mul < 0) Or (mul = 0) And
          (Dis(List[m], List[Stk[p]]) < Dis(List[i], List[Stk[p]]))
          Then m := i
    End;{for}
    Inc(p);{Pm 入栈}
    Stk[p] := m
  Until List[1]. y = List[m].y{直至当前凸包顶点 Pm 为最高点为 K}
End;{Left_And_Right}


Procedure Jarvis;
Var
  i, p, hi        : Integer;
Begin
  Left_And_Right(-1, p);{构造和输出左链上逆时针排列的凸包顶点}
  For i := 1 to p Do
    Writeln('(', List[Stk[i]].x:7:2, ',',  List[Stk[i]].y:8:2,')');
  hi := Stk[p];{暂存最高顶点}
  Left_And_Right(1, p);{构造左链上顺时针排列的凸包顶点}
  If Stk[p] = hi Then Dec(p);
  For i := p Downto 2 Do{按逆时针顺序输出右链上的凸包顶点}
    Writeln('(', List[Stk[i]].x:7:2, ',',  List[Stk[i]].y:8:2,')');
End;{Jarvis}

Begin
  Init;{输入点集 Q; 求凸包上的最低点 P0 和最高点 P1}
  Jarvis{通过步进法计算和输入凸包顶点}
End.{main}
```

§4.4 寻找最近点对

我们通常将平面上两点 $P_i=(X_i,Y_i)$ 和 $P_j=(X_j,Y_j)$ 之间的距离称为欧式距离

$$d_{ij}=\sqrt{(X_i-X_j)^2+(Y_i-Y_j)^2}$$

如果 P_i 和 P_j 重合，在这种情况下它们之间的距离 d_{ij} 为 0。

现在我们考虑在 $n>2$ 个点的集合 Q 中寻找最近点对的问题。这个问题有很大的实用价值，例如一个控制空中或海上交通系统可能需要了解两个最近的交通工具，以检测可能产生的相撞事故。

显然，n 个点的集合 Q 中共有 $c(n,2)$ 个点对。最原始的算法是通过计算所有这些点对的距离来找出最近点对。这种算法的效率直接依赖于 n 值，n 愈大，运行时间愈长。为了提高时效，我们将描述一种应用于该问题的分治算法。

算法每次递归调用的输入为点的子集 $P=(P_r..P_R)\subseteq Q$ 和数组 Y。P 中的所有点按其 Y 坐标单调递增的顺序排列，其编号序列存入数组 Y。同样，我们也得将所有点按 X 坐标单调递增的顺序排列，其编号序列存入数组 X。显然这种排序性质在每次递归调用中是保持不变的，因此我们只要在调用分治程序前进行一次预排序，分别求出数组 X 和数组 Y。

输入为 P、Y 的递归调用首先检查 $|p|<3$。如果是这样，则计算所有 $c(|p|,2)$ 个点对的距离并返回最近的点对及其距离。如果 $|p|>3$，则递归调用采取如下分治策略。

划分:

找出一条垂直平分线，将点集 P 划分为点数分别为 $P/2$（P 为偶数）或者 $[P/2]$ 和 $[P/2]+1$（P 为奇数）的两个点集 P_L 和 P_R。其中 P_L 上的所有点在垂直平分线上或其左侧；P_R 上的所有点在垂直平分线上或其右侧。数组 Y 被划分为两个数组 Y_L 和 Y_R。类似地，数组 X 也被划分为 X_L 和 X_R。划分后的 X_L 和 Y_L 包含了 P_L 中的点，并分别按 X 坐标和 Y 坐标单调递增的次序进行排序；X_R 和 Y_R 包含了 P_R 中的点，也按 X 坐标和 Y 坐单调递增的次序进行排序。

解决:

把 P 划分为 P_L 和 P_R 后再进行两次递归调用:

第一次: 找出 P_L 中的最近点对。调用的输入为子集 P_L 和数组 Y_L，返回最近点对的距离 $\&_L$;

第二次: 找出 P_R 中的最近点对。调用的输入为子集 P_R 和数组 Y_R，返回最近点对的距离 $\&_R$;

置 $\&=\min(\&_L,\&_R)$;

合并:

问题是可能还存在其距离小于 & 的点对。如果存在这样的一个点对的话，则点对的一个点在 P_L 中，另一个点在 P_R 中，两个点必定相距垂直平分线 & 单位之内，即它们必定处于以垂直平分线为中心、宽度为 2& 的垂直带形区域内。更精确一点讲，由于点对之间

的垂直距离也少于 &单位，因此，它们是在以垂直平分线为中心的某个 &*2& 的矩形区域内。

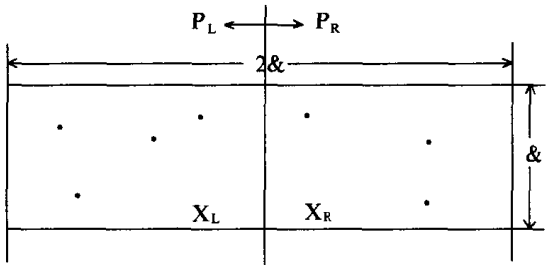


图 4.4-1

为了找出这样的点对（如果存在），算法要做如下工作：

1. 建立一个数组 Y'，Y' 仅含 Y 数组中所有在宽度为 2& 的垂直带形区域内的点，这些点按其 Y 坐标单调递增的顺序存储于 Y'；

2. 对数组 Y' 中的每个点 P，求出 P 到紧随 P 后的 7 个点间的距离，并纪录下 Y' 的所有点对中找出的最近点对的距离 &'。为什么对点 P，在 Y' 中仅需考虑紧随 P 后的 7 个点呢？

设最近点对为 $X_L \in P_L, X_R \in P_R$。由于 X_L 和 X_R 只能在某个 &*2& 的矩形区域内出现，且 P_L 中所有点对的距离和 P_R 中的所有点对的距离都至少为 &单位，因此 P 中至多有 8 个点可能位于该 &*2& 矩形内（垂直平分线上的点即可属于 P_L，亦可属于 P_R），而 Y' 中 X_L 位于 X_R 之前，那么即使 X_L 在 Y' 中尽可能早出现而 X_R 尽可能晚出现，X_R 也必定是跟随 X_L 的 7 个位置中的一个。

3. 如果 &' < &，则垂直带形区域内的确包含比我们根据递归调用所找出的最近距离 & 更近的点对，于是返回该点对及其距离 &'；否则就返回递归调用中发现的最近点对及其距离 &。

下面是程序题解：

```
  {$M 16520,0,655360}
Program Find_Minimum_Distance_Among_Points;

Const
  Maxn              =  1000;{点的最多个数}

Type
  PosType           =  Record{点的类型}
                       x, y        :  Real
                       End;
  OrdType           =  Array [1..Maxn] of Integer;{点的编号序列类型}

Var
```

```
  N, u, v                  :  Integer;{实际点数和最近点对}
  F                        :  Text;{文件变量}
  List                     :  Array [1..Maxn] of PosType;{点集合}
  X, Y, Tmp                :  OrdType;
{x,y ——按X坐标和Y坐标值递增顺序排列的编号序列;
 Tmp ——左右侧标志: Tmp[i]=┌1 点i在左侧;
                           └2 点i在右侧;}
  d                        :  Real;{最近点对的距离}

Function Dis(Var p1, p2 : PosType): Real;{点P1和点P2的距离的平方}
Begin
  Dis := Sqrt(Sqr(p1.x - p2.x) + Sqr(p1.y - p2.y))
End;
Function Comp (var p1,p2 : PosType; m : Integer): Shortint;
{输入P1和P2两点，以及轴向标志m。若m=0，比较两点在X轴上的值：若m=1，比较
两点在Y 轴上的值。在m对应的数轴上若P1 < P2，函数返回-1；若P1 = P2，函数返
回 0；若P1 > P2，函数返回1。}
Begin
  Case m of
    0 : If p1.x < p2.x
          Then Comp := -1
          Else If p1.x = p2.x Then Comp := 0
                              Else Comp := 1;
    1 : If p1.y < p2.y
          Then Comp := -1
          Else If p1.y = p2.y Then Comp := 0
                              Else Comp := 1
  End{case}
End;{comp}

Procedure Swap(Var a, b : Integer);{整数a和b交换}
Var
  t                :  Integer;
Begin
  t := a; a := b; b := t
End;{Swap}
```

```
Procedure Sort ( Var A : OrdType ; p, r, m : Integer );
{输入点的编号序列 A, 子区间的两个端点 P 和 R 以及轴向标志 m。按 m 对应的轴向上的
坐标值递增的顺序对 A[p..r]进行合并排序。输出排序后的 A}
Var
  i, j          : Integer ;
  x             : PosType ;
Begin
  If r - p + 1 <= 5 Then
{若 A[p..r] 的元素个数< 5, 则对 A[p..r]按 m 对应的数轴上的坐标值递增的顺序
进行排序}
    For i := p + 1 to r Do Begin
      j := i;
      While (j > 1) And (Comp(List[A[j]], List[A[j-1]], m) = -1) Do Begin
        Swap(A[j], A[j-1]);
        Dec(j)
      End {while}
    End {for}
  Else Begin
    x := List[A[p + Random(r - p + 1)]];
    {在 A[p..r]中随机取出一个元素，该元素对应的点为 X}
    i := p; j := r;
    Repeat
{将 A[p..r]分成两个子序列 A[p..J]和 A[J+1..R], 使得在 m 轴向上，序号为 A[J+
1..R] 的每个点的坐标值大于等于 X 的坐标值，而序号为 A[P..J]的每个点的坐标值小
于等于 X 的坐标值}
      While Comp(List[A[i]], x, m) = -1 Do Inc(i);
      While Comp(List[A[j]], x, m) = 1 Do Dec(j);
      If i < j Then Begin
        If Comp(List[A[i]], List[A[j]], m) = 0 Then Begin
          Dec(i); Dec(j)
        End {then}
        Else Swap(A[i], A[j])
      End {then}
    Until i >= j;
    Sort(A, p, j, m);{对 A[P..J]继续递归排序}
    Sort(A, j + 1, r, m){对 A[J+1..R]继续递归排序}
  End {else}
End ;{sort}
```

```
Function Find_Min( Var Y : OrdType; xa, xb : Integer;
                   Var u, v : Integer ): Real;
Var
  del, t        : Real;
  Yl, Yr        : ^OrdType;
  u1, v1, i, l,
  j, q          : Integer;
Begin
  If xb - xa+1 < 3 Then Begin
  {若 Y[Xb..Xa]的元素个数小于等于 3, 则求出该子区间所有点对中的最近点对(u, v),
返回其距离}
    del := 1e30;
    For i := xa to xb - 1 Do
      For j := i + 1 to xb Do Begin
        t := Dis(List[X[i]], List[X[j]]);
        If t < del Then Begin
          del := t;
          u := X[i]; v := X[j]
        End {then}
      End;{for}
    Find_Min := del;
    Exit
  End;{then}
  FillChar(Tmp, Sizeof(Tmp), 0);
  q := (xa + xb - 1) div 2;{求 xa..xb 的中间点 q}
  For i := xa to q Do Tmp[X[i]] := 1;{设 Xxa..xq 左侧标志}
  For i := q + 1 to xb Do Tmp[X[i]] := 2;{设 Xq+1..Xxb 右侧标志}
  New(Yl); New(Yr);
  u1 := 0; v1 := 0;
  For i := xa to xb Do
{求左侧 Xxa..Xq 对应的 Y 坐标序列 Yl 和右侧 Xq+1..Xxb 对应的 Y 坐标序列 Yr}
    If Tmp[Y[i]] = 1 Then Begin
      Inc(u1);
      Yl^[xa + u1 - 1] := Y[i]
    End {then}
    Else If Tmp[Y[i]] = 2 Then Begin
          Inc(v1);
          Yr^[q + v1] := Y[i]
```

```
        End ;{ then }
  del := Find_Min ( Yl^, xa, q, u, v );
{在分别求出左侧和右侧最小点对的距离后，求出两个点对距离的最小值 del 及其点对 ( U,
V )}
  t := Find_Min ( Yr^, q + 1, xb, u1, v1 );
  If t < del Then Begin
    del := t; u := u1; v := v1
  End ;{ then }
  {把数组 Y 中所有不在宽度为 2 * del 垂直带形区域内的点去掉后，求出新数组 Y1,Y1 的
元素个数为 L }
  t := ( List[X[q]].x + List[X[q+1]].x ) / 2;
  l := 0;
  For i := xa to xb Do
    If Abs ( List[Y[i]].x - t ) < del Then Begin
      Inc (l);
      Yl^[l] := Y[i]
    End ;{ then }
  For i := 1 to l - 1 Do Begin
  {对数组 Y1 中的每个点 P, 分别计算出 P 到其后 7 个点的距离，从中找出最近点对 del }
    j := i + 1;
    While (j - i <= 7) And (j <= l) Do Begin
      t := Dis ( List[Yl^[i]], List[Yl^[j]]);
      If t < del Then Begin
        del := t;
        u := Yl^[i]; v := Yl^[j]
      End ;{ then }
      Inc (j)
    End { while }
  End ;{ for }
  Find_Min := del ;{返回最近点对 del }
  Dispose ( Yl );{释放 Y1 和 Yr 数组}
  Dispose ( Yr )
End ;{ find_min }

Procedure Init ;
Var
  i              :  Integer ;
Begin
```

```
  Assign(F, ' INPUT.DAT' ); {输入文件名串与文件变量连接}
  Reset(F );{ 文件读准备}
  Readln(F, N);{ 读入点数}
  For i := 1 to N Do Begin
  {读入每个点的坐标，X 和 Y 序列初始化}
    Readln ( F, List [ i ]. x, List [ i ]. y );
    X[i] := i; Y[i] := i
  End ;{ for }
  Sort ( X, 1, N, 0 );{ 按 X 坐标单调递增的顺序对 n 个点进行合并排序，返回编号序列 X }
  Sort ( Y, 1, N, 1 ){ 按 Y 坐标单调递增的顺序对 n 个点进行合并排序，返回编号序列 Y }
End ;{ init }

Begin
  Init ;{ 输入点集，并分别按 X 坐标和 Y 坐标单调递增的顺序 n 个点进行合并排序}
  d := Find _ Min ( Y, 1, N, u, v );
{输入按 Y 坐标单调递增顺序排列的编号序列 Y [ 1.. n ], 返回最近点对 ( U, V ) 的距离}
  Writeln ( u, ' ', v, ' ', d : 8 : 2 ){ 输出最近点对及其距离}
End .{ main }
```

第五章　显式图的基本算法

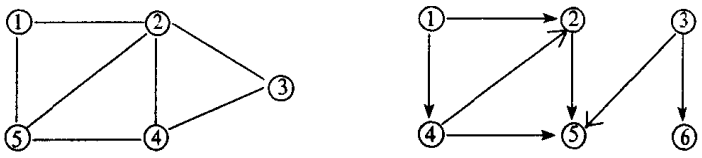
对一个已列表给出各顶点及边(包括标有权的边)的图进行搜索，实际上就是以一种系统的方式沿着图的边访问所有顶点。一个图形的搜索算法可以使我们发现图的许多结构信息。许多有关图的算法开始都通过搜索输入图来获取结构信息；另外还有一些图形算法也是由基本的图形搜索算法经过简单扩充而成的。因此图形搜索技术是图形算法领域的核心。

§5.1　显式图的表示

要表示一个图 G=(V, E) 有两种标准的方法。

一、邻接表

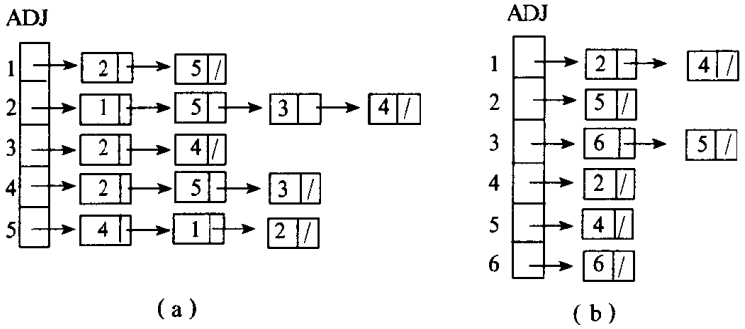
给定图 G=(V, E) 的邻接表表示由数组 ADJ[V_1..V_n]构成（G 的顶点个数为 n)。数组元素为顶点，每一个顶点 V_i($V_i \in V$)对应一个邻接表 ADJ[V_i]，包含（指向）所有满足条件（V_i, V_j）$\in E$ 的顶点 V_j($1 \leqslant j \leqslant n, i \neq j$)，即 ADJ[$V_i$]包含图 G 中所有和顶点 V_i 相邻接的顶点。每个邻接表中的顶点按任意次序存储。例如图 5.1 −1)分别列出了一个有向图和一个无向图:



(a)　　(b)

图 5.1−1

下面图 5.1 - 2(a)就是图5.1 - 1(a)中的无向图的邻接表表示。类似的图 5.1-2(b)就是图5.1 - 1(b) 中有向图的邻接表表示。



(a)　　(b)

图 5.1 - 2

由图 5.1 - 2(a)中可见，若 G 是无向图，那么所有邻接表的长度之和应为 2 |E|，因为

若(U, V)是一条无向边，那么顶点U一定出现在V的邻接表ADJ[V]中且反之亦然。同样，由图5.1-2(b)中可见，若G是有向图，那么所有邻接表的长度之和应为|E|。因为对于任何边(U, V)，顶点V只能出现在顶点U的邻接表ADJ[U]中一次。显然，邻接表的长度和取决于边数|E|，因而用它表示稀疏图($|E| \leq |V|^2/2$)比较简洁紧凑。

另外，对邻接表稍作变动就可用来表示加权图G，即对每一条边(U, V)∈E，权值W(U, V)和顶点V一起存储在U的邻接表ADJ[U]中。

当然邻接表也有潜在的不足之处。如要确定边(U, V)是否存在，只能在顶点U的邻接表ADJ[U]中搜索V而没有更快的方法。这一不足可以通过下述图的表示方法来弥补，但要以占用更多的存储空间为代价。

二、邻接矩阵

对一个具有n个顶点的图G=(V, E)，其邻接矩阵为一个nxn的矩阵$A=(a_{ij})$，且满足

$$a_{ij}=\begin{cases}1 & \text{若 } (i,j)\in E;\\ 0 & \text{其它。}\end{cases}$$

若G为加权图

$$a_{ij}=\begin{cases}w_{ij} & \text{若 } (i,j)\in E;\\ 0\text{或}\infty & \text{其它。}\end{cases}$$

显然，一个图的邻接矩阵的存储空间取决于顶点数n而和边数|E|无关，因此适用于存储稠密图($|E| > |V|^2/2$)。

图5.1-3(a)是图5.1-1(a)中的无向图的邻接矩阵，图5.1-3(b)是图5.1-1(b)中的有向图的邻接矩阵。

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

(a)

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

(b)

图5.1-3

由图5.1-3可见，有向图G的邻接矩阵不一定对称，因为a_{ij}不能等价于a_{ji}，即顶

点i与顶点j之间不一定存在环。而无向图G的邻接矩阵是对称的。在一些应用中，通常只存储其邻接矩阵对角线以上的部分，这样一来图所占用的存储空间几乎可以减少一半。另外，如果不是加权图，则邻接矩阵可以用N个N位二进制数表示，而不必用一个字的空间来存储每一个矩阵元素。

§5.2 宽度优先搜索

已知图G=(V, E)和一个起始顶点S，宽度优先搜索以一种系统的方式探寻G的边从而“发现”S所能到达的所有顶点，并计算S到所有这些顶点的最短矩离（最少边数）。该算法同时能生成一棵根为S且包括所有可达顶点的宽度优先树。对从S可达的任意顶点V，宽度优先树中从S到V的路径对应于图G中从S到V的最短路径，即包含最少边数的路径。该算法对有向图和无向图同样适用。

之所以称为宽度优先算法是因为算法自始至终一直通过已找到和未找到的顶点之间的边界向外扩展。就是说，算法首先搜索和S距离为K的所有顶点，然后再去搜索和S距离为K+1的其它顶点。 例如图5.1-(a)，从顶点1出发，通过宽度优先搜索可得到如下序列

$$1 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 4$$

按宽度优先搜索法访问图G后的遍历路径，形成一棵宽度优先生成树。

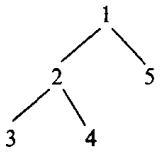


图5.2-1

为了保持搜索的轨迹，宽度优先搜索为每个顶点着色:

白色顶点——未被访问的顶点。初始时，所有顶点都为白色;

灰色顶点——第一次被访问的顶点。灰色顶点可以与一些白色顶点相邻接，代表着已访问顶点和未访问顶点的边界。算法中使用了一个先进先出的队列Q来存放当前灰色顶点的集合;

黑色顶点——已被访问过且与之相邻的顶点都被访问过的顶点。显然，若(U, V)∈E且顶点U为黑色，那么顶点V要么是灰色，要么是黑色;

下面的宽度优先搜索bfs假定输入图采用邻接表表示。对于图中的点U∈V，其色彩存于变量color[U]中；顶点U的父母存于P[U]。如果U没有父母（例如U=S或U还未访问)，则P[U]=NIL；由算法算出的初始顶点S和顶点U之间的距离存于变量d[U]中，U未被访问时，d[U]=∞。在搜索过程中，每从Q队列中移出一个灰色顶点后便访问它的所有儿子，计算它们的d值。 这些首次被访问的儿子由白色变为灰色，相继进入了Q队列。当所有儿子扩展完后，该灰色顶点被涂上了黑色。然后，再依照上述方法移出和扩展Q队列中的下一个顶点。显然，当Q队列变空，即所有顶点变黑时，宽度优先搜索结束。

从顶点S出发对图G进行宽度优先搜索的同时，建立了一棵以S为根的宽度优先树，这棵树由S可达的每个顶点的P指针组成。我们可从树中的任一顶点V出发，沿P指针递归打印从S到V最短（边数最少）路径:

```
procedure print_path(G, S, V);
```

```
begin
  if v=s
     then 打印顶点 s
     else if p[U]=NIL
          then 打印 s 到 v 无通路的信息
          else beign print_path(G, S, p[U]); 打印顶点 v; end;
end; {print_path}
```

由于每次递归调用的路径都比前一次调用少一个顶点，因此递归次数与打印路径上的顶点数相等。

```
Program B_First_Search;

Const
  Maxn                  =  100; {顶点个数的最大值}

Type
  Link                  = ^Node; {链指针类型}
  Node                  =  Record {链结点类型}
                           k          :  1..Maxn; {顶点序号}
                           next       :  Link {后继指针}
                         End;
  LType                 =  Record {顶点类型}
                           c          :  0..2; {颜色:0- 白色; 1-灰色; 2-黑色}
                           d, p       :  0.. Maxn;
                           {d- 初始顶点至该顶点的距离; p- 父顶点的序号}
                           s          :  Link
                           {所有与该顶点相邻接的顶点组成的邻接表（链表）的指
                           针}
                         End;

Var
  N, op, cl,
  i, u, start           :  Integer;
{N—顶点的实际个数; op, cl—队尾指针和队首指针; i—辅助变量; u—顶点序号;
start—初始顶点}
  F                     :  Text; {文件变量}
  List                  :  Array [1..Maxn] of LType; {图的邻接表}
  Q                     :  Array [1..Maxn] of 1..Maxn; {队列}
  p                     :  Link; {链指针}
```

```
Procedure Init;
Var
  u, v          : Integer;
  x             : Link;
Begin
  Assign (F, ' INPUT. DAT ');{文件名串与文件变量连接}
  Reset (F);{文件读准备}
  Readln (F, N, start);{读入顶点个数和初始顶点}
  For u := 1 to N Do Begin {所顶点的邻接表初始化}
    List[u]. c := 0;
    List[u]. d := Maxn;
    List[u]. s := nil;
    List[u]. p := 0
  End;{for}
  While Not Eof (F) Do Begin {若文件未读完，则循环}
    Readln (F, u, v);{读入边 (U, V)}
    New (x);{V 顶点插入 U 顶点对应的邻接表首}
    x^. k := v; x^. next := List[u]. s;
    List[u]. s := x
  End;{while}
  Close (F)
End;{init}

Procedure Print (v : Integer);{递归输出顶点 start 至顶点 V 的路径}
Begin
  If v = start Then Write (start)
     Else If List[v]. p = 0 Then Write (' No Way')
               Else Begin
                 Print(List[v]. p);
                 Write ('-->', v)
               End {else}
End;{print}

Begin
  Init;{输入和建立图的邻接表}
  Q[1] := start;{顶点 start 进入队列}
  List[start]. d := 0;{顶点 start 的距离为 0}
  op := 1; cl := 0;{队列的队尾和队首指针初始化}
```

```
  While cl <> op Do begin {若队列非空}
    u := Q[cl + 1]; {移出队首顶点 u 并求出 u 的邻接表指针}
    p := List[u]. s;
    While p <> nil Do Begin {重复搜索邻接表的每一个顶点}
      If List [p^. k]. c = 0 Then Begin
{若相邻的某顶点的颜色为白，则该顶点涂灰色，与初始点的距离+1，父顶点设为 u}
        List[p^. k]. c := 1;
        List[p^. k]. d := List[u]. d + 1;
        List[p^. k]. p := u;
        Inc (op); {相邻顶点进入队列}
        Q[op] := p^. k
      End; {then}
      p := p^. next {搜索 u 的邻接表的下一个顶点}
    End; {while}
    list[u]. c:=2; {u 顶点涂色}
    Inc (cl) {队首指针后移，准备移出下一个队首顶点}
  End; {while}
  For i := 1 to N Do Begin {依次打印顶点 start 至每个顶点的路径}
    Write (i: 5, ': ');
    Print (i);
    Writeln
  End {for}
End. {main}
```

宽度优先搜索算法是最简便的图形搜索算法之一，这一算法也是很多重要的图形算法的原型。例如，计算初始顶点到其余顶点间最短路径的 dijkstra 算法和计算最小生成树的 prim 算法都采用了和宽度优先搜索类似的思想。

【例 1】dijkstra 算法

给定一个带正权的有向图 D(V, E)（即每条边 (U, V) ∈ E, W(U, V) ≥ 0）和起始顶点 V_0。求从 V_0 到 D 中其余各顶点间的最短路径。

算法分析:

dijkstra 提供了一种按路径长度不减次序产生最短路径的算法:

令 S 表示已求出最短路径的顶点集合。对于不在 S 中的顶点 W，令 d(W) 表示从 V_0 开始且通过 S 中的顶点到达 W 的最短路径的长度。S 的初态为空集。若 V_0 通过 S 中的顶点到 W 有路径，则 d(W) 为 V_0 到 W 的最短的一条路径长度；若 V_0 没有经 S 中的顶点到 W 的路径，则 $d(W)=\infty$。初始时设 $d(V_0)=0$，除 V_0 外的所有顶点 V_i ($V_i \in V$ $V_i \neq V_0$)，$d(V_i)=\infty$。

在算法实现中设置了队列 Q, 该队列包含了除 S 集合外的所有顶点且各顶点都有相应的 d 值。初始时，优先队列 Q 包含了 D 中所有顶点。然后按照类似宽度优先搜索的方法求解：每次均从队列 Q 中移出一个其 d 值最小的顶点 U 并插入到集合 S 中（第一次从 Q 中移出 V_0 并插入 S），然后对以 U 为起点的每条边（U, V）进行判断（松驰操作）：如果可以经过 U 来改进到顶点 V 的最短路径，就更新 d[V] 以及 V 的父指针。每个顶点从 Q 移出并插入 S 一次，因此在进行了 |V| 次判断后，栈 Q 变空。此时所有 $U \in V$ 的 d[U] 值为 V_0 到 U 的最短路径长度。

例如图 5.2 - 2 例举了 dijkstra 算法的一个执行流程。图中最左边的 S 为初始顶点；每个顶点内为最短路径的估计；粗边说明了前驱的值：如果 (U, V) 为粗边，则 V 的父顶点为 U；黑色顶点属于集合 S; 白色顶点属于优先队列 Q（即除 S 集合外的顶点）。

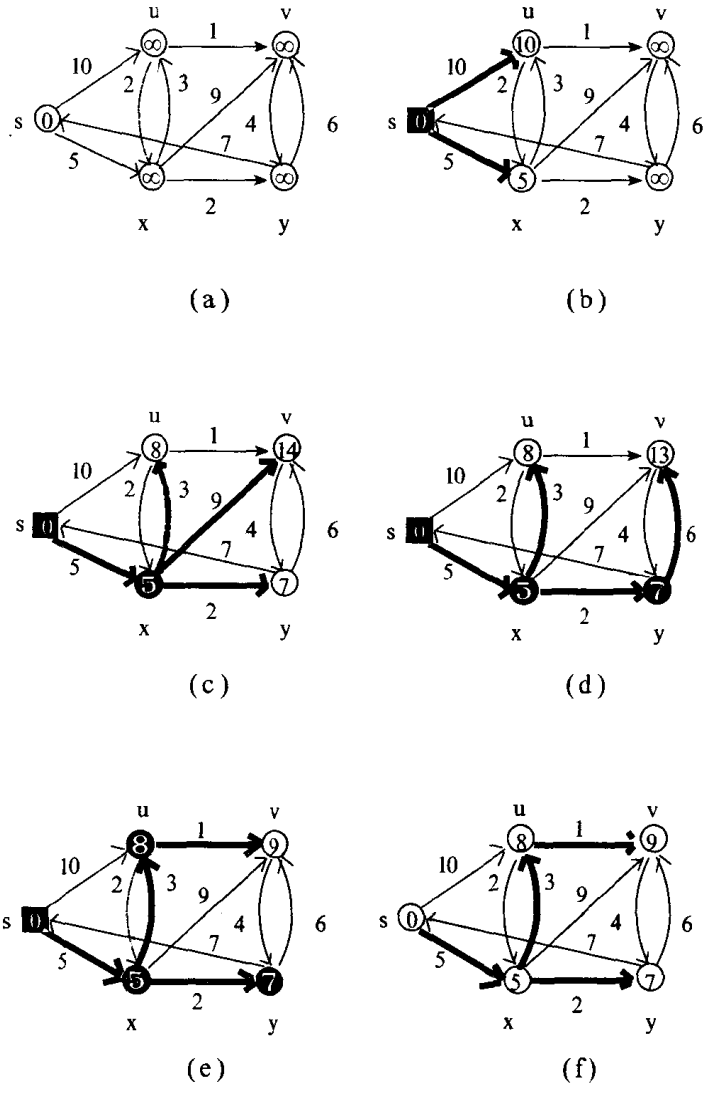


图 5.2 - 2

下面是程序题解。由于每次需从 Q 队列中，移出其 d 值最小的顶点，因此 dijkstra 算法的运行时间很大程度上取决于这一步排序操作。为了改善算法的效率，我们采用二叉堆

来实现队列 Q:

将 Q 看作是一棵按层次排列，同一层按自左到右排列的二叉树。即 Q[i] 的父结点是 Q[idiV 2]，左儿子是 Q[2i]，右儿子是 Q[2i+1]。若 2i>hs 或 2i+1〉 hs(hs——当前 Q 表的长度)，则与之相应的儿子不存在。Q 中各顶点的 d 值满足下式。

Q[i div 2]的 d 值 <Q[i]的 d 值

显然，堆阵 Q 所表示的二叉树的根结点 Q[1]的 d 值是最小的。

保持 Q 的堆性质由 heapfy 过程完成。每次从 Q 队列中移出表首顶点后调用一次 heapfy[1]，使以 Q[1]为根的二叉树成为一个堆，即 Q[1]的 d 值最小。另外，在对连接当前顶点 U 的边 (U, V) 进行松驰操作时，若 d[V]>d[U]+w[u, v]时，则通过调用decreass 过程将 d[v]修正为 d[u]+w[u, v]，并对顶点 V 在 Q 堆中的位置进行调整。

```
Const
  Maxn                  = 100; {顶点数的最大值}

Type
  Link                  = ^Node; {链 (邻接表) 指针}
  Node                  =  Record {链结点}
                             k, w       :  Integer; {顶点序号; 边的权}
                             next       :  Link {后继指针}
                           End;
  StType                =  Record {Q 堆的元素类型}
                             v, d, p    :  Integer
                           {顶点序号，最短距离估计，父顶点序号}
                           End;

Var
  N, root, i            :  Integer; {实际顶点数，源顶点，辅助变量}
  F                     :  Text; {文件变量}
  Gr                    :  Array [1..Maxn] of Link; {邻接表}
  Q                     :  Array [1..Maxn] of StType; {排序堆}
  Q_pos                 :  Array [1..Maxn] of Integer;
                           {Q_POS[i]—— 顶点 i 在 Q 堆中的位置}
  Hs                    :  Integer; {堆长度}

Procedure Insert (u, v, w : Integer);
{将权为 W 的边 (U, V) 插入顶点 U 对应的邻接表}
Var
  x             :  Link;
```

```
Begin
  New(x);
  x^.k := v; x^.w := w;
  x^. next := Gr[u]; Gr[u] := x
End; {Insert}

Procedure Init;
Var
  i, u, v        : Integer;
Begin
  Assign (F, ' INPUT. DAT' ); (输入文件名串与文件变量连接}
  Reset (F); {文件读准备}
  Readln (F, N, root); {读入顶点数和源顶点序号}
  For i := 1 to N Do Gr[i] := nil; {所有顶点的邻接表初始化}
  While Not Eof (F) Do Begin {依次读入每一条边及其权，建立邻接表 GR}
    Readln (F, u, v, i);
    Insert (u, v, i)
  End; {while}
  Close(F) {关闭文件}
End; {init}

Procedure Swap (a, b : Integer); {Q 堆中第 a 个顶点与第 b 个顶点交换}
Var
  i              : Integer;
  t              : StType;
Begin
  t := Q[a]; Q[a] := Q[b]; Q[b] := t;
  i := Q_pos[Q[a]. v]; Q_pos[Q[a]. v] := Q_ pos[Q[b]. v];
  Q_ pos[Q[b]. v] := i
End; {Swap}

Procedure Decrease (v, k : Integer);
{将 Q 堆中第 V 个顶点的 D 值设为 K, 并将该顶点放到 Q 堆的合适位置}
Begin
  Q[v]. d := k;
  While (v <> 1) And (Q[v]. d < Q[v div 2]. d) Do Begin
    Swap(v, v div 2);
    v := v div 2
```

```
  End {while}
End; {decrease}

Procedure Heapfy (i : Integer); {堆排序，使得以 i 为根的子树成为一个堆}
Var
  m                : Integer;
Begin
  m := i;
  Repeat
    i := m; {求顶点 i 与左儿子、右儿子三个顶点间 D 值最小的顶点序号 m}
    If (i * 2 <= Hs) And (Q[i * 2]. d < Q[m]. d) Then m := i * 2;
    If (i * 2 + 1 <= Hs) And (Q[i * 2 + 1]. d < Q[m]. d)
       then m := i * 2+ 1;
    If i <> m Then Swap(i, m) {顶点 i 的 D 值不是最小，则 Q 堆中顶点 i 与顶点 m 交换}
  Until i = m {直至顶点 i 的 D 值最小、以 i 为根的子树已是堆为止}
End; {heapfy}

Procedure Relax(u, v, w : Integer); {输入权为 W 的边 (U, V)，该边进行松驰操作}
Begin
  If Q[Q_Pos[v]]. d > Q[Q_Pos[u]]. d + w Then Begin
  {可通过 U 对迄今找出的到 V 的最短路径进行改进}
    Q[Q_Pos[v]]. p := u; {则 V 的父顶点设为 U}
    Decrease (Q_Pos[v], Q[Q_Pos[u]]. d+w) {修正 V 的 D 值并重新对 Q 进行堆排序}
  End {then}
End; {relax}

Procedure Dijkstra;
Var
  i, u             : Integer;
  p                : Link;
Begin
  For i := 1 to N Do Begin
{N 个顶点进入 Q 队列，每个顶点的最短路径估计为 ∞}
    Q[i]. v := i; Q[i]. d := Maxint;
    Q_Pos[i] := i
  End; {for}
  Q[root]. d := 0; Swap (1, root); {源点的 D 值设为 0 并移至 Q 表首}
  Hs := N; {Q 表长度初始化为 N}
```

```
  While Hs <> 0 Do Begin {若Q表非空，则循环}
    u := Q[1]. v; {移出Q的表首顶点}
    Swap(1, Hs);
    Dec(Hs); {Q的长度减1}
    Heapfy(1); {对队列Q按D值递增的顺序进行堆排序}
    p := Gr[u]; {求表首顶点U的邻接表指针}
    While p <> nil Do Begin {对Q队列中所有与U相连的边进行松驰操作}
      If Q_Pos[p^.k] <= Hs Then Relax(u, p^.k, p^.w);
      p := p^. next
    End {while}
  End {while}
End; {dijkstra}

Procedure Print(s : Integer);
{递归打印源顶点root至顶点S的最短路径}
Begin
  If s = root Then Write(root)
    Else Begin
      Print (Q[Q_Pos[s]]. p);
      Write (' ⟶ ', s)
    End {else}
End; {print}

Begin
  Init; {输入和建立邻接矩阵}
  Dijkstra; {计算单源最短路径}
  For i := 1 to N Do {依次输出源顶点至每个顶点的最短路径}
    If Q[Q_Pos[i]]. d = Maxint {若源顶点与顶点I之间不存在通路}
      Then Write(root, ' ⟶ ', i, 'NOWAY')
      Else Begin {否则输出源顶点与i顶点间的最短路径及其权}
        Print(i);
        Writeln ('d[', i, ']=', Q[Q_Pos[i]]. d)
      End {else}
End. {main}
```

【例2】prim算法

在一个带权无向连通图G=(V, E)中找出一个无回路子集T⊆E, 使其连接所有顶点且其权值之和$W(T)=\sum_{(u,v)\in T} W(U, V)$为最小。因为T无回路且连接了所有顶点，所以它必然是

一棵树；又因为所有边的权值之和最少，因此我们称 T 为最小生成树。

算法分析：

prim 算法的执行非常类似于寻找图的最短通路的 dijkstra 算法。设集合 A 是图 G 上最小生成树的边。树从任意根顶点 r 开始逐渐生成，直至该树跨越了 G 中所有顶点。在每一步连接 A 中某顶点的所有边中具有最小权值的边被加入到树中。当算法终止时，A 中的边就成为一棵最小生成树。

有效实现算法的关键是设法容易地选择一条新的边添加到由 A 的边集所形成的 A 中。为了做到这一点，我们设置了一个优先级基于 d 域的队列 Q，当前不在树中的所有顶点都驻留于 Q 中。对每个顶点 V，d[V] 是连接 V 到树中顶点的边所具有最小权值。当找到这样的边时，该边进入 A 集；若不存在这样的边，则 $d[V]=\infty$。初始时，$d[r]=0$，其它顶点的 d 值为 ∞，即 A 集为空。

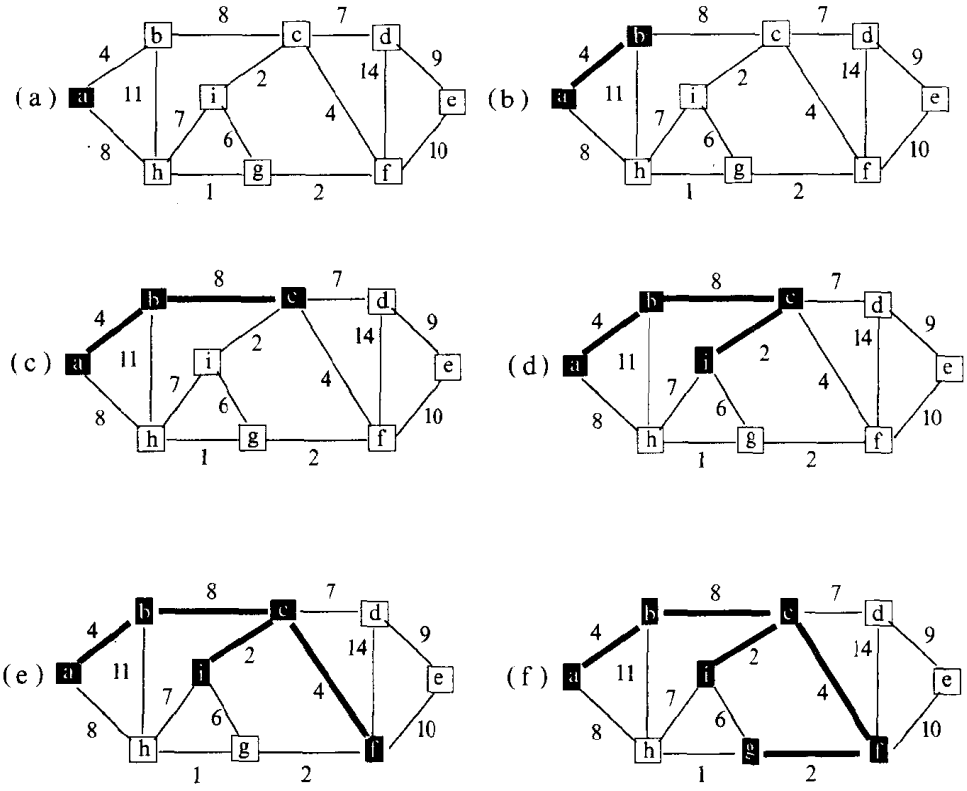
显然，A 集隐含地满足

A={(V, P[V])| V∈除 r 和 Q 队列外的顶点集合}　　(P[V]—树中 V 的 "父母")

算法开始时，所有顶点进入 Q 队列。然后依次从 Q 中移出 d 值最小的顶点。每移出一个顶点 U，分析与 U 相连的所有边 (U, V)：若 V 在队列中且 (U, V) 的权小于 d[V]，则设定 P[V]←U(即树中 V 的父母为 U)，d[V] 的值为该边的权。此时 (U, V) 边实际上进入了 A 集合。当所有顶点从优先队列 Q 中移出后，最小生成树 A 满足

A={(V, P[V])| V∈除 r 外的所有顶点集合}

图 5.2-3 例举了 prim 算法的一个执行流程。黑边属于正在生成的树，树中的顶点为黑点。在算法的每一步，相连树中某顶点，具有最小权值且不生成回路的边被加入树中。



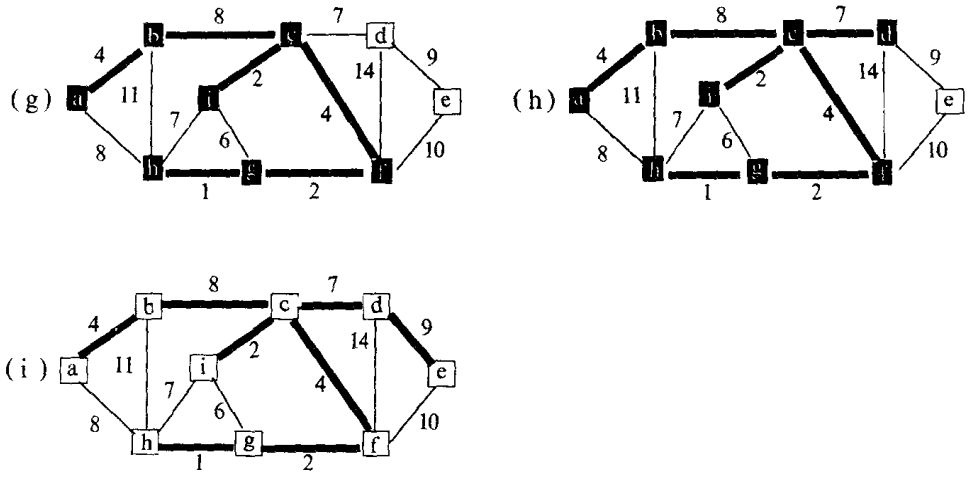


图 5.2-3

显然 prim 算法应和 dijkstra 算法一样，采用二叉堆来实现队列 Q。每次从 Q 中移出表首顶点 U 后，调用一次 heapfy(1), 保证以 Q(1) 为根的二叉树成为一个堆，即 Q(1) 的 d 值最小。另外在分析与 U 相连的所有边 (U, V) 时，若 V 在 Q 中且 W(U, V)<d(V) 则设定 P[V] ← U，并通过调用decrease(v, w[u, v]) 更新 d[v] 为W[U, V], 调整 V 在 Q 中的位置，以保持 Q 的堆性质。

下面给出程序题解:

```
Program Prime_Algorithm;

Const
  Maxn                  =  100;{顶点数的最大值}

Type
  Link                  = ^Node;{链 (邻接表) 指针}
  Node                  =  Record
                             k, w       :  Integer;{顶点序号，边的权}
                             next       :  Link {后继指针}
                           End;
  StType                =  Record {Q 堆的元素类型}
                             v, d, p    :  Integer;
                         {顶点序号，  连接到树中顶点的边所具有的最小权值；父顶点序号}
                           End;

Var
  N, root, Tot, i       :  Integer;{顶点数，根，生成树的权和，辅助变量}
```

```
  F                    : Text;{文件变量}
  Gr                   : Array [1..Maxn] of Link;{邻接表}
  Q                    : Array [1..Maxn] of StType;{排序堆}
  Q_pos                : Array [1..Maxn] of Integer;
                         {Q_POS[i]_顶点i在Q堆中的序号}
  Hs                   : Integer;{堆长度}

Procedure Insert (u, v, w : Integer);
{将权为W的边 (U, V) 插入顶点U对应邻接表}
Var
  x                : Link;
Begin
  New(x);
  x^. k := v; x^. w := w;
  x^. next := Gr[u]; Gr[u] := x
End; {insert}

Procedure Init;
Var
  i, u, v          : Integer;
Begin
  Assign (F,'INPUT. DAT');{输入文件名串与文件变量连接}
  Reset (F);{文件读准备}
  Readln (F, N, root);{读入顶点数和根}
  For i := 1 to N Do Gr[i] := nil;{所有顶点的邻接表初始化}
  While Not Eof(F) Do Begin{依次读入每一条边及其权，建立邻接表GR}
    Readln(F, u, v, i);
    Insert(u, v, i); Insert(v, u, i)
  End; {while}
  Close(F){关闭文件}
End; {init}

Procedure Swap(a, b : Integer); {Q堆中第a个顶点与第b个顶点交换}
Var
  i                : Integer;
  t                : StType;
Begin
  t := Q[a]; Q[a] := Q[b]; Q[b] := t;
```

```
  i := Q_pos[Q[a]. v]; Q_pos[Q[a]. v] := Q_pos[Q[b]. v];
  Q_pos[Q[b]. v] := i
End; {Swap}

Procedure Decrease (v, k : Integer);
{将Q堆中的第V个顶点的D值设为K，并将该顶点放到Q堆的合适位置}
Begin
  Q[v]. d := k;
  While (v <> 1) And (Q[v]. d < Q[v div 2]. d) Do Begin
    Swap(v, v div 2);
    v := v div 2
  End {while}
End; {decrease}

Procedure Heapfy (i : Integer); {堆排序，使得i为根的子树成为一个堆}
Var
  m              : Integer;
Begin
  m := i;
  Repeat
    i := m; {求顶点i与左儿子，右儿子的三个顶点间d值最小的顶点序号m}
    If (i * 2 <= Hs) And (Q[i * 2]. d < Q[m]. d) Then m := i * 2;
    If (i * 2 +1 <= Hs) And (Q[i * 2 +1]. d < Q[m]. d) Then m := i * 2+1;
    If i <> m Then Swap(i, m) {若i顶点的d值不是最小，则Q堆中i顶点与m顶点
交换}
  Until i = m {直至顶点i的d值最小，以i为根的子树已是堆为止}
End; {heapfy}

Procedure Mst_Prime;
Var
  u, v           : Integer;
  p              : Link;
Begin
  Tot := 0;
  For u := 1 to N Do Begin {所在顶点进入Q表，其D值为∞}
    Q[u]. v := u; Q[u]. d := Maxint;
    Q_pos[u] := u
  End; {for}
```

```
  Q[root]. d := 0; Q[root]. p := 0; {根的 d 值为 0, 父顶点为空}
  Swap(1, root); {Q 表的表首顶点与 root 顶点交换}
  Hs := N; {Q 表的长度最初为 N}
  While Hs > 0 Do Begin {若 Q 表非空, 则循环}
    u := Q[1]. v; {求 Q 的表首顶点 U}
    If u <> root Then {若该顶点非根顶点, 则打印 T 树中的边及其权}
      Writeln(Q[1]. p,'--', u,' ', Q[1]. d);
    Tot := Tot + Q[1]. d; {累计权和}
    Swap(1, Hs); {该顶点移出 Q 队列, Q 表长减 1}
    Dec(Hs);
    Heapfy(1); {对 Q 队列按 D 值递增的顺序进行堆排序}
    p := Gr[u]; {求 U 的邻接表指针}
    While p <> nil Do Begin {分析 Q 队列中所有与 U 相连的边 (U, V)}
      v := Q_pos[p^.k];
      If (v <= Hs) And (p^.w < Q[v]. d) Then Begin
    {若 V 属于 Q 队列且 W(U, V)<D(V)}
        Q[v].p := u; {则设 V 的父母为 U}
        Decrease(v, p^.w) {修正 V 的 D 值为 W(U, V) 并将 V 放到 Q 堆的合适位置}
      End; {then}
      p := p^.next {分析与 U 相连的下一条边}
    End {while}
  End {while}
End; {MST_prime}

Begin
  Init; {输入和构造图的邻接表}
  Mst_Prime; {计算和输出最小生成树}
  Writeln ('Tot = ', Tot) {输出最小生成树的权和}
End. {main}
```

§5.3 深度优先搜索

正如算法名称那样，深度优先搜索所遵循的搜索策略是尽可能“深”地搜索图形。在深度优先搜索中，对于最新发现的顶点，如果它还有以此为起点而未访问的边，就沿此边继续访问下去。当顶点 V 的所有边都已被访问过，则从最后被访问的顶点开始，依次返回新近被访问的且尚有邻接顶点未被访问过的顶点，再从该顶点重复上述搜索过程。直至访问了从起始顶点可达的所有顶点。如果还存在未被访问的顶点，则选择其中一个作为初始顶点，由此出发继续搜索。整个过程反复进行到所有顶点都被访问为止。

和宽度优先搜索不同的是，由于深度优先搜索可以由多个初始顶点开始重复进行，因

此可能产生一个由数棵深度优先树组成的森林。例如图 5.1-(b)，从顶点 1 出发对有向图进行深度优先搜索可得到如下序列 1→ 2→ 5→ 4 和 3→ 6。

按深度优先搜索法访问全图后的遍历路径，形成由两棵深度优先树组成的森林。

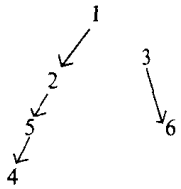


图 5.3-1

为了保持搜索轨迹，搜索过程中为每个顶点着色，以表示顶点的当前状态：每个顶点被访问前均为白色；首次被访问时置为灰色；当以其为根的子树上的所有顶点被访问之后，又被置成黑色。通过给顶点着色可以保证每一个顶点在搜索结束时只存在于一棵深度优先树上。

为了进一步帮助人们推算深度优先搜索的进程，我们同时为每个顶点加盖时间戳。每个顶点 V 有两个时间戳：当顶点 V 第一次被访问（并置成灰色）时记录下第一个时间戳 d(v)；当访问了以 V 为根的子树上的所有顶点后，记录下第二个时间戳 f(V)（并置 V 为黑色）。d(V)<f(V)，显然在时刻 d(V) 前，顶点为白色；在时刻 d(V) 和 f(V) 之间为灰色，以后就变成黑色。由于对每一个顶点 V 都对应一个发现事件（何时在变量 d(V) 中发现顶点 V）和一个完成事件（何时在变量 f(V) 中完成对以 V 为根的子树的访问），因此时间戳为 1 到 2*|V| 之间的整数。

依据各顶点的时间戳可以获得有关图的结构的大量信息。例如区间 d[V]..f[V] 完全包含于区间 d[U]..f[U] 之中（d[U]<d[V]<f[V]<f[U]），则顶点 V 是顶点 U 的后裔；又如区间 d[V]..f[V] 和区间 d[U]..f[U] 完全分离（f[V]<d[U] 或者 f[U]<d[V]），则 U 和 V 要么分属于两棵深度优先树，要么同属一棵深度优先树但不存在直系关系。

深度优先搜索的另一个令人感兴趣的特点就是可以通过搜索对图的边分类：

1. 树枝

深度优先森林中的边。如果 V 是在访问边（U, V）时第一次被发现（此时 V 是白色的），那么（U, V）是一个树枝。在树枝边中，我们称 U 为 V 的父母，记作 P(V)= U。

2. 反向边

在深度优先树中连接顶点 U 到它的祖先的那些非树枝边。环也被认为是反向边。显然，如果第一次访问(U, V)时 V 为灰色，则(U, V)为反向边。在对有向图的深度优先搜索中未发现反向边，则该有向图无回路。

3. 正向边

深度优先树中连接顶点 U 到它的后裔的非树枝边。当第一次访问（U, V）时，V 为黑色且 d[U]<d[V]，则（U, V）为正向边。

4. 交叉边

其它类型的边。它们既可以连接一棵优先树中的两个顶点，只要一顶点不是另一顶点的祖先，也可以连接分属两棵深度优先树的顶点。当第一次访问（U, V）时，V 为黑色且 d[U]>d[V]，则（U, V）为交叉边。

深度优先搜索无向图时，图的每条边要么是树枝边要么是反向边，原因在于边（U, V）和（V, U）互为等价。为了不失一般性的讨论，我们假定 d[U]<d[V]，即在完成对以 V 为根的子树的访问之后才完成对以 U 为根的子树的访问。（也可以用同样的方法对 d[V]<d[U] 的情况讨论）如果边（U, V）第一次是按从 U 到 V 的方向被访问到，则（U, V）必

是一树枝，因为 V 在访问前是白色的；如果边 (U, V) 第一次是按从 V 到 U 的方向被访问到，则由于该边被第一次访问时 U 依然是灰色顶点，所以 (U, V) 是一条反向边。由此可见，深度优先搜索一无向图时，是决计不会出现正向边和交叉边的。

深度优先搜索还可以统计深度优先树中每个顶点的度数：从灰色顶点 U 出发，每访问一条树枝边 (U, V) (此时 V 为白色)，则顶点 U 的度数 +1。当访问了以 U 为根的子树后，顶点 U 的度数便统计出来了。此时置 U 为黑色。

在通过深度优先搜索分析无向图的连通性时，经常用到标号函数 LOW[U], 用以确定顶点 U 或 U 的后代所能追溯到的最早 (最先开始被访问) 的祖先点 W 的 d(W) 值：

$$
\mathrm{LOW}[U]=\begin{cases}
d[U] & U\text{ 首次被访问；}\\
\min(\mathrm{LOW}(U),\ d(W)) & \text{访问反向边 }(U,\ W)\text{ 时；}\\
\min(\mathrm{LOW}(U),\ \mathrm{LOW}(S)) & U\text{ 的儿子 }S\text{ 的关联边全部被访问时。}
\end{cases}
$$

例如，我们对图 5.3 - 2中的有向图进行深度优先搜索。

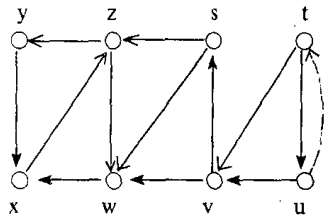


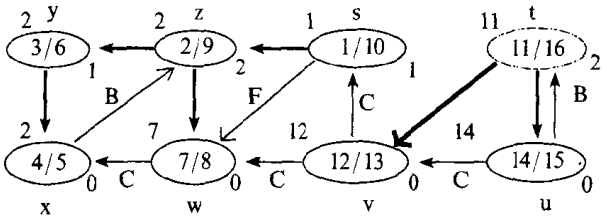
图 5.3 - 2

搜索的结果如图 5.3 - 3 (a)。被算法访问到的边要么为被阴影覆盖的树枝边，要么成虚线形式的非树枝边。这些边分别标明 B(反向边)，F(正向边)，C(交叉边)。我们以发现时刻/完成时刻的形成对顶点加盖时间戳。顶点下方为深度优先树中该顶点的度数。顶点上方为标号函数 LOW 的值。

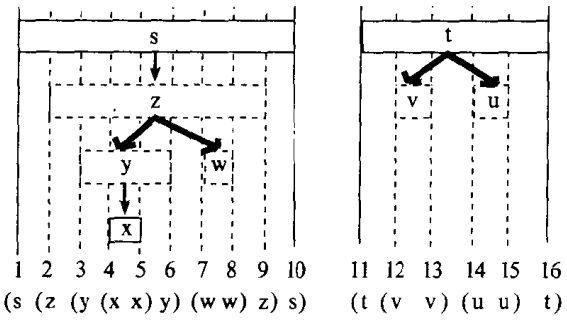
为了反映深度优先树中各顶点的上、下关系，我们在图 5.3 - 3(b) 中用括号表示对应于每个顶点的发现时刻与完成时刻所设定的区间。图中的每个矩形跨越相应顶点的发现时刻与完成时刻所设定的区间，图中还用阴影覆盖的边显示树枝。

为了更直观形像地描述深度优先树中的各类边，我们对图 5.3 - 3 (a) 重新绘制得出图 5.3 - 3(c)，使得深度优先树中的树枝边和正向边向下绘制，并使反向边向上绘制。

(a)



(b)



(c)

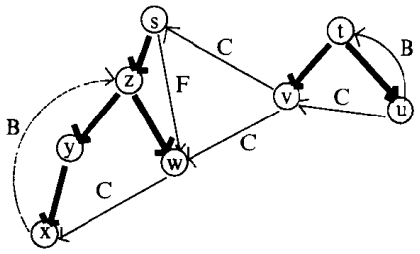


图 5.3-3

下面，我们给出对无向图进行深度优先搜索的程序题解。算法输入图的邻接矩阵 g，得出图的如下结构信息：

1. 边的类型数组gc。其中gc[i, j] 表示边 (i, j) 的类型：元素值 1 对应树枝边，2 对应反向边，3 对应正向边，4 对应交叉边。

2. 顶点的开始访问时间表 d 和完成访问时间表 f。其中 d[i] 和 f[i] 分别表示顶点 i 的开始访问时间和完成访问时间。

3. 顶点的颜色表 c。其中 c[i] 表示顶点 i 的当前颜色：0 值对应白色，−1 对应灰色，1 对应黑色。

4. 顶点的度数表 b。其中 b[i] 为深度优先树中顶点 i 的度数。

5. 顶点的标号函数表 L。其中 L[i] 为顶点 i 的 LOW 值。

6. 顶点的前趋表 P。其中 P[i] 为顶点 i 的父母序号。若 P[i] >0，则 (P[i], i) 为树枝边。

读者在使用深度优先搜索的算法求解各类图形问题时，可根据实际需要选择有关的结构信息。

```
const
  maxn=100; {最多顶点数}
var
  n: integer; {实际顶点数}
  g, gc: array [1..maxn, 1..maxn] of integer;

{g−邻接矩阵, g[i, j]=┌  0  顶点 i 至顶点 j 无边;
                      │  1  边 (i, j) 存在且允许访问
                      └ −1  边 g[i, j] 不许重复访问

                      ┌ 0  顶点 i 至顶点 j 无边;
                      │ 1  (i, j) 为树枝边;
gc −边的类型矩阵, g[i, j]= ┤ 2  (i, j) 为反向边;
                      │ 3  (i, j) 为正边;
                      └ 4  (i, j) 为交叉边; }
  d, f, c, p, l, b: array [1..maxn] of integer;
```

```
  {d - 顶点的开始访问时间表;
   f - 顶点的完成访问时间表;          c[i]= 0   白色
   c - 顶点颜色表;                          -1   灰色
                                             1   黑色
P - 顶点的前趋表;  L - 顶点的 LOW 值表;  b - 顶点度数表 . b[i] - 顶点 i 的度数; }
  time, color: integer; { 时间戳, 颜色 }

 procedure dfs _ visit (u: integer);
 { 递归搜索以 U 为根的深度优先树 }
  var
   v: integer;
  begin
 * c[u]:= -color; { 置 U 顶点为灰色 }
   inc(time); d[u]:=time; l[u]:=time; { 置 U 顶点的开始时间, U 顶点的 LOW 值初始
   化 }
 for v:=1 to n do { 依次搜索 G 中的每一个顶点 }
    if g[u, v]>0 then { 若 (U, V) 边存在且未访问 }
     if c[v]=0 then { 若 V 顶点为白色 }
      begin
     * g[v, u]:= -g[v, u]; { 置 (V, U) 不允许访问标志 }
       gc[u,v]:=1; 置 (U,V) 为树枝边 }
       inc(b[u]); { 累计 U 的度数 }
       p[v]:=u; { 置 V 的父母为 U }
       dfs _ visit(v); { 递归搜索 V 的后继 }
       if l[v]< l[u] then {U 的儿子 V 的关联边被检查后, 计算 LOW[U]=(LOW[V],
       LOW[U]}
        l[u]:=l[v];
       g[v, u]:= -g[v, u] { 恢复 (V, U) 可访问标志 }
      end {then}
     else
      if c[v]<0 then { 若 V 顶点为灰色 }
       begin
        if l[v]<l[u] then { 计算 LOW[U]={LOW[U], D[V]}
        l[u]:=l[v];
        gc[u, v]:=2 { 置 (U, V) 为反向边 }
       end {then}
      else { 若 V 为黑色 }
```

```
        if d[v]>d[u] then {置(U, V)为正向边}
         gc[u, v]:=3
        else
         gc[u, v]:=4; {置(U, V)为交叉边}
   c[u]:=color; {置U为黑色}
   inc(time); f[u]:=time {计算U的完成访问时间}
  end; {dfs_visit}

 procedure dfs;
  var
   u:integer;
  begin
   fillchar(gc, sizeof(gc), 0); {边类型表初始化}
   fillchar(c, sizeof(c), 0); {全部顶点为白色}
   fillchar(b, sizeof(b), 0); {全部顶点的度数为0}
   time:=0; color:=0; {时间戳和颜色初始化}
   for u:=1 to n do {依次搜索每个顶点}
    if c[u]=0 then {若顶点U为白色}
     begin
      inc(color); p[u]:=0; {准备颜色，设U顶点为根}
      dfs_visit(u) {递归搜索以U为根的深度优先树}
     end {then}
  end; {dfs}
```

* 注：无向图的邻接矩阵是对称的。即(U, V)等价于(V, U)。但是在图的搜索中每条边仅允许访问一次，因此一旦访问了(U, V)则不允许重复访问(V, U)。而有向图中(U, V)不等价于(V, U)(除环外)，因此上述算法在删去了*标志的语句后，同样适用于有向图的深度优先搜索。

目前，深度优先搜索算法已被深入开发，由此扩充出许多图形算法，显示出强大的生命力。下面介绍深度优先搜索的三个经典应用实例。

【例1】求有向无回路图G的拓扑排序

将有向无回路图G的所有顶点排成满足下列条件的拓扑序列：若G包含有向边(U, V)，则在序列中U出现在V之前，即该序列使得图中所有有向边均从左指向右。如果图是有回路的，就不存在这样的序列。试用深度优先搜索来求该序列。

算法分析：

我们可以通过深度优先搜索解决有向无回路图的拓扑排序问题：每完成对一个顶点的访问后，该顶点置黑并插入拓扑排序表的首端。结束搜索时，拓扑排序表中的顶点按完成时间递减的顺序由左向右排列。如果搜索过程中出现反向边，则说明图中有回路，失败

退出。

```
const
 maxn=100; {顶点数的最大值}
var
   n, i: integer; {实际顶点数，辅助变量,}
 g: array [1..maxn, 1..maxn] of integer; {邻接矩阵}
 c, p, a: array [1..maxn] of integer;
 {C -顶点颜色表; P -顶点前趋表;  A -拓扑排序表}
 color, m: integer; {颜色;  拓扑排序表的指针}

procedure dfs_visit (u: integer); {递归搜索以 U 为根的深度优先树}
 var
  v: integer;
 begin
  c[u]:=-color; {置 U 顶点为灰色}
  for v:=1 to n do {依次搜索 G 中的每一个顶点}
   if g[u, v]<>0 then {若 (U, V) 边存在且未访问}
    if c[v]=0 then {若 V 顶点为白色}
     begin
      g[v, u]:=-g[v, u]; {置 (V, U) 不许访问标志}
      p[v]:=u; {置 V 的父母为 U}
      dfs_visit(v); {递归搜索 V 的后继}
      g[v, u]:=-g[v, u]; {恢复 (V, U) 可访问标志}
     end {then}
  else
    if c[v]<0 then {若 V 为灰色, (U, V) 为反向边, 则失败退出}
      begin
        writeln ('Circle!'); halt
      end; {then}
  c[u]:=color; {置 U 为黑色并进入拓扑序列}
  a[m]:=u; dec(m)
 end; {dfs_visit}

procedure dfs;
 var
  u: integer;
 begin
```

```
  fillchar(c, sizeof(c), 0); {全部顶点颜色初始化为白色}
  color:=1; m:=n; p[1]:=0;
  for u:=1 to n do {依次搜索每个未访问的顶点}
    if c[u] = 0 then dfs_visit(u)
end; {dfs}

procedure input; { 输入邻接矩阵 }
 var
  f: text;
  i, j: integer;
 begin
  assign(f,'input2.txt'); reset(f);
  readln(f, n);
  for i:=1 to n do
   for j:=1 to n do
    read(f, g[i, j]);
  close(f)
 end; {input}

begin
 input; { 输入邻接矩阵 }
 dfs; { 深度优先搜索拓扑序列 }
 for i:=m + 1 to n do { 输出拓扑序列 }
  write(a[i],' ');
 writeln;
end.{main}
```

【例 2】 求一个无向连通图的割点和桥。

设 G=(V, E)是一无向连通图。如果去掉 G 的某顶点后，G 就不是连通图，这样的顶点称为割点，如果去掉某一边就不成为连通图，这样的边称为桥。试用深度优先搜索来确定一个无向连通图的所有割点和桥。

算法分析:

若我们对无向图实施深度优先搜索后找出一个割点 V，则 V 必然具备下述特征:

1. 若 V 是深度优先树的根，则该根至少有两个子女（见图 5.3-4(a)）;

2. 若 V 不是深度优先树的根，则 V 有一个后裔 U，从 U 或 U 的后代到 V 的祖先点之间不存在反向边，见图 5.3-4(b)。

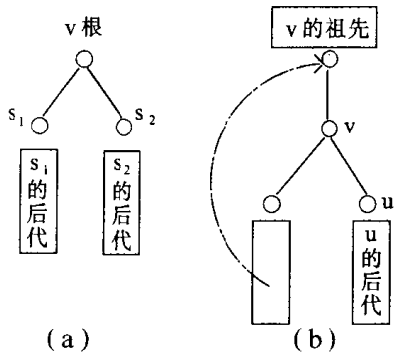


图 5.3-4

证明略去，读者可从图 5.3-4 中体会灵活的思路。图例中 V 是割点。

我们可以直接利用深度优先搜索过程中得出的结构信息: P(前趋表), b(度数表), L(标号函数表), d(发现时刻表)进行割点的识别。为了避免对割点重复输出，还在算法中增设了割点表 a。若 a[V]=0, 表示当前还未判断出 V 是割点；若 a[V]=1, 表示已判断出 V 是割点，应避免重复输出。

显然，对应于割点的第 1 个特征来说，若(P[V]=0) and(b[V]>1),则根结点 V 为割点；对应于割点的第 2 个特征来说，若 (P[P[V]]>0) and (a[P[V]]=0) and (L[V] ≥ d[P[V]]), 则非根结点 P[V]为割点。

同样我们也可以通过深度优先搜索得到的结构信息找出无向连通图的桥。所谓桥，是指那些不属于任何简单回路的边。在深度优先搜索中，当搜索到一条树枝边 (U, V) 时发现 V 及其后代不存在一条连接 U 的祖先点的返回边，即 LOW[V]>d[U],则说明(U,V)不属于图的任何简单回路，(U, V) 即为桥。

例如，应用深度优先搜索可找到图 5.3-5 中有向连通图的割点(黑色顶点)和桥(黑色边)。图中阴影区域的边为双连通子图，即没有割点的最大连通子图。

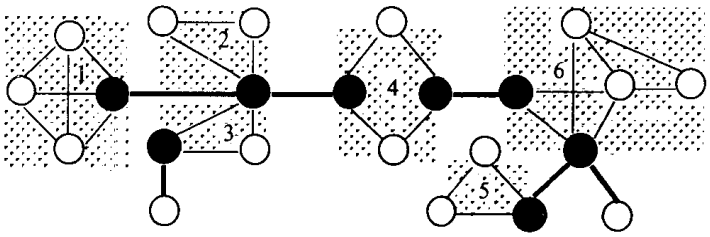


图 5.3-5

下面是程序题解:

```
const
 maxn=100; {顶点数的最大值}
var
 n: integer; {实际顶点数}
 g:array [1..maxn, 1..maxn] of integer; {邻接矩阵}
```

```
 d, c, p, l, b: array [1..maxn] of integer;
  {D－开始访问时间表;  C－顶点颜色表;  P－顶点前趋表;  L－顶点 LOW 值表;
B－顶点度数表}
  time, color: integer; {时间戳，颜色}

 procedure dfs_visit (u: integer); {递归搜索以 U 为根的深度优先树}
  var
   v: integer;
  begin
   c[u]:=－color; {置 U 顶点为灰色}
   inc(time); d[u]:=time; l[u]:=time; {置 U 顶点的开始时刻，U 顶点的 LOW 值初始化}
   for v:=1 to n do {依次搜索 G 中的每一个顶点}
    if g[u, v]>0 then {若(U, V)边存在且未访问}
     if c[v]=0 then {若 V 顶点为白色}
       begin
         g[v, u]:=－g[v, u]; {置(U, V)不许访问标志}
         inc(b[u]); {累计 U 的度数}
         p[v]:=u; {置 V 的父母为 U}
         dfs_visit(v); {递归搜索 V 的后继}
         if l[v]<l[u] then
           {U 的儿子 V 的关联边被检查后，计算 LOW[U]=min(LOW[V], LOW[U])}
         l[u]:=l[v];
         g[v, u]:=－g[v, u] {恢复(U, V)可访问标志}
       end {then}
     else
       if c[v]<0 then {若 V 为灰色((U, V)为反向边)}
       begin {计算 LOW[U]=min(LOW[U], D[V])}
        if l[v]<l[u] then l[u]:=l[v]
       end; {then}
     c[u]:=color {置 U 为黑色}
  end; {dfs_visit}

 procedure dfs;
  var
   u: integer;
  begin
    fillchar (c, sizeof(c), 0); {全部顶点颜色初始化为白}
    fillchar (b, sizeof(b), 0); {全部顶点度数初始化为 0}
```

```
  time:=0; color:=0; {时间戳和颜色初始化}
   for u:=1 to n do {依次搜索每个顶点}
    if c[u]=0 then {若顶点 U 为白色}
    begin
      inc(color); p[u]:=0; {准备颜色,设 U 顶点为根}
      dfs_visit(u) {递归搜索以 U 为根的深度优先树}
    end {then}
  end; {dfs}

procedure input;
 var
  f: text;
  i, j: integer;
 begin
  assign (f,'input. txt'); reset(f); {输入文件名串与文件变量连接,文件读准备}
  readln (f, n); {读入顶点数}
  for i:=1 to n do {读入邻接矩阵 G}
    for j:=1 to n do read (f, g[i, j]);
  close(f)
 end; {input}

procedure output;
 var
  i: integer;
  a: array [1..maxn] of 0..1;
 begin
   fillchar (a, sizeof(a), 0); {割点初始化}
   for i:=1 to n do
     if (p[p[i]]>0) and (a[p[i]]=0) and (l[i]>=d[p[i]]) then
     begin
  {若非根顶点 P[I] 目前未被识别为割点且 I 顶点及其后代不存在一条连接 P[I] 祖先点
的返回边, 则输出割点 P[I], 并置 P[I] 割点标志}
       write (p[i], ' '); a[p[i]]:=1
     end; {then}
  for i:=1 to n do
   if (p[i]=0) and (b[i]>1) then {若 I 顶点为根且子女多于 1 个, 则输出割点 I}
    write (i, ' ');
  writeln;
```

```
  for i:=1 to n do
    if p[i]>0 then
     if l[i]>d[p[i]] then
{若顶点I及其后代不存在一条连接P[I]的祖先点的返回边，则输出桥(P[I], I)}
       write (p[i], ' -', i, ' ');
   writeln
  end; {output}

 begin
  input; {输入邻接矩阵}
  dfs; {通过深度优先搜索找出割点和桥}
  output {输出割点和桥}
 end. {main}
```

【例3】求有向图的强连通分支

一有向图G=(V, E)的强连通分支是指满足如下条件的最大顶点集合U⊆V。对V集合中的任意一对顶点U, V, 它们之间互相可达。试通过深度优先搜索求G的所有强连通分支。

算法分析:

图G中的所有边改变方向后组成了G的转置图G^T。可由强连通分支中顶点间的互相可达性得出: G和G^T有着完全相同的强连通分支:即在G中U和V互为可达当且仅当在G^T中它们互为可达。

在对图G进行深度优先搜索后，每一个强连通分支内的所有顶点均在同一棵深度优先树中，并且有一个顶点为其中每个顶点的代表: 强连通分支中最先发现且最后完成的顶点，即深度优先搜索所计算出的具有最大完成时刻的顶点R。我们称R为强连通分支中每一个顶点的祖宗，但由于在G中R是完成时刻最大的顶点，所以R的强连通分支目前仅由那些可达R的顶点组成。我们再从强连通分支的祖宗R出发，对转置图G^T进行深度优先搜索，以求得从R可达的顶点集合。显然，这个顶点集合中的任意一对顶点都互为可达，属于R的强连通分支。

在识别出R的强连通分支后，算法又从不属于R的强连通分支且有着最大完成时刻的任何顶点R′出发，深度优先搜索G^T中未访问的顶点，以识别出属于R′的强连通分支的每个顶点。依次类推，直至取得图的所有强连通分支为止。例如，我们对图5.3-6中的有向图进行深度优先搜索。

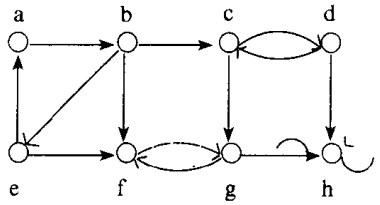


图5.3-6

搜索的结果如图 5.3 - 7。图 5.3 - 7 (a) 以发现时刻/完成时刻的形式给每个顶点加时间戳，黑边为树枝边。图 5.3 - 7 (b) 为对应的深度优先森林。

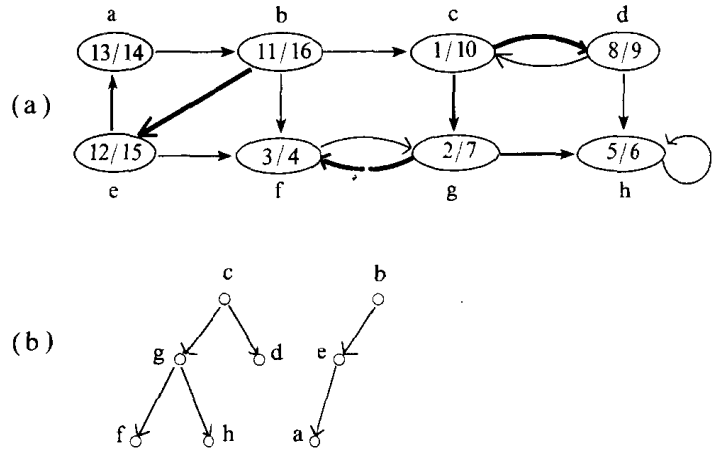


图 5.3 - 7

然后从完成时刻最大的 b 顶点出发，对 G^T 图进行深度优先搜索，得出属于 R 的强连通分支 {b, a, e}；再从不属于 b 的强连通分支且有着最大完成时刻的 c 出发，对 G^T 中未被访问的顶点进行深度优先搜索，识别出属于 G 的强连通分支 {c, d}；接下去从不属于 b 和 c 的强连通分支且有着最大完成时刻的 g 出发，继续搜索 G^T 中尚未访问的顶点，得出属于 g 的强连通分支 {g, f}；最后访问 h，得出最后一个强连通分支 h。图 5.3 - 8 给出了对转置图 G^T 进行深度优先搜索的结果。图 (a) 中黑色边为树枝边，黑色顶点 b, c, g, h 是强连通分支中每一个顶点的祖宗，也是对 G^T 进行深度优先搜索所产生的深度优先树的树根。图 (b) 是对应的深度优先森林，其中每一棵生成树对应一个强连通分支。

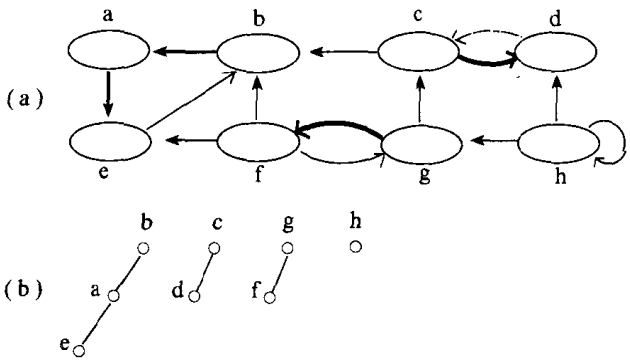


图 5.3 - 8

最后，我们把 G 的每个强连通分支缩减为单个顶点，得到一个无回路子图 G^{scc}。

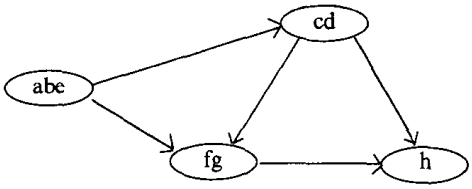


图 5.3 - 9

下面给出程序题解。算法中增加了一个完成时刻排序表 a_1。$a_1[i]$——具有第 i 个大的完

成时刻的顶点序号。初始时 $a_1[i]=i$ $(1 \leqslant i \leqslant n)$。在对 G 图进行深度优先搜索的过程中，第一个被涂黑的顶点序号送入 $a_1[n]$；第 2 个被涂黑的顶点序号送入 $a_1[n-1]$；…依次按照完成时刻的顺序类推，最后一个被涂黑的顶点序号送入 $a_1[1]$。G 图搜索结束后按 $a_1[1]..a_1[n]$ 的顺序搜索 G^T —— 只要当前 $a_1[i]$ 不属于 $a_1[1]..a_1[i-1]$ 中的任一个强连通分支，就从 $a_1[i]$ 出发对 G^T 中尚未访问的顶点进行深度优先搜索；否则往下考虑 $a_1[i+1]$。$(1 \leqslant i \leqslant n)$

```
const
  maxn=100; {最多顶点数}
var
  n, grt: integer; {实际顶点数}
  grt: integer; {转置标志}
  g: array [1..maxn, 1..maxn] of integer; {邻接矩阵}
  c, p, a1, a2: array [1..maxn] of integer;
  {c - 顶点颜色表; P- 顶点前趋表; a1 - 完成时刻排序表; a2 - 辅助表}
  color, m: integer; {颜色, 完成时刻排序表指针}

procedure dfs_visit (u: integer); {递归搜索以 U 为树的深度优先树}
  var
    v: integer;
  begin
    c[u]:=-color; {置 U 顶点为灰色}
    for v:=1 to n do {依次搜索 G 中的每一个顶点}
      if g[u, v]>0 then {若(U, V) 边存在且未访问}
        if c[v]=0 then  begin
          p[v]:=u; {置 V 的父母为 U}
          dfs_visit(v) {递归搜索 V 的后继}
        end; {then}
      c[u]:=color; {置 U 为黑色}
      if grt=1 then write (U,' ');
  {若是对 G^T 搜索, 则 U 作为当前强连通分支的一个顶点输出}
          else begin
            a1[m]:=u; dec(m)
{U 为第 m 大的完成时刻的顶点, 完成时刻排序表的指针前移}
          end {else}
  end; {dfs_visit}

procedure dfs;
  var
    u: integer;
```

```
begin
    a2:=a1;{完成时刻排序表转赋给 a2}
    fillchar(c, sizeof(c), 0);{全部顶点颜色初始化为白}
    color:=0; m:=n;{颜色初始化，首先访问最小完成时刻的顶点}
    for u:=1 to n do {按完成时刻递减的顺序访问每个顶点}
    if c[a2[u]]=0 then {若具有第 U 个大的完成时刻的顶点为白色}
    begin
      inc(color); p[a2[u]]:=0;
   {准备颜色，该顶点设为根，递归搜索其深序优先树}
      if grt = 1 then write ('{');
    {若对 G^T 进行深度优先搜索，则打印以该顶点为祖宗的强连通分支;}
      dfs_visit(a2[u]);
      if grt = 1 then write ('}')
    end;{then}
   writeln
end;{dfs}

procedure input;
 var
  f: text;
  i, j: integer;
 begin
   assign(f,'input1.txt'); reset(f);{输入文件名串与文件变量连接}
   readln(f, n);{输入顶点数}
   for i:=1 to n do {输入邻接矩阵 G}
     for j:=1 to n do read(f, g[i, j]);
   close(f);
   for i:=1 to n do {搜索 G 图前，完成时刻排序表按顶点序号排列}
   a1[i]:=i
end;{input}

procedure gt;{求 G 图的转置矩阵 G^T; 并设转置标志}
 var
  i, j, k: integer;
 begin
   for i:=1 to n do
    for j:=i+1 to n do
    begin
```

```
    k:=g[i, j]; g[i, j]:=g[j, i]; g[j, i]:=k
     end; {for}
    grt := 1
  end; {gt}

begin
   grt := 0;
   input;
   dfs;
   gt; {求转置矩阵 G_T}
   dfs; {按完成时刻递减顺序对 G_T 进行深度优先搜索，并输出强连通分支}
end. {main}
```

§5.4 有向图的最短路问题

这一章节讲一个比较简单但又是很重要的问题——最短路问题。 这个问题有着大量的生产实际的背景。事实上大至海陆空各种运输，小至一个人每天上班，都会遇到这一问题，甚至有些问题从表面上看与最短路问题没有什么关系，却也可以归结为最短路问题。下面就举一个这样的例子。

【例 1】渡河问题

一个人带了一只狼、一只羊和一棵白菜想要过河。河上有一只独木船，每次除了人以外，只能带一样东西。另外如果人不在旁时狼就要吃羊，羊就要吃白菜。 问应该怎样安排渡河，才能做到既把所有东西都带过河 ，而且在河上来回的次数又最少?

算法分析:

我们设变量 M 代表人，W 代表狼，S 代表羊，V 代表白菜。开始时设人和其它三样东西在河的左岸，这种情况用 MWSV 表示。

我们用一个集合表示目前左岸的情况。很显然，可能出现的情况有 16 种:

[MWSV]	[MWS]	[MWV]	[MSV]
[WSV]	[MW]	[MS]	[MV]
[WS]	[WV]	[SV]	[M]
[W]	[S]	[V]	[Φ]

剔除下述 6 种可能发生狼吃羊、羊吃白菜的情况:

[WSV] [MW] [MV] [WS] [SV] [M]

现在我们就来构造一个图 G, 它的顶点就是剩下的 10 种情况。G 中的边是按下述原则

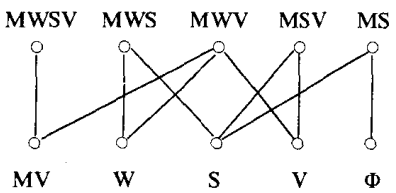


图 5.4 -1

来连的:如果甲经过一次渡河可以变成情况乙，那么就在情况甲与情况乙之间连一条边。

作了图G以后，渡河的问题就归结为下述问题了:“在G中找一条连接顶点MWSV与Φ，并且包含边数最少的路”。如果我们设G的与各边的长度都是1，那么也可以把渡河问题归结为:“找一条连接MWSV与Φ的最短路”。把问题转化为图的问题后，就可用一种系统的方法解决，而不是通常人们所用的凑的方法和凭经验的方法。

下面，我们可以给最短路问题下一个抽象的定义:

设G=(V, E)是一个有向图，它的每一条边(U, V)∈E都有一个权W(U, V)，在G中指定一个顶点S，要求把从S到G的每一个顶点V_j(V_j∈V)的最短有向路找出来(或者指出不存在从S到V_j的有向路，即S不可达V_j)。

在上述定义中，由于源顶点S是给定的，因此我们又形象地称为单源最短路径问题。

当然最短路径问题远不止上述一种，但它们一般都可用单源问题的算法来解决。例如:

1. 单目标最短路径问题:找出每一个顶点V_j(V_j∈V)到某指定顶点V_s的每条最短路径。把图中的每条边的方向反向，我们就可以把这一问题变成单源最短路径问题;

2. 单对顶点间最短路径问题:对于给定顶点U和V，找出一条从U到V的最短路径。如果我们解决了源顶点为U的单源问题，则这问题自然获得了解决。目前还没有一种比单源算法更快的算法来解决这一问题;

3. 每对顶点间最短路径问题:对于每对顶点U和V找出从U到V的最短路径。我们可以用单源算法将每个顶点作为源点运行一次就可以解决这一问题。但是，如果我们采用动态程序设计算法，运行的效率更高。我们将在本章节中介绍有关算法。

一、单源最短路径问题

我们曾在§5.2【例1】中给出了一种称为dijkstra的算法，该算法解决了有向加权图中单源最短路径问题，其条件是该图中所有边的权W(U, V)≥0((U, V)∈E)。但单源最短路径问题的实例中，可能存在权为负的边。如果图不包括从源S可达的负权回路，dijkstra算法求出的最短路径的权依然正确，即便它是一个负值也是如此。但如果存在一条从S可达的负权回路，最短路径的权值便无法计算了，因为我们总可以顺着找出的“最短”路径再穿过负权回路从而获得一条权值更小的路径，最短路径的权值实际上成了$-\infty$。此时dijkstra算法便爱莫能助了。如果读者对§5.3【例1】中的拓扑排序有所记忆的话，马上会想到一种算法:预先对有向图进行一次拓扑排序。如果图中存在从S可达的权为负的回路，则宣告该问题无解;否则用dijkstra算法求最短路及其权。

下面，我们另外介绍一种也能在一般情况下解决单源最短路径的算法。这种算法比较简洁，在我们即将介绍的关于稀疏图的每对顶点间最短路径的算法中，它将作为子程序被调用。

【例1】bellman _ford算法

该算法输入一个源顶点S和一个加权有向图G,边上的权可以为负。运行结果返回一个布尔值，表明图中是否存在一个从S可达的权为负的回路。若存在这样的回路，算法返回false，说明该问题无解;若不存在这样的回路，算法将产生最短路径及其权，并返回true。

算法分析:

算法中运用了松驰技术。对每一个顶点 v∈V，设置了一个变量 D[V] 来描述从 S 到 V 的最短路径的权的上界，称之为最短路径估计。另外设 P[V] 为 S 到 V 的最短路径中 V 的前趋，即 (P[V], V) 为最短路径上的一条边。在正式求最短路径之前，对所有顶点的最短路径估计和前趋初始化:

D[S]=0; D[v]=∞ (v∈V-{S}); P[v]= ∞ (v∈V);

因为 S 到每个顶点的最短路径是一条路径长度 K ≤|V|-1 的简单路径，且路径上的任何子路径都是最短路径，因此我们对图中的每条边(U, V)依次进行|V|-1 次松驰操作。每次松驰操作测试D[V]是否大于 D[U]+W(U, V)。如果是，则通过顶点U对迄今找出的到 V 的最短路径进行改进。更新 D[V] 和 P[V]:

D[V]←D[U]+W(U, V); P[V]←U;

第 i 次松驰操作后，使得由 S 出发的各条路径 (S, V_1, ... , V_i, ...) 上的 D[V_i]为实际的最短路径的权。如果有向图中无权为负的回路，则在对所有边进行了 |V|-1 次松驰后，S 到各顶点的最短路径的估计一定会单调递减到实际最短路径的权。例如图 5.4-2 中源顶点为 Z, bellman_ford 算法依次对每条边进行了四次松驰操作。图中给出了每一次操作后的状态。其中每个顶点内为该顶点的 D 值，黑边说明了 P 值。在该实例中 bellman_ford 算法返回 true。

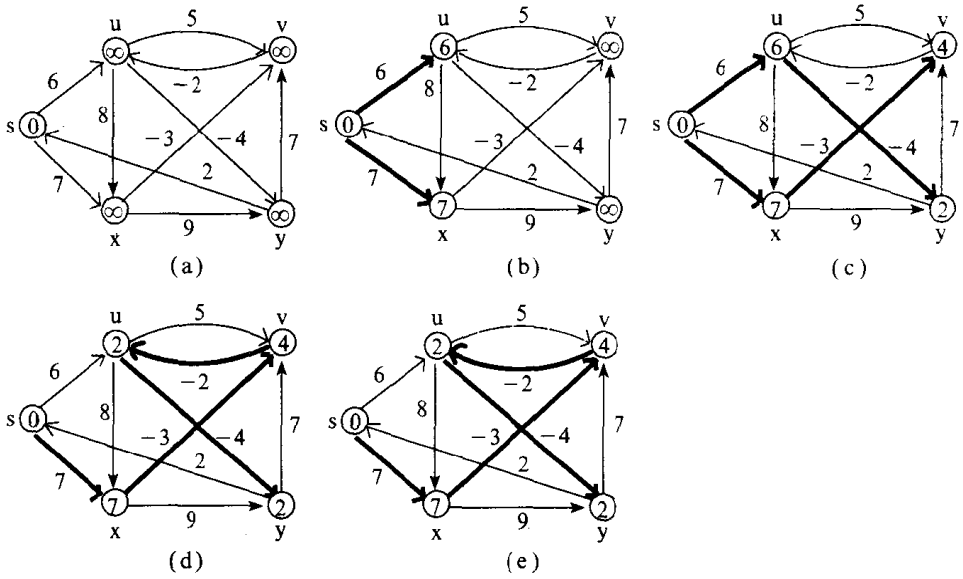


图 5.4-2

如果无向图中存在负边回路，则回路上的边(U, V)的 d[V]值一定大于 d[U]+W(U, V)，此时bellman_ford 算法返回 false。

下面是程序题解:

```
Program Bellman_Ford_Algorithm;
Const
  Maxn                    =   100; {顶点数的最大值}

Type
```

```
  Link                  = ^Nbour; {链指针}
   Nbour                  =  Record {链结点类型}
                             k, w        :  Integer;
                             {相连的顶点序号; 边的权}
                             next        :  Link {后继指针}
                           End;
   GrType                 =  Record {邻接表}
                             d, p        :  Integer;
                             {源点至该点的最短路径估计, 父顶点序号}
                             nbs         :  Link {链指针}
                           End;

Var
  N, root, i          :  Integer; {实际顶点数,源点序号,辅助变量}
  F                   :  Text; {文件变量}
  Gr                  :  Array [1..Maxn] of GrType; {邻接边}

Procedure Insert (Var Hd : Link; k, w : Integer);
{将以 K 顶点为弧头, W 为权重的边插入链表 HD 的表首}
Var
  x              :  Link;
Begin
  New(x);
  x^.k := k; x^.w := w;
  x^.next := Hd; Hd := x
End; {insert}

Procedure Init;
Var
  i, u, v        :  Integer;
Begin
  Assign (F, 'INPUT. DAT'); {输入文件名串与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N, root); {读入顶点数和源点}
  For i := 1 to N Do Gr[i]. nbs := nil; {邻接表初始化为空}
  While Not Eof(F) Do Begin
    Readln (F, u, v, i); {读入权为 i 的边(U, V), 将之插入顶点 U 的邻接表中}
    Insert (Gr[u]. nbs, v, i)
```

```
  End; {while}
  Close(F) {关闭文件}
End; {init}

Procedure Relax(u, v, w : Integer); {对权为 W 的边(U, V)进行松驰操作}
Begin
  If Gr[v]. d > Gr[u]. d + w Then Begin
    Gr[v]. d := Gr[u]. d + w;
    Gr[v]. p := u
  End {then}
End; {relax}

Function Bellman_Ford: Boolean;
Var
  i, u          : Integer;
  p             : Link;
Begin
  For i := 1 to N Do Begin {所有顶点 d 值设为∞, P 指针置空}
    Gr[i]. d := Maxint; Gr[i]. p := 0
  End; {for}
  Gr[root]. d := 0; {源点的 d 值为 0}
  For i := 1 to N - 1 Do {依次对图中的每条边进行 N-1 次松驰操作}
    For u := 1 to N Do Begin {依次考虑与顶点 1..顶点 N 相连的边}
        p := Gr[u]. nbs; {求顶点 U 的邻接表指针}
        While p <> nil Do Begin {依次对相连 U 的每一条边进行松驰操作}
          Relax(u, p^.k, p^.w);
          p := p^. next
        End {then}
      End; {for}
  For u := 1 to N Do Begin {依次分析每一个顶点}
    p := Gr[u]. nbs; {求顶点 U 的邻接表指针}
    While p <> nil Do Begin
      If Gr[p^. k]. d > Gr[u]. d + p^. w Then Begin
{若(U, V)中 D[V]未递减到实际最短路径的权, 则说明(U, V)在负边回路上, 返回 false}
        Bellman_Ford := False; Exit
      End; {then}
      p := p^.next
    End {while}
```

```
  End; {for}
   Bellman_Ford := True
{源点到各顶点的最短路径估计都递减到实际最短路径的权，则返回 true}
End; {bellman_ford}

Procedure Print(i : Integer);
{递归打印源点至顶点 i 的最短路径}
Begin
  If i = root Then Write(root)
    Else Begin
      Print(Gr[i]. p);
      Write ('——→ ', i)
    End {else}
End; {print}

Begin
  Init; {输入和构造图的邻接矩阵}
  If Not Bellman_Ford Then Writeln ('Error!') {若存在负边回路,则失败退出}
    Else For i := 1 to N Do Begin {否则打印源点至各顶点的最短路径及其权}
           Print(i);
           Writeln('d[', i, ']= ', Gr[i]. d)
         End {for}
End. {main}
```

二、每对顶点间的最短路径问题

现在我们再来讨论图中每对顶点间的最短路径问题。当然，可以把单源最短路径算法运行 |V| 次来求每对顶点间的最短路径，每次依次把图中的每个顶点作为源点。如果所有边非负，则对每个顶点运行一次 dijkstra 算法；若允许权为负的边存在，则对每个顶点运行一次 bellman_ford 算法。但这种算法的运行效率不如人意。下面我们将介绍几个时效更快的算法，这些算法对边的权的正负不加限制，可分别适用于中等规模的稠密有向图和大规模的稀疏有向图。

【例 2】flod_warshall 算法

输入有向图的权矩阵 W，其中 W[i, j] 为边(i, j)的权。算法输出两个矩阵 d 和 p: 其中 d[i, j] 是从顶点 i 到顶点 j 的最短路径的权。若顶点 i 与顶点 j 之间不存在通路，则 d[i, j]=∞；p[i, j] 表示 i 至 j 的最短路径上 j 的某个父母。若 i=j 或从 i 到 j 没有通路，则 p[i, j] 为 ∞。

算法分析:

P 矩阵实际上是每对顶点间最短路径的指针，有了 P[i, j]，我们就可以沿 P 指针递归

打印从顶点 i 到顶点 j 的一条最短路径，该最短路径的权为 d[i, j]:

```
procedure print_all_pairs_shortest_path (p, i, j);
begin
if i=j then  打印 i
        else if p[i, j]= ∞
                then  打印 i 与 j 之间不存在通路
                else begin
                        print_all_pairs_shortest_path(p, i, p[i, j]);
                        打印 j
                        end; {else}
end; {print_all_pairs_shortest_path}
```

flod_warshall 算法利用了当前最短路径 ($V_1, \ldots, V_k, \ldots, V_l$) 上的“中间”顶点 V_k。这个中间顶点是除 V_1 和 V_l 以外的任何顶点，即 $V_k \in \{V_2, \ldots, V_{l-1}\}$。

设 G 的顶点为 V={1, 2, …, n}，对任意一对顶点 i, j∈V，考察从 i 到 j 且其中间顶点皆属于集合 { 1, . . k } 的所有路径，其中的一条最短路径设为 p。

若 K 不是路径 P 的中间顶点，则 P 的所有中间顶点皆在集合 { 1, …, k−1 } 中。因此从顶点 i 到顶点 j 且满足所有中间顶点皆属于集合 { 1, …, k−1 } 的最短路径，也同样是从 i 到 j 且满足所有中间顶点皆属于集合 {1, … , k} 的一条最短路径;

若 K 是路径 P 的中间顶点。我们把 P 分解成 $P_1=\{i, \ldots, k\}$ 和 $P_2=\{k, \ldots, j\}$。显然 P_1 是从i 到 k 的一条最短路径且满足所有中间顶点均属于集合 { 1,…,k−1}。类似的，P_2 是从 k 到 j 的一条最短路径且其所有中间顶点皆属于集合 { 1,..,k−1}。

基于上述观察，我们得出了一个递推最短路径的公式:

设 $d_{ij}^{(k)}$ —— 从顶点 i 到顶点 j 且满足所有顶点均属于集合 { 1,…, k }的一条最短路径的权。显然 k=0 时，从顶点 i 到顶点 j 的路径中根本不存在中间顶点，因此它至多包含一条边，即 $d_{ij}^{(0)}=w_{ij}$。

$p_{ij}^{(k)}$ —— 从顶点 i 到顶点 j 且满足所有顶点均属于集合 {1,…, k}的一条最短路径上 j 的父母。 显然 k=0 时，若 (i, j) 边存在，则 $p_{ij}^{(0)}=i$; 否则 $p_{ij}^{(0)}=\infty$。

$$d_{ij}^{(k)}=\begin{cases} w_{ij} & \text{若 } k=0; \\ \min(d_{ij}^{(k-1)},\ d_{ik}^{(k-1)}+d_{kj}^{(k-1)}) & \text{若 } k>0; \end{cases}$$

$$p_{ij}^{(k)}=\begin{cases} i & (k=0)\text{且}((i,j)\text{ 边存在}); \\ \infty & (k=0)\text{ 且}((i,j)\text{ 不存在}); \\ p_{ij}^{(k-1)} & (k>0)\text{且}(d_{ij}^{(k-1)} \leqslant d_{ik}^{(k-1)}+d_{kj}^{(k-1)}); \\ p_{kj}^{(k-1)} & (k>0)\text{且}(d_{ij}^{(k-1)}>dik^{(k-1)}+dkj^{(k-1)})。 \end{cases}$$

矩阵 $D^{(n)}=(d_{ij}^{(n)})$ 出了最后解答。其中 $d_{ij}^{(n)}$ 为一对顶点 i、j 间的最短路径长度。调用 print_all_pairs_shortest_path(p, i, j) 就可以从 p[i, j] 指针出发递归输出从顶点 i 至顶点 j 的最短路径。这对所有 i, j 都成立，因为其所有中间顶点皆属于集合{1..n}。

例如我们用 flod_warshall 算法计算图 5.4-3 的有向图中每对顶点的最短路径。

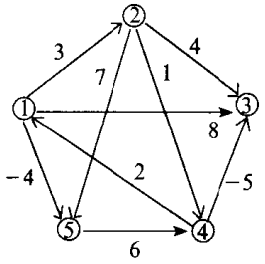


图 5.4-3

运行结果如下：

$$D^{(0)}=\begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \qquad P^{(0)}=\begin{bmatrix} \infty & 1 & 1 & \infty & 1 \\ \infty & \infty & \infty & 2 & 2 \\ \infty & 3 & \infty & \infty & \infty \\ 4 & \infty & 4 & \infty & \infty \\ \infty & \infty & \infty & 5 & \infty \end{bmatrix}$$

$$D^{(1)}=\begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & 2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \qquad P^{(1)}=\begin{bmatrix} \infty & 1 & 1 & \infty & 1 \\ \infty & \infty & \infty & 2 & 2 \\ \infty & 3 & \infty & \infty & \infty \\ 4 & 1 & 4 & \infty & 1 \\ \infty & \infty & \infty & 5 & \infty \end{bmatrix}$$

$$D^{(2)}=\begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \qquad P^{(2)}=\begin{bmatrix} \infty & 1 & 1 & 2 & 1 \\ \infty & \infty & \infty & 2 & 2 \\ \infty & 3 & \infty & 2 & 2 \\ 4 & 1 & 4 & \infty & 1 \\ \infty & \infty & \infty & 5 & \infty \end{bmatrix}$$

$$D^{(3)}=\begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \qquad P^{(3)}=\begin{bmatrix} \infty & 1 & 1 & 2 & 1 \\ \infty & \infty & \infty & 2 & 2 \\ \infty & 3 & \infty & 2 & 2 \\ 4 & 3 & 4 & \infty & 1 \\ \infty & \infty & \infty & 5 & \infty \end{bmatrix}$$

$$D^{(4)}=\begin{bmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix} \qquad P^{(4)}=\begin{bmatrix} \infty & 1 & 4 & 2 & 1 \\ 4 & \infty & 4 & 2 & 1 \\ 4 & 3 & \infty & 2 & 1 \\ 4 & 3 & 4 & \infty & 1 \\ 4 & 3 & 4 & 5 & \infty \end{bmatrix}$$

$$D^{(5)}=\begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix} \qquad P^{(5)}=\begin{bmatrix} \infty & 3 & 4 & 5 & 1 \\ 4 & \infty & 4 & 2 & 1 \\ 4 & 3 & \infty & 2 & 1 \\ 4 & 3 & 4 & \infty & 1 \\ 4 & 3 & 4 & 5 & \infty \end{bmatrix}$$

图 5.4-4

flod_warshall 算法的程序题解如下。从直观上看，可采用三维数组 D(k, i, j) 存储 $d_{ij}^{(k)}$。但实际上表示上标的一维数组是不必要的，可通过 k 循环中对 D(i, j) 重新赋值来递推 $d_{ij}^{(k)}$。

```
Program All_Pairs_Shortest_Path_as_multiplication;

Const
  Maxn                          =  100;

Type
  Node                          =  Record {矩阵解的元素类型}
                                     p, d        :  Integer
                                     {路径的权值和指针}
                                   End;

Var
  N, u, v                       :  Integer;
  F                             :  Text;
  Gr                            :  Array [1..Maxn, 1..Maxn] of Node;
{D矩阵和P矩阵解的组合。其中GR[i, j]. d——从i到j的最短路径的权;GR[i, j].p——
i到j的最短路径上j的父母}

Procedure Init;
Var
  i, j          :  Integer;
Begin
  Assign(F, 'INPUT. DAT'); {输入文件名串与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N); {读入顶点数}
  For i := 1 to N Do
    For j := 1 to N Do Begin
      Read(F, Gr[i, j]. d);
{依次读入每条边(i, j)的权，并作为初值赋予GR[i, j]. d}
      Gr[i, j]. p := i{GR[i, j]. p的初值为i}
    End {for}
End; {init}

Procedure Floyd; {递推D(n)和P(n)}
Var
  i, j, k       :  Integer;
```

```
Begin
  For k := 1 to N Do
    For i := 1 to N Do
      For j := 1 to N Do
        If (Gr[i, k]. d < Maxint) And (Gr[k, j]. d < Maxint) And
           (Gr[i, j]. d > Gr[i, k]. d + Gr[k, j]. d) Then Begin
           Gr[i, j]. d := Gr[i, k]. d + Gr[k, j]. d;
           Gr[i, j]. p := Gr[k, j]. p
        End {then}
End; {floyd}

Procedure Print(u, v : Integer); {沿 P 指针递归打印从 U 到 V 的最短路 }
Begin
  If u = v Then Write(u)
     Else Begin
       Print(u, Gr[u, v]. p);
       Write( ' ——→ ', v)
     End {else}
End; {print}

Begin
  Init; {读入图的邻接矩阵并求出 D(0) 和 P(0)}
  Floyd; {递推 D(n) 和 P(n)}
  For u := 1 to N Do Begin {输出图的邻接矩阵 }
    For v := 1 to N Do Write(Gr[u, v]. d: 8);
    Writeln
  End; {for}
  Repeat
    Write( ' u v = '); {读入顶点对}
    Readln(u, v); {输出两个顶点间的最短路径 }
    If (u <> 0) Then Print(u, v);
    Writeln
  Until u = 0 {重复，直至 U 为 0 为止 }
End. {main}
```

下面我们把问题再简化一些:对于任何顶点对 i, j ∈ V，只要判断是否存在一条从 i 到 j 的路径，而不需要了解最短路径的权和构成。显然读者可以直接得出问题的算法：对图中的每条边赋权为 1，然后对该图运行 floyd _ warshall 算法。如果从顶点 i 到顶点 j 存在一条

通路，我们将得到 $d_{ij}<n$; 否则，得到 $d_{ij}=\infty$。但对于简化后的问题而言，这种算法如同“杀鸡用牛刀”，得不偿失。下面我们来介绍一种类似的方法，这种方法在实际应用中可以节省时间和空间。

【例 3】求有向图的传递闭包

输入图 G，输出矩阵 T=[t(i, j)]。若 t[i, j]=1，则 i 到 j 有通路；若 t[i, j]=0，则 i 到 j 无通路。

算法分析:

所谓有向图的传递闭包是指可达性矩阵 T=[t(i, j)] 所对应的图 G*，其中

$$t[i,j]=\begin{cases}1 & \text{从 i 到 j 至少存在一条通路;}\\ 0 & \text{从 i 到 j 无通路。}\end{cases}$$

求有向图传递闭包的算法将 floyd _ warshall 中的算术运算操作 min 和 + 用相应逻辑运算∧和∨代替。对于 i, j 和 k=1, .. n, 如果图 G 中从顶点 i 到顶点 j 存在一条通路且所有中间顶点均属于集合 {1, .., k}，则定义 $t_{ij}^{k}=1$，即 (i, j) 边加入传递闭包 G*；否则 $t_{ij}^{k}=0$。

t_{ij}^{k} 的递归定义为

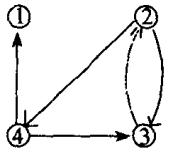
$$t_{ij}^{(0)}=\begin{cases}0 & \text{若 } i\neq j \text{ 且 } (i,j)\notin E;\\ 1 & \text{若 } i=j \text{ 或 } (i,j)\in E;\end{cases}$$

且对于 $K\geqslant 1$

$$t_{ij}^{(k)}=t_{ij}^{(k-1)}\vee(t_{ik}^{(k-1)}\wedge t_{kj}^{(k-1)})$$

由于布尔值的存储量少于整数值，位逻辑操作的执行速度快于对整数字长的数据的算术运算操作，因此传递闭包算法无论在时间上和空间上都优先于 floyd _ warshall 算法。

图 5.4-5 例举了传递闭包算法对一个有向图计算 $T^{(n)}$ 的过程。



$$T^{(0)}=\begin{pmatrix}1&0&0&0\\0&1&1&1\\0&1&1&0\\1&0&1&1\end{pmatrix}\quad T^{(1)}=\begin{pmatrix}1&0&0&0\\0&1&1&1\\0&1&1&0\\1&0&1&1\end{pmatrix}$$

$$T^{(2)}=\begin{pmatrix}1&0&0&0\\0&1&1&1\\0&1&1&1\\1&0&1&1\end{pmatrix}\quad T^{(3)}=\begin{pmatrix}1&0&0&0\\0&1&1&1\\0&1&1&1\\1&1&1&1\end{pmatrix}\quad T^{(4)}=\begin{pmatrix}1&0&0&0\\1&1&1&1\\1&1&1&1\\1&1&1&1\end{pmatrix}$$

图 5.4-5

下面给出程序题解:

```
Program All _ Pairs _ Shortest _ Path _ as _ multiplication;
cosnt
Maxn                    =   100; {最多顶点数}
```

```
Var
  N, u, v                    :  Integer; {实际顶点数和顶点序号}
  F                          :  Text; {文件变量}
  Gr                         :  Array [1..Maxn, 1..Maxn] of 0..1;
                                {邻接矩阵和传递闭包}

Procedure Init;
Var
  i, j          :  Integer;
Begin
  Assign(F, 'INPUT. DAT'); {文件名串和文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N); {读入顶点数}
  FillChar(Gr, Sizeof(Gr), 0); {邻接矩阵初始化}
  For i := 1 to N Do Gr[i, i] := 1; {读入邻接矩阵,传递闭包初始化}
  While Not Eof(F) Do Begin
    Readln(F, i, j);
    Gr[i, j] := 1
  End; {while}
  Close(F)
End; {init}

Procedure Transtive_Closure; {递推传递闭包 T(n)}
Var
  i, j, k       :  Integer;
Begin
  For k := 1 to N Do
    For i := 1 to N Do
      For j := 1 to N Do Gr[i, j] := Gr[i, j] Or (Gr[i, k] And Gr[k, j])
End; {transtive_closure}

Begin
  Init; {输入邻接矩阵, 传递闭包初始化}
  Transtive_Closure; {递推传递闭包 T(n)}
  For u := 1 to N Do Begin {输出传递闭包 T(n)}
    For v := 1 to N Do Write(Gr[u, v]: 2);
    Writeln
  End; {for}
```

```
Writeln ('************')
End. {main}
```

floyd _ warshall 算法适用于在一个中等规模的有向图中求每对顶点间的最短路径。如果在一个大规模的稀疏图上运行 floyd _ warshall 算法，其运行效率就不怎么理想了。下面，我们介绍一种关于稀疏图的 johnson 算法。

【例 4】johnson 算法

该算法输入一个稀疏图 G。算法执行后要么返回关于每对顶点间最短路径的权的矩阵 D，要么报告输入图中存在一负权回路。

算法分析:

johnson 算法运用了重赋权技术，一开始便构造了一个新的图 G'。G'中每条边的权非负，并且若原图 G 中从 U 到 V 有一条最短路径，则新图 G' 中同样对应一条由 U 到 V 的最短路径。

构造新图 G' 的办法是:

(1)增加一个新顶点 S;

(2)从 S 出发依次向原图 G 的每一个顶点引一条辅助边，该边的权为 0;

(3)从 S 出发对 G' 运行一次 bellman _ ford 算法。若算法报告 G' 包含一条负权回路，则 G 必包含负权回路，johnson 算法终止。否则

(4)新图 G' 中每条边的权 W' (U, V) 修正为

$$W'(U, V) = W(U, V) + (h(U) - h(V))$$

其中 h(U) 和 h(V) 分别为 bellman _ ford 算法计算出的从 S 到 U 和从 S 到 V 的两条最短路径的权。

例如，依照上述方法对图 5.4 - 3 的有向图重赋权后形成新图 G'。

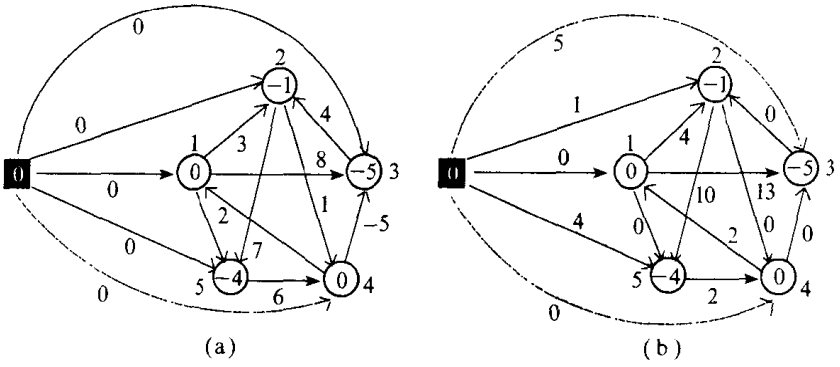


图 5.4-6

图 5.4-6 (a)和(b)中的黑色顶点为新顶点 s。顶点内的数字为 s 至该顶点 v 的最短路径长度h(v)。图(a)中各边的权依据公式 $w'(U, V) = w(u, v) + (h(u) - h(v))$ 进行了修正后形成了图(b) 中各边的权。

接下去把原图 G 中的每个顶点 u($u \in v(G)$)逐次作为源顶点调用 dijkstra 算法，计算出在 G' 中从 u 出发到其它顶点 v 的最短路径的权的值 d' (u, v)。由于图 G' 中 (u, v)边的权比原图 G 增加了 h(u)−h(v)，因此必须还原，即最后结果 d(u, v)=d' (u, v) −(h(u)−

h(v))。

我们对图 5.4 - 6 中形成的 G'继续运算的过程如下:

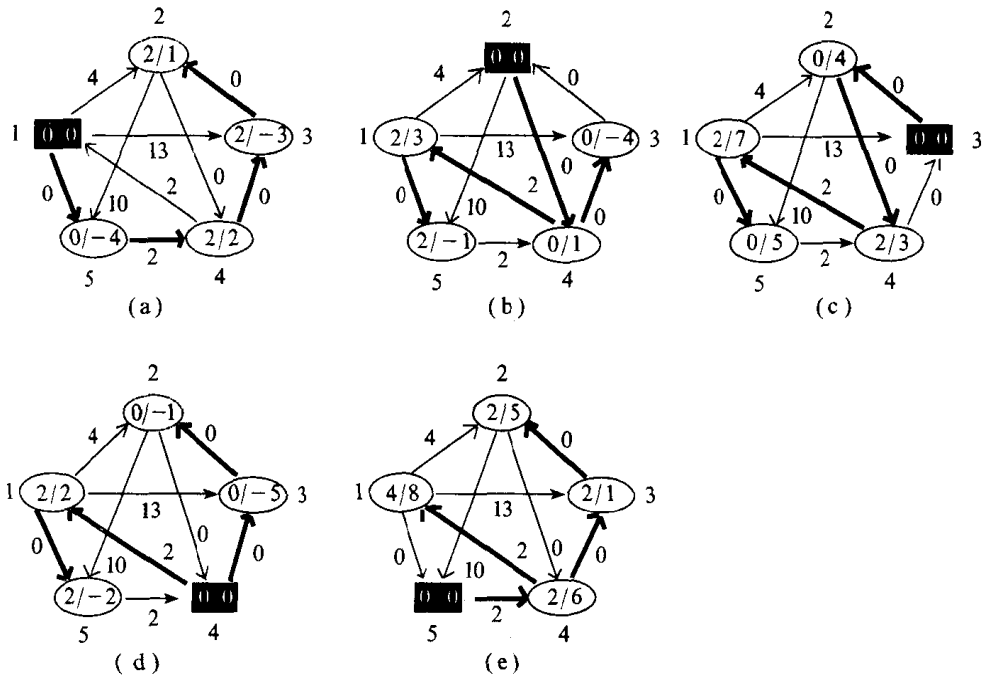


图 5.4 - 7

各图中的黑色顶点为当前调用 dijkstra 算法的源顶点; 黑色边表示从源点至其它顶点的最短路径; 顶点内的两个数字分别对应从源点至该顶点的最短路径在 G' 和 G 中的权。

下面是程序题解。

```
Program Johnson_Algorithm;

Const
  Maxn                  =   100;

Type
  Link                  = ^Node; {链 (邻接表) 指针}
  Node                  =   Record {链结点}
                              k, w, w2    :  Integer;
                            {顶点序号, 边的权, 权的修正值}
                              next        :  Link {后继指针}
                            End;
  GrType                =   Record
                              nbs         :  Link; {邻接表指针}
                              h           :  Integer
```

```
                        {新图中源点 N+1 至当前点的最短路径的权}
                        End;
  StType          =     Record {Q 堆的元素类型}
                          v, d, p     :  Integer
                        {顶点序号，最短距离估计，父顶点序号}
                        End;
  ListType        =     Record {顶点对最短路径矩阵的元素类型}
                          p, d        :  Integer {父指针，最短路径的权}
                        End;

Var
  N, u, v         :  Integer; {实际顶点数，顶点对序号}
  F               :  Text; {文件变量}
  Gr              :  Array [1..Maxn] of GrType; {邻接矩阵}
  Q               :  Array [1..Maxn] of StType; {堆}
  Q_pos           :  Array [1..Maxn] of Integer;
                     {Q-POS[I]-顶点 i 在 Q 堆中的位置}
  List            :  Array [1..Maxn, 1..Maxn] of ListType;
                     {顶点对的最短路径矩阵}
  Hs              :  Integer; {堆长度}

Procedure Insert(u, v, w : Integer);
  {将权为 W 的边 (U, V) 插入顶点 U 对应的邻接表}
Var
  x               :  Link;
Begin
  New(x);
  x^.k := v; x^.w := w;
  x^.next := Gr[u].nbs; Gr[u].nbs := x
End; {insert}

Procedure Init;
Var
  i, u, v         :  Integer;
Begin
  Assign(F, 'INPUT.DAT'); {输入文件名与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N); {读入顶点数}
```

```
 For i := 1 to N Do Gr[i]. nbs := nil; {所有顶点的邻接表初始化}
  While Not Eof(F) Do Begin {依次读入每一条边及其权，建立邻接表GR}
    Readln (F, u, v, i);
    Insert (u, v, i)
  End; {while}
  Close (F) {关闭文件}
End; {init}

Procedure Swap (a, b : Integer); {Q堆中第a个顶点与第b个顶点交换}
Var
  i              : Integer;
  t              : StType;
Begin
  t := Q[a]; Q[a] := Q[b]; Q[b] := t;
  i := Q_pos[Q[a]. v]; Q_pos[Q[a]. v] := Q_pos[Q[b]. v];
  Q_pos[Q[b]. v] := i
End; {Swap}

Procedure Decrease (v, k : Integer);
{将Q堆中第V个顶点的D值设为K，并在Q中将该顶点调整至合适位置}
Begin
  Q[v]. d := k;
  While (v <> 1) And (Q[v]. d < Q[v div 2]. d) Do Begin
    Swap(v, v div 2);
    v := v div 2
  End {while}
End; {decrease}

Procedure Heapfy (i : Integer); {堆排序，使得以i为根的子树成为一个堆}
Var
  m              : Integer;
Begin
  m := i;
  Repeat {求顶点i与左儿子、右儿子的三个顶点间D值最小的顶点序号m}
    i := m;
    If (i * 2 <= Hs) And (Q[i * 2]. d < Q[m]. d) Then m := i * 2;
    If (i * 2 + 1 <= Hs) And (Q[i * 2 + 1]. d < Q[m]. d) Then m := i*2+1;
    If i <> m Then Swap(i, m) {顶点i的D值不是最小，则Q堆中顶点i与顶点m交
```

```
换}
  Until i = m {直至顶点 i 的 D 值最小、以 i 为根的子树已是堆为止}
End; {heapfy}

Procedure Relax(u, v, w : Integer);
{在 dijkstra 算法中对权为 W 的边 (U, V) 进行松驰操作}
Begin
  If Q[Q_Pos[v]]. d > Q[Q_Pos[u]]. d + w Then Begin
  {可通过 U 对迄今找出的到 V 的最短路径进行改进}
    Q[Q_Pos[v]]. p := u; {V 的父顶点设为 U}
    Decrease(Q_Pos[v], Q[Q_Pos[u]]. d +w) {修正 V 的 D 值并重新对 Q 进行堆排序}
  End {then}
End; {relax}

Procedure Dijkstra(s : Integer); {计算源点 S 至各顶点的最短路径}
Var
  i, u          : Integer;
  p             : Link;
Begin
  For i := 1 to N Do Begin {N 个顶点进入 Q 队列，每个顶点的最短路径估计为 ∞}
    Q[i]. v := i; Q[i]. d := Maxint;
    Q_Pos[i] := i
  End; {for}
  Q[s]. d := 0; Swap(1, s); {源点的 D 值设为 0 并移至 Q 表首}
  Hs := N; {Q 表长设为 N}
  While Hs <> 0 Do Begin {若 Q 表非空，则循环}
    u := Q[1]. v; {移出 Q 的表首顶点，Q 的表长减 1}
    Swap(1, Hs);
    Dec(Hs);
    Heapfy(1); {对队列 Q 按 D 值递增的顺序进行堆排序}
    p := Gr[u]. nbs; {求表首顶点 U 的邻接表指针}
    While p <> nil Do Begin
  {对 Q 队列中的所有与 U 相连的边进行松驰操作}
      If Q_Pos[p^.k] <= Hs Then Relax(u, p^.k, p^.w2);
      p := p^.next
    End {while}
  End {while}
End; {dijkstra}
```

```
Procedure Relax_B(u, v, w : Integer);
{bellman_ford 算法中对权为 W 的边 (U, V) 进行松弛操作}
Begin
  If Q[v].d > Q[u].d+w Then Begin
    Q[v].d := Q[u].d+w;
    Q[v].p := u
  End {then}
End; {relax_b}

Function Bellman_Ford(s, N : Integer): Boolean;
{输入源点 S 和顶点数 N，若图中存在负边回路，则返回 false；否则计算出 S 至各顶点的最
短路径并返回 true}
Var
  i, u          : Integer;
  p             : Link;
Begin
  For i := 1 to N Do Begin {所有顶点的 D 值设为 ∞，P 指针置空}
    Q[i].d := Maxint; Q[i].p := 0
  End; {for}
  Q[s].d := 0; {源点的 D 值为 0}
  For i := 1 to N - 1 Do {依次对图中的每条边进行 N-1 次松弛操作}
    For u := 1 to N Do Begin
        p := Gr[u].nbs;
        While p <> nil Do Begin
          Relax_B(u, p^.k, p^.w);
          p := p^.next
        End {then}
      End; {for}
  For u := 1 to N Do Begin {依次分析每个顶点}
    p := Gr[u].nbs; {求顶点 U 的邻接表指针}
    While p <> nil Do Begin
      If Q[p^.k].d > Q[u].d+p^.w Then Begin
{若 (U, V) 中的 D[V] 未递减到实际最短路径的权，则说明 (U, V) 在负边回路上，返
回 false}
        Bellman_Ford := False; Exit
      End; {then}
      p := p^.next
    End {while}
```

```
  End ; {for}
  Bellman_Ford := True
  {源点到各顶点的最短路径估计都递减到实际最短路径的权，则返回 true}
End ; {bellman_ford}

Procedure Johnson;
Var
  u, v          : Integer;
  p             : Link;
Begin
  Gr[N+1].nbs := nil; {新增顶点 N+1 的邻接表为空}
  For u := 1 to N Do Insert(N + 1, u, 0);
  {从顶点 N+1 出发依次向原图的每一个顶点引一条权为 0 的辅助边．这些边进入顶点
N+1 的邻接表}
  If Not BellMan_ford (N + 1, N + 1) Then Writeln (' Error! Circle')
  {从顶点 N+1 出发对新图进行一次 bellman_ford 算法。若算法报告新图有负权回路，
则失败退出}
    Else Begin
      For u := 1 to N Do Gr[u].h := Q[u].d;
     {求从顶点 N+1 至每个顶点的最短路径的权 H}
      For u := 1 to N Do Begin
     {将每条边(U, V)的权修正为 W(U, V) + H(U) - H(V)}
        p := Gr[u].nbs;
        While p <> nil Do Begin
          p^.w2 := p^.w + Gr[u].h - Gr[p^.k].h;
          p := p^.next;
        End {while}
      End ; {for}
      For u := 1 to N Do Begin {每个顶点 U 逐次作为源点调用 dijkstra 算法}
        Dijkstra(u);
        For v := 1 to N Do Begin {U 至每个顶点的最短路径的权还原，并设定 P 指针}
          List[u, Q[v].v].d := Q[v].d + Gr[Q[v].v].h - Gr[u].h;
          List[u, Q[v].v].p := Q[v].p
        End {for}
      End {for}
    End {else}
End ; {johnson}
```

```
Procedure Print(u, v : Integer); {递归打印 U 顶点至 V 顶点的最短路径}
Begin
  If u = v Then Write(u)
    Else Begin
      Print(u, List[u, v]. p);
      Write ( ' ⟶ ', v)
    End {else}
End; {print}

Begin
  Init; {输入和构造图的邻接矩阵}
  Johnson; {运行 johnson 算法}
  For u := 1 to N Do Begin {输出每对顶点间最短路径的权}
    For v := 1 to N Do Write (List[u, v]. d: 8);
    Writeln
  End; {for}
  Repeat
    Write ('u v = '); {输入顶点对}
    Readln(u, v);
    If (u <> 0) Then Print(u, v); {输出顶点对之间的最短路径}
    Writeln
  Until u = 0
End. {main}
```

第六章　隐式图的基本算法

隐式图与第五章所讲的显式图不同：显式图列表给出图中的各结点及边（包括标有权的边），要求按某种规则系统地遍历图中的每个结点，以便得出图的结构信息；而隐式图仅给出初始结点、目标结点以及生成子结点的约束条件（一般由题意隐含地给出），要求将扩展规则应用于扩展结点的过程，找出其它结点，使得隐式图的足够大的一部分变成显式，直到包含目标结点为止。在构造这个隐式图的过程中，所要解决的问题是：如何从初始结点出发扩展结点，以找出一条由初始结点至目标结点的最佳的（或较好的）解答路径。

一般来讲，隐式图的基本算法有两种

1. 盲目搜索

结点的扩展次序不受目标结点位置的影响，扩展结点不估计路径代价。由于这种求解方法，原则上要求知道产生问题状态空间的全部结点及其相互关系，因此搜索效率低。

盲目搜索包括回溯法，广度优先搜索，双向广度优先搜索，分支定界法等；

2. 启发式搜索

结点的扩展次序受问题的性质和目标结点的一般方向的影响，扩展结点时要估计由初始结点经该结点至目标结点的路径代价。它原则上讲，只要求产生问题的全部状态空间的部分结点及其相互关系就可以求解问题了，因此搜索效率高。

启发式搜索包括 A* 算法，分阶段 A* 算法，博奕树等；

下面，逐一介绍这些算法。

§6.1　回溯法的讨论

需要用计算机求解的试题一般有两种类型：

1. 能够用简明正确的数学公式揭示问题；

对于这一类试题，我们尽量用解析法求解。因为一个好的数学模型建立了客观事物间准确的运算关系，运用这个数学模型求解是再合适不过了。组合数学对常见的计数问题都建立了数学模型。

2. 不能对给定问题建立数学模型，或即便有数学模型但解该模型的准确方法也不一定能运用现成算法。在要求枚举方案时，常会遇到这类问题；

对于第二类问题，我们一般采用搜索的方法解决，即从初始状态出发，运用题目给出的条件、规则扩展所有可能情况，从中找出满足题意要求的解答。回溯法是搜索算法中的一种控制策略，亦是求解特殊型计数题或较复杂的枚举题中使用频率最高的一种算法。

何谓回溯法，我们不妨通过一个具体实例来引出回溯思想与在计算机上实现的基本方法。

【例 1】n 皇后问题

一个 n× n 的国际象棋棋盘上放置 n 个皇后，使其不能相互攻击，即任何两个皇后都不能处在棋盘的同一行、同一列、同一条斜线上，试问共有多少种摆法?

一、如何求 n 皇后问题

在分析算法思路之前，先让我们介绍几个常用的概念:

1. 状态 (state)

状态是指问题求解过程中每一步的状况。n 皇后问题中，某行皇后所在的列位置 i(1 ≤ i≤ n)即为其时皇后问题的状态。显然，对问题状态的描述，应与待解决问题的自然特性相似，而且应尽量做到占用空间少，又易于用算符对状态进行运算。

2. 算符 (operater)

算符是把问题从一种状态变换到另一种状态的方法代号。 算符通常采用合适的数据来表示。N 皇后的一种摆法对应 1..n 排列方案 $(a_1,\cdots, a_n)$。排列中的每个元素 a_i 对应 i 行上皇后的列位置 (1≤ i≤ n)。由此想到，在 n 皇后问题中，采用当前行的列位置 i(1 ≤ i≤ n)作为算符是再合适不过了。由于每行仅放一个皇后，因此行攻击的问题自然不存在了，但在试放当前行的一个皇后时，不是所有列位置都适用。例如 (L, i) 位置放一个皇后，若与前 1..L−1 行中的 j 行皇后产生对角线攻击 (abs(L−j)= abs(j 行皇后的列位置− i)) 或者列攻击 (i=j 行皇后的列位置)，那么算符 i 显然是不适用的，应当舍去。因此，不产生对角线攻击和列攻击是 n 皇后问题的约束条件，即排列 (排列 $a_1,\cdots, a_i,\cdots, a_j,\cdots, a_n$)必须满足条件 ($|j-i|\neq |a_j-a_i|$) and ($i\neq a_j$) ($1\leq i,j\leq n$)。

3. 结点 (node)

用以表明某状态特征及关联方式的基本信息单元。结点的数据结构一般为记录类型 type

```
 node=record                                { 结点类型 }
  operater : 算符类型;
  state : 状态类型;
  end ;
var
   stack : array [ 1..maxdepth ] of node ;{ 结点数不超过 maxdepth 的一条路径}
```

由于 stack 数组下标 (n 皇后问题中的行序号) 指明各结点间的逻辑关系，所以毋需另辟指针域了。在 n 皇后问题中，状态和算符同指当前行的元素值，结点类型最终简化为一个短整型。描述当前方案的路径可以直接定义为:

```
var
  stack : array [ 1.. 20 ] of integer ;
```

现在让我们先来观察一个简单的 n 皇后问题。设 n=4, 初始状态显然是一个空棋盘。

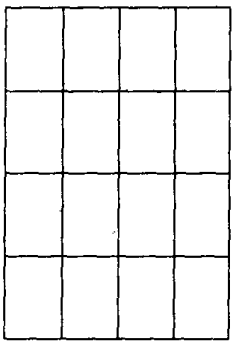


图 6.1 - 1

此时第一个皇后开始从第一行第一列位置试放，试放的顺序是从左至右，自上而下。为了说明这个事实，我们引进了 4 个结点，每个结点表征相应的状态信息，见图 6.1 - 2。

(× × × ×)

图 6.1 - 2

每个结点共有 4 个数据。第 i(1 < i < 4)个数据指明当前方案中第 i 个皇后置放在第 i 行的列位置。若该数据为 0, 表明所在行尚未放置皇后。从初始的空棋盘出发， 第 1 个皇后可以分别试放第 1 行的 4 个列位置，扩展出 4 个子结点。

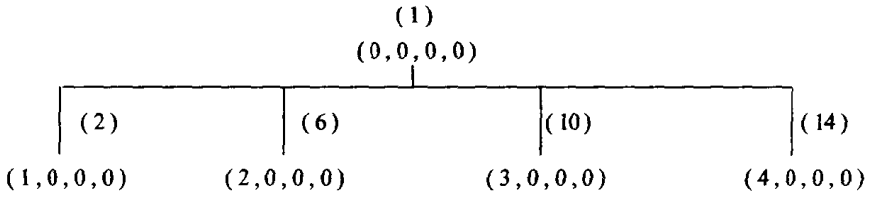


图 6.1 - 3

图 6.1 - 3 中，在结点右上方给出按回溯法扩展顺序定义的结点序号。现在我们也可以用相同方法找出这些结点的第二行的可能列位置，如此反复进行， 一旦出现新结点的四个数据全非空，那就寻到了一种满足题意要求的摆法。当尝试了所有可能方案， 即获得了问题的解答，于是得到了图 6.1 - 4 的图形。

该图形象一棵倒悬的树。其初始结点 V_1 叫根结点，而最下端的结点 V_3、V_5、V_9、V_{13}、V_{16}、V_{17}、称为叶结点，其中 4 个数据全非零的叶结点，亦即本题的目标结点。由根结点到每一个目标结点之间，揭示了一种成功摆法的形成过程。显然，4 皇后问题存在由 V_9、V_{13} 表示的二种方案。图 6.1 - 4 被称作解答树。 树中的每一结点都是当前方案中满足约束条件的元素状态。除了根结点、叶结点以外的结点都称作分枝结点。分枝结点愈接近根结点者，辈分愈高；反之，愈远离根结点者，辈分愈低。图 6.1 - 4 中结点 V_7 是结点 V_8 的父结点 (又称前件)，结点 V_{15} 是结点 V_{14} 的子结点 (又称后件)。某结点所拥有的子结点的个数称作该结点的次数。显而易见，所有叶结点的次数为 0。树中各结

点次数最大值，被称作为该树的次数。算符的个数即为结解答树的次数。由图 6.1 - 4 可见，4 皇后的解答树是 4 次树。

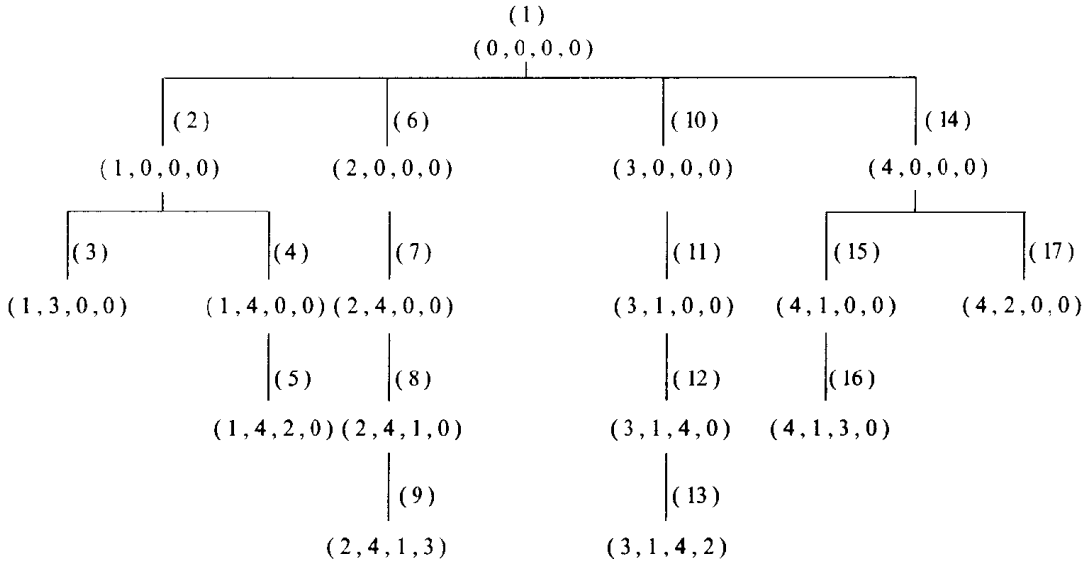


图 6.1 - 4

一棵树中的某个分枝结点也可视作为“子根”，以此结点为根的树则称作“子树”。

由以上讨论可以看出解答树的结构:

1. 初始状态构成（主）树的根结点;

2. 除根结点以外，每个结点都具有一个、且只有一个父结点。对应于 n 皇后问题来说，置放 i 行皇后的子结点，只有在置放了前 i−1 行皇后的一个父结点基础上产生;

3. 每个非根结点都有一条路径通往根结点，其路径长度（代价）定义为这条路径的边数。对应于 n 皇后来说，当前行序号即为路径代价。当路径代价为 n+1 时，说明 n 个皇后已置放完毕，一种成功的摆法产生。

有了以上的基础知识和对 n 皇后问题的初步分析，我们已经清楚地看到，求解 n 皇后问题，无非就是做两件事:

1. 从左至右逐条树枝地构造和检查解答树 t;

2. 检查 t 的结点是否对应问题的目标状态。

上述两件事同时进行。为了加快检查速度，一般规定:

1. 在扩展一个分枝结点前进行检查，只要它不满足约束条件，则不再构造以它为根的子树;

2. 已处理过的结点若以后不会再用，则不必保留。即回溯过程中经过的结点不再保留。

例如图 6.1 - 4 当我们求出第一种摆法 $V_1 - V_2 - V_3$ 后，由于皇后置放第三行任何列位置都会产生攻击，因此舍弃该摆法，开始寻求第二种摆法。从图 6.1-4 可看出，第二条路径为 $V_1 - V_2 - V_4 - V_5$，V_3 在第二种摆法中不再用到，不必保留，应当退回到 V_2 状态，从那里选择尚未使用过的列位置 4, 扩展出 V_4。一般来说，当求出一条路径后，必须从叶结点开始，沿所在路径回溯，回溯至第一个还剩有适用算符的分枝点（亦称为尚未

使用过的通向右边方向的结点），从那里换上一个新算符，继续求下一条路径。

按上述规定，对照图 6.1 - 4，我们来具体分析 4 皇后的置放过程。初始状态（0,0,0,0）作为根结点 V_1，由此出发，置第 1 个皇后于第 1 行第 1 列位置。从（1,0,0,0）开始，第 2 个皇后相继选择了第 2 行的 1、2 列位置，由于会产生攻击，因此选择该行的列位置 3 放入，产生状态（1,3,0,0）。但是第 3 个皇后无论放入第 3 行哪列位置都难逃攻击，因此只得沿第一条路径回溯至第一个尚未用过的通向右边方向的分枝点 V_2，以寻求第二种摆法。从（1,0,0,0）状态换上新的列位置 4，产生（1,4,0,0）。从（1,4,0,0）选择列位置 2（由于列位置 1 产生攻击），产生（1,4,2,0）。由于第 4 个皇后无论置放第 4 行哪列位置都会产生攻击，第二种摆法失败，同样再从 V_5 开始，沿第二条路径回溯。由于 V_2，V_4 都没有未使用的满足约束条件的算符（列位置）了，因此第一个分枝点是 V_1，从 V_1 的（0,0,0,0）换上位置 2，产生 V_6 的（2,0,0,0）。这样依次使用满足约束条件的算符扩展下去，又得出第三条路径 $V_1 - V_6 - V_7 - V_8 - V_9$。可见，$V_9$ 的（2,4,1,3）是一种成功的摆法。

按上述规律不断回溯检查，直至得出第六条路径 $V_1 - V_{14} - V_{17}$。沿路径从 V_{17} 回溯，由于 V_{14} 选择尚未用过的列位置 3、4 都会产生攻击，因此不再剩有适用的列位置了，只得回溯至 V_1。又因为 V_1 已经选择了列位置 4 而无法再扩展，至此，求出了 4 皇后的所有可能摆法。

二、回溯法的算法分析和程序框架

如上节所述，从左至右逐条树枝地构造和检查查找解答树，已处理过的结点若以后不会再使用则不必保留（一般说来，检查长度为 n 的树枝，只要保留 n 个结点就够了）。若按这种方式得到一条到达树叶的树枝 t，实际上就得到了一条路径，然后沿树枝 t 回溯到第一个尚未使用过通往右边路径方向上的分枝点，并由此分枝点向右走一步，然后再从左至右地逐个进行构造和检查，直至达到叶子为止，这时又得到一条路径。按这种方法搜索下去，直至求出所有路径。显然用这种方法检查，在树枝左边的一切结点都已检查过，树枝右边的一切结点尚未产生出来。我们把这种不断“回溯”查找解答树中目标结点的方法，称作“回溯法”。

由上述算法思想，我们很容易想到，应选择怎样一种数据结构来存放当前路径上各结点的状态和算符？它应具有“后进先出”的特征，就象食堂里的一叠盘子，每次只许一个一个地往顶上堆，一个一个地从顶上往下取。这就是我们通常所说的栈。

栈是一种线性表，所有进栈或出栈的数据都只能在表的同一端进行，就象堆盘子和拿盘子一样，都只能在顶端“堆上”或“取下”。这顶端叫“栈顶”，另一端叫“栈底”。pascal 编译系统内部，保留一部分内存用作栈区，存放过程和函数的值参以及过程和函数内部所说明的局部变量。每当一个过程和函数被启用时，系统就在栈顶分配一组值参和局部变量（进栈）。而当该过程或函数退出时，这些局部变量或值参就被消除（退栈）。

我们为回溯法设计的一个递归（自己自调用己）过程 make(L)，就是利用系统的这一特性。

1. 在全局变量说明中，分配一个连续的数组区域 stack[1..maxdepth]，顺序存放栈中表目，即当前路径的结点。栈元素为结点，一般用路径最大长度 maxdepth 作为栈容量，

直接用当前数组下标 L 指向栈顶，但 L 不允许是全局变量。

2. 栈顶指针 L 作递归程序 make 的值参数，指出待扩展结点在当前路径序列 stack 中的顺序，算符作 make 过程的局部变量。

调用 make (L) 后，栈顶指针 L 和当前结点的算符进入系统栈区，算符作用于 stack [L－ 1] 结点状态，产生当前路径序列的第 L 个结点 stack [L]；退出 make (L) 后，L 指针和 stack [L] 结点的算符出系统栈区，回溯至调用前的父结点 stack [L－1]。

3. 递归过程 make (L) 的边界条件是

① 若 stack [L] 是目标结点；

② 若 stack [L] 再无可使用的算符，即无法再扩展。

4. 只要当前结点能扩展，则递归调用 make (L＋1)，沿所在路径扩展子结点 stack [L＋1]，直至到达边界条件为止。然后通过返回调用过程的形式回溯（恢复调用前的指针 L 和算符），以寻求下一条路径。

5. 主程序中调用 make (1)，从初始结点出发回溯搜索。递归结束返回主程序时（此时L 恢复为 1) 表明所有路径搜索完毕。

```
program 程序名;
  …… …… …… ……
var { 全局变量说明}
stack:array [ 1.. maxdepth ] of node ;
  …… …… …… ……
procedure make ( L : integer );
    var
       算符的变量说明;
     …… …… …… ……
    begin
     …… …… …… ……
       make ( L+1 );
     …… …… …… ……
    end ; { make }
begin
  …… …… …… ……
   make ( 1 );
  …… …… …… ……
end . { main }
```

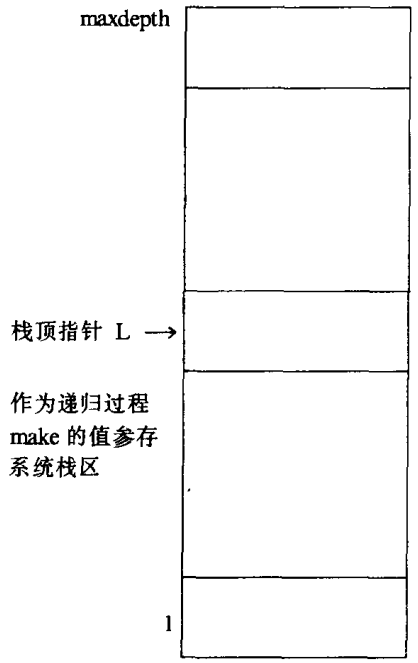


图 6.1 - 5

由于递归程序 make 中栈指针 L 和算符以值参和局部变量形式存在栈区，L 直接指明当前路径序列中所有结点间的逻辑关系，比较其它搜索算法中设置指向父辈结点的指针场的做法，其存储量小，访问速度快，结构简洁。特别是得到了一条到达树叶的树枝 t（即求得一条路径）后，按返回调用过程的形式，沿树枝 t 的路径回溯到第一个尚未用过

通向右边方向的分枝点。实际上，此时的栈中不再保留下一条路径不会再用的结点，栈中结点个数即为当前路径的实际长度，因此一般不会发生内存溢出。可以说，回溯法是求所有路径试题时可采用的算法中最好的一种。

下面给出用回溯法求所有路径的算法框架。

```
program 程序名;
const maxdepth = × × × ;
type statetype = … … … ;          { 状态类型定义 }
     operatertype = … … … ;       { 算符类型定义 }
     node = record                { 结点类型 }
          state:statetype ;       { 状态域 }
          operater : operatertype ;    { 算符域 }
          end ;
{ 注: 结点的数据类型可以根据试题需要简化 }
var
   stack : array [ 1.. maxdepth ] of node ;   { 存当前路径 }
   total : integer ;   { 路径数 }
procedure make ( L : integer );
   var
      i : integer ;    { 子结点个数 }
   begin
      if stack [ L－1 ] 是目标结点 then
         begin
            total := total + 1;      { 路径数＋1 }
            打印当前路径 [ 1.. L－1 ];
            exit ;  { 回溯。若试题仅要求一条路径，则 exit 改为 halt 即可 }
         end ; { then }
      for I := 1 to 解答树次数 do
         begin
            生成 stack [ L ]. operater ;
            stack [ L ]. operater 作用于 stack [ L－1 ]. state, 产生新状态 stack [ L ]. state ;
            if stack [ L ]. state 满足约束条件 then make ( k＋1 );
             { 若不满足约束条件，则通过 for 循环换一个算符扩展 }
              { 递归返回该处时，系统自动恢复调用前的栈指针和算符，再通过 for 循环
              换一个算符扩展 }
              { 注: 若在扩展 stack [ 1 ]. state 时曾使用过全局变量，则应插入若干语句，
              恢复全局变量在 stack [ －1 ]. state 时的值。}
         end ; { for }
```

```
        {再无算符可用，回溯}
    end ;{ make }
begin
    total:=0 ;    { 路径数初始化为 0 }
    初始化处理;
    make ( 1 );
    打印路径数 total ;
end .{ main }
```

三、应用算法框架解题

```
program queen ;
uses crt ;  { 调用屏幕单元 }
var
  stack : array [ 1.. 20 ] of byte ; { stack [ i ] ——第 i 行皇后的列位置 1 ≤ i ≤ n }
  n, total : integer ;              { 棋盘规模和摆法数 }
procedure make ( l : integer );
{ 从 l 行出发，递归搜索所有摆法 }
var i , j : integer ; { i——子结点个数，同指列位置算符; j——辅助变量 }
    att : boolean ; { 攻击标志 }
begin
  if l=n+1 then  { 产生一种成功摆法 }
   begin
    inc ( total ); writeln (' No ', total ); { 累计方案数并输出该方案 }
    for j:=1 to n do writeln (' ': stack [ j ],' * ');
    writeln ;
    exit ; { 回溯， 求下一方案 }
   end ; { then }
  for i:=1 to n do
   begin
    att : = false ; stack [ l ]: = i ; { 置非攻击标志，( L , i ) 试放一个皇后 }
    for j:=1 to l-1 do        { 检查前 L-1 行，若产生攻击，置 att 为 true }
     if ( abs ( l-j ) = abs ( stack [ j ] -i )) or ( i = stack [ j ] ) then
       begin
        att :=true ; j :=l-1;
       end ;{ then }
    if not att then make ( l+1 ); { 若 ( L , i ) 置放皇后不产生攻击，则搜索 L+1 行 }
   end ;
end ;{ make }
```

```
begin
  clrscr;{清屏}
  total:=0;{ 方案数初始化 }
  fillchar(stack,sizeof(stack),0);{ 栈初始化 }
  write('n=');readln(n);{ 读入棋盘规模 }
  make(1);{ 从第1行开始搜索所有摆法 }
  writeln('Total=':9,total);{ 打印方案数 }
end.{main}
```

四、回溯法的深入

由上述解题过程可以看出，有了回溯法的算法框架，搜索一条路径或所有路径的程序流程就比较清晰了。但这仅是在一般情况下。若求解过程中出现下述情况：

1. 求满足某个特定条件的一条最佳路径；
2. 扩展结点时参与运算的变量有多个，构成多元化的约束条件；
3. 需对当前状态分情形进行递归搜索。

就不能简单套用回溯法的算法框架了，必须具体问题具体分析。

下面，我们再举一个例题。

【例2】L形骨牌覆盖

对于任意一个 m × n 的矩阵，求L形骨牌覆盖后所剩方格数最少的一个方案。

算法分析：

根据数论的整除性质，可以得出这样一个结论：

任意 m × n(m,n ≥ 3)的矩阵，被L形骨牌覆盖后所剩的最少空格数

$$Step = \begin{cases} 4 & ((m*n)\bmod 4=0)\ \text{and}\ ((m*n)\bmod 8 <>0); \\ (m*n)\bmod 4 & \text{其它。} \end{cases}$$

本题就是求一个剩余空格数为Step的覆盖方案。

我们定义结点的状态为骨牌覆盖后的矩阵状态。一个L形骨牌若覆盖某方格，则该方格值为1，否则为0。经过旋转或翻转使之吻合的两种L形骨牌算是同一形状，共得L形骨牌的八种形状。

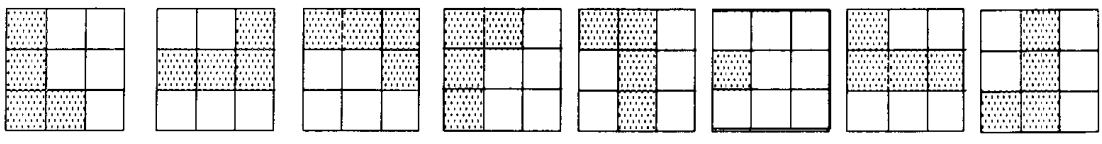


图 6.1－6

结点的算符为L形骨牌的形状序号。试放当前骨牌时，不是所有形状的骨牌都适用。若形状i(1 ≤ i ≤ 8)的骨牌放入以某空格为右下角的 3 × 3 的方阵后，伸出矩形之

外或者与其它骨牌重叠，那么算符 i 显然是不适用的，应当舍去。因此，不产生越界和重叠是算符采用的约束条件。

扩展结点时参与运算的变量主要有四个:

1. 搜索的方格座标 x，y

我们按自上而下，由左至右的顺序逐格搜索。若某格 (x，y) 为空格，就在以其为左上角的 3 × 3 方阵内，逐一试放形状 1..形状 8 的骨牌;

2. 当前骨牌覆盖后所剩的空格数 Blank

搜索前，我们根据 m、n 值预先确定最少空格数 Step，以此作为槛值。在搜索过程中，若当前的 Blank 超过这个边界值，则废弃最后一个 L 形骨牌的放法，回溯至上一格，重新试放下一形状的 L 形骨牌，直至满足空格数为 Step 的覆盖方案产生为止。

3. 已用的骨牌数 Lt

由于是按顺序逐格搜索第 Lt 块骨牌的放入位置，因此，每搜索一格需递归调用一次。若当前格被覆盖，则递归搜索下一格 (x'，y') 时，Blank 和 Lt 不变; 若任何形状的 L 形骨牌未能放入某空格对应的 3 × 3 方阵，则递归搜索下一格 (x'，y') 时，Blank + 1, Lt 不变; 若当前空格对应的 3 × 3 方阵放入某形状的骨牌，则递归搜索下一格 (x'，y') 时，Blank 不变，Lt + 1。

显然: 搜索过程与上述四个变量的当前值密切相关，因此我们将它们作为递归过程 Run 的四个值参。

```
Run (x, y, Blank, Lt);
begin
  if Blank > Step then Exit;
  if 搜索了矩阵的所有格子 then 打印结果并退出程序;
  if (x, y) 已覆盖
    then Run (x', y', Blank, Lt)
    else begin
      在 (x, y) 对应的 3 × 3 方阵中试放形状 1..形状 8 的骨牌;
      if 未能放入所有形状的骨牌
        then Run (x', y', Blank + 1, Lt)
        else begin
          在 (x, y) 对应的 3 × 3 的方阵内放入某形状的骨牌;
          Run (x', y', Blank, Lt + 1);
          恢复第 Lt + 1 块骨牌放入前的矩阵状态;
        end;
    end;
end;
```

另外，为了提高搜索效率，我们不仅在搜索前根据 m 和 n 值确定槛值 Step，而且还先设定了矩阵的行数 > 列数，即 n > m 时 n 与 m 互换。在方案产生后再将覆盖方案旋转 90°

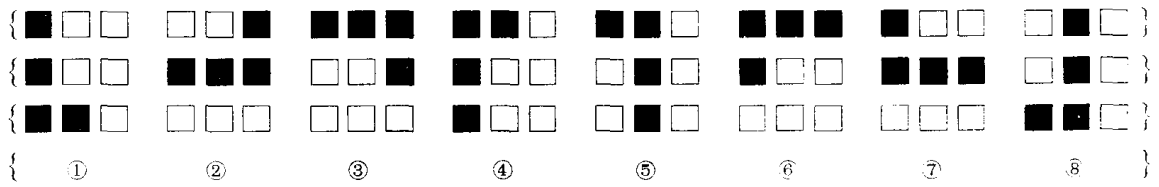
后输出，使得与输入保持一致。
下面，给出程序题解：

```
program L_gu_pai;

const
  Startxy : array [1..8] of 1..3
{ Startxy[k]——第 k 种形状的 L 形骨牌所对应的 3 * 3 矩阵的第 1 行列序号 }
          = (1, 3, 1, 1, 1, 1, 1, 2);

  Ls : array [1..8, 1..3, 1..3] of 0..1
{ Ls[k, i, j]——第 k 种形状的 L 形骨牌所对应的 3 * 3 矩阵中，(i, j)方格的值 }
      = (((1, 0, 0), (1, 0, 0), (1, 1, 0)), ((0, 0, 1), (1, 1, 1), (0, 0, 0)),
         ((1, 1, 1), (0, 0, 1), (0, 0, 0)), ((1, 1, 0), (1, 0, 0), (1, 0, 0)),
         ((1, 1, 0), (0, 1, 0), (0, 1, 0)), ((1, 1, 1), (1, 0, 0), (0, 0, 0)),
         ((1, 0, 0), (1, 1, 1), (0, 0, 0)), ((0, 1, 0), (0, 1, 0), (1, 1, 0)));

    { Ls——L 形骨牌的八种形状: }
```




```
  Maxm = 10;
  Maxn = 10;

type
  SquareType = array [1..Maxm, 1..Maxn] of integer; { 矩阵类型 }

var
  m, n, i, j, Step : integer;   { M, N  -  矩阵的长和宽        }
                                {i, j  -  辅助变量             }
                                {Step  -  最佳解中的空格数     }
  Change : boolean;             { 交换标志                     }
  Square : SquareType;          { M * N 的矩阵，即覆盖方案 }

procedure Init;
var i : integer;
begin
```

```
  Write('m, n = ');
  repeat
    Readln(m, n);
  until (m >= 1) and (m <= Maxm) and (n >= 1) and (n <= Maxn);
  if (m < 2) or (n < 2) then begin
  { 若M小于2, 或N小于2, 则该矩阵无法放入L形骨牌 }
    Writeln('Blank = ', m × n); { 矩阵为空 }
    Writeln('Total = ', 0);     { 所用L形骨牌数为0 }
    for i := 1 to m do begin    { 打印空矩阵 }
      for j := 1 to n do Write(0 : 4);
      Writeln;
    end;
    Readln;
    Halt; { 退出程序 }
  end;
  { 根据M和N, 设Step的值 }
  if ((m * n) mod 4 = 0) and ((m * n) mod 8 <> 0)
    then Step := 4      { 求出被L形骨牌覆盖后所剩的最少空格数 }
    else Step := (m * n) mod 4;
  { 交换M和N的值, 使得M >= N }
  if m < f m < n then begin
    Change := true; { 若交换了M和N的值, 则置交换标志为true }
    i := m; m := n; n := i;
  end
  else Change := false;
  FillChar(Square, Sizeof(Square), 0); { 矩阵初始化为0 }
end; { Init }

procedure Run(Posx, Posy, Blank, Lt : integer);
{ 递归搜索试放第Lt块L形骨牌的方案。(当前空格数为Blank, 试放位 }
{置为(Posy, Posx)右下角的3*3矩阵, 当前已用的L形骨牌数为Lt)}

var No : integer;

function Can : boolean;

{若当前位置(Posx, Posy)右下角的3*3矩阵能放入第No种形状的L形骨牌 }
{则返回True, 并生成覆盖方案Square; 否则返回False }
```

```
var
  sq : SquareType;
  row, col, dx, dy : integer;
begin
  Can := false;
  dy := Posy - 1;
  dx := Posx - Startxy[No];
  sq := Square;
  for row := dy + 1 to dy + 3 do
    for col := dx + 1 to dx + 3 do
      if Ls[No, row - dy, col - dx] = 1 then
        if ((row < 1) or (row > m) or (col < 1) or (col > n))
        or (sq[row, col] <> 0)
        then exit
        else sq[row, col] := Lt;
  Square := sq;
  Can := true;
end; { Can }

procedure Restore;
{废弃当前L形骨牌的放法，恢复Square }
var row, col, Max : integer;
begin
  if Posy + 3 > m then Max := m else Max := Posy + 3;
  for row := Posy to Max do
    for col := 1 to n do
      if Square[row, col] = Lt then Square[row, col] := 0;
end; { Restore }

begin
  if Blank > Step then Exit; { 空格数超过边界值，回溯 }
  if Posy >= m + 1 then begin
   { 所有行搜索完毕，则打印空格数所用骨牌数以及覆盖方案 }
    Writeln('Blank = ', Blank);
    Writeln('Total = ', Lt - 1);
    if Change
    then for j := 1 to n do begin
     { 若M和N的值被交换过，则把所得方案旋转90度后打印 }
```

```
            for i := m downto 1 do Write(Square[i, j] : 4);
            Writeln;
          end
      else for i := 1 to m do begin
      { 若M和N的值未被交换过，则把所得方案原样打印 }
            for j := 1 to n do Write(Square[i, j] : 4);
            Writeln;
          end; { for }
      Readln;
      Halt; { 退出程序 }
    end; { then }
    if Square[Posy, Posx] <> 0
     { 若当前格(Posy, Posx)已覆盖，则递归搜索下一格 }
      then Run(Posx mod n + 1, Posy + Posx div n, Blank, Lt)
      else for No := 1 to 9 do
          { 当前格(Posy, Posx)未覆盖，则递归试放形状1——形状8的L形骨牌 }
            if No = 9
        { 所有形状的L形骨牌试放失败，则(Posy, Posx)为空格，递归搜索下一格 }
              then Run(Posx mod n + 1, Posy + Posx div n, Blank + 1, Lt)
            else if Can then begin { 若L形骨牌放入，则递归搜索下一格 }
                   Run(Posx mod n + 1, Posy + Posx div n, Blank, Lt + 1);
                   Restore; { 回溯，恢复L形骨牌放入前的状态 }
                 end; { else }
end;

begin
  Init; { 初始化 }
  Run(1, 1, 0, 1);
   { 从位置(1, 1)试放第1块骨牌开始，递归搜索最佳覆盖方案 }
end. { main }
```

由于上述程序一开始就根据m和n值计算出最少空格数，并以此作为搜索边界，因此大大减少了搜索空间，提高了效率。如果没有这个边界值，盲目搜索所有放置方案，其笨拙程度是可以想象的。

§6.2 广度优先搜索

从初始结点开始，应用算符生成第一层结点，检查目标结点是否在这些后继结点中。若没有，再用算符将所有第一层的结点逐一扩展，得到第二层结点，并逐一检查第二层结

点中是否包含目标结点。若没有，再用算符逐一扩展第二层的所有结点……，如此依次扩展、检查下去，直至发现目标结点为止。这就是所谓的广度优先搜索。

在广度优先搜索中，解答树上结点的扩展是沿结点深度的“断层”进行的，也就是说，结点的扩展是按它们接近起始结点的程度依次进行的。长度为 n+1 的任一结点进行扩展之前，必须先考虑长度为 n 的每种可能的算符序列。因此，对于同一层结点来说，求解问题的价值是相同的。我们可以按任意顺序来扩展它们，这里采用的原则是先生成的结点先扩展。只要目标结点在解答树的有限层上，广度优先搜索第一次扩展到该结点，便保证找到一条通向它的最佳路径。

为了满足先生成的结点先扩展的原则，我们将存储待扩展结点的 open 表设计成队列的数据结构。

队列是一种线性表，对于它所有的插入都在表尾的一端进行，所有的删除都在表首的一端进行。如同现实生活中的等车、买票的排队，新来的成员总是加入队尾，每次离开的总是队列头上的人，即当前“最老”的成员。因此，队列也被称为“先进先出表”（FIFO 表）。

在广度优先搜索中，我们将 open 表、closed 表一起存储在一个称为 list 的数组里，数组设两个指针 open、closed，分别是 open 表的队尾指针和队首指针。当 open＝closed 时，则表示 open 表已空；当 open＝listmax 时，则表空间溢出，见图 6.2 - 1。

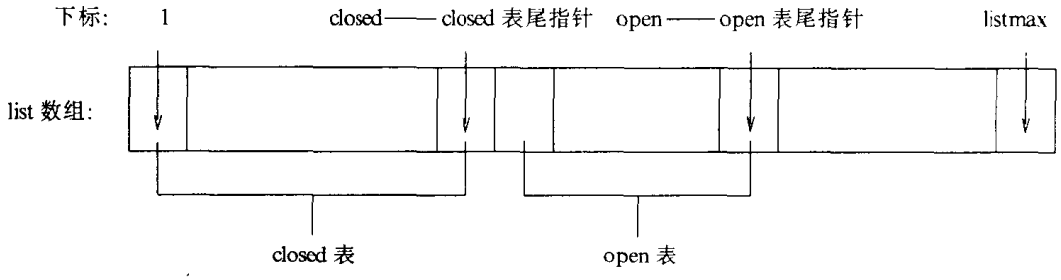


图 6.2 - 1

List 数组的元素为结点。结点的信息包括状态和指针。所谓指针实际上是指其父结点在 closed 表中的下标。

```
type
  node＝record
      state : statetype ;{状态}
      father : integer ; {指针}
      end ;
 var
 list : array [ 1.. listmax ] of  node ;
```

广度优先搜索的算法流程如下：

open ← 1; closed ← 0 ; list [open] . state ← 初始状态； list [open] . father ← 0 ;

```
while (closed<open) and (open ≤ listmax) do
        begin
        closed ← closed+1;
       expand (list[closed]);
       end;{while}
if open ≥ listmax
   then 输出内存溢出信息
   else 找到了初始结点所能到达的所有结点 list[1]..list[closed];
```

采用广度优先搜索的试题一般有两种类型。试题要求不同，扩展结点的方式（expand过程的内容）也不同：

1. 求初始结点所能到达的所有结点

```
procedure expand (q : node);
var
successor : node;
begin
 while (q 结点可扩展) and (open ≤ listmax) do
       begin
       产生 q 的一个子结点 successor;
      open ← open+1; list[open] ← successor;
      end;{while}
end;{expand}
```

显然，初始结点可达的结点个数不应超过 listmax 个。若超过，则只能求出其中 listmax 个。

2. 求初始结点到某目标结点的最短路径

求最短路径的广度优先搜索算法，在扩展结点时有如下不同之处：

（1）每扩展出一个子结点 succesor，需通过其 father 指针确认其与 list[closed] 的父子关系；

（2）需判别扩展出的子结点 succesor 是否为目标结点。若是目标结点，则从 succesor 的father 指针出发，递归输出初始结点至该结点的最短路径，并退出程序；

（3）如果产生重复子结点的概率很大，则在生成子结点 succesor 后，判别其是否在 open 表和 closed 表（list[1]..list[open]）中重复：若不重复，succesor 入队 open 表；否则由于重合在两表中的结点是位于 successor 的上面层或同一层，其路径代价必小于或等于 succesor 的路径代价，因此不再重复以 successor 为根的子树，避免了多余子结点的计算。但如果产生重复子结点的概率小，则不必进行重复判断，因为对所有子结点判断重复的代价和是相当大的。

```
procedure expand (q : node);
```

```
var
 successor : node ;
tot : integer ;
begin
 while (q 结点可扩展) and (open ≤ listmax) do
        begin
        successor . fathe ← closed ;
       if successor 是目标结点
          then begin
                 tot ← 0 ;
                print (successor );
                输出路径代价 tot－1;
                halt ;
                end ;{ then }
       if successor 不重复于 list [ 1 ] .. list [ open ]
       {若产生重复子结点的概率小时，则避免使用该判断}
          then begin
                 open ← open + 1; list [ open ] ← successor ;
                end ;{ then }
end ;{ expand }
```

其中 print 的过程说明为:

```
procedure print (q : node );
begin
 if q . father = 0 then begin {输出根结点}
                        tot ← tot + 1;
                        输出 q . state ;
                        end { then }
                  else begin
                         print ( list [ q . father ] );
                        {递归输出初始结点至目标结点 q 的路径}
                        tot ← tot + 1;
                        输出 q . state
                        end ;{ else }
end ;{ print }
```

从广度优先搜索的算法流程可以看出，若当 open 队列空时还未搜索到目标结点，则说明初始结点与目标结点之间根本不存在任何路径，失败退出。下面举两个应用的实例。

【例 1】房间问题

(图 6.2 - 2) 示出了一张建筑平面图，编程计算:

1．该建筑中有多少个房间;

2．最大的房间有多大;

3．拆除建筑中的某一堵墙，以形成一个尽可能大的房间。指出该墙。

该建筑分成 m· n 个方块 (m＜ 50, n＜ 50), 每个方块可有 0 ～ 4 堵墙 (0 表示无墙)。

输入数据:

平面图以数字形式存储在 INPUT . TXT 文件中，用一个数字表示一个方块。

● 文件开始是北南方向的方块数，其次是东西方向的方块数。

● 后面的行中，每个方块中墙的特征由数字 P 来描述 (0＜ P＜ 15)。数字 P 是下面的可能取的数字之和:

1 (西墙 west)

2 (北墙 north)

4 (东墙 east)

8 (南墙 south)

室内的墙被定义两次: 例如方块 (1, 1) 中的南墙也被位于其北面的方块 (2, 1) 定义了一次。

● 建筑中至少有两个房间。

在我们的例子中文件 INPUT . TXT 的内容为:

```
4
7
11 6 11 6 3 10 6
7 9 6 13 5 15 5
1 10 12 7 13 7 5
13 11 10 8 10 12 13
```

输出数据:

在输出文件 OUTPUT . TXT 中，

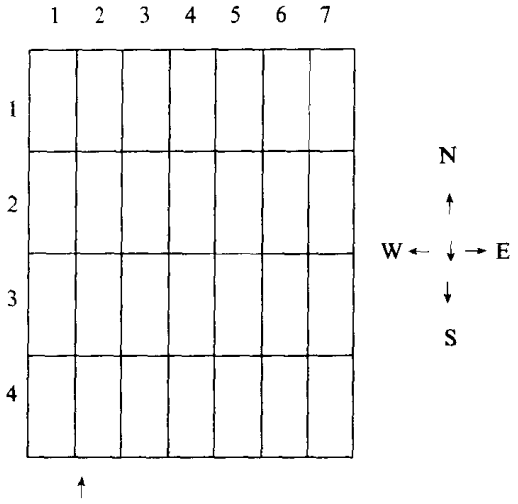
第 1 行写房间总数;

第 2 行写最大房间的面积 (方块数);

第 3 行写你的建议，指明应拆除位于哪两个方格间的一堵墙。

在我们的例子中，第 3 行是 (4, 1)(4, 2), 这是各种解中的一个，你只要给出其中一个即可。

```
5
9
(4,1)(4,2)
```



图中箭头指明被拆的墙（根据本例的输出文件）

图 6.2 - 2

算法分析:

该题要求编程者根据建筑平面图中每个方块的墙的特征字，得出

1. 房间的分布情况;

2. 求最佳拆墙方案。

问题 1 的解是解问题 2 的基础。因为只有计算出各房间拥有的方块数以及房间之间的隔墙情况，才能明确拆除哪堵墙后可形成尽可能大的房间。

1. 明确每个方块中的设墙情况

程序从文件中输入 m×n 个 0..15 的数，每个数字描述了一个方块中墙的特征——即下面的可能取的数字之和:

1. 西墙 (west)
2. 北墙 (north)
3. 东墙 (east)
4. 南墙 (south)

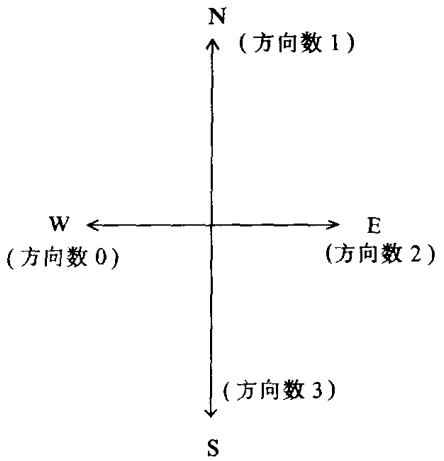


图 6.2 - 3

我们将方格中墙的特征字转换成一个四位二进制数，每一位对应一个方向

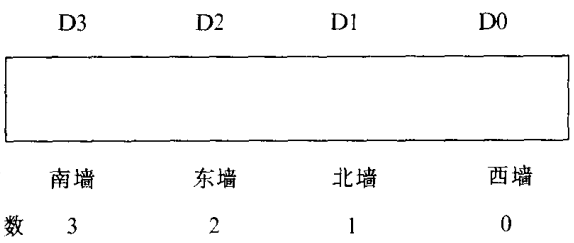


图 6.2 - 4

若 D_i 位为 1，表示该方块的 i 方向上有一堵墙；

D_i 位为 0，表示 i 方向上无墙，该方块与 i 方向上的相邻块同属一个房间；

$(1 \leqslant i \leqslant 3)$

例如：(4,7)方块的墙的特征字为 13。

$(13)_{10} = (1101)_2$

即：(4,7)方块的南面、东面、西面各有一堵墙。

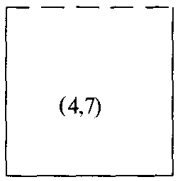


图 6.2 - 5

2. 求房间的分布

虽然我们求出了每个方块中的设墙情况，但由于除周边外的中间方块中，每堵墙被位于墙两面的方块各定义一次，因此还不能得出有多少个房间，每个房间拥有哪些方块。

接下来的问题，是将建筑平面划分为若干连通的闭区域，这种区域亦称面。面由墙围成，两两面之间无公共方块。我们对每一面着一种序号的颜色，这个颜色序号对应该面内方块所在的房间序号。对建筑平面图的着色方案构勒出建筑中房间的分布情况，其颜色数即为房间总数。

我们从(1,1)方块出发，由上而下、由左至右搜索一个目前未着色的块(x,y),运用宽度优先搜索的方法求(x,y)所在的面，对该面着一种新颜色。

在宽度优先搜索过程中设两张表:

open 表——以队列形式存储待扩展的方块。在扩展某方块时，其四个方向上的相邻块中与该方块间无墙且未确定房间号的相邻块，作为同一房间内的方块进入 open 表。然后按“先进先出”的顺序扩展这些方块。若 open 表空，则说明房间内的所有方块已确定，完成了对建筑平面图中一个面的着色；

closed 表——存储已扩展的方块。当方块的四个方向上的相邻块都已检查，该方块进入closed 表；

运用一次宽度优先搜索，可确定一个房间内的所有方块，完成一个面着色。反复搜索，直至求出所有面为止。

```
procedure main;
 begin
  搜索的初始位置为(1,1); 房间数 ← 1;
   repeat
    房间数+1;
```

```
搜索未确定房间的方块 (x, y);
  if 所有方块确定房间
    then begin
      房间数－1;输出房间数和每房间的面积; exit;
    end; {then}
  (x, y) 进入 open 表;
  (x, y) 的房间号← 房间数; 该房间的面积初始化为 1;
while open 表非空 do
    begin
    (x, y) 移入 closed 表;
    for k=0 to 3 do
     if ((x, y) 在 k 方向无墙) and (k 方向的相邻块未确定房间)
       then begin
            (x, y) k 方向的相邻块 (x', y') 进入 open 表;
            (x', y') 的房间号← 房间数; 该房间的面积＋1;
            end; {then}
    end; {while}
  until 所有方块确定房间;
end; {main}
```

例如，用一个 4×7 的矩阵存储建筑平面图，图内每个方块中墙的特征字如下:

$$\begin{bmatrix} 11 & 6 & 11 & 6 & 3 & 10 & 6 \\ 7 & 9 & 6 & 13 & 5 & 15 & 5 \\ 1 & 10 & 12 & 7 & 13 & 7 & 5 \\ 13 & 11 & 10 & 8 & 10 & 12 & 13 \end{bmatrix}$$

运用 main 算法过程，可得出建筑平面图的各房间的分布情况

3. 求最佳拆墙方案

有了房间的分布情况，求最佳拆墙方案便不难了。例如，由 (图 3.2 - 5)

可以看出

房间 1 的面积 ＝9;

房间 2 的面积 ＝3;

房间 3 的面积 ＝8;

房间 4 的面积 ＝1;

房间 5 的面积 ＝7;

由于面积最大的房间 1 不与面积次大的房间 3 相邻，而与房间 5 相隔，因此只要拆除房间 1 与房间 5 的任一堵墙，即可形成面积为 9＋7＝16 的大房间。

	1	2	3	4	5	6	7
1	1	1	2	2	3	3	3
2	1	1	1	2	3	4	3
3	1	1	1	5	3	5	3
4	1	5	5	5	5	5	3

方格内的数字即房间号（面的颜色序号）

图 6.2 - 6

搜索最佳拆墙方案的算法如下:

设 best —— 目前最大房间的面积，搜索前初始化为 0;

我们按由上而下，由左至右的顺序搜索每一方块的相邻格 (i,j):

若 (i,j) 与 (i,j)k 方向 (0＜ k＜ 3) 的相邻块 (x,y) 分属两个房间且两个房间的面积和大于 best，则

best ← (i,J) 所属房间面积 + (x,y) 所属房间面积;

记下两个方块 (x,y) 和 (i,j)

按上述算法搜索了所有方块后，若 best 为 0，则不需要拆墙 (整个建筑为一个房间)；否则输出拆墙方案——拆除位于 (i,j) 和 (x,y) 间的一堵墙后，形成面积为 best 的一个最大的房间。

下面给出程序题解。

```
PROGRAM Room;
  USES
    Crt;    {调用屏幕单元}
  TYPE
    PosType          = array[1..2] of integer;{结点 (即方块坐标) 类型}
  CONST
    Dig              : array[0..3] of integer = (1,2,4,8);
                       {Dig[i]——第 i 位二进制数的权}
    Way              : array[0..3,1..2] of integer
                     = ((1,0),(0,1),(-1,0),(0,-1));
            {Way[i,1]——方向 i 的垂直增量; Way[i,2]——方向 i 的水平增量;}
    VAR
      List           : array[1..2500] of PosType;
                       {open 表和 close 表}
```

```
  Area                : array[1..2500] of integer;
                        {Area[i]——第i个房间的面积}
  M,N,rnum            : integer; {建筑平面图的规模，房间数}
  G                   : array[1..50,1..50,0..3] of integer;
                        {G[I,J,K]= 1  方块(i,J)在K方向有墙;
                                   0  方块(i,J)在K方向无墙;}
  Map                 : array[1..50,1..50] of integer;
    {若Map[i,J]=0，则(i,J)方块未被检查过，否则为(i,J)方块所属的房间号}
  f                   : text;     {文件变量}
PROCEDURE Init;
  var
    i,j,k,s : integer;
    fn : string;
  begin
    clrscr;          {清屏}
    writeln('Input File Name : '); readln(fn);     {读入文件名}
    assign(f,fn); reset(f);     {文件名与文件变量连接，读准备}
    readln(f,M,N);                  {读入建筑平面图的规模}
    for i:=1 to M do for j:=1 to N do begin
      read(f,s);                    {读入(i,j)方块的墙的特征字}
      for k← 3 down to 0 do       {将特征字转换为四周设墙的情况}
        if dig[k]<=s
        then begin
          G[i,j,k]← 1;
          dec(s,dig[k]);
        end {then}
        else G[i,j,k]← 0;
    end; {for}
    close(f);                     {关闭文件}
  end; {init}
PROCEDURE Main;
  var
    x,y,open,closed,k : integer;
  begin
    fillchar(Map,sizeof(Map),0);       {置所有方块未检查标志}
    fillchar(area,sizeof(area),0);    {所有房间的面积初始化为0}
    x← 1; y← 0; rnum← 0;                {从左上角开始搜索}
    repeat
```

```
    inc(rnum);
    repeat                                  {搜索未确定房间的方块(x,y)}
      inc(y);
      if y>N then begin y←1; inc(x); end;
    until (x>M) or (Map[x,y]=0);
    if x>M then begin
      {若所有方块都已检查，则输出房间数与每个房间的面积}
      dec(rnum);
      writeln('Number of rooms :', rnum);
      writeln('Size of each room :');
      for x:=1 to rnum do
        writeln('Room ', x,' : ', Area[x]);
      exit;
    end; {then}
    open←1; closed←0; List[1,1]←x;
     List[1,2]←y;                          {(x,y)进入open表}
    Map[x,y]←rnum; Area[rnum]←1;
      {房间数作为(x,y)的房间序号，该房间的面积初始化为1}
    while closed<open do begin          {若open表非空}
      inc(closed);                         {则(x,y)进入closed表}
      for k:=0 to 3 do        {顺序搜索(x,y)的四个方向的相邻方块}
        if (G[List[closed,1],List[closed,2],k]=0)
          and (Map[List[closed,1]+Way[k,1],List[closed,2]+Way[k,2]]=0)
        then begin     {若(x,y)在k方向无墙且k方向的相邻方块未确定房间}
          inc(open); {相邻方块进入open表}
          List[open,1]←List[closed,1]+Way[k,1];
          List[open,2]←List[closed,2]+Way[k,2];
          Map[List[open,1],List[open,2]]←rnum;
            {当前房间数作为相邻方块的房间号}
          inc(Area[rnum]);       {该房间的面积+1}
        end; {then}
      end; {while}
    until True=False;        {直到检查了所有方块}
  end; {main}
PROCEDURE Search;
  var
    MaxArea, b_i, b_j, b_x, b_y, i, j, k, x, y : integer;
  {maxarea——最大房间面积; (b_i, b_j),(b_x, b_y) ——应拆墙两端的方块坐标;
```

```
    (x,y)——相邻方块坐标;; k——方向数; i,j——辅助变量}
  begin
    MaxArea ← 0;          {最大房间面积初始化为零}
    for i:=1 to M do for j:=1 to N do for k:=0 to 3 do
      if G[i,j,k]=1 then begin  {若(i,j)k方向有墙}
        x ← i+way[k,1]; y ← j+way[k,2]; {求k方向的相邻方块坐标(x,y)}
        if(x>0)and(y>0)and(x<=M)and(y<=N)and(Map[x,y]<>Map[i,j])
          and (Area[Map[x,y]]+ Area[Map[i,j]]>MaxArea)
  {若(x,y)和(i,j)分属两个房间且两个房间的面积和大于当前最大房间面积}
        then begin
             MaxArea ← Area[Map[x,y]]+Area[Map[i,j]];
             {两个房间面积和作为当前最大房间面积}
             b_i ← i; b_j ← j; b_x ← x; b_y ← y;
          {拆除(i,j)和(x,y)间的一堵墙可以形成目前最大的房间}
          end; {then}
      end; {then}
    if MaxArea=0 {若整个建筑为一个房间，则不需拆墙}
    then writeln('No walls needed to be put down !')
    else begin
     {否则， 拆除(b_i,b_j)和(b_x,b_y)间的一堵墙，形成面积最大的房间}
      writeln('Put down the wall between (',bi,',',bj,') and (',bx,',',
      by,')');
      writeln('Size of new room :', MaxArea);
    end; {else}
  end; {search}
BEGIN
  Init; {读入建筑平面图，求每个方块的设墙情况}
  Main; {求房间的分布}
  Search;{计算和输出最佳拆墙方案}
END. {main}
```

【例 2】最短编号序列

表 A 和表 B 各含 k (k≤ 20)个元素，元素编号从 1 到 k。两个表中的每个元素都是由 0、1 组成的字符串。(不是空格) 字符串的长度≤ 20。例如下面的两个表，每个都含 3 个元素 (k=3)。

表 a

元素编号	字符串
1	1
2	10111
3	10

表 b

元素编号	字符串
1	111
2	10
3	0

图 6.2 - 7

对于表 A 和表 B，存在一个元素编号的序列 2 1 1 3，分别用表 A 中的字符串和表 B 中的字符串去置换相应的元素编号，可得相同的字符串序列 1 0 1 1 1 1 1 1 0，见下表：

元素编号序列	2	1	1	3
用表 A 的字符串替换	10111	1	1	10
用表 B 的字符串替换	10	111	111	0

图 6.2 - 8

对表 A 和表 B，具有上述性质的元素编号序列称之为 S (AB)。对于上例 S (AB)=2 1 1 3。

编写程序：从文件中读入表 A 和表 B 的各个元素，寻找一个长度最短的具有上述性质的元素编号序列 S (AB)。

注意：如果对于表 A 和表 B 不存在 S (AB)，即找不到相同元素编号序列对应有相同的长度＜ 100 的由 0、1 组成的字符串序列，这时应输出“No Answer”（无解）。

输入数据：

输入文件名由键盘输入，该文件

第 1 行为 K 的值；

第 2 行至第 K＋1 行为表 A 的内容（依次是元素编号从 1 到 K 的相应 0、1 字符串）；

第 K＋2 行至第 2K＋1 行为表 B 的内容（依次是元素编号从 1 至 K 的相应 0、1 字符串）

输出数据：

输出文件名为 OUTPUT . TXT，该文件只有一行。或是 S (AB) 的值，或是“No Answer”（无解时）。S (AB) 是元素编号序列，输出时每个编号占一行。

输入输出范例：

输入文件内容:
3
1
10111
10
111
10
0

输出文件内容:
2
1
1
3

图 6.2-9

算法分析:

1. S(AB)的性质

首先我们必须明确，所谓S(AB)是指满足下述性质的元素编号序列:

(1) S(AB)中的元素编号在表A和表B存在;

(2) 取出S(AB)中的前i个元素编号 ($1 \leq i \leq K$),分别用表A中的字符串和表B中的字符串去置换，得到A_i串和B_i串，则要未A_i是位于B_i左端的子串，要未B_i是位于A_i左端的子串，两者必居其一。

例如对于试题示例中的表A和表B，存在S(AB)={2113}。取S(AB)的一个元素编号2，得到

A_1='10111'

B_1='10'

显然B_1是A_1首字符开始的一个长度为2的子串。

取S(AB)的前二个元素编号2、1，得到

A_2='101111'

B_2='10111'

同样A_i和B_i向左对齐并截去相同的子串后，剩余串要末是A_i的子串A_i'，要末是B_i的子串B_i'，两者也必居其一。显然

A_1' ='111', B_1' ='';

A_2' ='1', B_2' ='';

(3) A_i' (或B_i') + S(AB)中第i+1个元素编号对应表A(或表B)的字符串，与S(AB)中第i+1元素编号对应表B(或表A)的字符串之间也有子串关系。

例如S(AB)中的第3个元素编号为1。表A的元素1为'1'，表B的元素1为'111'

A_2' + 表A中的元素1='1'+'1'='11'

显然是表B元素1的子串。

(4) 当顺序取出S(AB)的所有N(1＜N＜K)个元素编号后，$A'_N = B'_N$。

由此可见，对于一个元素编号序列，当我们分别用表A中的字符串和表B中的字符串去顺序置换的过程中，不需要存贮目前置换的结果A_i和B_i(1＜i＜N,N＜K)，只需要记住

① 序列中待置换的第i+1个元素编号d；

② A_i的长度L_a和B_i的长度L_b。因为在L_a＞100或L_b＞100时还未找到S(AB)，应输出无解；

③ 剩余串Lev和标志K

K：1　Lev=A_i'，即A_i中剩余；

K：2　Lev=B_i'，即B_i中剩余；

有了上述条件，便可以判别出元素编号序列是为S(AB)。例如，对于元素编号序列{2113}，判别过程如下：

元素编号	用表A字符串替换	用表B字符串替换	LA	LB	lev	K
2	10111	10	5	2	111	1
1	1	111	6	5	1	1
1	1	111	7	8	1	2
3	10	0	9	9		

↓

得出{2113}为S(AB)

图6.2-9

2. 搜索S(AB)

上面我们给出了判别一个元素编号序列是否为S(AB)的方法。但试题的要求是对于任意的两张表A和B(表A和表B的元素个数以及每个元素的字串值未知)，或者输出长度最短的S(AB)或者输出无解信息。因此不可能找出一种数学解析式，只能用搜索的方法来依次确定序列中的元素编号。

设当前确定序列第i个元素编号的结点为q。结点信息包括

q.f ——父指针，指向确定序列第i-1个元素编号的结点；

q.d ——序列中第i个元素的编号；

q.La ——A_i的长度；

q.Lb ——B_i的长度；

q.Lev ——剩余串A_i(或B_i)；

q.K——剩余串标志;

q.d 只能在 1…… k 中选择同时满足下述条件的一个编号 J(1＜ J＜ K)

① (q.La+ 表 A 中元素 J 的串长＜ 100) and (q.Lb+ 表 B 中元素 J 的串长＜ 100);

② (A_i' = ' ') or (B_i' = ' ');

其中 A_i' 和 B_i' 按如下方法求出

$$Sa=\begin{cases}(q.f).Lev+\text{表 A 中的元素 J} & (q.f).K=1;\\ \text{表 A 中的元素 J} & (q.f).K=2;\end{cases}$$

$$Sb=\begin{cases}\text{表 B 中的元素 J} & (q.f).K=1;\\ (q.f).Lev+\text{表 B 中的元素 J} & (q.f).K=2;\end{cases}$$

A_i'——在 Sa 中删去位于左端的 Sb 子串后的剩余串;

B_i'——在 Sa 中删去位于左端的 Sa 子串后的剩余串;

若满足上述条件，则选择编号 J 作为序列的第 i 个元素，扩展出结点 q；否则试选 J+1 作编号。

在扩展结点的过程中，一旦发现当前结点的 (A_i'=' ') 和 (B_i'=' '), 则说明编号序列已为 S(AB), J 为 S(AB) 序列中的最后一个编号。

3. 用广度优先搜索求最短的 S(AB)

由于试题要求寻找长度最短的 S(AB), 因此不妨采用广度优先搜索的算法求解:

从空序列出发，在 1…… K 中逐一选择满足条件的编号作为序列的第一个元素，生成第一层结点；再依次扩展第一层的每一个结点，即在 1…… K 中逐一选择满足条件的编号作为序列的第二个元素，……，这样扩展出第二层；…… 如此扩展下去直至 A_i' =' '且 B_i'=' '，便找出一个最短的 S(AB)。序列中的编号由初始结点至最后一个扩展结点的路径上各顶点的 q.d 组成。

在广度优先搜索中，我们将存贮待扩展结点的 open 表、存贮已扩展结点的 close 表放在一个 list 数组里，list 的表长为 maxn。open 表采用“先进先出”的队列结构，cl 为队首指针，op 为队尾指针，当 op＞ maxn 时，list 数组空间溢出；当 op＜ cl L 时 open 表空。

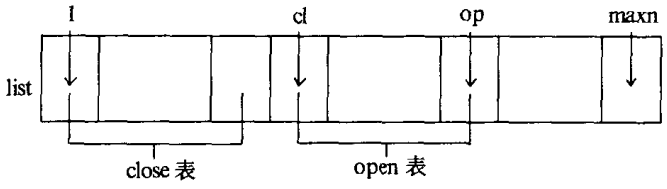


图 6.2－10

广度优先搜索的算法如下:

```
LIST[1].LEV←' ';{空序列进入 OPEN 表}
CL← 1;OP← 2;
WHILE (OP<MAXN) AND (CL<OP) DO {LIST 未溢出且 OPEN 表非空}
  BEGIN
    Q← LIST[CL];{移出 OPEN 表首结点}
    FOR I:=1 TO K DO {顺序在 1…… K 中选择序列的下一个编号}
      BEGIN
        求剩余串 A' 和 B';
        IF 编号 I 满足条件 THEN
          BEGIN
            LIST[OP].F← CL;
            LIST[OP].D← I;
            LIST[OP].LA← Q.LA+ 表 A 第 I 个元素的串长;
            LIST[OP].LB← Q.LB+ 表 B 第 I 个元素的串长;
            IF A'=' 'THEN BEGIN LIST[OP].K← 2;LIST[OP].LEV←
                          A' END;
                          ELSE BEGIN LIST[OP].K← 1;LIST[OP].LEV← B'
                          END;
            IF (A'=' ') AND (B'=' ') THEN
              BEGIN
              从 LIST[OP]结点出发沿 F 指针递归打印初始结点到 LIST[OP]结
              点路径上各结点的 B 域值
              HALT;
              END;{THEN}
            END;{THEN}
        OP← OP+1;{表尾指针后移}
      END;{FOR}
    CL← CL+1;{表首指针后移，准备移出下一个表首结点}
  END;{WHILE}
输出无解信息;
```

由上述算法流程可以看出，表 A 和表 B 的元素愈多，则扩展的结点愈多，搜索的代价愈大。广度优先搜索的最大弊端是效率低。如果从初始结点到达最近的目标结点的长度为 L，每一个结点可扩展的子结点的个数为 m,则它必须在解答树上扩展完 L－1 层的所有结点，而不管目标结点位于哪条树枝上。在整个搜索过程中，扩展的结点数为 m(m2－1)/(m－1)(略去由于不满足约束条件而不允许扩展的结点数，即以完全 m 叉树计算) 由此看出结点数基本上以指数增长，L 愈长，搜索空间愈大，时间愈慢。

为了解决搜索效率问题，我们将在下一章节中介绍一种双向广度优先搜索寻找最短的S(AB)的算法。

§6.3 双向广度优先搜索

所谓双向搜索指的是搜索沿两个方向同时进行:

正向搜索: 从初始结点向目标结点方向搜索;

逆向搜索: 从目标结点向初始结点方向搜索;

当两个方向的搜索在Y层生成同一子结点时，终止此搜索过程

$$Y=\begin{cases} \lfloor L/2 \rfloor +1 & \text{当最佳路径长度 L 为奇数时;} \\ L/2 & \text{当最佳路径长度 L 为偶数时;} \end{cases}$$

无论从正向或逆向，双向广度优先搜索一定能搜索到Y层的全部子结点。如果确实存在一条从初始结点至目标结点的最佳路径，那未双向搜索在深度为Y的搜索边域处，一定会生成同一子结点而相交，最终使双向搜索终止在最终路径上。

双向广度优先搜索过程所扩展的结点数要比单向搜索少得多。如果每一结点可扩展的子结点数为M，不计约束条件，以完全M叉树计算，单向广度优先搜索必须在解答树上扩展完L-1层的所有结点，扩展的结点数为$M(M^L-1)/(M-1)$。对于双向广度优先搜索，正向搜索的结点数为$M(M^Y-1)/(M-1)$，逆向搜索扩展的结点数为$(M^{L-Y}-1)/(M-1)$，双向搜索扩展的结点数为$M(M^Y+M^{L-Y}-2)/(M-1)$。设L为偶数(Y=L/2)，双向搜索扩展的结点数约是单向搜索的$2/(M^{L/2}+1)*100\%$，相对减少$(M^{L/2}-1)/(M^{L/2}+1)*100\%$。

双向广度优先搜索的数据结构要比单向的广度优先搜索复杂一些。由于双向广度优先搜索在搜索过程形成两棵扩展方向相反的解答树，因此必须设置四张表:

OPEN0表，CLOSED0表——存储正向搜索中产生的待扩展结点和已扩展结点;

OPEN1表，CLOSED1表——存储逆向搜索中产生的待扩展结点和已扩展结点;

四张表存储在一个称为LIST的二维数组里

LIST数组设四个整数指针CL[0],OP[0],CL[1],OP[1]

LIST[0,1]-LIST[0,CL[0]-1]存储CLISED0表;

LIST[0,CL[0]]-LIST[0,OP[0]C]存储OPEN0表;

LIST[1,1]-LIST[1,CL[1]-1]存储CLISED1表;

LIST[1,CL[1]]-LIST[1,OP[1]]存储OPEN1表;

OPEN0和OPEN1都为先进先出的队列结构，CL[0]为正向队首指针，OP[0]为正向队尾指针；CL[1]为逆向队首指针，OP[1]为逆向队尾指针。为了区分当前搜索方向，设方

$$S_t=\begin{cases} 0 & \text{当前处于正向搜索;} \\ 1 & \text{当前处于逆向搜索;} \end{cases}$$

向标志

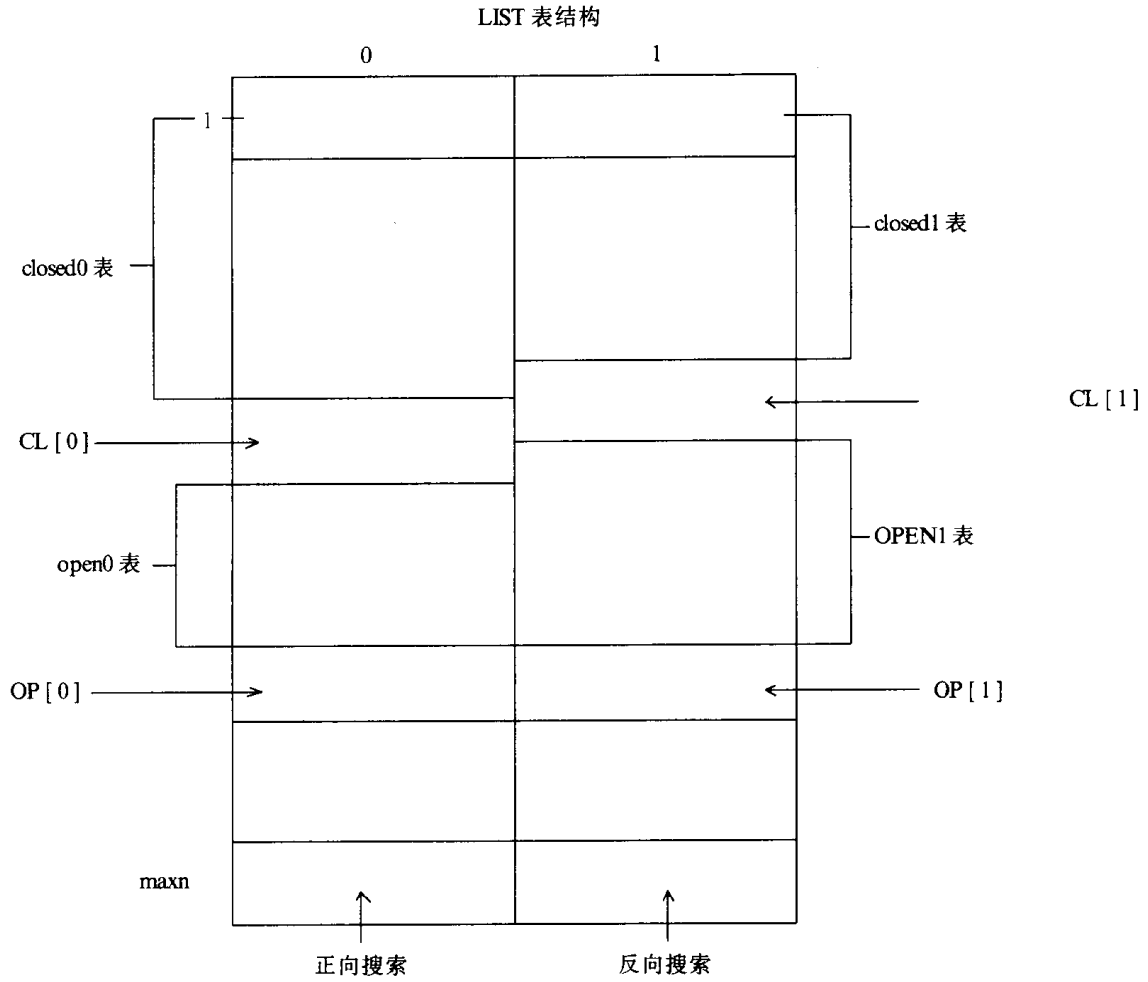


图 6.3-1

在双向搜索过程中，若出现 OP[ST]>MAXN，表示 ST 方向的 LIST 空间溢出；若出现 CL[ST]>OP[ST]表示 ST 方向的 OPEN 队列空，该方向的搜索失败。另外，扩展结点不是机械地按照正方向扩展一层再逆方向扩展一层、两个方向依层交替扩展的方法，而是每扩展一层后，下一层则选择结点个数较少的那个方向先扩展，克服了两方向结点的生成速度不平衡的状态，明显提高了效率。

双向搜索一般不受初始状态和目标状态的影响。同时从这两个状态出发进行双向搜索，无论哪个方向的搜索失败，只要另一方向的搜索还能继续，则搜索沿另一方向进行下去，直至求出最佳路径或者搜索失败为止。但一般要求，正向扩展结点或逆向扩展结点应使用同一规则，即调用同一个 expand 过程，否则会使问题变得复杂得多。

下面给出双向广度优先搜索的算法流程:

```
procedure print(st,k:integer);
begin
if k<>1 then
   begin
```

```
        if st=1 then  输出 list[st,k].state;
        {若逆向搜索，则顺 f 指针打印 list[1,k]..list[1,1]的路径}
        print(st,list[st,k].f);
        if st=0 then  输出 list[st,k].state;
        {若正向搜索，则递归打印 list[0,1]..list[0,k]的路径}
        end;{then}
      end;{print}

    procedure check(st:0..1);
    var
    i:integer;
    begin
     for i:=1 to op[1.st]-1 do {顺序检查 st 相反方向扩展出的每一个结点}
        if list[st,op[st]].state 相交于 list[1-st,i].state then
         begin
            if st=0 then begin print(o,op[st]);print(1,i);end{then}
{若当前为正向，则先递归打印 list[0,1]..list[0,op[st]]的路径，后顺 f 指针打印
list[1,i]..list[1,1]的路径}
                    else begin print [0,i];print(1,op[st]);end;{else}
{若当前为逆向，则先递归打印 list[0,1]..list[0,i]的路径，后顺 f 指针打印
list[1,op[st]]..list[1,1]的路径}
             halt;{退出程序}
        end;{then}
    end;{check}
    procedure expand(st:0..1);
    begin
     q← list(st,cl[st]);{移出 st 方向的 open 队列的队首结点}
    while (Q 结点还可扩展) and (op[st]<maxn)do
      begin
        沿 st 方向扩展出 q 的子结点状态 q '.state;
       list[st,op[st]].state← q '.state;list[st,op[st]].father← cl[st];
       check(st);{若两个方向的搜索相交于 q ；则输出最佳路径并退出程序}
       op[st]← op[st]+1;{st 方向的队尾指针后移}
      end;{while}
    cl[st]← cl[st]+1;{st 方向的队首指针后移}
    end;{expand}

    begin
```

```
list[0,1].state←起始状态; list[0,1].father← 0;op[0]← 2;cl[0]← 1;
list[1,1].state←目标状态; list[1,1].father← 0;op[1]← 2;cl[1]← 1;
while((op[0]≤ maxn)and(cl[0]<op[0]))or((op[1]≤ maxn)and(cl[1]<op[1]))do
{只要某方向的 list 表未溢出且 open 队列非空，则循环}
  if op[0]<op[1] then expand(0)
                 else expand(1);
输出无解信息;
end.{main}
```

下面我们采用双向广度优先搜索的算法求解最短编号序列。

【例 1】最短编号序列

试题内容见§6.2 中的【例 2】。

算法分析:

所谓双向搜索指的是搜索沿两个方向同时进行。

正向搜索——先确定序列的第 1 个元素编号，然后依次考虑第 2 个编号，第 3 个编号……

逆向搜索——先确定序列的最后一个元素编号，然后依次考虑倒数第 2 个编号，倒数第 3 个编号……

正向搜索扩展结点的规则如§6.2 中【例 2】的算法分析所述。而逆向搜索中扩展结点的规则如下:

若当前选择编号 J 作为序列中倒数第 I 个元素:

$$S_a' = \begin{cases} \text{表 A 中的元素 J+(q.f).lev} & (q.f).K=1; \\ \text{表 A 中的元素 J} & (q.f).K=2; \end{cases}$$

$$S_b' = \begin{cases} \text{表 B 中的元素 J} & (q.f).K=1; \\ \text{表 B 中的元素 J+(q.f).lev} & (q.f).K=2; \end{cases}$$

A_i' ——在 S_a' 删去位于右端的 S_b' 子串后的剩余串;

B_i' ——在 S_b' 删去位于右端的 S_a' 子串后的剩余串;

扩展出的结点 q 若满足下述两个条件

(1)((q.La+表 A 元素 J 的串长)≤100)AND((q.lb+表 B 元素 J 的串长)≤100);

(2)(Ai'='') or (Bi'='')

则选择编号 J 作为序列中倒数第 i 个元素。

至所以在两个方向上采用这两种不同的扩展规则，是为了顺利对接，即只要存在 S(AB)，就一定能够在双向搜索的过程中将之找出来。

例如，对于下述两张表

表 A

元素编号	字符串
1	110
2	0
3	1101

表 B

元素编号	字符串
1	1
2	110110
3	01

图 6.3-2

在正向搜索中，我们依次选择 2113 作为编号序列的前 3 个元素

元素编号序列	用表 A 字符串替换	用表 B 字符串替换	LA	LB	LEV	K
2	0	110110	1	7	110110	2
1	110	1	4	8	1101	2
1	110	1	7	9	11	2

图 6.3-3

逆向搜索中，我们依次选择 31 作为编号序列倒数 2 个元素

元素编号序列	用表 A 字符串替换	用表 B 字符串替换	LA	LB	LEV	K
3	1101	01	4	2	11	1
1	110	1	7	3	11	1

图 6.3-4

由此可见，对于表 A 和表 B，存在一个最短的 S(AB)=2113。正向搜索和逆向搜索相交于元素编号序列的第 3 个元素。

若推广到一般情况，则“相交”的概念可以这样描述：当前已扩展出的结点 q 与相反方向扩展出来的所有结点中的某结点 q’ 之间，若同时满足下述关系

(1) q.k<>q’.k

(2) (q.La+q’.La < 100) and (q.Lb+q’.Lb < 100)

(3) q.Lev = q’.Lev

则说明两个方向的搜索在某搜索边域处生成同一个子结点 q(q’) 而相交。

例如，对于（图 6.3 - 2)中的表 A 和表 B, 双向广度优先搜索的过程如下:

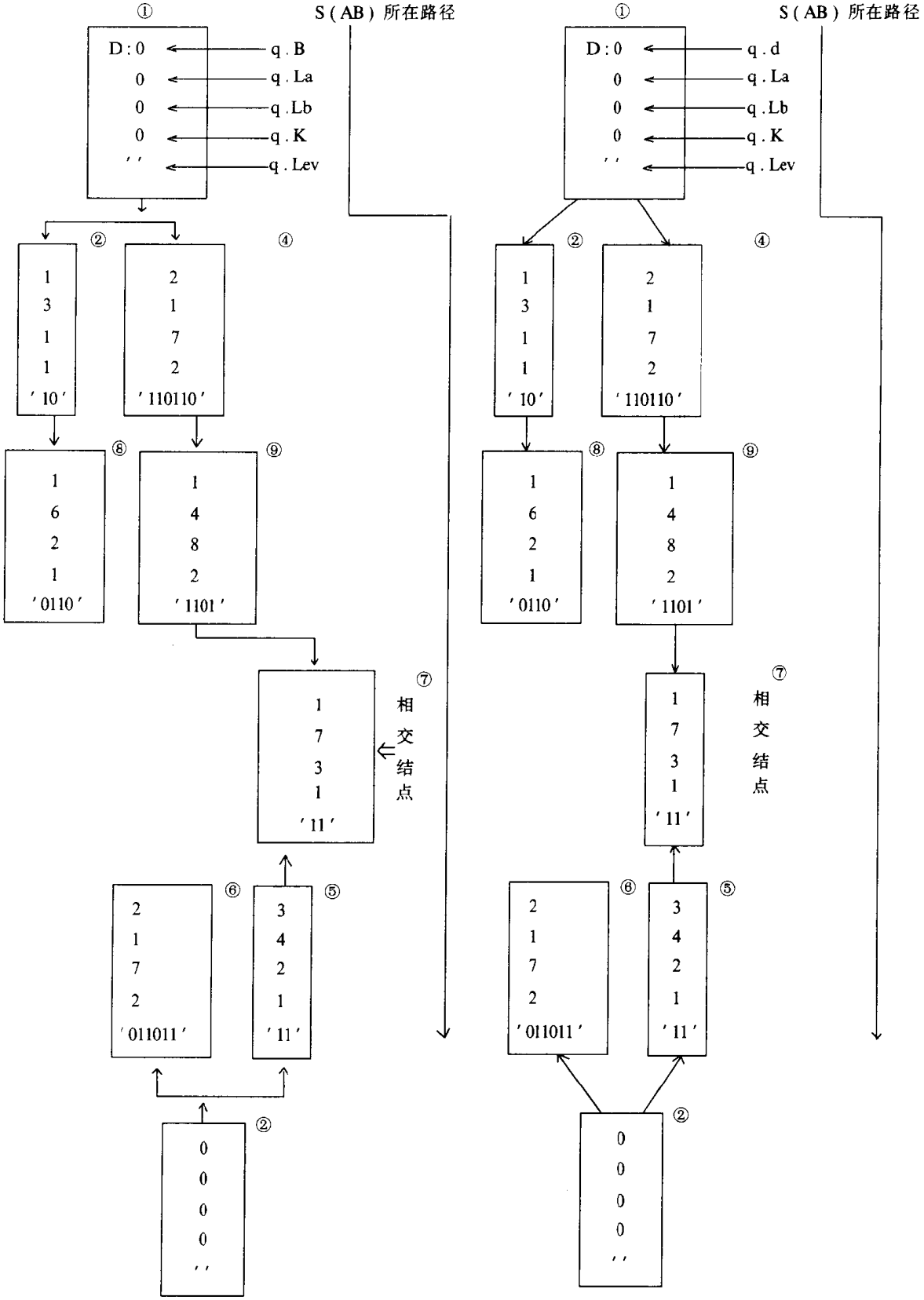


图 6.3 – 5

注：结点右上方● 内为扩展结点的顺序

下面给出程序题解．

```
Program S(AB);

Const
  Ln                    =  100;{置换后的字符串的最大串长}
  Maxn                  =  1600;{LIST 数组的容量}
Type
  LevStr                =  String[Ln];
  Node                  =  Record
                             f          : Integer;{父指针}
                             d, la, lb,{序列中第 i 个元素的编号;
                                        Ai 的串长;  Bi 的串长;}
                             k          : Shortint;
                             lev        :^LevStr
                           {Lev-剩余串，K-剩余串标志
                                    ┌ 1     Lev=Ai'，即 Ai 中剩余
                              K=    │
                                    └ 2     Lev=Bi'，即 Bi 中剩余 }
                           End;

Var
  F                     :  Text;{文件变量}
  Fn                    :  String;{文件名串}
  K                     :  Integer;{表 A 和表 B 的元素个数}
  Strs                  :  Array[1..2, 1..20] of String[20];
  {Strs[1,i]——表 A 的第 i 个元素; Strs[2,i]——表 B 的第 i 个元素;
  List                  :  Array[0..1, 1..Maxn] of Node;
  {List[0]——正向搜索表; List[1]——逆向搜索表}
  Cl, Op                :  Array[0..1] of Integer;
  {CL[0],OP[0]——正向搜索中 OPEN0 表的队首、队尾指针;
   CL[1],OP[1]——逆向搜索中 OPEN1 表的队首、队尾指针}

Procedure Init;
Var
  i, j : Shortint;
Begin
  Write(' File name = ');{输入文件名串}
  Readln(Fn);
```

```
  Assign(F, Fn);{文件变量与文件名串连接}
  Reset(F);{文件读准备}
  Readln(F, K);{读入表A和表B的元素数}
  For i := 1 to 2 Do{读入表A和表B的每一个元素}
    For j := 1 to K Do Readln(F, Strs[i, j]);
  Close(F){关闭文件}
End; { Init }

Procedure Print(st, k : Integer);
{从结点K出发，顺序打印ST方向搜索中产生的元素编号}
Begin
  If k <> 1 Then Begin
    If st = 1 Then Writeln(F, List[st, k].d);
    {若是逆向搜索，顺f指针打印结点K至结点1的路径}
    Print(st, List[st, k].f);
    If st = 0 Then Writeln(F, List[st, k].d)
    {若是正向搜索，则递归打印结点1至结点K的路径}
  End { Then }
End; { Print }

Procedure Check(st : Shortint);
{检查ST相反方向扩展出的每一个结点，是否与当前ST方向上扩展出的结点相交}
Var
  i : Integer;
Begin
  For i := 1 to Op[1-st]-1 Do{顺序检查ST相反方向扩展出的每一个结点}
    If (List[st, Op[st]].k <> List[1-st, i].k) And
      (List[st, Op[st]].lev^ = List[1-st, i].lev^) And
      (List[st, Op[st]].la + List[1-st, i].la <= Ln) And
      (List[st, Op[st]].lb + List[1-st, i].lb <= Ln)
      {若相反方向的结点i与ST方向的当前结点相交}
      Then Begin
            If st = 0 Then Begin
    {若当前处于正向搜索，则先递归打印正向的结点1至当前结点的路径}
              Print(0, Op[st]);
              Print(1, i){后顺f指针打印逆向的结点i至结点1的路径}
            End { Then }
            Else Begin
```

```
    {若当前处于逆向搜索，则先递归打印正向的结点 1 至结点 i 的路径}
                Print ( 0 , i );
                Print ( 1, Op [ st ]){后顺 f 指针打印逆向的当前结点至结点 1 的路径}
              End ; { Else }
              Close ( F );
              Halt
            End { Then }
End ; { Check }
Procedure Expand ( st : Shortint );
{沿 ST 方向扩展出新的一层结点}
Var
  sa, sb : LevStr ;{A' 和 B' }
  i : Integer ;{ 辅助变量}
Begin
  For i := 1 to k Do { 顺序考虑 1 …… K 作为序列中的当前编号}
    If ( List [ st , Cl [ st ]]. la + Length ( Strs [ 1, i ]) <= Ln ) And
       ( List [ st , Cl [ st ]]. lb + Length ( Strs [ 2, i ]) <= Ln ) Then Begin
{ST 方向的 OPEN 队首结点的元素编号序列在加入 i 后，分别用表 A 的字符和表 B 的字串
替换．若 A 串长和 B 串长都不超过 100，则}
      sa := '';  sb := ''; {求 A' 和 B' }
      If List [ st , Cl [ st ]]. k = 1 Then sa := List [ st , Cl [ st ]]. lev^
                                   Else sb := List [ st , Cl [ st ]]. lev^;
      If st = 0 Then Begin
        sa := sa + Strs [ 1, i ]; sb := sb + Strs [ 2 , i];
        While ( sa <> '' ) And ( sb <> '' ) And ( sa [ 1 ] = sb [ 1 ]) Do Begin
          Delete ( sa , 1, 1 ); Delete ( sb , 1, 1 )
        End { While }
      End { Then }
      Else Begin
        sa := Strs [ 1, i ] + sa ; sb := Strs [ 2 , i ] + sb ;
        While ( sa <> '' ) And ( sb <> '' ) And
          ( sa [ Length ( sa )] = sb [ length ( sb )]) Do Begin
          Delete ( sa , Length ( sa ), 1 ); Delete ( sb , Length ( sb ), 1 )
        End { While }
      End ; { Else }
      If ( sa = '' ) or ( sb = '' ) Then
  {若 ( A' =' ' ) 或 ( B' =' ' )，编号 i 可以成为序列中的一个元素，则 ST 方向扩展
出一个结点进入 OPEN 队列尾}
```

```
        With List[st, Op[st]] Do Begin
          f := Cl[st];
          d := i;
          la := List[st, Cl[st]].la + Length(Strs[1, i]);
          lb := List[st, Cl[st]].lb + Length(Strs[2, i]);
          New(lev);
          If sa = '' Then Begin
            k := 2; lev^:= sb
          End { Then }
          Else Begin
                 k := 1; lev^:= sa
               End; { Else }
          Check(st);{检查该结点是否与ST相反方向扩展出的某结点相交}
          Inc(Op[st]);{ST方向的队尾指针后移}
        End { With }
    End; { Then }
  Inc(Cl[st]){ST方向的队首指针后移}
End; { Expand }

Procedure Main;
Begin
  FillChar(List, Sizeof(List), 0);{LIST表初始化}
  {空序列分别进入两个方向的OPEN队列}
  New(List[0, 1].lev); List[0, 1].lev^:= '';
  New(List[1, 1].lev); List[1, 1].lev^:= '';
  Op[0] := 2; Op[1] := 2;
  Cl[0] := 1; Cl[1] := 1;
  While (Op[1] <= Maxn) And (Op[0] <= Maxn) And (Cl[0] < Op[0]) And
        (Cl[1] < Op[0]) Do
  {若两个方向的LIST表空间未溢出且OPEN队列未空，则循环}
    If Op[0] <= Op[1] {下一层选择扩展出的结点数较少的方向先扩展}
       Then Expand(0)
       Else Expand(1);
  Writeln(F, ' No Answer'); {某一方向的OPEN队列空或LIST空间溢出，输出无解信息}
  Close(F){关闭文件}
End; { Main }
Begin
```

```
  Init;{输入表 A 和表 B}
  Assign(F,' OUTPUT.TXT');{输出文件名串与文件变量连接}
  Rewrite(F);{文件写准备}
  Main{通过双向广度优先搜索求最短的 S(AB)}
End.
```

§6.4 分支定界法

采用回溯法和广度优先搜索算法求解的试题一般都指定某个目标状态，要求找出一条由初始结点至目标结点的最佳的（或较好的）解答路径。但是，有时试题并未指定目标状态，而仅给出约束条件和求某个目标函数最小值的要求。在这种情况下，我们一般采用分支定界法求解。

一、分支定界法算法思想

我们结合具体实例分析分枝定界法的算法思想。

【例 1】最佳工作序列

有 N 件工作。输入每件工作的费时、最后完成的期限及工作的价值，试求可能的一个完成工作的序列，使价值和最大。

算法分析:

我们通过仔细分析，可以得出一些规律:

1. 求完成工作的价值和最大的问题，可以等价为求剩余价值和（所有工作的价值和减去已完成的工作的价值和）最小的问题。因此，每完成一项工作，需求剩余价值和;

2. 当第 i－1(i＞1) 项工作完成的时刻加上第 i 项工作的费时小于等于第 i 项工作的最后完成的期限时，才允许完成第 i 项工作。因此，在分析问题前，应按各项工作最后完成期限由小到大的顺序排列工作序列，分析问题时，根据该顺序逐一考虑每项工作；每完成一项工作，要记下该项工作最后完成的时刻;

3. 若当前完成的最后一项工作的序号为 Last，那么，可能完成的下一项工作的序号应在 (Last＋1)…N 之间，这就要求在该范围内逐一分析各项工作。当分析第 i 项工作时，当且仅当余下的 N－i 项 ((i＋1)…N) 工作的价值和小于等于当前最小剩余价值和，才可根据规律 2 考虑完成第 i 项工作的可能性问题；否则第 i 项工作不予完成，继续往后考虑第 i＋1 项工作 ……。由此可见，每分析一项工作，需求该项工作后的所有工作的价值和；每次新完成一项工作，需记下其序号 Last;

4. 当前工作序列中，每项工作完成的时间必须小于等于该项工作的最后完成的期限；最后一项工作完成的时刻必须小于等于各项完成的工作中最后完成期限的最大值。

根据上述规律，我们设计了一种算法:

我们将结点表征为当前工作序列:

结点的信息包括:

1. 1..N 项工作的完成情况，即完成工作序列:

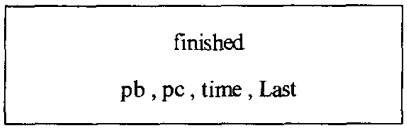


图 6.4 -1

$$finished[i] = \begin{cases} true & 第\ i\ 项工作完成 \\ flase & 第\ i\ 项工作未完成 \end{cases}$$

2. 完成的最后一项工作的序号 Last;

3. 最后一项工作完成的时间 time;

4. (Last + 1)…N 项工作的价值和 pc;

5. 剩余价值和 pb;

根据工作时间有序性的要求，我们将输入的 N 项工作，按最后完成期限由小到大排序，开成一个新的工作序列，按该顺序逐一考虑完成每项工作。根据上述规律，首先，我们设所有工作未完成，即 finished[1..N] 为 flase;Last 为 0; pc,pb 为 N 项工作的价值和。从这个初始结点出发，按先生成的结点先扩展的原则进行扩展，其方法是从表示当前工作序列的某结点出发，逐一考虑完成 (Last + 1)…N 范围内的每项工作的可能性。若考虑第 i 项工作时，剩下的 ((i + 1)…N 间)N − i 项工作的价值和小于等于当前最小剩余价值和且完成了该项工作的时刻小于等于该项工作的最后期限，则完成第 i 项工作; 否则第 i 项工作不予完成。这样，生成了该结点的下一层子结点。从表面上看，这个算法十分相似于宽度搜索，但有两点显著区别:

1. 设置了一个下界 pmin——目前为止剩余价值和最少的值

在分支定界法中，每扩展一个结点后，都要重新估计下界。若子结点中相关于目标函数的域值小于下界，则取该域值为下界，并记下子结点的信息。这样使得扩展过程中，其下界越来越小，越来越接近目标函数的要求。本题中，下界 pmin 取为目前扩展出来的所有结点中剩余价值和最小的那个值。初始时，对应于 N 项工作未完成的情况，pmin 值设为 N 项工作的价值和。在搜索过程中，每扩展一个子结点，若剩余价值和 pb 小于下界 pmin，则 pmin 取该剩余价值和，并记下该子结点的 finished。这样，当所有待扩展结点都已扩展完时，pmin 对应的完成工作序列，就是最佳工作序列。

2. 设置了一个界变量 pc——第 i 项工作后的剩余工作的价值和 (Last + 1 ≤ i ≤ N)

在分枝定界法中必须按照目标函数的要求设计界变量，作为分枝标准。每扩展一个结点之前，都要根据当时的扩展情况计算界变量。本题中，结点的界变量 pc 取为当前第 i 项工作完成后的剩余工作的价值和，初始时为 n 项工作的价值和。在搜索过程中，若当前结点的 pc 值 ≤ pmin，且完成第 i 项工作后的时间小于等于第 i 项工作最后完成的期限，则允许第 i 项工作完成，扩展出子结点。否则不再考虑完成第 i 项工作，因为以

这个子结点为根的子树上的所有结点，不可能产生比已求得的 pmin 更好的方案，而是应该转向考虑第 i + 1 项工作。

例如输入下列四项工作：

项＼工作	费时	最后完成期限	价值
1	4	10	2
2	3	8	3
3	2	4	3
4	1	2	6

图 6.4 -2

按各项工作最后完成的期限由小到大顺序排列后，形成工作序列 4，3，2，1，(序号为 4 的工作为第 0 项，序号为 3 的工作为第 1 项工作，序号为 2 的工作为第 2 项工作，序号为 1的工作为第 3 项工作)，按该顺序逐一考虑完成每项工作。

首先，我们将初始工作序列定义为结点 V0，finished [0] = … = finished [3] = false，简写为 ffff，pb = pc = 2 + 3 + 3 + 6 = 14，time = 0，Last = －1，下界 pmin = 14。

接下去，从 V0 结点出发，逐一考虑完成 0..3 项工作的可能性。若完成第 0 项工作，剩余工作的价值和 2 + 3 + 3 = 8，小于下界 pmin，并且完成该项工作的时间 1 小于该项工作的最后期限 2，则允许完成该项工作，扩展出结点 V1，即 V1 的 finished = tfff，pb = pc = 8，time = 1，Last = 0。由于下界 pmin 大于 V1 的 pb，则 pmin 设置为 8。显然，完成其他某项工作时也满足这两个条件，因此也允许完成。这样，依次扩展出结点 V2，V3，V4 (这些结点的信息见图 6.4 -3)，形成了第一层结点。在扩展完第一层结点后，下界 pmin = 8。

在完成了某一项工作的基础上，我们再考虑完成二项工作的可能性。首先从 V0 出发，在完成第 0 项工作后，依次考虑完成 1..3 项工作。若完成第 1 项工作，剩余工作的价值和 3 + 2 = 5，小于下界 pmin，完成该项工作的时间 1 + 2 = 3，小于该项工作的最后期限 4，则允许完成该项工作，扩展出结点 V5，即 V5 的 finished = ttff，pb = pc = 5，time = 3，Last = 1，下界 pmin 大于 V1 的 pb，pmin 设置为 5。由于完成第 2 项、第 3 项工作相继满足了上述条件，因此允许完成，依次扩展出结点 V6，V7 (这些结点的信息内容见图 6.4 -3)，形成了 V1 的子结点。显然，扩展出这些结点后，下界 pmin = 5。然后再分别扩展 V2、V3 结点，生成子结点 V8，V9 和 V10 (由于 V4 已完成了第 3 项工作，按工作序列顺序，不再回头考虑完成其它小于该序号的工作)，形成了解答树的第二层结点。扩展完第二层后，下界 pmin 已变成 5。接着，依次从第二层中那些 Last 值小于 3 (还剩有未完成的工作) 的结点出发，考虑完成三项工作的可能性。由

于结点 V7，V9，V10 已完成了第 3 项工作，因此不再扩展。而结点 V5，V6，V8 满足扩展的约束条件，依次扩展出子结点 V11，V12，V13，V14，形成解答树的第三层。扩展完第三层后，下界 pmin 为 2。

最后，从第三层中那个 Last 值小于 2 的 V11 结点出发，考虑完成最后一项工作的可能性。由于结点 V11 的 time = 6，完成第四项工作后的时间 6 + 4 = 10，满足该项工作的最后期限，且无剩余工作，剩余价值和的值为零，显然小于下界 pmin。因此允许完成该项工作。至此，得出了一个完成工作的最佳工作序列 4→3→2→1，价值和为 14。

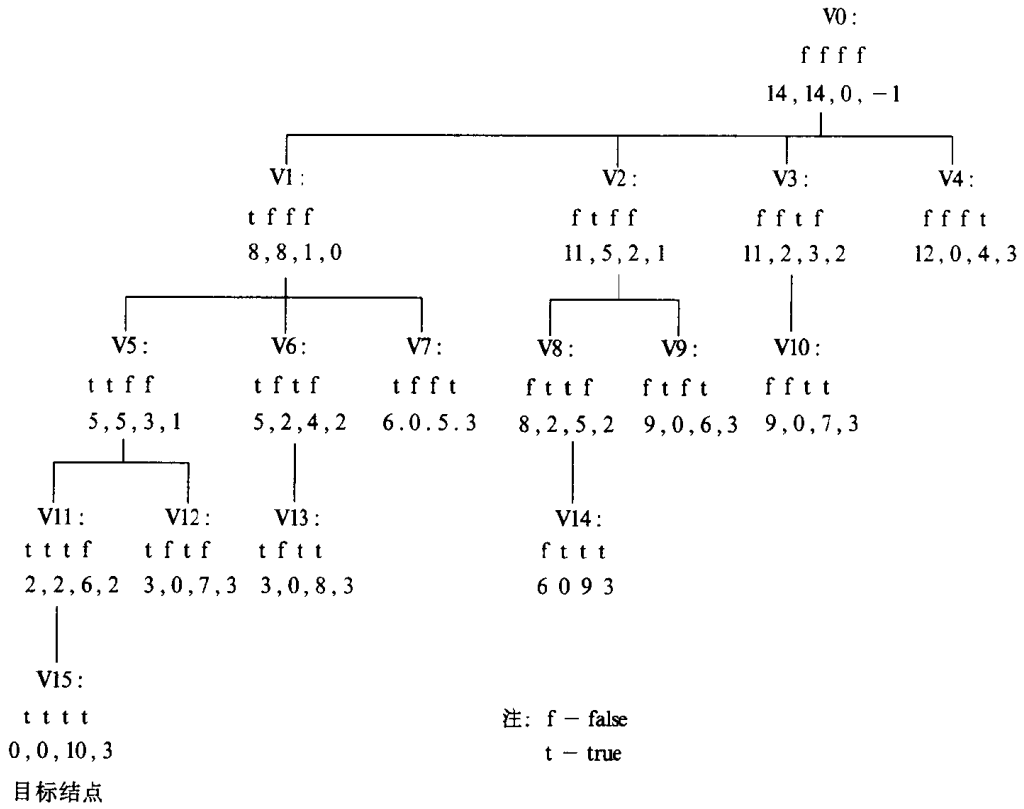


图 6.4 -3

由此可见，分枝定界法虽然在扩展结点的顺序上类同于广度优先搜索（也可以类同回溯法），但由于分枝定界法是求解目标函数的最小值，因此它需要设定界变量，估计下界。在搜索过程，若发现某子结点的界变量值小于等于目前下界且满足约束条件，则该子结点进入解答树；否则不再重复构造以该结点为根的子树，因为由此扩展下去，不可能求出目标函数的最小值。当扩展了所有的待扩展结点时，算法结束，对应于下界的子结点信息就是目标函数的解。

二、分支定界法的算法框架

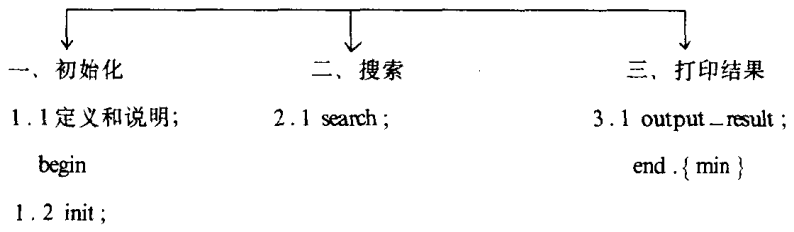
由于分枝定界法扩展结点的顺序如同宽度优先搜索，因此，我们通过一个采用队列结构的 open 表来存储待扩展结点，open 表元素为动态数据类型。设动态指针 open_list 和 tail，分别指向 open 表的表首结点和表尾结点的地址。open_list = NIL 时，表示

open 表空。

这里说明一点，算法框架中的下界和界变量未具体指向，它们仅是一个抽象的概念。如何估计下界，取结点的那个状态域为界变量，由编程者根据对目标函数的要求和结点状态中与目标函数相关的量的了解，具体规定。

下面，我们采用逐步求精的办法给出分枝定界法的算法框架

第一层:



第二层:

1．求精 1.1——定义和说明

```
type
 link = ^node;
  node = record
          statel : statetypel;
           ..............;
           statei : statetypei;{界变量}
           ..............;
          staten : statetypen;
          next : link;
          end;
var
 open_list, tail : link; {open 表的表首指针和表层指针}
 best : besttype; {符合目标函数要求的解答}
 min_delimiter : integer; {下界}
```

根据题意定义各约束变量等;

至于函数和过程说明，则在下面给出。

2. 求精 1.2——init 的过程说明

```
procedure init;
begin
```

按输入格式输入初始状态;

根据要求设置下界 min_delimiter 的初始值;

```
new(tail); {初始结点进入 open 表}
```

```
open  list ← tail;
open_list^ 的 statel...staten ← 初始状态;
open_lise^.next ← Nil;
end;{init}
```

3. 求精 2.1——search 的过程说明

```
procedure search;
var
 delimiter: integer; {界变量}
 temp: link; {子结点指针}
begin
 while open_list <> Nil do {open 表非空}
  begin
   delimiter ← open_list^.statei;
   while open_list^ 还能扩展 do
     begin
      根据当前扩展情况计算 delimiter;
      if delimiter < = min_delimiter then
      {其子结点可能产生比当前下界更好的方案}
        begin
         if 子状态符合约束条件 then {生成子结点 temp, 并进入 open 表尾}
          begin
           new(temp);
           tail^.next ← temp;
           tail ← tail^.next;
           子状态进入 temp^ 的 statel...staten;
           temp^.next ← nil;
           if min_delimiter > temp^.statei {重新估计下界}
           then begin
                  min_delimiter ← temp^.statei;
                  best ← temp^ 与解相关的状态;
                end;{then}
          end;{then}
         end;{then}
      end;{while}
      temp ← open_list^.next; {释放 open 表首结点}
      dispose(open_list);
      open_list ← temp;
```

```
    end ;{ while }
  end ;{ search }
```

四．求精 3．1——output_result 的过程说明

按输出格式打印 best 中的信息。

三、应用框架解题

主程序完全类同于算法框架的第一层，这里不再赘述。下面根据试题的具体要求，求精第二层。

1. 求精 1．1——定义和说明

```
  const
    max_job = 4 ; {最多的工作件数}
  type
    job = record
            time : integer ; {费时}
            time_ltd : integer ; {最后完成的期限}
            value : integer ; {价值}
            code : integer ;
{由于需要按 time_ltd 关键字升序的顺序重新排列工作序列，因此通过 code 来存储该项工作在输入时的顺序号}
              end ;
          used = array [ 0 .. max_job ] of boolean ; {当前工作序列的完成情况}
          link = ^node ; {link 为指向 node 类型变量的指针}
          node = record {定义结点类型}
                   finished : used ; {当前完成工作的序列}
                   pb : integer ; {剩余价值和}
                   last : integer ; {完成的最后一项工作的序号}
                   pc : integer ; {last + 1 ... total_jobs 项工作的价值和}
                   time : integer ; {完成工作所需所时间}
                   next : link ;
                   end ;
    var
      total_jobs : integer ; {实际的工作件数 1 ≤ total_jobs ≤ max_job }
      pmin : integer ; {目前为止剩余价值和的最小值作为下界}
      best : used ; {最佳完成工作的序列}
      jobs : array [ 0 .. max_job ] of job ; {工作序列}
      open_list , tail : link ; {open 表首指针和表尾指针}
```

至于函数和过程说明，则在下面逐一给出。

2. 求精 1.2——init 的过程就明

```
procudre init;
var
 i,j,1:integer;
 temp:job;
begin
 repeat    {输入工作件数 total_jobs}
  write('how many jobs:')  ;readln(total_jobs);
  if(total_jobs > max_job)or(total_jobs < 1)then
  writeln('invalid input.try again.');
until(total_jobs > = 1)and(total_jobs < = max_job);
pmin ← 0;
for i ← 0 to total_jobs - 1 do
{输入每件工作的费时、期限、价值，设置输入顺序号，累计所有工作的价值和，作为下界 pmin 的初始值}
          begin
            writeln('the no.:',i,'job:');
            writeln('time to finish it:');readln(jobs[i].time);
            writeln('time to hand it:');readln(jobs[i].time-ltd);
            writeln('value of it:');readln(jobs[i].value);
            jobs[i].code ← i + 1;
            pmin ← pmin + jobs[i].value;
          end;{for}
          for i ← 0 to total_jobs-2 do
{所有工作按最后完成的期限由小到大的顺序排列，形成新的工作序列}
            begin
              1 ← i;
              for j ← i + 1 to total_tobs - 1 do
              if jobs[j].time_ltd < jobs[1].time_ltd then 1 ← j;
              if 1 <> i then begin
                 temp ← jobs[i];jobs[i] ← jobs[1];jobs[1] ← temp;
                          end;{then}
            end;{for}
            new(tail);        {首结点进入 open 表}
            open_list ← tail;
            for i ← 0 to tatol_jobs - 1 do open_list^.finished[i] ← false;
            open_list^.pb ← pmin;
            open_list^.pc ← pmin;
```

```
        open_list^.time ← 0;
        open_list^.last ← 1;
        open_list^.next ← nil;
    end;{init}
```

3. 求精 2.1——search 的过程说明

```
procedure search;
var
 i,j:integer;
 pc:integer;{考虑余下工作的价值和，作为界变量}
 time:integer;{前一项工作最后完成的时间}
 temp;link;{子结点}
begin
 while open_list <> nil do
  begin
   pc ← open_list^.pc;
   time ← open_list^.time;
   for i ← open_list^.last + 1 to total_jobs - 1 do
{逐一考虑剩下的每一项工作，以生成 open 表首结点的下一层子结点}
     begin
       pc ← pc - jobs[i].value;{计算剩余工作的价值和，作为界变量}
       if pc < = pmin then
{界变量小于下界，说明扩展下去有可能产生比下界更好的方案}
         begin
          if time + jobs[i].time < job[i].time_ltd {时间上允许完成第 i 项工作}
          then begin {生成子结点 temp，进入 open 表尾}
          tail^.next ← temp;
          tail ← tail^.next;
          temp^.finished ← open_list^.finished;{确定第 i 项工作完成}
          temp^.finished[i] ← ture;
          temp^.pb ← open_list^.pb-jobs[i].value;
          temp^.pc ← pc;
          temp^.time ← time + job[i].time;
          temp^.last ← i;
          temp^.next ← nil;
          if pmin > temp^.pb then {重新估计下界}
           begin
             pmin ← temp^.pb;
```

```
                best ← temp^ . finished ;
              end ;{ then }
            end ;{ then }
          end ;{ then }
        end ;{ for }
        temp ← open_list^ . next ; {释放 open 表首结点}
        dispose ( open_list );
        open_list ← temp ;
      end ;{ while }
    end ;{ search }
```

4. 求精 3 . 1——output_result 的过程说明

```
procedure output_result ;
var
  i , j ; integer ;
  value : integer ;
begin
  value ← 0 ;
  writeln (' then jobs table :'); {按输入顺序打印工序表}
  writeln (' no . time_used time_ltd value '');
  for i ← 1 to total_jobs do
    for j ← 0 to total_jobs − 1 do
      if job [ j ] . code = i then
        begin
        write ('', jobs [ j ] . code : 2 ,'', jobs [ j ] time : 5 ,'',
              jobs [ j ] . time_ltd : 5 ,'' jobs [ j ] . value : 3 );
      end ;{ then }
    writeln (' − − −
```

§6.5 A*算法

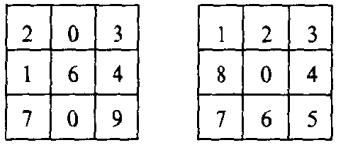
A*算法属于一种启发式搜索。它扩展结点的次序类似于广度优先搜索，但不同的是，每生成一个子结点需要计算估价函数 F，以估算起始结点的约束经过该结点到达目标结点的最佳路径代价；每当扩展结点时，总是在所有待扩展结点中选择具有最小 F 值的结点作为扩展对象，以便使搜索尽量沿最有希望的方向进行。因此，A* 算法只要求产生问题的全部状态空间的部分结点及其关系，就可以求解问题了，搜索效率较高。

我们不妨通过一个具体实例引出对 A*算法思想的讨论。

【例 1】8 数码问题

8 张编有数码 1 ～ 8 的滑牌，能在 3 * 3 的矩阵中滑动，矩阵中有 1 个是空位，用 0 表示，空位也可以移动。

(图 6.5 - 1) 是数码滑牌在矩阵中的二种状态



(a)　　(b)

图 6.5 - 1

在图 6.5 - 1 中，(a) 为初始状态，(b) 为目标状态，找出由初始状态转换到目标状态的滑牌移动序列。

具体要求：

1. 每次输入一个初始状态和一个目标状态的数据，例如

first state: 2 8 3 1 6 4 7 0 5

last state: 1 2 3 8 0 4 7 6 5

2. 实现从初始状态到目标状态的转换(如不能实现，程序应给出不能实现的原因)；

3. 输出结果。每移动一步都必须在屏幕上显示移动序号，并以 3 * 3 表格形式显示状态；

4. 要求使移动的步数尽可能少。

算法分析：

仔细分析题意，我们可以看出，相对于一种状态来说，空位与相邻的滑牌交换产生一个子状态。若空位未在周界上，则可产生 4 个子状态(上，下，左，右移动)，即该题的解答树为 4 次树。在需要移动很多步数的情况下，若采用盲目的广度优先搜索来寻找最佳方案，很可能产生内存溢出；若采用回溯法求解，其时间效率也低得令人难以忍受。迫不得已，我们选择了一种全局择优的启发式搜索算法。

一、计算估价函数 F(N)

每当空位与相邻滑牌交换一次，都要估算一下，从初始状态约束经过交换后状态，再转换至目标状态的最小步数。这个估价值称之为估价函数 F(N) 的值。

形式：$F(N) = G(N) + H(N)$

其中 G(N)——从起始结点 S 到任意结点 N 的最佳路径的实际代价 $G^*(N)$ 的估算值 ($G(N) \geqslant G^*(N)$)。一般为解答树中 S 到 N 这段路径的代价。这一代价可以从 N 出发，沿着各结点的父指针把所遇到的各段路径代价加起来得出。G(N) 在 F(N) 中的比重越大，搜索越倾向于横向趋势；G(N)=0，则只希望找到目标结点而不考虑会付出代价。

H(N)——从结点 N 到目标结点 T 的最佳路径的实际代价 $H^*(N)$ 的估算值 ($H^*(N) > H(N)$)。H(N) 在 F(N) 中比重越大，越强调启发因素，搜索越趋向于纵向趋势；反之比重越小，则越突出横向搜索的特性。H(N)=0，则为广度优先搜索。

启发函数 H(N) 的选择是至关重要的。它无固定的数学模型，主要依赖于对被解问题特性的先验认识和求解的策略窍门。但是有两个限制：

(1) H(N) 一定要小于等于结点 N 至目标结点 T 的实际代价，即 ($H(N) \leqslant H^*(N)$)；

(2) 任意结点的 F 值必须大于等于父辈结点的 F 值，即 F 值单调递增；

当估价函数 F 满足上述两个限制时，按正确的启发式搜索算法，一定会得出问题的最优解，并尽可能少地产生结点；否则可能失去最佳解甚至全部解。因此使用 A* 算法的前提是设计一个好的（即满足两个限制条件）F 函数。

对于 8 数码问题，我们是这样设计 F 函数的：

H——8 个编有 1 ～ 8 的滑牌〞离家〞（目标状态）的距离总和。这个 H 是满足 $H \leqslant H^*$ 的。因为对于每个滑牌来说，从当前位置移至目标位置的最少步数总是大于等于它们之间的距离；

G——从初始状态移至当前状态的步数，即该结点的深度值；

相对当前结点的子结点来说，G 增加 1，而 H 值至多减少 1，因此子结点的 F=G+H 总是大于或等于它的父结点的，显然 F 为单调递增。使用这个满足 $H^* > H$ 和 F 单调递增的估价函数，从理论上说，是一定能够搜索出由初始状态转换为目标状态的最佳走步序列的。

二、按 F 值递增顺序排列待扩展结点

由估价函数 F(N) 的定义可知，F 值愈小，该结点位于最佳路径的希望程度愈大。因此每生成一个子结点后，必须根据其 F 值的大小插入到待扩展结点序列的适当位置，等待扩展； 每次扩展结点时总是选择 F 值最小的结点扩展，尽量使搜索沿最佳路径方向进行。如何对待扩展结点序列进行排序，是影响 A* 算法效率的一个重要因素。

我们设置一个堆阵 DTS[1..N]（N——当前待扩展结点个数）来存贮待扩展结点。堆阵可看作是一棵按层次由左到右排列的二叉树，即 DTS[I] 的父结点是 DTS[I DIV 2]，左子

结点是 DTS [2I], 右子结点是 DTS [2I + 1]。若 2I + 1 > N 或 2I > N, 则与之相应的子结点不存在。并且各结点的 F 值满足下式

DTS [I DIV 2] 的 F 值 ≤ D [I] 的 F 值

例如堆阵 DTS [1..8] 的 F 值序列 = { 13, 38, 27, 49, 76, 65, 49, 97 }, 对应的完全二叉树如下图

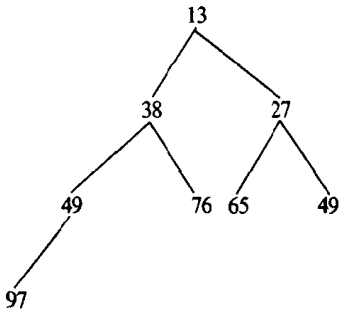


图 6.5 -2

由此可知，堆阵所表示的二叉树的根结点 DTS [1] 是最小的。

实现对待扩展结点序列的堆排序需要解决两个问题:

(1) 在取出堆顶元素 DTS [1] 扩展后，如何调整剩余元素使之成为一个新的堆。堆顶元素出堆扩展后，以堆的最后一个元素 DTS [N] 替代之。例如从图 6.5－2 的堆中取出元素 13 后得新堆 (图 6.5－3 (a))。

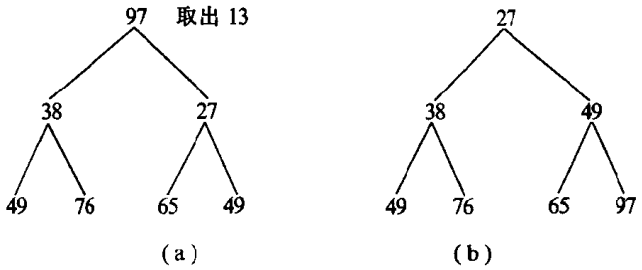


图 6.5 -3

此时根结点的左，右子树均为堆，则仅需自上而下进行调整即可:

首先堆顶元素和其左、右子树根的值比较。由于右子树根结点的值小于左子树根结点的值且小于根结点的值，则将 27 和 97 相交换。由于 97 替代 27 之后虽然堆顶点成为 N－1 个元素中的最小值，但破坏了右子树的″ 堆″，则需进行和上述相同的调整，直至叶结点，见图 6.5 -3 (b)。

(2) 扩展的子结点进入序列后如何保持堆性质。

每生成一个子结点后，将该结点插入堆尾 DTS [N + 1], 即在二叉树的最后一层加一片叶子。例如在图 6.5 -3 (b)的堆中加入 F 值为 15 的结点

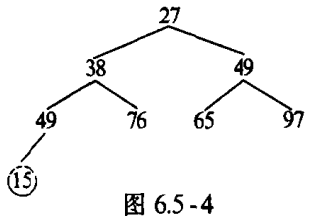


图 6.5 -4

然后由下而上遍历由新的叶结点至根的路径，该路径的结点被向下复制，直至找到合适的位置插入，见图 6.5-5(a),(b),(c)

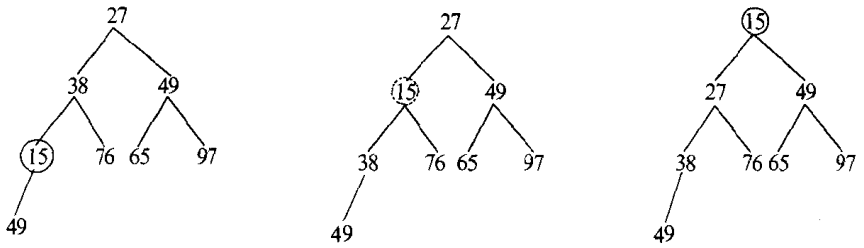


图 6.5-5

每当从堆中选择 F 值最小的 DTS[1]进行扩展前，都要判别其是否为目标结点。若是目标结点，便确认初始结点通往这个目标结点的路径是最佳的，从而结束了搜索过程；若堆空时还未找到目标结点，则 A* 搜索失败。

三、检查调整重合状态

若某结点扩展出一个子结点 newnode，而其状态与以前已经扩展过的结点或当前堆中某一个待扩展结点 Q 的状态重合时，怎么办。在经过同一种状态的两个路径方案中，究竟选择哪一个方案能达到最佳的要求？如果让两种方案同时存在，则会引起不少麻烦。例如 8 数码问题中，若重合的 Q 结点是待扩展结点，是由某结点中的空位移动一步后生成的子状态，那未将来扩展 newnode 时，空位沿同一方向移动一步后生成的子状态又与 Q 的父结点状态重合；若 Q 是已扩展结点，那未将来 newnode 的子结点不仅可能同 Q 的父结点、而且可能同 Q 的子结点重合，使问题变得更复杂。

为了遏制住多余子结点的繁衍，当发现新生成的子结点 newnode 的状态与以前生成的 Q 结点状态重合时，我们便按下述规律及时调整：

若 Q 结点是待扩展结点且 nwenode 的结点 G 值小于 Q 结点的 G 值，则 newnode 取代堆中的 Q 结点，即在经过同一状态的两个方案中。选择经由 newnode 的路径；否则不让 newnode 进入堆，选择经由 Q 结点的路径。这样做可在一定程度上遏制住由于 F 函数偏差所带来的负效应；

若 Q 结点是已扩展结点，则 newnode 不进入堆。因为 F 单调递增，所以根本不会产生 newnode 的 F 值好于任一已扩展结点的 F 值的可能。即使 F 不呈单调递增，通常情况下也是 G 的 F 值小于 newnode 的 F 值，故 newnode 不必进入堆。

由此可见，每生成一个子结点都要检查一下重合问题，而目前为止生成的子结点都属于被查询的范围。因此查找效率也是影响 A* 算法效率的又一个重要因素。

为了提高查找效率，我们将目前生成的所有状态各异的子结点存贮在一棵查找树上。查找树也是一棵二叉树，它或者是空树，或者满足下列条件：

(1)如果 T 的左子树非空，那么 T 的左子树中所有结点的关键值都小于 T 的根结点的关键值；

(2)如果 T 的右子树非空，那么 T 的右子树中所有结点的关键值都大于 T 的根结点的

关键值;

(3)T 的左右子树也是查找树;

例如图 6.5-6 就是一棵查找树

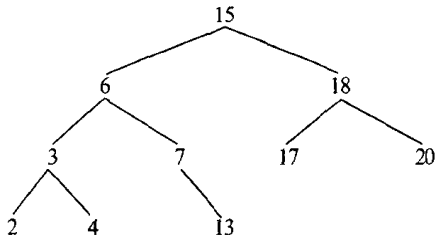


图 6.5-6

这里的问题是如何定义不同结点状态间的"值"大小。在 8 数码问题中，我们通过两棋盘间由上而下，由左至右的第一个不同数码的大小来确定两状态的大小关系。

每扩展出一个新状态，则建立一个新结点，并按下述原则对查找树进行查询和插入操作:

若查找树为空，则新结点为查找树的根结点；若查找树非空，则将新结点的状态和根结点状态比较：若相等，返回根结点的位置指针；若小于关系，则递归左子树；若大于关系，则递归右子树。

每次插入的新结点都是查找树的叶结点，无需移动其它结点，仅改动原来某叶结点的左（右）儿子指针，使之指向新结点即可。这就相当于在一个有序序列上插入一个元素而毋需移动其它元素，查询元素的个数也不超过对应查找树的深度。由此可见，在这种动态数据结构上的查找效率要比静态数组的顺序查找快许多。

综合上述几点，我们得出 A* 算法的基本思想:

从初始结点出发，自顶而下地进行解答树的构造和检查。每次扩展结点时，总是选择 F 值最小的堆顶结点作为扩展对象。若扩展出的子结点不是以前生成的重合结点，则分别进入堆和查找树；否则按上述规律进行调整。如果在搜索过程中发现当前未剩待扩展结点了（即堆空），则宣告不可能从初始状态到达目标状态；反之一旦找到一个目标状态，便可以得出一条由初始状态至目标状态的最佳路径方案。

四、A* 算法框架

下面采用逐步求精的方法给出 A* 算法框架

```
                一、初始化
                { $1024, 0, 655360 }
                Program A*;
                1.1 定义和说明;
第一层          Begin
                1.2 init;{输入初始状态和目标状态}
                二、求解
                2.1 main ;{计算和输出最佳路径方案}
                End.
```

第二层

1、求精 1.1——定义和说明

```
const
   maxheap          = xxxxx ;{堆的最大容量}
type
   statetype        = xxxx ;{状态类型}
   nodeptr          = ^node {堆指针类型}
   node             = record {堆结点类型}
                      g , f    : integer ;{深度，估价函数}
                        p      : nodeptr ;{路径的前趋指针}
                       hp      : integer ;{堆序号}
                     state     : statetype ;{状态}
                     end ;
  listptr          = ^listp ;{查找树的指针类型}
  listtp           = record ;{查找树的结点类型}
                       p       : nodeptr ;{堆指针}
                       l , r , f  : listtp {左指针，右指针，父指针}
                     end ;
  heapty           = object {堆操作对象的类型定义}
                       hpsize    : integer ;{堆的实际长度}
                       dts       : array [ 1.. maxheap ] of nodeptr ;{堆序列}
procedure heapfy ( i : integer );{将序号为 i 的结点下降到堆中的合适位置}
procedure decrease ( i : integer );{将序号为 i 的结点上升到堆中的合适位置}
procedure heap__insert ( newnode : nodeptr );{将 newnode 结点插入堆}
function extrace__min : nodeptr ;{取出堆顶元素}
end ;
var
  stt , trm                    : statetype ;{初始状态和目标状态}
  root                          : listptr ;{查找树的根指针}
  op                            : heaptp ;{堆操作对象}
  tot                           : integer ;{生成的结点数}
```

2、求精 1.2——init 的过程说明

```
procedure init ;
var
  f                             : text ;
begin
   assign (f, 输入文件名串); {输入文件名串与文件变量连接}
```

```
    reset(f);{文件读准备}
    writeln('First State:'); 从文件F中读入初始状态stt;
    writeln('Last State:'); 从文件F中读入目标状态trm;
  end;{init}
```

3、求精 2.1 —— main 的过程说明

```
  procedure main;
  var
    x                                    : nodeptr;{堆指针}
    succeed                              : boolean;{成功标志}
  begin
    tot ← 0;{子结点个数初始化}
    new(x);{为堆的初始根申请内存并设定参数}
    with X^ do begin
      state ← stt; g ← 0; p ← nil;
2.1.1  f ← calculate(state);{计算初始根的估价函数}
    end;{with}
    new(root);{为查找树的初始根申请内存并设定参数}
    root^.L ← nil; root^.r ← nil; root^.p ← x;
    op.hpsize ← 1; op.dts[1] ← x;{堆的初始根作为堆顶元素}
    succeed ← false;{成功标志初始化}
    while (not succeed) and (op.hpsize <> 0) do begin
      {若未找到目标状态且堆非空，则循环}
2.1.2  x ← op.extract_min;{取出堆顶结点X}
      x^.f ← -x^.f;{置x已扩展标志}
2.1.3  if comp(x^.state, trm)='='
{若x为目标状态，则设成功标志，并递归打印起始结点至x的最佳路径}
        then begin
          succeed ← true;
2.1.4     print(x);
          end{then}
2.1.5     else expand(x);{否则扩展x的所有儿子}
    end;{while}
   if not succeed then writeln('No answer');{若堆空，则输出无解信息}
  end;{main}
```

第三层

1. 求精 2.1.1 ——calculate(state)的函数说明

```
输入状态 state，计算和返回该状态的启发函数值
function calculate (var state : statetype ):shortint;
var
   c                              : shortint;
beign
   c← 0;
  累计状态 state 的启发函数值 C;
  calculate ← C;
end;{calculate}
```

2、求精 2.1.2——op.extract_min 的函数说明

取出堆顶元素，并使得剩余结点仍保持堆的性质。

```
function heaptp.extract_min:nodeptr;
begin
   extract.main← dts[1];{返回堆顶结点的指针}
   dts[1]← dts[hpsize];{堆中的尾结点移至堆顶}
  dec(hpsize);{堆长度减 1}
2.1.2.1 heapfy(1);{将堆顶结点下降至合适位置，以保持堆性质}
end;{heapty.extract_min}
```

3、求精 2.1.3——comp (x^.state, trm) 的函数说明

输入 state1 状态和 state2 状态。若两状态重合，则返回' ='；若 state1 状态值大于 state2 状态值，则返回' >'；若 state1 状态值小于 state2 状态值，则返回' <'。

```
function comp (var state1, state2 : statetype ): char;
begin
   if 状态 state1 <> 状态 state2 then begin
     if state1 的状态值 > state2 的状态值 then comp ← ' >'
                                        else comp ← ' <';
       exit;
    end;{then }
  comp ← ' =';
end;{comp}
```

4、求精 2.1.4——print (X) 的过程说明

输入目标结点指针 K。顺 K^.P 指针递归打印初始结点至 K 结点的最佳路径方案。

```
procedure print (k : nodeptr);
begin
   if k <> nil then begin
```

```
        print ( k^. p ):
      writeln (' step = ', k^. g );
      按输出格式打印 k^. state ;
    end ;{ then }
  end ;{ print }
```

5、求精 2.1.5 —— expand (x) 的过程说明

输入结点指针 fnode, 逐一扩展 fnode 的子结点。若扩展出的子结点非重合结点，进入堆和查找树；否则进行调整。

```
  procedure expand ( var fnode:nodeptr );
  var
    substate                    : statetype ;{ 子状态}
    x                           : nodeptr ;{ 堆的新结点指针}
    y                           : listptr ;{ 查找树中的重合结点指针}
  begin
     while fnode 结点还可扩展 do begin
       new ( x );
      在 fnode^. state 的基础上生成子状态 sbcbstate ;
      x^. state ← substate ;{ 设置子结点 X 的参数}
      x^. g ← fnode^. g + 1;
      x^. f ← x^. g + calculate ( x^. state );
      x^. p ← fnode ;
2.1.5.1 y ← find__insert ( x );
{ 在查找树中寻找 X 的重合结点。若无重合，则 X 插入查找树并返回 nil; 否则返回重合结点的指针}
2.1.5.2 if y = nil then op . heap__insert ( x ){ 若无重合，则 X 插入堆}
                  else begin { 若有重合，则}
                    if (y^. p^. f > 0 ) and ( x^. g < y^. p^. g ) then begin
{ 若重合结点为待扩展结点且处解答树下方，则调整重合结点的参数，使路径经由 x 结点}
                    y^. p^. g ← x^. g ;
                      y^. p^. f ← x^. f ;
                      y^. p^. p ← fnode ;
2.1.5.3               op . decrease ( y^. p^. hp );{ 调整重合结点在堆中的位置}
                    end ;{ then }
                    dispose ( x );{ 释放 X 结点}
                  end ;{ else }
       end ;{ while }
```

```
end ;{ expand }
```

第四层

1、求精 2.1.2.1 ——heapfy (1)的过程说明

输入堆的结点序号 i。将该结点下降至合适位置，使序列保持堆性质。

```
procedure heaptp . heapfy ( i : integer );
var
   l , r , m                              : integer ;
 {左根序号，右根序号，i 结点与左根，右根中 F 值最小的结点序号}
   t                                      : nodeptr ;{ 辅助指针}
begin
   m ← i;
   while ( m <> 0 ) and ( i < = hpsize div 2 ) do begin
{若 i 结点未找到合适位置且有左、右儿子，则求 i 和左右儿子中 F 值最小的结点序号 m }
      l ← i * 2 ;    r ← l+1 ;
      if ( l< =hpsize ) and ( dts [ l ]^. f < dts [ m ]^. f ) then m ← l;
      if ( r< =hpsize ) and ( dts [ r ]^. f < dts [ m ]^. f ) then m ← r;
      if m=i then m ← 0 { 若 i 结点的 F 值最小，则找到合适位置}
             else begin { F 值最小的 m 结点与 i 结点对换}
               t ← dts [ i ]; dts [ i ] ← dts [ m ]; dts [ m ] ← t ;
               dts [ i ]^. hp ← i ; dts [ m ]^. hp  ← m ;
               i ← m { 继续对该子树进行堆排序}
             end ;{ else }
    end ;{ while }
end ;{ heaptp . heapfy }
```

2、求精 2.1.5.1 ——find__insert (x) 的函数说明

输入结点指针 newnode。在查找树中查询与 newnode 状态重合的结点。若无重合结点，则 newnode 插入查找树中某叶结点位置，并返回 nil；否则返回查找树中重合结点的指针。

```
function find__insert ( newnode : nodeptr ): listper ;
var
   c                                    : char { 状态值的比较字符}
  x,y                                   : listptr ; { 当前被查询的结点及其父结点 }
begin
   y ← nil ;  x ← root ;{ 从查找树的根结点开始沿树下降。y 指向 x 的父结点}
  repeat
    c ← comp ( newnode^. state , x^. p^. state );{ 比较 newnode 和 X 的状态值大小}
```

```
      y← x;
      if c=' <' then x← x^.L{newnode的状态值小，则向左转}
                else x← x^.r{否则向右转}
    until (x=nil)or (c=' ');{直至找到插入位置或重合状态为止}
    find_insert← x;{返回重合结点指针。若无重合状态，则返回nil}
    if x=nil then begin {若查找树中无重合状态，则在y结点后插入叶结点newnode}
       new(x); x^.l← nil ;x^.r← nil ; x^.f← y; x^.p← newnode;
       if c=' <' then y^.l← x
                 else y^.r← x;
       inc (tot);{累计扩展的子结点个数}
     end;{then}
  end;{find_insert};
```

3、求精2.1.5.2——op.heap_insert(x)的过程说明

输入结点指针newnode，将之插入堆中的合适位置。

```
procedure heaptp.heap_insert(newnode:nodeptr);
var
   i                                    : integer;
begin
    if hpsize = maxheap then begin
  {若堆中结点个数超过最大容量，则输出堆溢出信息，并退出程序}
      write ('heap overflow'); halt;
  end;{then}
  inc(hpsize);{堆长度+1}
  dts[hpsize]← newnode;{在堆的最下层加一片叶子}
  decrease(hpsize);{遍历该叶结点通向根的路径，以找到合适的位置插入}
 end;{heaptp.heap_insert}
```

4、最后求精2.1.5.3——op.decreas(y^.p^.hp)的过程说明

输入堆序号i。自下而上地遍历i结点通往根的路径，以找到合适的位置插入。

```
procedure heaptp.decrease(i:integer);
var
   t                                    : nodeptr;{辅助指针}
begin
    t← dts[i];{暂存堆中序号为i的结点}
  while (i>1)and(t^.f<dts[i div 2]^.f) do begin
{若i的父结点的F值大于T的F值，则i的父结点向下复制，并继续往上追踪}
     dts[i]← dts[i div 2];dts[i]^.hp← i;
```

```
    i ← i div 2;
  end;{while}
  dts[i] ← t;t^.hp ← i;{i的父结点的F值小于等于T结点的F值，则T结点插入第i个空位}
end;{heaptp.decrease}
```

五、使用 A* 算法求 8 数码问题

有了对 8 数码问题的算法分析和 A* 算法的程序框架，我们不难得出其程序题解：

主程序不变。常量说明，类型说明和变量说明中增加了一些专为 8 数码问题设置的参数：

```
const
  tbisize                    = 3 {棋盘规模}
  maxheap                    = 3000;{堆的最大容量}
  md                         : array[1..4,0..1]of shortint
                             = ((1,0),(0,1),(-1,0),(0,-1));
{md[i,0]——i方向的水平增量; md[i,1]——i方向的垂直增量; (1< i< 4)}
type
  statetype                  = array[1..tbisize,1..tbisize]of  shortint;
                             {棋盘状态类型}
var
  pos                        : array[0.. tbisize * tbisize-1, 1..2] of
shortint;
{pos[i,1]——数码i的X坐标; pos[i,2]——数码i的Y坐标}
```

函数和过程说明中，查找树和堆操作的函数过程不变。例如 find_insert 函数，heaptp.heapfy 过程，heaptp.decrease 过程，heaptp.heap_insert 过程 heaptp.extract_min 过程。其它过程和函数在框架基础上根据解题要求增添了相应内容：

```
Function Comp(Var STATE1, STATE2: STATETYPE):Char;
{自上而下，自左而右地检查棋盘state1和state2。若state1中(i,j)的数码大于state2(i,j)的数码，则返回'>'; 若小于，则返回'<'; 若state1和state2的状态相同，则返回'='}
Var
  i, j          : Shortint;
Begin
  For i := 1 to TblSize Do For j := 1 to TblSize Do
    If STATE1[i, j] <> STATE2[i, j] Then Begin
      If STATE1[i, j] > STATE2[i, j] Then Comp := '>'
                                     Else Comp := '<';
```

```
        Exit
      End ;{ then }
    Comp := '='
End ;{ comp }

Function Calculate ( Var STATE : STATETYPE ): Shortint ;
Var
  i , j , c        : Shortint ;
{输入棋盘状态 state, 计算各数码离家 (目标状态) 的距离和, 并作为 state 的启发函数值返回}
Begin
  c := 0 ;
  For i := 1 to TblSize Do For j := 1 to TblSize Do
    If STATE [ i , j ] <> 0 Then
      Inc ( c , Abs ( Pos [ STATE [ i , j ], 1 ] - i ) + Abs ( Pos [ STATE [ i , j ], 2 ] - j ));
  Calculate := c
End ;{ calculate }

Procedure Expand ( Var FNode : NodePtr );
Var
  x              : NodePtr ;
  y              : ListPtr ;
  i , u , v      : Shortint ;
Begin
  For i := 1 to 4 Do Begin {逐一分析空位的四个方向}
    u := FNode^. zi + Md [ i , 0 ]; v := FNode^. zj + Md [ i , 1 ];
    {求空位 i 方向的相邻格 ( U , V )}
    If ( u > 0 ) And ( u <= TblSize ) And ( v > 0 ) And ( v <= TblSize ) Then Begin
      New ( x );{若 ( U , V ) 在棋盘内, 则空位移至 ( U , V ), 生成子结点 X }
      x^. state := FNode^. state ;{ 设置子结点 X 的参数}
      x^. state [ FNode^. zi , FNode^. zj ] := x^. state [ u , v ];
      x^. state [ u , v ] := 0 ;
      x^. zi := u ; x^. zj := v ; x^. g := FNode^. g + 1;
      x^. f := x^. g + Calculate ( x^. state );
      x^. p := FNode ;
      y := Find__Insert ( x );
    {在查找树中查询重合结点。若没有与 X 重合的状态, 则 X 分别插入堆和查找树}
      If y = nil Then Op . Heap__Insert ( x )
```

```
        Else Begin
{若X重合于查找树中某待扩展结点且重合结点处于X的下方，则修正重合结点的参数
并调整其在堆中的位置}
          If (y^.p^.f > 0) And (x^.g < y^.p^.g) Then Begin
            y^.p^.g := x^.g; y^.p^.f := x^.f;
            y^.p^.p := FNode;
            Op.Decrease(y^.p^.Hp)
          End;{then}
          Dispose(x){释放X}
        End{else}
    End{then}
  End{for}
End;{expand}

Procedure Print(k : NodePtr);
{顺k^.p指针递归打印初始结点至k结点的路径方案}
Var
  i, j          : Shortint;
Begin
  If k <> nil Then Begin
    Print(k^.p);
    Writeln('Step = ', k^.g);
    For i := 1 to TblSize Do Begin
      For j := 1 to TblSize Do Write(k^.state[i, j]:2);
      Writeln
    End{for}
  End{then}
End;{print}

Procedure Main;
Var
  x             : NodePtr;
  Succeed       : Boolean;
Begin
  tot := 0;{扩展的结点个数初始化}
  New(x);
  With x^Do Begin{设置初始结点X参数}
    STATE := stt; zi := Pos[0, 1]; zj := Pos[0, 2];
```

```
    g := 0; f := Calculate(Tbl); p := nil;
  End;{with}
  New(Root);{X作为查找树的初始根}
  Root^.l := nil; Root^.r := nil; Root^.p := x;
  Op.HpSize := 1;{X作为堆顶元素}
  Op.Dts[1] := x;
  Succeed := False;{成功标志初始化}
  While Not Succeed And (Op.HpSize <> 0) Do Begin
   {若未找到目标结点且堆非空，则循环}
    x := Op.Extract_Min;{取出堆顶元素X}
    x^.f := - x^.f;{X设已扩展标志}
    If Comp(x^.STATE, trm) = ' =' Then Begin
   {若X是目标结点，则设成功标志，递归打印初始结点至X的路径}
      Succeed := True; Print(x)
    End{then}
    Else Expand(x);
  End;{while}
  If Not Succeed Then Writeln(' No Answer!')
End;{main}

Procedure Init;
Var
  F              : Text;{文件变量}
  i, j           : Shortint;
Begin
  Assign(F, 'INPUT.DAT');{文件变量与文件名串连接}
  Reset(F);{文件读准备}
  Writeln('First State :');{读入初始棋盘状态}
  For i := 1 to TblSize Do For j := 1 to TblSize Do Begin
    Read(F, stt[i, j]);
    If stt[i, j] = 0 Then Begin{设置空位的初始坐标}
      Pos[0, 1] := i; Pos[0, 2] := j
    End{then}
  End;{for}
  Writeln('Last State :');{读入目标棋盘状态}
  For i := 1 to TblSize Do For j := 1 to TblSize Do Begin
    Read(F, trm[i, j]);
    If trm[i, j] <> 0 Then Begin{设置各滑牌的目标坐标}
```

```
      Pos[trm[i, j], 1] := i; Pos[trm[i, j], 2] := j
    End {then}
  End {for}
End; {init}
```

六、 分阶段 A* 算法

八数码问题中的启发函数是很粗糙的，它不能精确估价出交换两个滑牌位置的困难程序，不能使搜索集中指向目标，启发能力差，各 F 值相差也不大，因此 A* 搜索的范围就变得相当大。特别是，一旦给出一个需要转换步数很多的初始条件，则往往会在找到最佳路径之前出现有效存储空间被用光的情况。为了避免前功尽弃，就得修剪解答树，丢弃 F 值大的一些结点，腾出它们所占据的存储空间，以推进搜索向纵深发展。

那么如何推进纵深发展，一句话，分阶段搜索。

我们定义一个内存溢出的临界值，例如 1000 个字节。我们每扩展一个结点后，通过调用标准函数 memavail 查询当前内存的可用字节数。当其小于临界值时，便标志一阶段搜索完成。下一阶段搜索开始前，先从堆中挑选出一定数量的结点（结点个数记为常量 maxleft），这些结点的 F 值最小。在查找树和堆中，初始结点通往这些结点的路径上的所有结点置保留标记（八数码问题中，保留结点的标记设为空位的某坐标（X 或 Y）值取负。）然后丢弃堆和查找树中的非保留结点，腾出所占的存储空间。A* 算法由新堆中的这些最优结点出发，重新开始下一阶段的搜索。这个过程反复进行下去，一直至产生下述三种情况之一：

（1）找到一个目标结点，搜索成功；

（2）堆空，说明初始状态不可能到达目标状态；

（3）释放非保留结点后，内存的可用字节数还是小于临界值，失败退出；

分阶段 A* 算法的程序流程基于 A* 算法。原来的常量、类型定义、变量、函数、过程以及主程序依然保留。另外

1. 定义一个常量 maxleft，为分阶段时堆中保留的结点数；增设一个数组 pack：array [..maxleft] of nodeptr，以存储保留的结点指针；增加阶段数变量 stage；

2. 修改 A* 算法的主过程 main，增加了分阶段的内容：

```
procedure main;
var
  x                                  : nodeptr; {结点指针}
  succeed                            : boolean; {成功标志}
begin
  tot ← 0; stage ← 1; {子结点数和阶段数初始化}
  new (x) {为堆的初始根申请内存并设定参数}
  with x^ do begin
    state ← stt;
```

```
    g ← 0 ; p ← nil ;
    f ← calculate ( state );
  end ; { with }
  new ( root ); { 为查找树的初始根申请内存并设定参数}
  root^. l ← nil ;
  root^. r ← nil ;
  root^. p ← x ;
  op . hpsize ← 1;
  op . dts [ 1 ] ← x ; { 堆的初始根作为堆顶元素}
  succeed ← false ; { 成功标志初始化}
  while ( not succeed ) and ( op . hpsize <> ) do begin
  { 若未找到目标状态且堆非空，则循环}
    x ← op . extract__min ; { 取出堆顶结点 X }
    x^. f ← −x^. f ; { 置 X 已扩展标志}
    if comp ( x^. state , trm ) = ' =' 
    { 若 X 为目标状态，则设成功标志，并递归打印起始结点至 X 的最佳路径}
       then begin
            succeed ← true ; print ( x );
       end { then }
       else begin
          expand ( x ); { 否则扩展 X 的所有儿子}
         if memavail <1000 then begin
          { 若内存可用字节数小于临界值，则分阶段}
            打印子结点数 tot 和刚结束的阶段序号 stage ;
          * cut ;
{ 保留堆中 maxleft 个 F 值最小的结点，重新组合成新堆。查找树和堆中除初始结点通往这些结点的路径上的结点保留外，其它结点释放}
            inc ( stage ); { 下一阶段搜索开始}
            if memavail <1000 then begin
{ 若修剪解答树后的可用字节数还是小于临界值，则无法分阶段去，失败退出}
              输出内存溢出信息;
              halt ;
             end ; { then }
             end { else }
       end ; { while }
end ; { main }
```

3. main 过程在执行分阶段任务时调用了一个 cut 子过程，以保留堆中若干个 F 值最

小的结点。初始结点通往它们的路径上的结点也在查找树中保留下来。查找树和堆中的非保留结点被释放。

```
procedure cut ;
var
  i,t                                    : integer;
  x                                      : nodeptr;{保留路径上的结点指针}
begin
   i←0;{保留的结点数初始化}
  while (i < maxleft)and(hpsize <> 0)do begin
   {若未保留下足够的结点且堆非空，则循环}
    inc{i};{累计保留的结点数}
    x← op.extract_min;{取出F值最小的堆顶元素X,暂存pack数组}
    pack[i]← x;
    while (x^的状态未置保留标记) and (x <> nil) do begin
       x^的状态置保留标记; x← x^.p;
    end;{while}
  * packing(root);{删去查找树和堆中未置保留标志的结点}
    hpsize← 0;{堆置空}
    for t← 1 to i do op.insert(pack[k]);
   {将原堆中保留下的结点重新组合成新堆}
end;{cut }
```

其中被调用的 packing 子过程通过后序遍历查找树的方法，删除未被置保留标志的结点。

```
procedure packing(x:listptr);
begin
   if x <> nil then begin
      packing (x^.l); packing(x^.r);{后序遍历左，右子树}
    *if x^.p 的状态未置保留标记 then tree __ delete(x)
                                  else 撤去x^.p的保留标记;
  end;{then}
end;{packing}
```

这样，又引出一个问题——在保持查找树性质的前提下如何删除树中的一个结点。packing 调用的子过程 tree__delete 就是用来解决这个问题的。

4、在一棵查找树中删除一个结点 X,无非有三种情况：

(1)如果 X 没有子女，则把 X 删除即可，例如（图 6.5－7）。

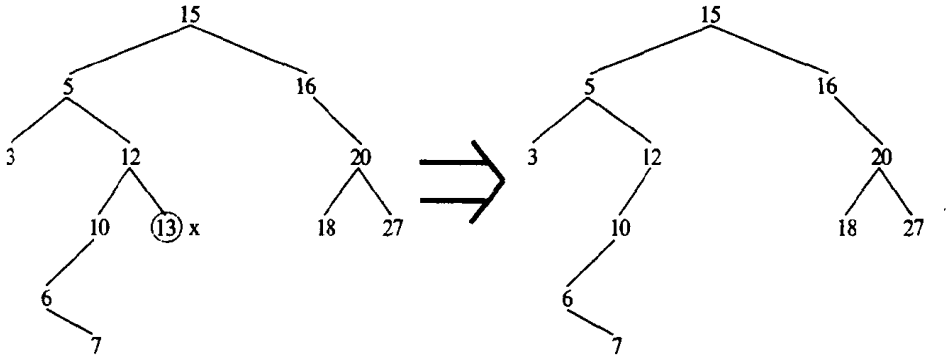


图 6.5-7

(2)如果 X 只有一个子女，则把 X 抽掉即可，例如（图 6.8－8）。

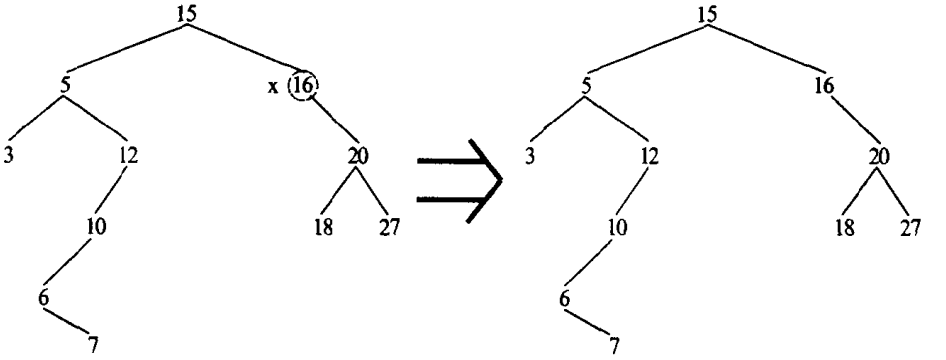


图 6.5-8

(3)如果 X 有两个子女，则先抽掉中序下 X 的后继 Y, 它至多有一个子女。然后用 Y 的内容代替 X 的内容，例如（图 6.5-9）。

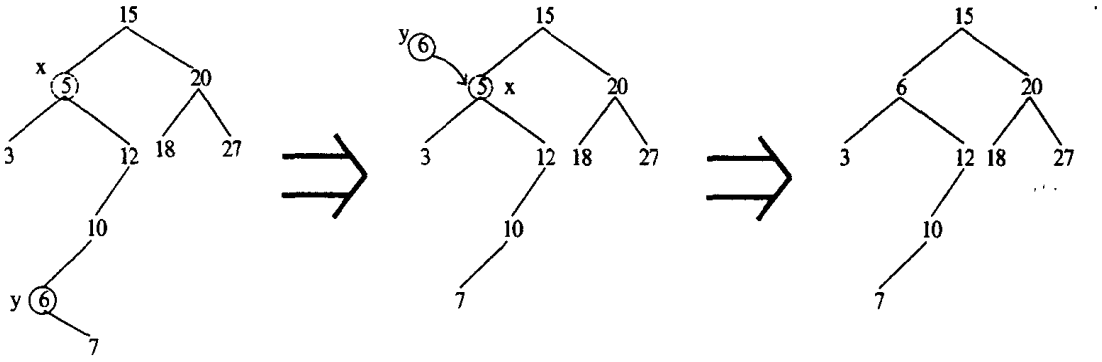


图 6.5-9

tree__delete 过程综合上述三种情况，给出了在查找树中删除一个结点的算法描述。

```
procedure tree__delete ( x : listptr );
var
  y , z                                    : listptr ;
  { 被删位置的指针和其子女的指针 }
begin
   if ( x^. r = nil ) or ( x^. l = nil )
{ 确定要删除的结点 Y。若 X 至多一个子女，则 Y 为 X; 若 X 有两个子女，则 Y 为中序下
```

```
X的后继}
  then y← x
  else begin
     y← x^.r;
    while y^.l <> nil do y← y^.l;
  end;{else}
  dispose(x^.p);
  x^.p← y^.p;{将X从堆中删除}
  if y^.l=nil then z← y^.r{Z被置为Y的非nil子女。或者当Y无子女时Z被置为nil}
  if y=y^.f^.l then z← y^.f^.l← z{通过修改Y的父结点的子女指针将y删除}
                else y^.f^.r← z;
  if z <> nil then z^.f← y^.f;{Y的非nil子女的父指针指向Y的父亲}
  dispose(y);{释放Y}
  dec(tot);{子结点数减1}
end;{tree_delete}
```

§6.6 博奕树

实现计算机与人对奕，诸如象棋、围棋等双方对垒的游戏，是一件饶有趣味的事情。在这一章节里，我们将讨论实现这种人机完备信息博奕时常用的树型结构及其基本算法。

所谓人机完备信息博奕，是指人机对垒，依次轮流走步。为了引起对奕者的兴趣，计算机一方必须尽可能多地知道对方已经走过的棋步和将可能走的棋步。这一切都是依据比赛规则确定，没有任何机遇。对于编程者来说，要使得计算机能懂得比赛规则并记住当前格局，就得建立恰当的数据结构；要使得计算机能在对奕中选择最可能致胜的走步，就得为之设计一个"聪明"的算法。博奕树适应了这两个需要，它将对垒的初始格局作为根结点，把所有由初始格局经过合法一步得到的新格局作为其孩子结点，然后以新格局为根，由此新格局产生的格局为其孩子结点，依次类推，得到一棵包罗对垒中各种可能格局变化的树，并通过对这棵树的遍历分析确定计算机的走步。这棵树就是所谓的博奕树。

一、博奕树的数据结构和算法思想

下面，我们结合井字棋的例子来具体讨论博奕树的数据结构和算法。

【例1】井字棋游戏

问题描述:

从一个空的N*N的棋盘（例如N=3）

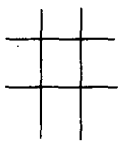


图 6.6-1

开始，甲乙二人轮流放置棋子到棋盘上未被占据的方格中。如果甲第一个放，他把棋子放在中央方格里:

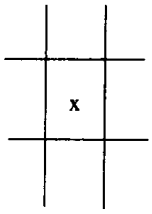


图 6.6-2

然后轮到乙放，他把棋子放在第一行中间的方格里:

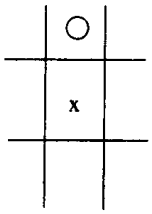


图 6.6-3

于是又轮到甲放，……，如此进行下去，判定胜负的方法是：若某一游戏者若有N(N = 3)枚棋子占据了一横线或一竖列，或一对角线，则该游戏者获胜；若至整个棋盘被占满还没有一方获胜，则为平局。

算法分析:

我们让计算机模拟甲方，称为max选手，与计算机对手的乙方称min选手，上述博奕由这两位选手对垒，双方依次轮流走步。

图6.6-6的树以棋盘状态作为结点，描述了从min方走后的棋盘状态（图6.6-4）

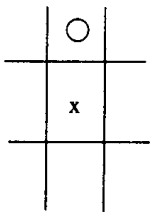


图 6.6-4

出发可能出现的所有格局。初始状态（图6.6-5)

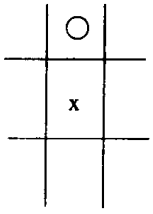


图 6.6-5

作为树根，这时轮到max方放。max选择某个空格放置棋子后的棋盘所有可能状态，都作为根的子女出现在树的第一层。第二层的结点是轮到min方时的棋盘状态。min方

从第一层的某个结点出发，选择某个空格放置棋子后的所有可能棋盘状态都作为该结点的子女出现在第二层（由于版面限制，第二层的结点未全部画出）。图 6.6 - 6 的树称作博奕树，这棵树的第二层并不是游戏终止时的棋盘状态，我们还可以画出第三层、第四层、…… 的结点，但在一般情况下，由于计算机存储器大小和运算速度的限制，不能把博奕树构造到游戏终止时的棋盘状态，而只能构造到一定深度。图 6.6 - 6 是井字棋的一棵深度为 2 的博奕树。

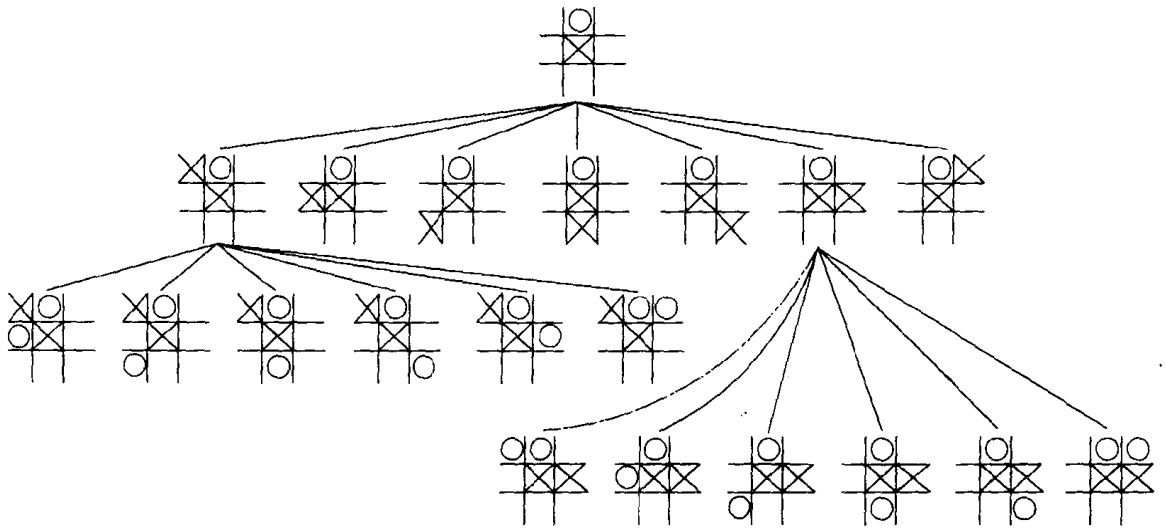


图 6.6-6

由图 6.6 - 6 可以看出，博奕树具有如下特点：

1、奇层（min 层）结点，是 max 方的走后状态，下一步轮 min 方走步。

偶层（包括 0 层在内的 max 层）结点是 min 方的走后状态（除初始状态外），下一步轮max 方走步。

2、博奕树中任意结点的度最从等于不同的合法移动数。例如对井字棋的任一格局来说，有9－x（x——当前占据的方格数）种合法移动，任意结点的度为 9－x。

3、从根结点到当前格局的一条路径，即为计算机和对方依次轮流走过的棋步。博奕树的深度不超过该游戏最长的有效序列的长度。例如井字棋的博奕树深度不超过 9，即从空棋局开始最多放 9 次棋就可以决定胜负。

4、博奕树的叶子表示对垒的终结状态，即某方取胜或者和局。在奇层到达终结状态时，min 方才能取胜或和局，在偶层到达终结状态时，max 方或者胜或者和局。

在构造一棵博奕树时，计算机的目的是寻找一步好棋，然后走完这一步，等待对方回手，并从这个新的位置上再次寻求一步好棋。由于时空限制，终局条件除获胜和和局外，还需要指出希望产生的深度为多少。一般说来，博奕树的实际深度不可能达到游戏的最长的有效序列的长度。例如井字棋要放 9 次棋则要产生 9＋9＊8＋9＊8＊7＋ … ＋9！个结点，计算机无论如何满足不了这种储存量。在对垒过程中，max（计算机）方要战胜对方，它必须有一个使它获胜机会最大的优先走步的估计，这个估计可以通过对博奕树的叶结点应用一种静态估价函数的办法来实现。

$$\text{evlation}(q)=\begin{cases}\text{maxint} & q\text{ 是 max 方获胜的终局状态}\\ -\text{maxint} & q\text{ 是 min 方获胜的终局状态}\\ \text{max 方获胜的希望程度值} & q\text{ 是其它格局状态}\end{cases}$$

静态估价函数把一个用数字表示的值赋予格局状态 q，是估计状态 q 的值，或价值的估量。因此，在 max 方有一个获胜机会的状态，静态估价函数值就大，反之就小。或者在 max 方获胜的状态，静态估价函数有最大值。而在确保 min 方获胜的状态，静态估价函数有最小值，而对 max 方和 min 方都无获胜的博奕位置，则产生接近于 0 的函数值。

对井字棋游戏来说，我们是这样设计估价函数的:

$$\text{evlation}(q)=\begin{cases}\text{maxint} & q\text{ 是 max 方获胜的终局棋盘状态}\\ -\text{maxint} & q\text{ 是 min 方获胜的终局棋盘状态}\\ \text{rcdc}-\text{rcdo} & q\text{ 是其它棋盘状态}\end{cases}$$

其中：rcdc——所有空格都放上 max 的棋子 (x) 后，max 的 N 个棋子所组成的行、列或对角线的总和。rcdo——所有空格都放上 min 的棋子 (o) 后，min 的 N 个棋子所组成的行、列或对角线的总和。

由上可见，一个棋盘状态对计算机愈有利，它的 evlation 值愈高、计算机获胜的终局状态 evlation (q) 值取 maxint 。但是不是说，计算机在所有可能下法中取 evlation (q) 值最大的一个，就可以必胜呢？不一定。例如:

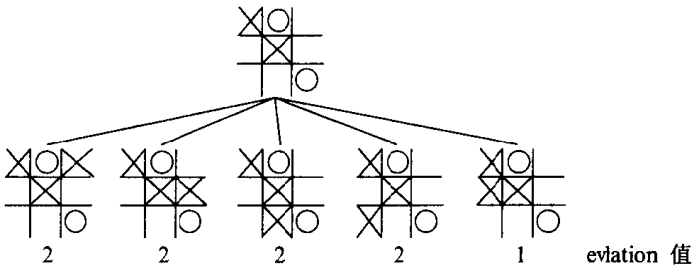


图 6.6-7

从根结点的棋盘状态出发有五种可能下法。这些棋盘状态中有四个取相同的最大值 2，这些状态对计算机有利的程度是不同的，第 4 种状态是计算机必胜状态，而 1、2、3 种状态都可能由于对方的努力而最终成为平局。事实上第 5 种状态，虽然其 evlation 的值低，但它也是必胜状态。这说明利用深度为 1 的博奕树来确定下一步棋的走法，也就是说只向前看一步就决定怎样走是难于保证取胜的，应尽可能加大博奕树的深度，往前多看几步。

我们再来定义一个极大极小过程 search (q)，通过加大博奕树深度的办法来重新估算当前棋盘状态 q 对计算机有利的程度。设 c1 ... cd 为非终局棋盘状态 q 在博奕树中的后继棋盘状态 (由 d 种可能下法产生):

$$\text{search}(q)=\begin{cases}\text{evlation}(q) & \text{当 q 是终局状态}\\ \max\{\text{search}(ci)\}\ (1<i<d) & \text{当 q 是 min 方的非终局状态，下一步轮 max 方走}\\ \min\{\text{search}(ci)\}\ (1<i<d) & \text{当 q 是 max 方的非终局状态，下一步轮 min 方走}\end{cases}$$

定义这个极大极小过程的道理是显而易见的。我们设想，如果 max 方真的在叶结点之间进行选择，那么它就会选择具有最大估价值的结点，即对于计算机最有利的状态。所以 min 层结点的父结点（max 结点）所赋的倒推值等于叶结点估价值中的最大值；另一方面，如果 min 方在叶结点之间进行选择，那么他大概总是选择具有最小估价值的结点，即对于计算机最不利的状态。所以 max 叶结点的父结点（min 结点）所赋的倒推值等于叶结点估价值中的最小值。在所有叶结点的各个父结点都已经赋好倒推值之后，我们再向上倒推一层的值，还是假设 max 总是选择具有最大倒推值的结点而 min 总是选择具有最小倒推值的结点。我们继续一层一层地向上计算倒推值，最后直至根结点被赋值为止。根结点则在第一层结点中选择具有相同倒推值的那一个结点作为它的下一步。这种从叶结点开始，逐层倒推赋值至根结点的过程，就是后序遍历博奕树的过程。

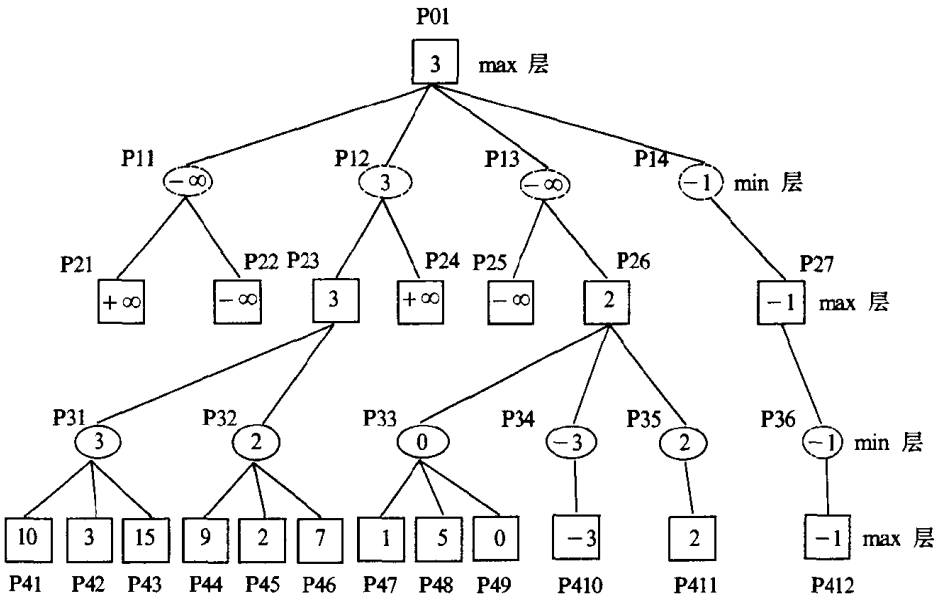


图 6.6-8

例如图 6.6-8 中各结点中标的值就是用极大极小过程确定的。方形结点所在的层称为 max 层，对于 max 层的所有结点来说，轮到计算机走下一步。圆形结点所在的层称为 min 层，对于 min 层的所有结点来说，轮到对方走下一步。树根代表的棋盘状态 P_{01} 的值为 3，就是说从该棋盘状态出发，计算机所能保证的最好结果是达到一个值为 3 的棋盘状态，也是为了避免立即败北而被迫走的一步。如果计算机从 P_{01} 状态出发选择下一个状态 P_{11}，企图达到 P_{21} 而获胜，但下一步轮对方走，min 方选择 P_{22}，结果计算机只好拱手认输。在状态 P_{01} 处，计算机的最好选择是状态 P_{12}。

在按照后序遍历的次序估算各结点值，构造博奕树的过程中，一旦求出了根的值和确定了下一步的走法，当前构造的博奕树就没有用了，到确定以后的走法时，需要构造的已是博奕树的另外部分了。搜索的时空效率取决于每次被构造的博奕树的规模。是否一定要构造整个博奕树后才能正确地求出根的 search 值和确定下一步的走法呢？不是的。我们可以分析一下，在求 search（P_{01}）的过程中，当求出 search（31）=3 以后，因为 P_{23} 处于 max 层，所以就已经知道 search（23）的值至少等于 3；接着，当求出 search（P_{45}）=2 以

后，因为 P_{32} 处于 min 层，所以就已经知道 search (P_{32}) 的值至多等于 2。(P_{32}) 的值不可能影响 search (P_{23})，而不管 P_{32} 的其余子女的 search 值为多大或多小。由此可见、不生成结点 P_{46}，绝对不会影响 max 的最好的优先走步。(图 6.6 - 8) 的博奕树的其它部分，也同样存在类似情况。

那么，如何在博奕树中裁剪去这些多余的枝节，减少搜索工作量呢？这里，我们提出一种称作 $\alpha - \beta$ 裁剪的优化措施。

α 裁剪—— 一个 max 层结点的 α 值定义为该结点的最小可能值。如果已经确定一个 min 层结点的值小于或等于它的父母结点的 α 值，则可以停止产生这个 min 层结点的其它子女。

例如图 6.6 - 8 中，一旦 P_{31} 的值确定为 3，则 P_{23} 的 α 值成为 3，search (P_{45}) 小于 P_{23} 的 α 值，就意味着 P_{46} 可以不必产生了。

β 裁剪—— 一个 min 层结点的 β 值定义为该结点的最小可能值。如果已经确定一个 max 层结点的值大于或等于它的父母结点的 β 值，则可以停止产生这个 max 层结点的其它子女。

例如图 6.6－8 中，一旦 search (P_{25}) 的值确定为 $-\infty$，则 P_{12} 的 β 值成为 $-\infty$，search (P_{47})、search (P_{48})、search (P_{49}) 算出后，确定了 search (P_{33}) 的值为 0，大于 P_{13} 的 β 值，从而确定了 P_{26} 的 α 值大于 P_{13} 的 β 值，于是 P_{26} 的其余子女就不必产生了。

$\alpha - \beta$ 裁剪——α 裁剪与 β 裁剪的规则结合。

$\alpha - \beta$ 裁剪用于图 6.6 - 8，事实上整个 P_{26} 为根的子树都不必产生。这是因为当 search (P_{12}) 确定为 3 时，P_{61} 的 α 值成为 3，search (P_{25}) 小于 P_{01} 的 α 值，确定 P_{13} 的 α 值小于 P_{01} 的 α 值，于是进行了 α 裁剪。

一般地讲，若当前结点的倒推值为 value，它的父结点的倒推值为 oldvalue，走步对象用 mode 表征 (mode 值为 max 或 min)，则 $\alpha - \beta$ 裁剪过程可以描述如下：

```
if not (((mode=max) and (value>=oldvalue)) or ((mode=min) and (value<=oldvalue)))
    then 生成子结点，继续极大极小过程
    else 裁剪以当前结点为根的子树;
```

当根结点及其子结点的最终倒推值全都求出时，上述过程便宣告结束，而最好的优先走步就是走向具有相同倒推值的那个子结点。在搜索深度相同的条件下，采用这个过程所获得的走步总是和一般的极大极小过程一样的好。区别在于由于增加了 $\alpha - \beta$ 裁剪，通常不必构造整个博奕树便可以正确地求出根结点的倒推值和确定下一个最好的优先走步。在同样的存储需求的条件下，博奕树的深度比较一般的极大极小过程可以增加一倍。

二、博奕树的算法框架

求精分析算法框架：

第一层

```
1．初始化
    1.1 定义和说明
    1.2 init;
        step ← 0;   {对垒步数初始化}
        begin
2．对垒
    2.1 while (goal(s) = 0) and (step < maxstep) do
        {若s非终局状态且对垒步数未超过最大值}
        begin
         case mode of
    2.2 max:compter__do;{计算机走一步}
        min:begin
            repeat
            writeln ('It is your turn!Please do it!');
            按格式输人对方的走步信息;
            until 走步合理;
            s.state ← 新格局;
            mode ← max;{置下一走步对象为计算机}
            end;{min:}
        end;{case}
    2.3 print(s); step ← step+1;        {打印新格局，累计对垒步数}
        end;{while}
3．博奕结果
 if goal(s) = -maxint
    then writeln('congrration!!you win')  {打印对方赢}
    else if goal(s) = maxint then writeln('what a pity!!I win。')
     {打印计算机赢}
                                  else writeln('OK!We are draw!');{打印和局}
 readln;
 end.{main}
```

第二层

1、求精 1.1——定义和说明

```
Const
 maxsuccessor = ...;      {博奕树第一层结点的最多个数}
 maxdepth = ...;           {博奕树的深度界限}
 maxstep = ...;            {对垒步数的最大值}
Type
```

```
statetype = **...;          {定义格局的数据类型}
node=record                 {博奕树的结点类型}
      state:statetype;  {格局}
      value:integer;    {倒推值}
      end;
modetype = (max, min); {当前走步对象的数据类型。mode ┬ max 轮计算机走步
                                                      └ min 轮对方走步
Var
                s:node;                          {表征对垒的结点}
   successor:array[1..maxsuccessor]of node;  {存储第一层子结点}
successornum:integer;                          {第一层子结点的实际个数}
          mode:modetype;                       {当前走步对象}
         value:integer;                        {当前倒推值}
          step:integer;                        {当前对垒步数}
```

关于过程和函数说明，则在下面逐一给出。

2、求精 1.2 ——init 的过程和说明

```
Procedure init;
var
 x:char;
begin
 writeln('The game now begin!!');
  s.state ← 输人初始格局;
   writeln('Who is first? Y(you) or C(computer)');
    readln; read(x); readln;                    {输人谁先走信息}
   if upcase(x)='Y'  then mode ← min            {设置走步对象}
                     else mode ← max;
  if mode=max then s.value ← -maxint            {设置根结点的初始倒推值}
              else s.value ←  maxint;
end; {init}
```

3、求精 2.1 ——goal(s)的函数说明

```
Function goal(q:node):integer;
begin
 if q.state 是计算机获胜格局
    then goal ← maxint
    else if q.state 是对方获胜格局
           then goal ← -maxint
           else goal ← 0;      {goal 为零表示 q 是非终局状态或和局状态}
end; {goal}
```

4、求精 2.2——computer_do 的过程说明

```
Procedure computer_do ;
begin
 计算第一层的子结点个数 successornum , 逐一产生根结点的子结点,
 顺序送入 successor [1..successornum];
  value ← -maxint ;
  for i:=1 to successornum do
    begin
2.2.1  successor [i]. value ← search (successor [i], min , value , 0);
                          {逐一求出第一层中每个子结点的倒推值}
2.2.2  value ← select ( value , successor [i], value , max );
                          {取其中最大的倒推值赋根结点}
    end ; {for }
  for i:=1 to successornum do
    if value=successor [i]. value
    then begin
           s ← successor [i];
{求出具有与根结点相同倒推值的那个子结点存入 s , 作为下一走步}
           mode ← min ;  {下一步轮对方走}
           writeln (' I have put a chess !');
           exit ;
         end ;{ then }
end ; { computer_do }
```

5、求精 2.3——print (s) 的过程说明

```
Procedure print (q : node);
begin
 writeln ;
  writeln (' Now the stste is :');
 按输出格式打印 q . state ;
end ; { print }
```

第三层

1、求精 2.2.1——search (successor [i], min , value , 0) 的函数说明

```
Function search (q : node ; mode : modetype ; oldvalue , depth : integer ): integer ;
      {q- 待扩展结点; mode- 走步对象; oldvalue- 父结点的倒推值; depth- 当
前深度}
  var
```

```
    successor : node ;                {q 的子结点}
        value : integer ;             {目前求出的结点的倒推值}
begin
 if ( depth < maxdepth ) and ( goal ( q ) = 0 )
                                  {当前未越出深度界限且 q 非一方获胜的终局状态}
    then begin
          if mode = max                        {根据走步对象，设倒推值}
            then value ← −maxint
            else value ← maxint ;
          while q 结点未扩展完 do
           begin
            if not ((( mode = max ) and ( value > oldvalue )) or (( mode = min ) and ( value
              < oldvalue )))
              then begin     {若 q 结点不满足 α − β 裁剪条件}
                   根据走步对象 mode 值，扩展出 q 结点的子结点 successor ;
                   if mode = max
                     then value ← select ( value ,
                          search ( successor , min , value , depth + 1 ), max , )
                     else value ← select ( value ,
                          search ( successor , max , value , depth + 1 ), min );
                                  {通过后序遍历求出 q 结点的当前倒推值}
                 end  {then}
           end ;{while}
          search ← value ;
        end {then}
    else begin
          if goal ( q ) <> 0 then search ← goal ( q )
        {若 q 结点状态为一方获胜的格局，则 q 结点的倒推值为极大（或极小）值}
2.2.1.1                 else search ← evlation ( q );
                              {若达到深度界限，则叶结点 q 的倒推值即为估价函数值}
         end ; { else }
end ;{ search }
```

search 即为带 $\alpha-\beta$ 裁剪的极大极小过程。通过后序遍历的方法构造以 q 结点为根的博奕树，

计算并返回 q 结点的倒推值。

2、求精 2.2.2 —— select (value , successor [i] . value , max) 的函数说明

```
Function select ( a , b : integer ; mode : modetype ): integer ;
```

```
{a－结点当前的倒推值；b－其子结点的倒推值；mode－倒推值为a的那个结点
所对应的走步对象}
    begin
     if ((a>b) and (mode=max)) or ((a<b) and (mode=min))
        then select ← a
        else select ← b;
    end; {select}
{若轮max方走步，则在倒推值为a和b的两个结点之间选择具有最大倒推值的结
点，该最大值作为max结点的当前倒推值；若轮min方走步，则在倒推值为a和b的两
个结点之间选择具有最小倒推值的结点，该最小值作为min结点的当前倒推值}
```

第四层

最后求精 2.2.1.1 ——evlation(q)的函数说明

```
Function evlation (var q: node): integer;
var
  e: integer;
begin
 计算叶结点q.state对计算机有利的程度值e;
 evlation ← e;
end; {evlation}
```

三、应用框架解题的一个实例

在井字棋的程序题解中，程序结构和大部分程序行与算法框架如出一辙。为了节省篇幅和方便读者阅读，我们还是采用求精分析的方法。相同的程序行不再赘述，由'.....'表示。而算法框架中的文字部分则由针对井字棋问题而言的程序描述取代。

第一层

第一层的程序结构和大部分程序行与算法框架一致，不再赘述。这里，仅对文字部分给出程序描述。

```
1. 初始化
  1.1 定义与说明;
  1.2 init;
  …… …… …… ……
2. 对垒
  2.1 while (goal(s) = 0) and (step <=n*n) do
        begin      {若s结点是非终局状态且对垒步数未达到最大值，继续对垒}
          case mode of
    2.2   max: computer_do;
          min: begin
```

```
                repeat
                  writeln('It is your turn! Please do it!');
                  write('column:'); readln(col);      {输入走步信息}
                  write('line:'); readln(lin);
                until(s.state[lin,col]='.');          {直至走步合理}
                s.state[lin,col]←'O';                 {新格局置入s结点的状态域}
                mode← max;
              end; {min:}
          end; {case}
      2.3 print(s); step← step+1;
        end; {while}
  3. 博弈结果
     …… …… …… ……
end. {main}
```

第二层

1、求精 1.1——定义与说明

```
Const
  maxsuccessor =10;          {博弈树第一层结点的个数}
  maxdepth =3;               {博弈树的深度界限}
```

{由于棋盘矩阵的阶数 n 随机输入，因此 maxsuccessor 只能预先约定一个最大值，而 maxstep(n*n) 无法定义为常量}

```
Type
  statetype = array[1..4,1..4] of char;     {格局的数据类型}
      node = record                          {结点的数据类型}
           state: statetype;
           value: integer;
           end;
  modetype = (max,min);                     {走步对象的数据类型}
Var
  n: integer;                {棋盘矩阵的阶数}
  col,lin: integer;          {棋子的放置位置（列和行）}
  s: node;                   {下述各变量的意义见算法框架}
  successor: array[1..maxsuccessor] of node;
  step,successornum,value: integer;
  mode: modetype;
```

至于函数和过程说明，则在下面逐一给出。

2、求精 1.2——init 的过程说明

```
Procedure init;
var
  x:char;
  i,j:integer;
begin
  writeln('The game now begin!!');
  writeln('Computer__chesses are "*" and your chesses are "O"s.');
  writeln('How many columns?'); read(n);    {输入棋盘矩阵的阶数}
  for i:=1 to n do                          {设置初始格局——空棋局}
    for j:=1 to n do
      s.state[i,j]←' .';
      …… …… …… …… ……                      {下述内容见算法框架}
end; {init}
```

3、求精 2.1——goal(s)的函数说明

```
Function goal(q:node):integer;
 var
   w1,w2,w3,w4:integer;
   x1,x2,x3,x4:char;
   {w1——左对角线相同元素的个数;  x1——左对角线首元素}
    {w2——右对角线相同元素的个数;  x2——右对角线首元素}
    {w3——行相同元素的个数;  x3——行首元数};
    {w4——列相同元素的个数;  x4——列首元数};
  i,j:integer  {循环变量}
 begin
   w1←0;w2←0;w3←0;w4←0;  {行、列、对角线相同元素的个数初始化}
   x1←q.state[1,1];          {左对角线的首元素赋 x1}
   x2←q.state[n,1];          {右对角线的首元素赋 x2}
   for i:=1 to n do
     begin
      if q.state[i,i]=x1 then w1←w1+1;  {累计左对角线的相同元素的个数 w1}
      if q.state[n+1-i,i]=x2 then w2←w2+1;{累计右对角线的相同元素的个数 w2}
      x3←q.state[i,1];                  {i行的首元素赋 x3}
      x4←q.state[1,i];                  {i列的首元素赋 x4}
      for j:=1 to n do
        begin
          if q.state[i,j]=x3 then w3←w3+1;   {累计 i 行的相同元素的个数 w3}
```

```
        if q.state[j,i]=x4 then w4← w4+1;    {累计i列的相同元素的个数w4}
      ens; {for}
    if ((w3=n) and (x3='*')) or ((w4=n) and (x4='*'))
      then begin          {若i行或者i列占满'*'，则返回maxint}
             goal← maxint;
             exit;
           end; {then}
    if ((w3=n) and (x3='O')) or ((w4=n) and (x4='O'))
      then begin          {若i行或者i列占满'O'，则返回-maxint}
             goal← -maxint;
             exit;
           end; {then}
    w3← 0;w4← 0;          {下一行、列的相同元素个数初始化}
  end; {for}
  if ((w1=n) and (x1='*')) or ((w2=n) and (x2='*'))
      then begin          {若左对角线或者右对角线占满'*'，则返回maxint}
             goal← maxint;
             exit;
           end; {then}
  if ((w1=n) and (x1='O')) or ((w2=n) and (x2='O'))
      then begin          {若左对角线或者右对角线占满'O'，则返回-maxint}
             goal← -maxint;
             exit;
           end; {then}
  goal← 0;          {无一对角线、行、列占满'*'或者'O'，则返回0}
end; {goal}
```

4、求精2.2——computer_do的过程说明

```
Procedure computer_do;
 var
   successors:node;          {s结点的子结点}
   i,j:integer;
 begin
   successornum← 0;          {子结点个数初始化}
   for i:=1 to n do          {计算第一层结点的个数successornum}
     for j:=1 to n do        {逐一产生s结点的子状态，分别送入successor[1..successor
                               num]}
       begin
```

```
        if s.state[i,j]= ' . '
          then begin
                 successors ← s;
                 successors.state[i,j]←'*';
                 successornum ← successornum+1;
                 successor [successornum] ← successors;
                 end; {then}
      end; {for}
      value ← -maxint;                    {下述内容如算法框架}
      …… …… …… …… …… …… …… ……
2.2.1 successor[i].value ← search(successor[i], min, value, max);
2.2.2 value ← select(value, successor[i].value, max);
      …… …… …… …… …… …… …… ……
      end; {computer_do}
```

5、求精 2.3——print(s)的过程说明

```
procedure print(s: node);
var
  i,j: integer;
begin
  writeln;
  writeln('now the state is:');
  for i:=1 to n do
    begin
      for j:=1 to n do
        write(s.state[i,j],' ');          {打印棋局的i行状态}
      writeln;                            {换行}
    end;{for}
end;{print}
```

第三层

1、求精 2.2.1——search(successor[i], min, value, 0)的函数说明

```
Function search(q: node; mode: modetype; oldvalue, depth: integer): integer;
    {各形式参数的意义见算法框架}
var
  successor: node;        {q的子结点}
  value: integer;         {q的当前倒推值}
  i,j: integer;
```

```
begin
  if (depth < maxdepth) and (goal(g) = 0) then
     begin
      if mode=max then value← -maxint
                  else value← maxint;
      for i:=1 to n do          {在循环结构中逐一扩展和处理 q 的子结点}
        for j:=1 to n do
          begin
            if q.state[i,j]='.'  then
               begin
                if not(((mode=max) and (value > oldvalue) or ((more=min) and
                   (value < oldvalue)))
                then begin
   {若 q 结点不满足 α-β 裁剪条件，则扩展 q 的子结点 successor}
                     successor← q;
                     if mode=max then successor.state[i,j]← '*'
                                 else successor.state[i,j]← 'O';
                     if mode=max      {下述内容如算法框架}
                        then value← select(value,search(successor,min,v
                                                    alue,depth+1),max)
                        else value← select(value,search(successor,max,v
                                                    alue,depth+1),min);
                   end; {then}
               end; {then}
          end; {for}
      search← value;
    end  {then}
  else begin
        if goal(q) <> 0 then search← goal(q)
2.2.1.1                 else search← evlation(q);
       end; {else}
end; {search}
```

2、求精 2.2.2——select(value, successor[i], value, max)的函数说明

具体函数说明见算法框架

第四层

最后求精 2.2.1.1——evlation(q)的函数说明

```
Function evlation(q:node)integer;
```

```
var
  e : integer ;              {叶结点 q 的静态估价函数值}
  r : node ;
  i , j : integer ;          {循环变量}
  w1 , w2 , w3 , w4 : integer ;      {左、右对角线及行、列的相同元素个数}
  x1 , x2 , x3 , x4 : char ;         {左、右对角线及行、列的首元素}
begin
  e ← 0 ;        {估价函数值初始化}
  r ← q ;
  for i := 1 to n do  {所有空格处填' * ', 以统计计算机还可占据的行、列和对角线}
    for j := 1 to n do
      if r . state [ i , j ] = ' . '  then r . state [ i , j ] ← ' * ' ;
  w1 ← 0 ; w2 ← 0 ; w3 ← 0 ; w4 ← 0 ; {置左、右对角线及行、列相同元素的个数为 0}
  x1 ← r . state [ 1 , 1 ] ; x2 ← r . state [ n , 1 ] ;  {左、右对角线的首元素分别赋 x1 和 x2}
  for i := 1 to n do
    begin
      if r . state [ i , i ] = x1 then w1 ← w1 + 1 ;     {累计左、右对角线上相同元素的个数 w1 , w2}
      if r . state [ n + 1 - i , i ] = x2 then w2 ← w2 + 1 ;
      x3 ← r . state [ i , 1 ] ;                          {i 行和 i 列的首元素赋 x3 和 x4}
      x4 ← r . state [ 1 , i ] ;
      for j := 1 to n do
        begin
          if r . state [ i , j ] = x3 then w3 ← w3 + 1 ;  {累计 i 行和 i 列的相同元素的个数 w3 , w4}
          if r . state [ j , i ] = x4 then w4 ← w4 + 1 ;
        end ;  {for}
      if (( w3 = n ) and ( x3 = ' * ' ))  then e ← e + 1 ; {若计算机还可占据 i 行, 则 e + 1}
      if (( w4 = n ) and ( x4 = ' * ' ))  then e ← e + 1 ; {若计算机还可占据 i 列, 则 e + 1}
      w3 ← 0 ; w4 ← 0 ;                          {下一行、列相同元素的个数初始化}
    end ;  {for}
  if (( w1 = n ) and ( x1 = ' * ' ))  then e ← e + 1 ;    {若计算机还可占据左对角线, 则 e + 1}
  if (( w2 = n ) and ( x2 = ' * ' ))  then e ← e + 1 ;    {若计算机还可占据右对角线, 则 e + 1}
  r ← q ;
  for i := 1 to n do {所有空格处填' O ', 以统计对方还可占据的行、列和对角线}
```

```
    for j:=1 to n do
      if r.state[i,j]='.' then r.state[i,j]←'O';
  w1←0;w2←0;w3←0;w4←0;{置左、右对角线及行、列相同元素的个数为0}
  x1←r.state[1,1];              {左、右对角线的首元素分别赋x1和x2}
  x2←r.state[n,1];
  for i:=1 to n do
    begin
      if r.state[i,i]=x1 then w1←w1+1;  {累计左、右对角线上相同元素的个数
w1,w2}
      if r.state[n+1-i,i]=x2 then w2←w2+1;
      x3←r.state[i,1];                  {i行和i列的首元素赋x3和x4}
      x4←r.state[1,i];
      for j:=1 to n do
       begin
         if r.state[i,j]=x3 then w3←w3+1;   {累计i行和i列的相同元素的个
数 w3,w4}
         if r.state[j,i]=x4 then w4←w4+1;
       end;  {for}
      if ((w3=n) and (x3='O')) then e←e-1; {若对方还可占据i行，则e-1}
      if ((w4=n) and (x4='O')) then e←e-1; {若对方还可占据i列，则e-1}
    end; {for}
  if ((w1=n) and (x1='O')) then e←e-1;      {若对方还可占据左对角线，则
e-1}
  if ((w2=n) and (x2='O')) then e←e-1;      {若对方还可占据右对角线，则
e-1}
  evlation←e;                               {e值作为q结点的估价函数值返回}
end; {evlation}
```

第七章　网络流的算法

许多系统包含了流量问题。例如，公路系统中有车辆流，控制系统有信息流，供水系统中有水流，金融系统中有现金流等等。这些流从一个产生该物质的“源”出发，经过一个系统流向另一个消耗该物质的“汇”。“源”以某固定的速度产生该物质，并且“汇”用同样的速度消耗该物质。所谓流量是指该物质在系统中的运行速度。

【例 1】求输送量最大的运输方案

图7.1－1(a)是联结产品产地V_1和销地V_6的交通网，每一弧(V_i, V_j)代表从V_i到V_j的运输线，产品经这条弧由V_i输送到V_j，弧旁的数字表示这条运输线的最大通过能力(以后简称容量)。产品经过交通网从V_1输送到V_6。现在要求制定一个运输方案，使V_1运到V_6的产品数量最多。

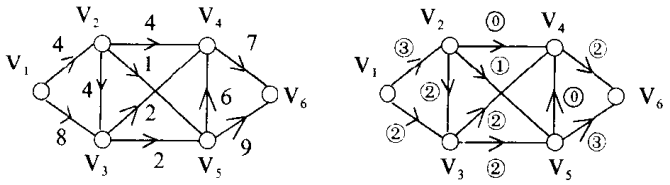


图 7.1-1(单位:百吨)

卡车沿运输线输送产品的速度是一个典型的流。对于运输商来说，他们无权控制交通网和每条运输线的容量。他们关心的是每天有尽可能多的物资离开产地，并且每天有同样数量的物资到达销地。落实到具体运输方案无非就是从产地运出多少物资，其中多少吨沿这一条路送到销地，多少吨沿那一条路送到销地 …… 图 7.1-1(b) 就给出了这样一种运输方案。2 百吨物资沿有向路$P_1(V_1,V_2,V_3,V_4,V_6)$运到销地，2 百吨物资沿有向路$P_2(V_1,V_2,V_3,V_5,V_6)$运到销地，1 百吨物资沿有向路$P_3(V_1,V_2,V_5,V_6)$运到销地。每条弧旁的数字表示方案中每条运输线上的实际运输量。这些数必须满足一些条件，即:

1. 实际运输量不能是负的;

2. 每条弧的实际运输量不能大于该弧的容量;

3. 除了起点V_1和终点V_6外，对其它顶点来说，所有指向V_i的弧上的运输量的和应该等于所有从V_i出发的弧上的运输量的和。

这三个条件中的前两个条件的必要性是显然的，条件(3)也是不难理解的，因为，除了起点和终点以外，其余顶点都不过是一些“中转站”，它们既不能产生物质也不能囤积物质，因此运到它那儿的物质是多少吨，从它那儿运走的也应该是多少吨，不能多也不能少。显然，方案中的所有数满足了这三个条件，因此方案是可行的。但问题是，用图7.1-(b)的方案将 5 百吨的物质从V_1运到V_6。在这个交通网上网的输送量是否还可以增多，或者说这个运输网中，从V_1到V_6的最大输送量是多少呢？本章节即要研究类似这样的问题。

§7.1 基本概念与基本定理

一、网络与流

网络 D=(V, A, C)

设 D 是一个简单有向图，(D=(V, A))。在 V 中指定了一个顶点，称为源点（记为 V_s）和另一个顶点，称为汇点（记为 V_t），对于每一条弧 $(V_i, V_j) \in A$，对应有一个 $C_{ij} \geq 0$，称为弧的容量。通常我们就把这样的 D 叫做一个网络，记作 D=(V, A, C)。

网络流 F

所谓网络上的流是指定义在弧集合 A 上的一个函数 $F=\{F_{ij}\}$，并称 F_{ij} 为弧 (V_i, V_j) 上流量。

例如图 7.1-1(a) 即一个网络，指定 V_1 是源点，V_6 是汇点，其它的顶点是中间点。弧旁的数字为 C_{ij}。图 7.1-1(b) 所示有运输方案，就可看作是这个网络上的一个流，每个弧上的运输量即该弧上的流量即：$F_{12}=3$，$F_{13}=2$，$F_{23}=2$，$F_{24}=0$，$F_{25}=1$，$F_{34}=2$，$F_{35}=2$，$F_{46}=2$，$F_{54}=0$，$F_{56}=3$

二、可行流与最大流

可行流（可行流的流量 V(F)）

满足下述条件的流 F 称为可行流：

1. 容量限制条件：

对每一条弧 $(V_i, V_j) \in A$，$0 \leq F_{ij} \leq C_{ij}$

2. 平衡条件：

对于中间点：

流出量 = 流入量，即对每个 i($i \neq s, t$)有

V_i 的流出量 $-V_i$ 的流入量 $= \sum_{(V_i, V_j)\in A} F_{ij} - \sum_{(V_j, V_i)\in A} F_{ji} = 0$

对于源点 S:

V_s 的流出量 $-V_s$ 的流入量 $= \sum_{(V_s, V_j)\in A} F_{sj} - \sum_{(V_j, V_s)\in A} F_{js}$ = 源点的净输出量 V(F)。

对于汇点 T:

V_t 的流出量 $-V_t$ 的流入量 $= \sum_{(V_t, V_j)\in A} F_{tj} - \sum_{(V_j, V_t)\in A} F_{jt} = (-1)*$汇点的净输入量 V(F)。

式中 V(F) 称为这个可行流的流量，即源点的净输出量(或汇点的净输入量)。可行流总是存在的。例如：令所有弧的流量 $F_{ij}=0$，就得到一个其流量 V(F)=0 的可行流(称为零流)。

最大流问题即求一个流 $\{F_{ij}\}$，使其流量 V(F) 到最大，并且满足：

$$0 \leq F_{ij} \leq C_{ij}, \quad (V_i, V_j) \in A$$

$$\sum F_{ij} - \sum F_{ji} = \begin{cases} V(F) & (i=s) \\ 0 & (i \neq s, t) \\ -V(F) & (i=t) \end{cases}$$

三、可改进路 P

若给一个可行流 $F=(F_{ij})$，我们把网络中使 $F_{ij}=C_{ij}$ 的弧称为饱和弧，使 $F_{ij}<C_{ij}$ 的弧称为非饱和弧，把 $F_{ij}=0$ 的弧称为零流弧，$F_{ij}>0$ 的弧称为非零流弧。

如图 7.1-1(b) 中，(V_2, V_4) 和 (V_5, V_4) 为零流，(V_3, V_4) 和 (V_3, V_5) 是饱和弧，其它非零流弧为非饱和弧。

若 P 是网络中联结源点 V_s 和汇点 V_t 和一条路，我们定义路的方向是从 V_s 到 V_t，则路上的弧被分成两类：一类是弧的方向与路的方向一致，叫做前向弧。前向弧的全体记为 P^+。另一类弧与路的方向相反，叫做后向弧。后向弧的全体记为 P^-。

图 7.1−(a) 中，在路 $P=\{V_1, V_3, V_2, V_4, V_6\}$ 中

$P^+=\{(V_1, V_3), (V_2, V_4), (V_4, V_6)\}$

$P^-=\{(V_3, V_2)\}$

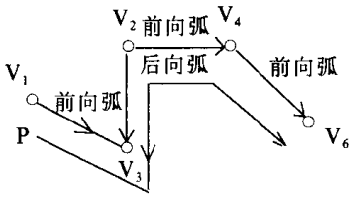


图 7.1-2

下面引出可改进路 P 的定义：

设 F 是一个可行流，P 是从 V_s 到 V_t 的一条路，若 P 满足下列条件：

1. 在 P 的所有前向弧 (V_i, V_j) 上，$0 \leqslant F_{ij}<C_{ij}$，即 P^+ 中的每一条弧是非饱和弧。

2. 在 P 的所有后向弧 (V_i, V_j) 上，$0<F_{ij} \leqslant C_{ij}$，即 P^- 中的每一条弧是非零流弧。

则称 P 为关于可行流 F 的一条可改进路。显然图 7.1-2 给出的路 P 满足上述两个条件，是一条可改进路。那么，为什么将具有上述特征的路 P 叫做可改进路，原因是可以通过修正 P 路上所有流量 F_{ij} 来把现有的可行流 F 改进成一个值更大的流 F′。下面我们具体地给出一种方法，利用这种方法就可以把 F 改进成一个更好的流 F′。这种方法是：

1. 不属于可改进路 P 的弧 (V_i, V_j) 上的流量一概不变，即 $F'_{ij}=F_{ij}$；

2. 可改进路 P 上的所有弧 (V_i, V_j) 上的流量按下述则规变化：

在前向弧 (V_i, V_j) 上，　　$F'_{ij}=F_{ij}+a$；

在后向弧 (V_i, V_j) 上，　　$F'_{ij}=F_{ij}-a$；

a 称谓可改进量，它应该按照下述原则确定：a 即要取得尽量大，又要使变化后的 F_{ij} 仍满足可行流的两个条件——容量限制条件和平衡条件。不难看出，按照这个原则，a 即不能超过每条前向弧的 $C_{ij}-F_{ij}$，也不能超过每条后向弧的 F_{ij}。因此 a 应该等于前向弧上的 $C_{ij}-F_{ij}$ 与后向弧上的 F_{ij} 的最小值。

图 7.1−2 给出一条可改进路 $P(V_1, V_3, V_2, V_4, V_6)$。现在就按照上面讲的方法将流 F 改进成一个更好的流。首先应该定出改进量 a，先看 P 的前向弧集合 $P^+=\{(V_1, V_3), (V_2, V_4), (V_4, V_6)\}$

$$C_{13}-F_{13}=8-2=6;$$

$$C_{24}-F_{24}=4-1=3;$$

$$C_{46}-F_{46}=7-3=4;$$

再看 P 的后向弧集合 $P^- = \{(V_2, V_3)\}$，在这条弧上 $F_{23} = 2$。

总起来说，a 至多取 2，这样既可以使改进后的前向弧上的流量有所增加，又可以使改进后的后向弧上的流量在减少 a 之后不变负数。这个改进过程见图 7.1-3，改进后的流的值增加为 8。

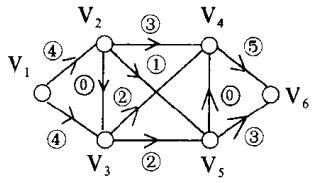
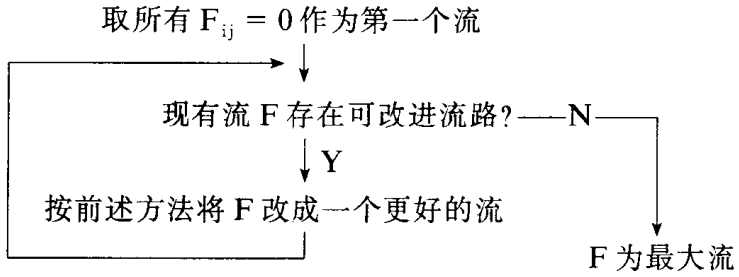


图 7.1-3

上面我们讲了在什么情况下，一个流可以改进和如何改进，但还有一点没有讲——怎样检查一个流是不是最大的。下面我们给出一个重要结论：

设 F 是网络 D 的一个流，如果不存在从 V_s 到 V_t 关于 F 可改进路 P，那么 F 一定是最大流。至此，我们不难得出求最大流的基本思路：



四、截集与截量

将网络 D=(V,A,C) 中的顶点集合 V 划分为两个非空子集 S 和 T：我们把弧的始点在 S、终点在 T 中 ($V_s \in S$, $V_t \in T$) 的所有弧构成的集合 (S, T) 称为是分离 V_s 和 V_t 的截集。把截集 (S, T) 中所有弧的容量之和

$$C(S, T) = \sum_{(v_i v_j) \in (s, t)} C_{i,j}$$

称截集的容量，简称截量。穿过截集 (S, T) 的网络流被定义为 F(S, T)。

从直观上讲，截集 (S, T) 是从 V_s 到 V_t 的必经之道。若把某一截集的弧从网络中丢弃，则从 V_s 到 V_t 便不存在路。例如图 7.1-1(a)所示的网络中，$S=\{V_1, V_2, V_3\}$，$T=\{V_4, V_5, V_6\}$，$(S, T) = \{(V_2, V_4), (V_2, V_5), (V_3, V_4), (V_3, V_5)\}$。

$$C(S, T) = C(V_2, V_4) + C(V_2, V_5) + C(V_3, V_4) + C(V_3, V_5)$$
$$= 4 + 1 + 2 + 2$$
$$= 9$$

对于 (图 7.1-3) 给出的可行流

$$F(S, T) = F(V_2, V_4) + F(V_2, V_5) + F(V_3, V_4) + F(V_3, V_5)$$
$$= 3 + 1 + 2 + 2$$
$$= 8$$

显然，通过截集 (S, T) 的网络流 F(S, T)= |F|。也不难看出，任一个可行流 V(F) 都不会超过任一截集的容量。即

$$V(F) \leqslant C(S, T)$$

当 V(F)=C(S,T) 时，则可行流 F 必是最大流，而 (S,T) 必定是网络 D 的所有截集中容量最小的一个，即最小截集。在计算最大流的过程中，我们利用下述方法来定义 S:

令 $V_s \in S$。 若 $V_i \in S$ 且 $F_{i,j} < C_{ij}$，则令 $V_j \in S$; 若 $V_i \in S$ 且 $F_{ji} > 0$，则令 $V_j \in S$;

一旦 V_t 进入 S 集合，就表明找到一条可改进路；如果 S 集合扩展不下去而 V_t 又尚未进入 S 集合，则说明不存在可改进路。此时，除 S 外的顶点进入 T 集合

$$F_{ij} = \begin{cases} C_{ij} & (V_i V_j) \in (S, T) \\ 0 & (V_i V_j) \in (T, S) \end{cases}$$

于是有如下重要的最大流最小截量定理:

任一个网络 D 中从 V_s 到 V_t 的最大流的流量等于分离 V_s 和 V_t 的最小截集的容量。

五、多个源和多个汇的网络

一个最大流问题中可以有 M 个源和 N 个汇 (M, N ≥ 1) 而并非仅有一个源和一个汇。

【例 2】求多个源和汇的网络的最大流

图 7.1-4 (a) 是一种商品供应系统，弧的方向表示商品流向，弧上的数字表示商品从某地至某地的最大通过能力。商品的货源来自 S_1, S_2, …, S_5, 并在商场 T_1, T_2, T_3 出售。问是否所有的需求皆可同时满足。

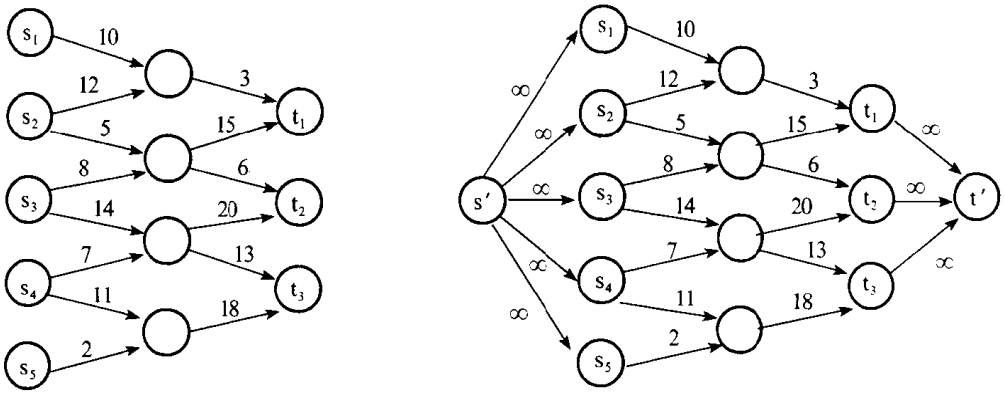


图 7.1-4

我们能够把在具有多个源和多个汇的网络中确定最大流的问题转换成单个源和单个汇的最大流问题。方法如图 7.1-4 (b) 所示:

增加一个超级源 S, 并且对原网中的 M 个源分别加入有向边 (S, S_i), 其容量为 $C(S, S_i) = \infty$ ($1 \leqslant i \leqslant m$)；同时增加一个超级汇 T, 并且对原网中的 n 个汇分别加入有向边 (T_j, T), 其容量为 $C(T_j, T) = \infty$ ($1 \leqslant j < n$)。直观上网络 (a) 中的任意流均对应于网 (b) 中的一个流，反之亦然。单源 S 对多个源 S_i 提供其所需要的任意大的流，同样单汇 T 对多个汇 T_j 消耗其所需要的同样数量的流。这两个问题是等价的。

不仅求多个源和多个汇的网络最大流可以转换为对应的单源和单汇的网络最大流问题，而且求源点至汇点的最小流问题也可以通过求汇点至源点的最大流来解决，因为这个最大流恰为由源点到汇点的最小流。由此可见，最大流的算法是网络流问题的核心和基础。

求网络的最大流的关键和难点是如何寻找可改进路，特别是对于顶点数和弧数多的网络，寻找的效率至关重要。是否能找到一种系统地寻找可改进路的方法，这种方法在存在可改进路时能很快地将它找出来，当不存在可改进路时能很快地告诉你“不用找了，肯定不会有”。下一章节讲的标号法（ford - fulkerson 算法）就是这样一种方法。

§7.2 寻求最大流的标号法

从一个可行流出发（若网络中没有给定 F，则可以设 F 是零流），经过标号过程和调整过程。

1. 标号过程

在这个过程，网络中的顶点或者是标号点（又分为已检查和未检查两种），或者是未标号点。每个标号点的标号包含两个部分。第一标点指明它的标号从哪一顶点得到，以便找出可改进量；第二个标号是为确定可改进量 a 用的。

标号过程开始，总先给 V_s 标上 $(0, +\infty)$，这时 V_s 是标号而未检查的顶点，其余都是未标号点。一般地，取一个标号而未检查的标号 V_i，对一切未标号点 V_j：

(1)若在弧 (V_i, V_j) 上 $F_{ij}<C_{ij}$，则给 V_j 标号 $(V_i, L(V_j))$，这里 $L(V_j)=\min[L(V_i), C_{ij}-F_{ij}]$。这时顶点 V_j 成为标号而未检查的顶点。

(2)若在弧 (V_j, V_i) 上 $F_{ij}>0$，则给 V_j 标号 $(-V_i, L(V_j))$，这里 $L(V_j)=\min[L(V_i), F_{ij}]$。这时顶点 V_j 成为标号而未检查的顶点。

在 V_i 的全部相邻顶点都已标号后，V_i 成为标号而已检查过的顶点。重复上述步骤，一旦 V_t 被标上号，表明得到一条从 V_s 到 V_t 的可改进路 P，转入调整过程。

若所有标号都已检查过致使标号过程无法继续时，则算法结束，这时的可行流即最大流。

2. 调整过程

采用“倒向追踪”的方法，从 V_t 开始，利用标号点的第一个标号逐条弧地找出可改进路 P，并以 V_t 的第二个标号 $L(V_t)$ 作为改进量 a，改进 P 路上的流量。

例如设 V_t 的第一个标号为 V_k（或 $-V_k$），则弧 (V_k, V_t)（或相应的 (V_t, V_k) 是 P 上的弧。接下来检查 V_k 的第一个标号，若为 V_i（或 $-V_i$）则找出 (V_i, V_k)（或相应的 (V_k, V_i)）。再检查 V_i 的第一个标号，查到 V_s 为止。这时被找出的弧就构成了可改进路 P。令改进量 a 是 $L(V_t)$，即 V_t 的第二个标号。

$$F'_{ij}=\begin{cases} F_{ij}+a & (V_i, V_j)\in P^+ \\ F_{ij}-a & (V_i, V_j)\in P^- \\ F_{ij} & (V_i, V_j)\in P \end{cases}$$

去掉所有的标号，对新的可行流 $F'=\{F'_{ij}\}$，重新进入标号过程。

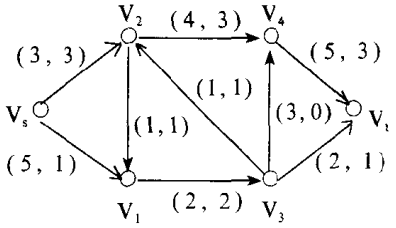
显然，我们可以采用宽度优先搜索的办法求最大流：

设队列 Q——存储当前未检查的标号点，队首顶点出队后，成为已检查的标号点；

path——存储当前已标号点的可改进路。

```
repeat
  path 置空;
  源点标号并进入 path 和 Q;
  while(Q 非空)and(汇点未标号)do
  begin
    移出 Q 的队首顶点 X;
    分析 X 出发的所有弧;
    if(相邻点未标号)and(对应弧的流量可改进) then 相邻点进入 Q 和 path;
  end;{while}
  if 汇点已标号 then 从汇点出发修正可改进路的流量;
until 汇点未标号;
```

【例 1】用标号法求图 7.2-1 所示网络的最大流。



弧旁的数是 (C_{ij}, F_{ij})

图 7.2-1

算法分析：

1. 标号过程

(1) 首先给 V_s 标上 $(0, +\infty)$

(2) 检查 V_s。弧 (V_s, V_2) 上，$F_{s2} = C_{s2} = 3$，不满足标号条件；弧 (V_s, V_1) 上，$F_{s1} = 1 < C_{s1} = 5$，则 V_1 的标号为 $(V_s, L(V_1))$，其中 $L(V_1) = \min[L(V_s), (C_{s1} - F_{s1})] = \min[-\infty, 5-1] = 4$

(3) 检查 V_1。弧 (V_1, V_3) 上 $F_{13} = C_{13} = 2$，不满足标号条件；弧 (V_2, V_1) 上，$F_{21} = 1 > 0$，则给 V_2 记下标号为 $(-V_1, L(V_2))$，其中 $L(V_2) = \min[L(V_1), f_{21}] = \min[4, 1] = 1$

(4) 检查 V_2。在弧 (V_2, V_4) 上 $F_{24} = 3 < c_{24} = 4$，则给 V_4 标号 $(V_2, L(V_4))$，其中 $L(V_4) = \min[L(V_2), C_{24} - F_{24}] = \min[1, 1] = 1$

(5) 在 V_3，V_4 中任选一个进行检查。例如弧 (V_3, V_t) 上，$F_{3t} = 1 < C_{3t} = 2$，给 V_t 标号 $(V_3, L(V_4))$，其中 $L(V_t) = \min[L(V_3), (C_{34} - F_{34})] = \min[1, 1] = 1$

由于 V_t 有了标号，所以转入调整过程。

2. 调整过程

按顶点的第一个标号找到一条可改进路，如图 7.2-2 中的粗箭头线所示。

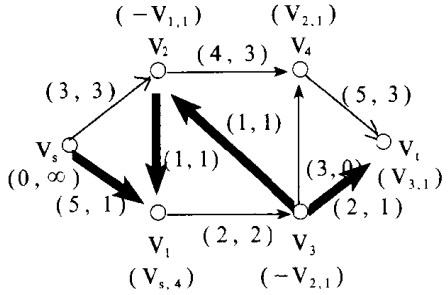


图 7.2-2

易见，$P^+=\{(V_s, V_1), (V_3, V_t)\}$

$P^-=\{(V_2, V_1), (V_3, V_2)\}$

按 a=1 在 p 上调整

P^+ 上：$F_{s1}+a=1+1=2$　　$F_{3t}+a=1+1=2$

P^- 上：$F_{21}-a=1-1=0$　　$F_{32}-a=1-1=0$

其余的 F_{ij} 不变。

调整后得图 7.2-3 所示的可行流，对这个可行流进入标号过程，寻找可改进路。

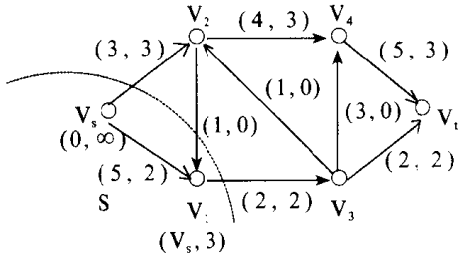


图 7.2-3

开始给 V_s 标以 $(0, +\infty)$，于是检查 V_s，给 V_1 标以 $(V_s, 3)$。检查 V_1，弧 (V_1, V_3) 上，$F_{13}=C_{13}$。而弧 (V_2, V_1) 上，$F_{21}=0$，均不符合条件，标号过程无法继续，算法结束。这时的可行流（图 7.2-3）即为所求的最大流

$$F=F_{s1}+F_{s2}=F_{4t}+F_{3t}=5$$

与此同时可找到最小截集 (S, T)。其中 S 为标号点集合：$S=\{V_s, V_1\}$；T 为未标号点集合：$T=\{V_2, V_3, V_4, V_t\}$；$(S, T)=\{(V_s, V_2), (V_1, V_3)\}$

$$C(S, T)=C(V_s, V_2)+C(V_1, V_3)=3+2=5$$

由此可见，用标号法找可改进路以求最大流的结果，同时得到一个最小截集。

下面给出标号法的程序

```
Program Ford-Fulkerson-Edmonds-Karp;

Const
  Maxn                    =  1000; {最多顶点数}

Type
  Link                    =  ^Nbour; {N网的邻接表的指针}
```

```
  Nbour          =  Record{N网的邻接表结点}
                      k          :  1..Maxn; {顶点序号}
                      f, c       :  Integer; {流量，容量}
                      next, g    :  Link{前向弧指针，后向弧指针}
                    End;
  WType          =  Record {可改进路的结点类型}
                      p, d       :  Integer; {标号(指针，当前可改进量}
                      w          :  Link {弧指针}
                    End;

Var
  N, stt, trm    :  Integer; {顶点数，源点，汇点}
  F              :  Text; {文件变量}
  Nbs            :  Array [1..Maxn] of Link; {N网的邻接表}
  Pth            :  Array [1..Maxn] of WType; {可改进路}

Procedure Insert(u, v, c : Integer);
{在U对应的邻接表中插入流量为0、容量为C的前向弧(U, V)}
{在V对应的邻接表中插入流量为0,容量为0的后向弧(U, V)}
Var
  x              :  Link;
Begin
  x := Nbs[u];
  While (x <> nil) And (x^.k <> v) Do x := x^.next;
  If x <> nil Then x^.c := c
    Else Begin
      New(x);
      x^.k := v; x^.c := c; x^.f := 0;
      x^.next := Nbs[u]; Nbs[u] := x;
      New(x^.g);
      x^.g^.k := u; x^.g^.c := 0; x^.g^.f := 0; x^.g^.g := x;
      x^.g^.next := Nbs[v]; Nbs[v] := x^.g
    End{else}
End; {Insert}

Procedure Init;
Var
  u, v, c        :  Integer;
```

```
Begin
  Assign(F, ' INPUT.DAT' ); {输入文件名串与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N, stt, trm);{读入顶点数，源点和汇点}
  For u := 1 to N Do Nbs[u] := nil;{N网的邻接表初始化为空}
  While Not Eof(F) Do Begin{依次读入每条弧及其容量，构造N网的邻接表}
    Readln(F, u, v, c);
    Insert(u, v, c)
  End;{while}
  Close(F)
End;{init}

Procedure Ford;
Var
  Q              :  Array [1..Maxn] of 1..Maxn; {队列}
  op, cl, u, d   :  Integer;{队列首指针，尾指针，顶点序号，可改进量}
  x              :  Link;{辅助指针}
Begin
  Repeat
    FillChar(Pth, Sizeof(Pth), 0);{可改进路置空}
    Pth[stt].d := Maxint; Pth[stt].p := stt; {源点进入可改进路，并为之标号}
    cl := 0; op := 1; Q[op] := stt; {源点进入队列}
    While (cl < op) And (Pth[trm].p = 0) Do Begin
    {若队列非空且汇点未标号，则循环}
      Inc(cl); {队首顶点U出队}
      u := Q[cl];
      x := Nbs[u];
      While x <> nil Do Begin{分析顶点U出发的每一条弧}
        d := x^.c - x^.f;
        If (Pth[x^.k].p = 0) And (d > 0) Then Begin
        {若相邻点未标号且弧流量可改进}
          Inc(op); Q[op] := x^.k; {则相邻点进入队列和可改进路，并为之标号}
          Pth[x^.k].p := u; Pth[x^.k].w := x;
          If Pth[u].d < d Then Pth[x^.k].d := Pth[u].d
                          Else Pth[x^.k].d := d;
        End; {then}
        x := x^.next{分析与顶点U相连的下一条弧}
      End{while}
```

```
    End; {while}
    If Pth[trm].p <> 0 Then Begin{若汇点已标号}
      u := trm; d := Pth[u].d; {则从汇点出发，倒向修正可改进路的流量}
      Repeat
        Pth[u].w^.f := Pth[u].w^.f + d;
        Pth[u].w^.g^.f := - Pth[u].w^.f;
        u := Pth[u].p
      Until u = stt{直到倒推至源点为止}
    End{then}
  Until Pth[trm].p = 0{直至不存在可改进路为止}
End; {ford}

Procedure Answer;
Var
  i, tot        : Integer;
  x             : Link;
Begin
  tot := 0; {最大流的流量初始化}
  For i := 1 to N Do Begin{分析每一个顶点}
    x := Nbs[i];
    While x <> nil Do Begin{分析顶点 i 出发的每一条弧}
      If x^.f > 0{若该弧的流量大于 0,则打印该弧及其流量}
        Then Writeln(i:4,'-->', x^.k:3, x^.f:6);
      If (i = 1) And (x^.f > 0) Then Inc(Tot, x^.f); {累计最大流的流量}
      x := x^.next{分析顶点 i 出发的下一条弧}
    End{while}
  End; {for}
  Writeln('************** tot =', tot){打印最大流的流量}
End; {answer}

Begin
  Init; {输入数据，构造 N 网}
  Ford; {计算最大流}
  Answer{输出结果}
End.{main}
```

上面讨论的标号法是求一种标准网络上的最大流问题，这种网络的每一条弧 e 对应了一个弧容量 C(e) ≥ 0。现在，我们将网络结构再修改一下。

【例 2】求容量有上下界的网络的最大流。

设网络 D=(V, A, B, C)。其中 A 中的每条弧 e 对应两个数字 B(e) 和 C(e)，分别表示弧容量的上界和下界，那么如何求满足下列条件的最大流：

1. $B(e) \leqslant F(e) \leqslant C(e)$;

2. $\sum_{e\in\beta(V)} F(e) - \sum_{e\in\alpha(V)} F(e) = 0 \quad (v \neq s, t)$

(注：$\alpha(V)$——以 V 为尾的弧集；$\beta(V)$——以 V 为头的弧集)

算法分析：

标号法例举的网络结构是有上下界的网络的一种特例，即 B(e)=0。当 B(e)>0 时，这种有上下界的网络就不一定存在流了。例如，图 7.2－4 所示的网络，弧上的第 1 个数字为 B(e)，第二个数字为 C(e)。由于 $0 \leqslant Fsa \leqslant 1$, $2 \leqslant Fat \leqslant 3$, 对于顶点 a 来讲，$Fat - Fsa \neq 0$，所以该网络不存在流。

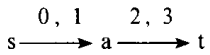


图 7.2-4

那么如何判断一个有上下界的网络 N 有可行流呢？我们很容易想到一种思路——将容量有上下界的 N 网转换成“每条弧仅对应一个容量 $C(e) \geqslant 0$”的附加网 N，然后用标号法对其求最大流，根据求解结果判断 N 网是否有可行流 F。具体的转换方法如下：

1. 新增加两个顶点 $\bar{s}$ 和 $\bar{t}$, $\bar{s}$ 称为附加源，$\bar{t}$ 称为附加汇；

2. 对原网络 N 的每个顶点 U, 加一条新弧 $e=U\bar{t}$, 这条弧的容量为所有以 U 为尾的弧的容量下限之和，即 $\bar{C}(e)=\sum_{e\in\alpha(U)} B(e)$;

3. 对原网络 N 的每个顶点 U, 加一条新弧 $e=\bar{s}U$, 这条弧的容量为所有以 U 为头的弧的容量下限之和，即 $\bar{C}(e)=\sum_{e\in\beta(U)} B(e)$;

4. 原网络 N 的每条弧在 N 仍保留，弧容量 $\bar{C}(e)$ 修正为 $C(e)-B(e)$;

5. 再添两条新弧 e=st, e'=ts。e 的容量 $\bar{C}(e)=\infty$，e'的容量 $\bar{C}(e')=\infty$;

用标号法求附加网 N 的最大流，若结果能使流出 $\bar{s}$ 的一切弧皆满载（即 $\bar{s}$ 出发的所有弧 e, $\bar{C}(e)=\bar{F}(e)$), 则 N 网有可行流 $F(e)=\bar{F}(e)+B(e)$。否则 N 网无可行流。

若 $\bar{N}$ 网的最大流满足上述条件，我们则可以再使用 §6.1 节所述的标号法，将可行流 F 放大，最终求出 N 网的最大流。

例如，图 7.2-5 为一个容量有上下界的网络 N，弧上的数第一个数是 B(e)、第二个数是 C(e)。

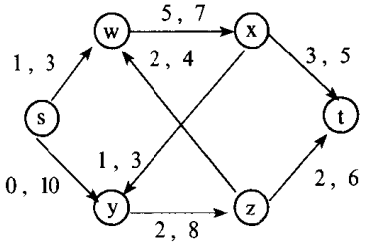


图 7.2-5

作附加网络 $\bar{N}$，用标号法求 $\bar{N}$ 网上的最大流 $\bar{F}$，见图 7.2-6。

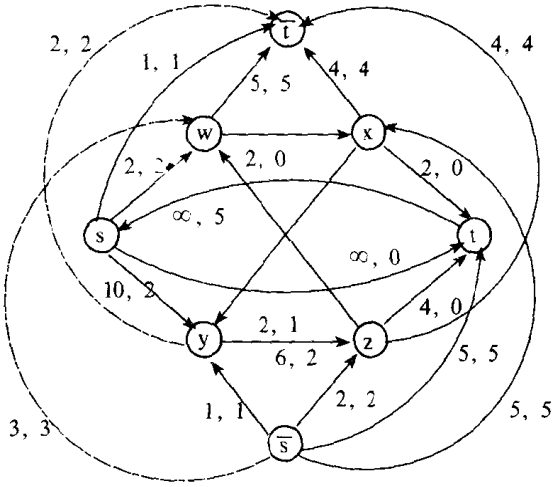


图 7.2-6

弧上的第一个数为 $\bar{C}(e)$，第二个数为 $\bar{F}(e)$。由于流出 $\bar{s}$ 的弧皆满载，可以得出图 7.2-5) 的网 N 中有可行流 $F(e)=\bar{F}(e)+B(e)$。图 7.2-7 将 $\bar{F}$ 化成 N 网的可行流，弧上的三个数依次为 B(e)、C(e)、F(e)。

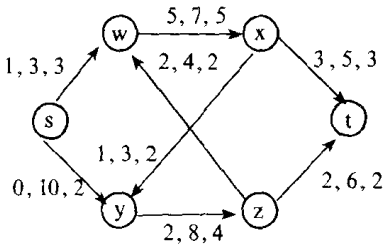
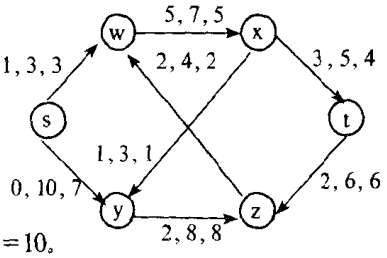


图 7.2-7

用标号法将图 7.2-7 中的可行流放大，得最大流如图 7.2-8。



最大流的流量 F=10。

图 7.2-8

下面，我们给出求容量有上下界的网络的最大流的程序。

```
Program Max__Flow__With__Limit;

Const
  Maxn                    =   1000; {最多顶点数}

Type
```

```
  Link                  = ^Nbour; {邻接表指针类型}
  Nbour                 =  Record{邻接表结点类型}
                           k          :  1..Maxn ; {顶点序号}
                           f, b, c, t :  Integer ;
                          {流量，容量的下界和上界，可行流的恢复增量}
                           next, g    :  Link ; {前向弧指针，后向弧指针}
                         End ;
  WType                 =  Record{可改进路上的结点类型}
                             p, d     :  Integer; {标号 (指针，当前可改进量)}
                             w        :  Link; {该弧在邻接表的指针}
                         End ;

Var
  N, stt, trm           :  Integer ; {顶点数，源点，汇点}
  F                     :  Text ; {文件变量}
  Nbs                   :  Array [1..Maxn] of Link; {N网的邻 接表}
  Pth                   :  Array [1..Maxn] of WType; {可改进路}
  aa, bb                :  Array [1..Maxn] of Integer;
  {aa[v]-B[v]的容量下限和; bb[v]-a[v]的容量下限和}

Procedure Insert(u, v, b, c : Integer);
{在U对应的邻接表中插入下界为0,上界为c-b,流量为0的前向弧(U, V),可行流的恢复
增量为b;在V对应的邻接表中插入下界为b-c,上界为0,流量为0的后向弧(U, V), 可行
流的恢复增量为-b}
Var
  x             :  Link;
Begin
  New(x);
  x^.k := v; x^.c := c - b; x^.b := 0; x^.t := b; x^.f := 0;
  x^.next := Nbs[u]; Nbs[u] := x;
  New(x^.g);
  x^.g^.k := u; x^.g^.c := 0; x^.g^.b := b - c; x^.g^.t := - b;
  x^.g^.f := 0; x^.g^.g := x;
  x^.g^.next := Nbs[v]; Nbs[v] := x^.g
End; {Insert}

Procedure Init;
Var
```

```
  u, v, b, c     :  Integer;
Begin
  Assign(F, 'INPUT.DAT'); {输入文件名串与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N, stt, trm); {读入顶点数, 源点和汇点}
  For u := 1 to N Do begin
   {网的邻接表初始化为空, 各顶点的 α[V] 和 β[V] 的容量下限和清零}
    Nbs[u] := nil; aa[u] := 0; bb[u] := 0
  End;
  While Not Eof(F) Do Begin
    Readln(F, u, v, b, c);
    {读入弧 (U, V) 及其下界和上界}
    Insert(u, v, b, c); {该弧插入 N 网中}
    Inc(bb[u], b); Inc(aa[v], b){累计 β[U] 和 α[V] 的容量下限和}
  End; {while}
  Close(F)
End; {init}

Procedure Ford(stt, trm : Integer);
Var
  Q              :  Array [1..Maxn] of 1..Maxn;
  op, cl, u, d   :  Integer;
  x              :  Link;
Begin
  Repeat
    FillChar(Pth, Sizeof(Pth), 0); {可改进路置空}
    Pth[stt].d := Maxint; Pth[stt].p := stt; {给源点标号}
    cl := 0; op := 1; Q[op] := stt; {源点进入队列}
    While (cl < op) And (Pth[trm].p = 0) Do Begin
    {若 Q 队列非空且汇点未标号, 则循环}
      Inc(cl); {移出 Q 的队首顶点 U}
      u := Q[cl];
      x := Nbs[u];
      While x <> nil Do Begin{分析 U 出发的每一条弧}
        d := x^.c - x^.f;
        If (Pth[x^.k].p = 0) And (d > 0) Then Begin
       {若相邻点未标号且对应弧流量可改进}
          Inc(op); Q[op] := x^.k; {则相邻点进入队列和可改进路,,并为之标号}
```

```
            Pth[x^.k].p := u; Pth[x^.k].w := x;
             If Pth[u].d < d Then Pth[x^.k].d := Pth[u].d
                             Else Pth[x^.k].d := d;
          End; {then}
          x := x^.next{分析U的下一个相邻点}
        End{while}
      End; {while}
      If Pth[trm].p <> 0 Then Begin{若汇点已标号}
        u := trm; d := Pth[u].d; {则从汇点出发,倒向修正可改进路上的流量}
        Repeat
          Pth[u].w^.f := Pth[u].w^.f + d;
          Pth[u].w^.g^.f := - Pth[u].w^.f;
          u := Pth[u].p
        Until u = stt{直到倒推至源点}
      End
    Until Pth[trm].p = 0{直至不存在可改进路为止}
End; {ford }

Procedure Answer;
Var
  i              : Integer;
  x              : Link;
Begin
  For i := 1 to N Do Begin{依次分析每个顶点}
    x := Nbs[i];
    While x <> nil Do Begin{依次分析与i顶点相连的每条弧}
      If x^.f > 0{若该弧的流量大于0}
        Then Writeln(i:4,' ->', x^.k:3, x^.f:6);
       {则打印该弧及其流量}
      x := x^.next{分析相连i顶点的下一条弧}
    End{while}
  End{for}
End; {answer}

Procedure Ford_Limit;
Var
  i              : Integer;
  x              : Link;
```

```
Begin
  Nbs[N + 1] := nil; Nbs[N + 2] := nil; {附加源和附加汇的邻接表初始化为空}
  For i := 1 to N Do Begin
    Insert(N + 1, i, 0, aa[i]);
    {附加源分别向每一个顶点引一条有向弧,容量为β[i]的容量下限和}
    Insert(i, N + 2, 0, bb[i])
    {每一个顶点向附加汇引一条有向弧,容量为α[i]的容量下限和}
  End;
  Insert(stt, trm, 0, Maxint);
  {源点与汇点间加入容量为∞的有向弧}
  Nbs[trm]^.c := Maxint;
  Ford(N + 1, N + 2); {求附加网的最大流}
  x := Nbs[N + 1]; {分析连接附加源的流量是否满载}
  While (x <> nil) And (x^.f = x^.c) Do x := x^.next;
  If x <> nil Then Writeln('No Answer'){若不满载,则宣布无解}
    Else Begin{否则为满载}
      Nbs[stt] := Nbs[stt]^.next; {撤去源点与汇点之间的两条弧}
      Nbs[trm] := Nbs[trm]^.next;
      For i := 1 to N Do Begin{分析原网中的每一个顶点}
        Nbs[i] := Nbs[i]^.next^.next; {撤去该顶点与附加汇的一条弧}
        x := Nbs[i]; {分析该顶点出发的所有弧}
        While x <> nil Do Begin
          x^.f := x^.f + x^.t; {求弧的可行流}
          x^.b := x^.t; x^.c := x^.c + x^.t; {恢复弧的下界和上界}
          x := x^.next{分析下一条相连弧}
        End{while}
      End; {for}
    * Ford(stt, trm); {在原网中放大可行流}
      Answer{输出结果}
    End{else}
End; {ford_limit}

Begin
  Init; {输入数据构造N网}
  Ford_Limit; {计算和输出容量有上下界的最大流}
  Writeln('***********');
End.{main}
```

对于一个容量有上下界的网络 N, 我们已经有了对其求最大流的算法，因此不难得出求 N 网的最小流的算法:

依照前述方法构造附加网 N, 并用标号法求 N 的最大流。若流出附加源 S 的一切弧都满载，则求出 N 网的一个可行流

$$F(e) = F(e) + B(e)$$

上述工作完全与求容量有上下界的网络的最大流一致。 与之不同的是接下来放大可行流 F 时倒了一个方向，以 t 为源点，S 为汇点，倒向求出的最大流即为从 S 到 T 的最小流。 因此只要将 ford－limit 过程中的一行语句

″ ford (stt , trm);″ (＊标注的行)

改为″ford (trm , stt);″

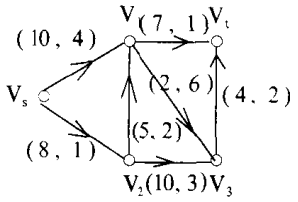
使得 N 网中的源点、汇点变成倒向求解时的汇点和源点，原程序便可以彻底更新为最小流算法了。

§7.3 最小费用最大流问题

上面讨论了寻找网络中的最大流问题。在实际生活中，涉及“流”的问题时， 人们考虑的还不只是流量，而且还有“费用”的因素。

【例 1】求输送量最大且费用最少的运输方案

图 7.3 - 1 是一个公路网，V_s 是仓库乙所在地，即物资的起点，V_t 是甲地，即物资的终点。另外，每一条弧旁都写了两个数字。 第一个数字表示某一时间里通过这段公路的最多吨数，第二个数字表示每吨物资通过该公路的费用。 现在的问题是怎样安排运输才能即使得从起点 V_s 运到终点 V_t 的物资最多，又使得总的运输费用最少?



(弧旁的数字为容量，单位流量费用)

图 7.3 - 1

算法分析:

显然图 7.3 - 1 是一个网络。这个网络的每一条弧 (V_i , V_j) 除给定的容量 c_{ij} 外，还给了一个单位流量费用 $B_{ij} \geqslant 0$。上述问题的数学模型即要求一个最大流 F, 使流的总输送费用

$$B(F) = \sum_{(V_i, V_j) \in A} B_{ij} * F_{ij}$$

取极小值

这即所谓的最小费用最大流问题。

§7.2 节中可知，寻找最大流的方法是从某个可行流出发，找到关于这个流的一条可改进路 P, 沿着 P 调整 F, 对新的可行流试图寻找关于它的可改进路，如此反复直至求得最大流。现在要寻求最小费用的最大流，我们首先考察一下，当沿着一条关于可行流 F 的可改进路 P， 以 a＝1 调整 F, 得到新的可行流 F′ (显然 V (F′) ＝V (F′) ＋1) ,B (F′) 比 B (F) 增加多少?

不难看出:

$$B(F')-B(F)$$

$$=[\sum_{P+}B_{ij}*(F'_{ij}-F_{ij})-\sum_{P^-}B_{ij}*(F'_{ij}-F_{ij})]$$

$$=\sum_{P+}B_{ij}-\sum_{P^-}B_{ij}$$

我们把 $\sum_{P+}B_{ij}-\sum_{P^-}B_{ij}$ 称为这条可行路 P 的“费用”。

显然,若 F 是流量为 V(F) 的所有可行流中费用最小者,而 P 是关于 F 的所有可改进路中费用最小的可改进路,那么沿 P 去调整 F,得到的可行流 F',即流量为 V(F') 的所有可改进流中的最小费用流。这样,当 F 是最大流时,它也即我们所要求的最小费用最大流了。

注意到,由于 $B_{ij}\geqslant 0$,所以 F=0 必是流量为 0 的最小费用流。这样,总可以从 F=0 出发求最小费用最大流。一般地,设已知 F 是流量 V(F) 的最小费用流,余下的问题如何去寻求关于 F 的最小费用可改进路。为此我们将原网中的每条弧 (U,V) 变成两条方向相反的弧:

前向弧 <U, V>,其容量 C 和费用 B 不变,流量 F 为 0;

后向弧 <V, U>,其容量 C 为 0,费用为 -B,流量 F 为 0;

每一个顶点设置一个参数 CT,表示源点至该顶点的通路上的费用和。如果我们得出一条关于 F 的最小费用可改进路时,则该路上每一个顶点的 CT 值相对其它可改进路来说是最小的。每一次寻找最小费用可改进路前,源点的 CT 值为 0,其它顶点的 CT 值为∞。

设 cost 为流的运输费用。初始时由于 F=0,则 cost=0。我们每求出一条关于 F 的最小费用可改进路,则通过 cost ← cost + $\sum_{e\in p}B(e)*d$ (d 为 p 的可改进量) 累计流的运输费用的增加量。

显然,当求出最小费用最大流时, cost 便成为最大流的运输费用了。

另外设置布尔变量 break 为最小费用可改进路的延伸标志。在搜索了网中的每一个顶点后,若 break=true 表示可改进路还可以延伸,还需要重新搜索网中的顶点;否则说明最小费用的可改进路已经找到或者最大流已经求出。

```
cost ← 0;
repeat
  可改进路撤空;
  设源点的 CT 值为 0 并进入可改进路;
   repeat
     break ← false;
     for u:=1 to N do begin
        分析 U 出发的所有弧(U, V);
        if ((U, V)的流量可改进)and(源点至 U 有通路)and
```

(U 的 CT 值 +(U, V) 的费用 <V 的 CT 值) then begin

break ← true;

V 的 CT 值← U 的 CT 值 +(U, V) 的费用;

V 进入可改进路并为之标号;

end; {then}

end; {for}

until break = false;

if 汇点已标号 then begin

从汇点出发倒向修正可改进路的流量;

cost ← cost + $\sum_{e\in p}$ B(e) * d (d 为 p 的可改进量);

end ; {then}

until 汇点未标号;

例如图 7.3 - 2 为例，求最小费用最大流。弧旁的数字为 (B_{ij} , C_{ij})

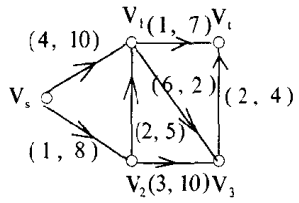


图 7.3 - 2

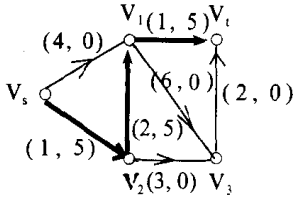
(1) 取 $F^{(10)} = 0$ 为初始可行流，求出关于 $F^{(10)}$ 的最小费用可改进路 $P_1 = \{V_s, V_2, V_1, V_t\}$。在 P_1 上进行调整，得出可行流 $F^{(1)} = 5$。cost = 5 * 1 + 2 * 5 + 1 * 5 = 20 (图 7.3 - 3 (a))。

(2) 取 $F^{(1)} = 5$ 为可行流，求出关于 $F^{(1)}$ 的最小费用可改进路 $P_2 = \{V_s, V_1, V_t\}$。在 P_2 上进行调整，得出可行流 F(2) = 7。cost = 20 + 4 * 2 + 1 * 2 = 30 (图 7.3 - 3 (b))。

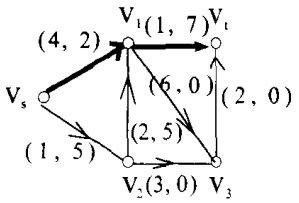
(3) 取 $F^{(2)} = 7$ 为可行流，求出关于 $F^{(2)}$ 的最小费用可改进路 $P_3 = \{V_s, V_2, V_3, V_t\}$。在 P_3 上进行调整，得出可行流 $F^{(3)} = 10$。cost = 30 + 3 * 1 + 3 * 3 + 2 * 3 = 48 (图 7.3 - 3 (c))。

(4) 取 $F^{(3)} = 10$ 为可行流，求出关于 $F^{(3)}$ 的最小费用可改进路 $P_4 = \{V_s, V_1, V_2, V_3, V_t\}$。在 P_4 上进行调整，得出可行流 $F^{(4)} = 11$。cost = 48 + 1 * 4 − 1 * 2 + 1 * 3 + 1 * 2 = 55 (图 7.3 - 3 (d))。

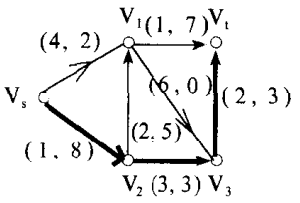
由于不存在关于 $F^{(4)}$ 的最小费用可改进路，所以 $F^{(4)}$ 为最小费用最大流，流的总运输费用为 55。



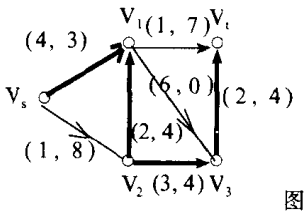
(a)



(b)



(c)



(d)

图中弧旁数字为 (b_{ij}、f_{ij})
粗线为最小费用可改进路

图 7.3-3

下面，我们给出求最小费用最大流的程序

```
Program Minimum_cost_And_Maximum_flow;

Const
  Maxn              =  1000;{最多顶点数}

Type
  Link              = ^Nbour;{邻接表指针类型}
  Nbour             =  Record{邻接表结点类型}
                         k          :  1..Maxn;{顶点序号}
                         f, c, w    :  Integer;{流量，容量，费用}
                         next, g    :  Link{前向弧指针，后向弧指针}
                       End;
  WType             =  Record{可改进路的结点类型}
                         p, d, ct   :  Integer;
                       {标号指针，当前可改进量，路径代价}
                         w          :  Link{该边在邻接表的指针}
                       End;

Var
  N, stt, trm, cost     :  Integer;{实际顶点数，源点，汇点，总输送费用}
  F                     :  Text;{文件变量}
  Nbs                   :  Array [1..Maxn] of Link;{N网的邻接表}
  Pth                   :  Array [1..Maxn] of WType;{可改进路}

Procedure Insert(u, v, c, w : Integer);
{在U对应的邻接表中插入前向弧(U, V),其容量为C，费用为W，流量为0}
{在V对应的邻接表中插入后向弧(U, V),其容量为0、费用为-W、流量为0}
Var
  x             :  Link;
Begin
```

```
  New(x);
  x^.k := v; x^.c := c; x^.f := 0; x^.w := w;
  x^.next := Nbs[u]; Nbs[u] := x;
  New(x^.g);
  x^.g^.k := u; x^.g^.c := 0; x^.g^.f := 0;
  x^.g^.w := - w; x^.g^.g := x;
  x^.g^.next := Nbs[v]; Nbs[v] := x^.g
End; {insert}

Procedure Init;
Var
  u, v, c, w     :  Integer;
Begin
  Assign(F, 'INPUT.DAT');{输入文件名串与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N, stt, trm); {读入顶点数,汇点,源点}
  For u := 1 to N Do Nbs[u] := nil; {N网的邻接表初始化为空}
  While Not Eof(F) Do Begin{依次读入每条弧及其容量和费用,构造N网}
    Readln(F, u, v, c, w);
    Insert(u, v, c, w)
  End; {while}
  Close(F)
End; {init}

Procedure Ford(stt, trm : Integer);
{输入源点和汇点,计算最小费用最大流及其总的运输费用}
Var
  i, u, d        :  Integer;
  break          :  boolean;
  x              :  Link;
Begin
  cost := 0; {总的运输费用初始为空}
  Repeat
    For i := 1 to N Do Begin{可改进路撤空}
      Pth[i].p := 0; Pth[i].d := Maxint; Pth[i].ct := Maxint
    End;
    Pth[stt].p := stt; Pth[stt].ct := 0; {源点进入可改进路}
    repeat
```

```
    break :=false;
     For u := 1 to N Do Begin{依次分析每一顶点}
       x := Nbs[u];
       While x <> nil Do Begin{分析顶点 U 出发的所有弧}
         break:=true;
         d := x^.c - x^.f;
      If (d>0)And(Pth[u].ct<Maxint)And(Pth[u].ct+x^.w<Pth[x^.k].ct)Then Begin
    {若该弧的流量可改进且源点至 U 点有通路且源点经 U 顶点至相邻点的路目前最短}
           Pth[x^.k].ct := Pth[u].ct + x^.w; {修正源点至相邻点的路径代价}
           Pth[x^.k].w := x; Pth[x^.k].p := u; {相邻点进入可改进路并为之标号}
           If d < Pth[u].d Then Pth[x^.k].d := d
                           Else Pth[x^.k].d := Pth[u].d
         End; {then}
         x := x^.next{分析顶点 U 出发的下一条弧}
       End{while}
      end{for}
    until not break ;
   If Pth[trm].p <> 0 Then Begin{若汇点已标号}
     u := trm; d := Pth[u].d;
    {从汇点出发,倒向修正可改进路的流量并累计总的输送费用}
     Repeat
       cost := cost + d * Pth[u].w^.w;
       Pth[u].w^.f := Pth[u].w^.f + d;
       Pth[u].w^.g^.f := - Pth[u].w^.f;
       u := Pth[u].p
     Until u = stt{直到倒推至源点为止}
   End{then}
 Until Pth[trm].p = 0{直至不存在可改进路}
End; {ford}

Procedure Answer; {输出 N 网的最小费用最大流及其总的运输费用}
Var
  i              : Integer;
  x              : Link;
Begin
  For i := 1 to N Do Begin{依次分析每一个顶点}
    x := Nbs[i];
    While x <> nil Do Begin{依次分析顶点 i 出发的每一条弧}
```

```
    If x^.f > 0{若该弧流量大于0,则输出该弧及其弧的流量}
      Then Writeln(i:4,' -->', x^.k:3, x^.f:6);
    x := x^.next{分析顶点i出发的下一条弧}
  End{while}
 End;{for}
 Writeln('Total cost = ', cost){输出总的运输费用}
End; {answer}

Begin
 Init; {输入数据,构造N网}
 Ford(stt, trm); {计算N网的最小费用最大流及其总的运输费用}
 Answer{输出结果}
End.{main}
```

§7.4 网络流算法的应用

网络流有许多实际的应用。有些问题看起来与流量问题风马牛不相干，但只要能构造出与问题要求对应的附加网，就可以归结到网络流的算法。

【例1】二分图的最大匹配

二分图G(X, Y, E)是这样的一种图：G的顶点划分成两个部分X和Y，G中的每条边的两个端点一定是一个属于X而另一个属于Y。所谓二分图的匹配是指边集E的一个子集M(M⊆E)，M中的任意两条边都没有公共端点。例如图7.4-1中的粗边为一个匹配。试找出包含边数最多的匹配M。

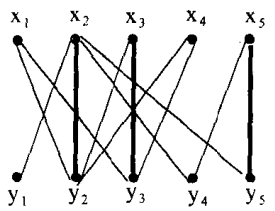


图7.4-1

算法分析：

求二分图最大匹配的算法是以求网络最大流的标号法为基础的。具体的说

1. 从二分图G出发构造一个网络N

(1)增加一个源点S和一个汇点T；

(2)从S向X的每一个顶点引一条有向弧；从Y的每一个顶点向T引一条有向弧；

(3)原来G中的边都改成有向弧，方向是从X顶点指向Y顶点；

(4)令所有弧的容量都等于1；

2. 对网络N运行一次标号算法；

3. 从X的顶点指向Y的顶点的弧集中，弧流量为1的弧对应二分图的匹配边，最大流量F对应二分图的最大匹配的边数 |M|。

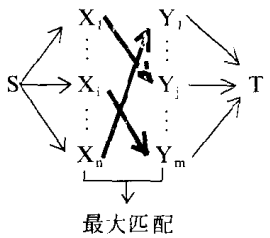


图 7.4-2

上述算法的道理很简单。对 M 中的每条边 XY，通过路径 S-X-Y-T，从 S 到 T 流过 1 个单位。显然，M 中的各条边分布在各自不同的路径上，所以最大流量 $F \geq |M|$。反过来，所有从 S 到 T 的路径皆形如 S-X-Y-T。若它从 S 至 T 传递一个流量，则不会再有边 XY′ 和 X′Y 能被用来通过流量，因此使得 F(X, Y) = 1 的边 (X, Y) 的集合组成二分图的一个匹配。于是 $|M| \geq F$。由此可见 $F = |M|$。编程时，可在读二分图 G 的同时直接构作网络 N:

已知 $|X| = n_1$, $|Y| = n_2$。设顶点 1, … 顶点 n_1 为 X; 顶点 $n_1 + 1$.. 顶点 $n_1 + n_2$ 为 Y; 源点 S 为顶点 $n_1 + n_2 + 1$; 汇点 T 为顶点 $n_1 + n_2 + 2$。先读入二分图，即读入顶点 1..n_1 中指向顶点 $n_1 + 1$..$n_1 + n_2$ 的所有弧 $(i, j)(1 \leqslant i \leqslant n, n_1 + 1 \leqslant j \leqslant n_1 + n_2)$，每条弧的容量为 1; 然后源点 S 向 X 的每一顶点引出有向弧 $(n_1 + n_2 + 1, 1)$, $(n_1 + n_2 + 1, 2) \cdots (n_1 + n_2 + 1, n_1)$，每条弧的容量为 1; Y 的每一个顶点向汇点 T 引出有向弧 $(n_1 + 1, n_1 + n_2 + 2)$, $(n_1 + 2, n_1 + n_2 + 2) \cdots (n_1 + n_2, n_1 + n_2 + 2)$ 每条弧的容量为 1。

构造出附加网后，直接使用标号法对其求最大流。最后检查顶点 1..n_1，指向顶点 $n_1 + 1$..$n_1 + n_2$ 的弧集，凡是 $F(i, j) = 1$ 的弧 $(1 \leqslant i \leqslant n_1, n_1 + 1 \leqslant j \leqslant n_1 + n_2)$ 对应二分图的一个匹配 $(X_i, Y_j - n_1)$。

这个程序留给读者编写。

【例 2】最小路径覆盖

有向图 G=(V, E) 的一个路径覆盖是一个其结点不相交的路径集合 P，图中的每一个结点仅分别包含于 P 中的一条路径。路径可从任意结点开始和结束，且长度也为任意值，包括 0。请写出一个有效算法，找出一个包含尽可能少的路径的路径覆盖。

算法分析:

求最小路径覆盖的算法也是以标号法为基础的。

1. 构造一个 N 网 (N=(V′, E′, C′))

将每一个顶点 i ($1 \leqslant i \leqslant n$) 拆成两个顶点 X_i 和 Y_i。若 $(i, j) \in E$, 则 $(X_i, Y_j) \in E'$。且该弧的容量为 1;

增加一个源点 X_0, X_0 分别向 X_i 引一条容量为 1 的有向弧 $(X_0, X_i)(1 \leqslant i \leqslant n)$;

增加一个汇点 Y_0，每一个 Y_j 向 Y_0 引一条容量为 1 的有向弧 (Y_j, Y_0) $(1 \leqslant j \leqslant n)$;

2. 用标号法求 X_0 到 Y_0 的最大流;

3. 如果 N 网有最大流 F, 则图 G 上有一个最小路径覆盖 P, 且 $|P| + |F| = |V|$。根据 F 可构造出 P:

若 $F(X_0, X_i) = F(X_i, Y_j) = F(Y_j, Y_0) = 1$，则 $(i, j) \in P$ 中某条路径；

若 $F(X_0, X_i) = F(Y_i, Y_0) = 0$，则 $\{i\}$ 是 P 中一条长度为 0 的路径；

上述算法思想的根据是什么呢？分析如下：

若 (i, j) 在 P 中某条路径上，则 $F(X_0, X_i) = F(X_i, Y_j) = F(Y_j, Y_0) = 1$，N 中其它边的流为 0，显然满足了容量的上下界要求。又因为每个结点在 P 中出现一次，因此同时也满足了流出量 = 流入量的平衡条件。对于任意顶点 $i \in V$，若 i 是 P 中某条路径的最后一个顶点，则 $F(X_0, X_i) = 0$；若 i 是 P 中某条路径的“前缀”顶点，则 $F(X_0, X_i) = 1$，所以 $\sum_{i \in V} f(x_0, x_i) = |f|$ 是路径覆盖中“前缀”顶点的总和。又因为每条路径 $P_i \in P$ 无圈，都有一个最后顶点（即便是 $|P_i| = 1$，也是一条长度为 0 的路径），所以得出 $|F| + |P| = |V|$。

下面，我们用反证法求证 P 中顶点的不相交性。设 P 中的两条路径 P_a 和 P_b 分别包含公共顶点 U，$(V_1, U) \in P_a$，$(V_2, U) \in P_b$。由此得出 $F(X_{v1}, Y_u) = 1$，$F(X_{v2}, Y_u) = 1$，这与 $C(Y_u, Y_0) = 1$ 矛盾，因此图中的每一个顶点仅包含于 P 中的一条路径。

综上所述，若求出 N 的最大流 F，那么路径覆盖 $|P| = |V| - |F|$ 必然是最小的。

编程时，可在读图的信息的同时直接构造网络 N：

设源点 X_0 为顶点 2N+1，汇点 Y_0 为顶点 2N+2；原图中的顶点 i 拆分成顶点 i 和顶点 i+N（$1 \leqslant i \leqslant N$）。每读一条边 (i, j)，则加入一条容量为 1 的有向弧 (i, N+j)；源点 2N+1 分别向顶点 1..顶点 N 引出容量为 1 的有向弧；顶点 N+1..顶点 2N 也分别向汇点 2N+2 引出容量为 1 的有向弧。用标号法对 N 网求最大流后，网中流量为 1 的弧 (i, j) ($1 \leqslant i \leqslant N$, $N+1 \leqslant J \leqslant 2N$) 对应最小路径覆盖中某条路径的边 <i, j−n>。

下面，给出程序题解：

```
Program Paths__Covering__A__Graph;
Const
  Maxn              =  1000; {最多顶点数}

Type
  Link              =  ^Nbour; {N网的邻接表的指针类型}
  Nbour             =  Record{N网的邻接表结点的类型}
                         k          :  1..Maxn; {顶点序号}
                         f, c       :  Integer; {流量，容量}
                         next, g    :  Link; {前向弧指针，后向弧指针}
                       End;
  WType             =  Record
                         p, d       :  Integer;
                       {标号前趋指针，当前可改进量}
```

```
                    w          : Link{弧指针}
                  End;

Var
  N                : Integer; {实际顶点数}
  F                : Text; {文件变量}
  Nbs              : Array [1..Maxn] of Link; {N网的邻接表}
  Pth              : Array [1..Maxn] of WType; {可改进路}
  Ans              : Array [1..Maxn, 1..2] of Integer;
  {路径指针。若F(U, V)=1则ANS[U, 1]=V, ANS[V, 2]=U}

Procedure Insert(u, v, c : Integer);
Var
  x          : Link;
  {在U对应的邻接表中插入流量为0,容量为C的前向弧(U, V);}
  {在V对应的邻接表中插入流量为0容量为0的后向弧(U, V);}
Begin
  x := Nbs[u];
  While (x <> nil) And (x^.k <> v) Do x := x^.next;
  If x <> nil Then x^.c := c
    Else Begin
      New(x);
      x^.k := v; x^.c := c; x^.f := 0;
      x^.next := Nbs[u]; Nbs[u] := x;
      New(x^.g);
      x^.g^.k := u; x^.g^.c := 0; x^.g^.f := 0; x^.g^.g := x;
      x^.g^.next := Nbs[v]; Nbs[v] := x^.g
    End{else}
End; {insert}

Procedure Init;
Var
  u, v, c      : Integer;
Begin
  Assign(F, 'INPUT.DAT');{文件变量与输入文件名串连接}
  Reset(F); {文件读准备}
  Readln(F, N); {读入图的顶点数}
  For u := 1 to 2 * N + 2 Do Nbs[u] := nil; {N网的邻接矩阵初始化为空}
```

```
  While Not Eof(F) Do Begin
    Readln(F, u, v); {读入弧(U, V)}
    Insert(u, N + v, 1){将容量为1的(XU, YV)加入N网}
  End; {while}
  For u := 1 to N Do Begin
    Insert(2 * N + 1, u, 1); {源点向XU引一条容量为1的有向弧}
    Insert(N + u, 2 * N + 2, 1){YU向汇点引一条容量为1的有向弧}
  End; {for}
  Close(F)
End; {init}

Procedure Ford(stt, trm : Integer);
Var
  Q              : Array [1..Maxn] of 1..Maxn; {队列}
  op, cl, u, d   : Integer; {队首指针,队尾指针,当前顶点序号,可改进量}
  x              : Link; {辅助指针}
Begin
  Repeat
    FillChar(Pth, Sizeof(Pth), 0); {可改进路设空}
    Pth[stt].d := Maxint; Pth[stt].p := stt;{源点进入可改进路并与入标号}
    cl := 0; op := 1; Q[op] := stt; {源点进入队列}
    While (cl < op) And (Pth[trm].p = 0) Do Begin
    {若Q队列未空且汇点未标号,则循环}
      Inc(cl); {队首顶点U出队}
      u := Q[cl];
      x := Nbs[u];
      While x <> nil Do Begin{分析U出发的每一条弧}
        d := x^.c - x^.f;
        If (Pth[x^.k].p = 0) And (d > 0) Then Begin
        {若相邻点未标且该弧流量可改进}
          Inc(op); Q[op] := x^.k; {则相邻点进入队列}
          Pth[x^.k].p := u; Pth[x^.k].w := x; {相邻点标号并进入队列}
          If Pth[u].d < d Then Pth[x^.k].d := Pth[u].d
                          Else Pth[x^.k].d := d;
        End; {then}
        x := x^.next{分析U出发的下一条弧}
      End{while}
    End; {while}
```

```
    If Pth[trm].p <> 0 Then Begin{若汇点已标号}
      u := trm; d := Pth[u].d; {则从汇点出发,倒向修正可改进路的流量}
      Repeat
        Pth[u].w^.f := Pth[u].w^.f + d;
        Pth[u].w^.g^.f := - Pth[u].w^.f;
        u := Pth[u].p
      Until u = stt {直至倒推至源点}
    End{then}
  Until Pth[trm].p = 0 {直至不存在可改进路}
End; {ford}

Procedure Answer;
Var
  u, v, t        : Integer;
  x              : Link;
Begin
  For u := 1 to N Do Begin{ANS 指针初始化}
    Ans[u, 1] := u; Ans[u, 2] := 0
  End;
  For u := 1 to N Do Begin
    x := Nbs[u]; {分析 U 出发的每一条弧的流量}
    While (x <> nil) And (x^.f < 1) Do x := x^.next;
    If x <> nil
    {若与 XU 相边的弧(XU, YJ)的流量为 1,则设定 ANS[U, 1]=J ANS[J, 2]=U}
      Then Begin
             Ans[u, 1] := x^.k - N; Ans[x^.k - N, 2] := u
           End{then}
  End; {for}
  For u := 1 to N Do
    If Ans[u, 2] = 0 Then Begin
    {若顶点 U 为路径起点, 则顺 ANS 指针输出开始于 U 的一条路径}
      v := u; Write(u);
      While v <> Ans[v, 1] Do Begin
        v := Ans[v, 1];
        Write('->', v)
      End; {while}
      Writeln
    End{then}
```

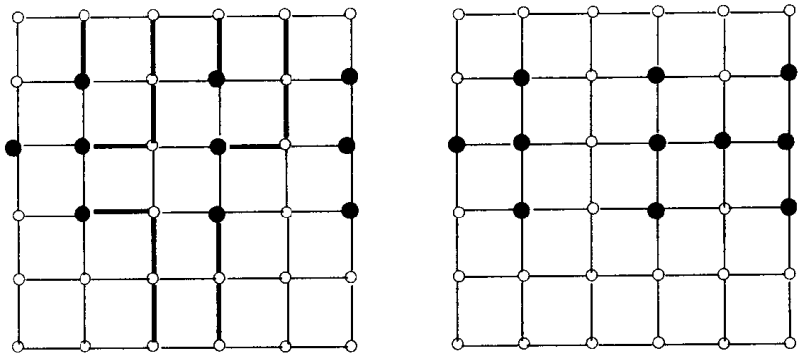
```
End ; {answer}

Begin
  Init ; { 读入数据, 构造附加网 }
  Ford ( 2 * N + 1, 2 * N + 2 ); { 计算附加网的最大流 }
  Answer{ 输出最小路径覆盖中的所有路径 }
End . {main}
```

【例 3】逃脱问题

一个 N*N 格栅是由 N 行和 N 列结点组成的一个无向图，如图 7.4-3 所示。我们用 (i, j) 表示处于第 i 行第 j 列的结点。除了边界结点，即满足 i=1, i=n, j=1, 或 j=n 的结点 (i, j), 格栅中的所有其它结点都有四个相邻结点。

处于格栅中的 M<N*2 个起始结点 (X_1, Y_1), (X_2, Y_2), $\cdots (X_m, Y_m)$, 逃脱问题就是确定从起始结点到边界上的任何 M 个相异的结点间是否存在 M 条其结点不相交的路径。例如图 7.4-3 (a) 中的格栅包含一个逃脱，而图 7.4-3 (b) 中的格栅中没有逃脱。



(a)　　(b)

开始点为黑色，其它均为白色。

图 7.4-3

描述一种有效算法以解决逃脱问题。

算法分析:

逃脱问题也是以标号法为基础。

1. 构造一个网 N

① 将每个格栅结点 (X, Y) 折分成两个结点 U_{XY} 和 V_{XY}，另外增加一个源点 S 和一个汇点 T;

② (X, Y) 对应的 U_{XY} 和 V_{XY} 之间边一条容量为 1 的有向边；若 (X_1,Y_1) 与 (X_2,Y_2) 在格栅中相邻，则 $V_{X1, Y1}$ 与 $U_{X2, Y2}$ 连一条容量为 1 的有向边；若 (X, Y) 是格栅的起点，则 S 向 $U_{X, Y}$ 连一条容量为 1 的有向边；若 (X, Y) 是格栅的终点，则 $V_{X, Y}$ 与 T 连一条容量为 1 的有向边。

2. 用标号法求 N 的最大流。若最大流的流量为 M, 则格栅包含一个逃脱，否则格栅中没有逃脱。

如果找到了流量为M的最大流，我们可以通过下述方法找出一个由M个起点到边界上的M个相异终点间的路径集合，且这些路径没有公共结点：

因为流量是M,且源点S发出M条容量为1的边，所以这些边的流量都为1。对于第i个起始结点(X_i,Y_i)来说，$U_{xi,yi}$的流入量为1($F(S,U_{xi,yi})=1$),则第i条路径P_i的起始结点为(X_i,Y_i)。用以下递归方法得出整个P_i:已知路径P_i的最后一个结点为(X_{i1},Y_{i1})对应N网中的顶点为$U_{xi1,yi1}$,它的流入量为1。所以$V_{xi1,yi1}$的流入量也为1,那么必有唯一的一条边$(V_{xi1,yi1},u)$,其流量也为1。若U是某个U_{xy},则把(X,Y)作为P_i的新结点，否则U只能是汇点T。当生成P_i时，其最后一个结点(X_{i1},Y_{i1})是在$U_{xi1,yi1}\rightarrow V_{xi1,yi1}\rightarrow T$这一段流量为1的子路径上，因此$U_{xi1,yi1}$只能是一个边界点。由此可见，$P_i$是一条以$(X_i,Y_i)$为起点、以某个边界点$(X_{i1},Y_{i1})$为终点的路径。

下面我们用反证法验证P集合中结点的不相交性。假设P_i与P_j中有共同的结点(X,Y)且(X,Y)是P_i中和第一个与P_j相交的结点，它在P_i中是第a个结点$P_{i,a}$,在P_j中是第b个结点$P_{j,b}$。因为(X,Y)是P_i中第一个与P_j相交的结点，所以有$P_{i,a-1}\neq P_{j,b-1}$。但生成P_i和P_j的过程中，只有当$U_{pi,a-1}\rightarrow V_{pi,a-1}\rightarrow U_{pi,a}$情况时，$P_{i,a}$才会加入$p_i$;也只有发生$U_{pj,b-1}\rightarrow V_{pj,b-1}\rightarrow U_{pj,b}$时，$P_{j,b}$才会加入$P_j$。因为$P_{i,a}=P_{j,b}$,意味着流入同一结点$U_{pi,b}$的流是为2，超出了容量限制，这是不可能的。由此可证，P中M条路径上没有重复结点.

总之，N中有流量为M的最大流与格栅中有M条满足条件的路径$P_1\cdots P_m$相对应。我们可以根据流量是否为M来判定格栅中是否包含一个逃脱。

下面给出程序题解。程序中每个格栅结点(X，Y)对应N网的一两个顶点：U_{XY}和V_{XY}。其中

U_{XY}的顶点序号为$(X-1)*N+Y$;

V_{XY}的顶点序号为$U_{XY}+N^2=(X-1)*N+Y+N^2$

源点的顶点序号为$2N^2+1$,汇点的顶点序号为$2N^2+2$。

按算法分析的要求构造出N网后对其求最大流。若流量=M,则分别从M个起点出发，顺着流量为1的弧一直找到汇点。一条流量为1的路径对应一条逃脱路线，其上的每一个顶点V_{XY}表示这条逃脱路线上的某个栅格结点(X,Y)。其中$X=(V_{XY}-1)\,divn+1$; $Y=(V_{XY}-1)\,modn+1$。

```
Program Escape_From_A_Net;

Const
  Maxn                  =  2000; {最多顶点数}
  Md                    :  Array [1..4,1..2] of Shortint
                        =  ((1, 0), (-1, 0), (0, 1), (0, -1));
  {MD[i, 1]-i方向上的水平增量; MD[i, 2]-i方向上的水平增量(1≤i≤4)}
Type
```

```
  Link                    =^Nbour; {N网的邻接表指针类型}
  Nbour                   =  Record{N网的邻接表的结点类型}
                             k          :  Integer; {顶点序号}
                             f, c       :  Shortint; {流量,容量}
                             next, g    :  Link{前向弧指针,后向弧指针}
                           End;
  WType                   =  Record
                             p, d       :  Integer;
  {标号,(指针,当前可改进量)}
                             w          :  Link{弧指针}
                           End;
  Point                   =  Record{格栅结点类型}
                             x, y       :  Byte{座标}
                           End;

Var
  N, M, stt, trm, Tot     :  Integer; {格栅规模,起始点个数,源点,汇点,流量和}
  F                       :  Text; {文件变量}
  Nbs                     :  Array [1..Maxn] of Link; {N网的邻接矩阵}
  Pth                     :  Array [1..Maxn] of WType; {可改进路}
  Sps                     :  Array [1..Maxn] of Point; {格栅的结点序列}

Procedure Insert(u, v, c : Integer);
{在U对应的邻接表中插入流量为0,容量为c的前向弧(U, V)}
{在V对应的邻接表中插入流量为0,容量为0的后向弧(U, V)}
Var
  x               :  Link;
Begin
  x := Nbs[u];
  While (x <> nil) And (x^.k <> v) Do x := x^.next;
  If x <> nil Then x^.c := c
    Else Begin
      New(x);
      x^.k := v; x^.c := c; x^.f := 0;
      x^.next := Nbs[u]; Nbs[u] := x;
      New(x^.g);
      x^.g^.k := u; x^.g^.c := 0; x^.g^.f := 0; x^.g^.g := x;
      x^.g^.next := Nbs[v]; Nbs[v] := x^.g
```

```
    End  {else}
End;  {insert}

Procedure Init;
Var
  i, j, k, u, v :  Integer;
Begin
  Assign(F, 'INPUT.DAT');{输入文件名串与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N, M); {读入格栅的规模和起点个数}
  stt := 2 * N * N + 1; trm := stt + 1; {设置N网的源点和汇点}
  For i := 1 to trm Do Nbs[i] := nil; {N网的邻接表设空}
  For i := 1 to M Do Readln(F, Sps[i].x, Sps[i].y); {读入M个起点}
  For i := 1 to N Do
    For j := 1 to N Do Begin
      u := (i - 1) * N + j; {计算(i, j)的结点序号U}
      Insert(u, u + N * N, 1); {Uu与Vu之间连一条容量为1的有向边}
      For k := 1 to 4 Do{寻找U的相邻点}
        If (i + Md[k, 1] in [1..N]) And (j + Md[k, 2] in [1..N])
       {若结点u与结点k相邻,则Vu与Uk连一条容量为1的有向边}
          Then Insert(N * N + u, u + Md[k, 1] * N + Md[k, 2], 1);
      If (i = N) Or (i = 1) Or (j = N) Or (j = 1)
     {若结点U在格栅边界,则V u与汇点连一条容量为1的有向边}
        Then Insert(N * N + u, trm, 1)
    End; {for}
  For i := 1 to M Do
    Insert(stt, (Sps[i].x - 1) * N + Sps[i].y, 1);
{源点分别向m个起始点 V_{x,y} 连一条容量为1的有向边}
  Close(F)
End; {init}

Procedure Ford(stt, trm : Integer); {求附加网的最大流}
Var
  Q              :  Array [1..Maxn] of 1..Maxn;
  op, cl, u, d   :  Integer; {队列的首、尾指针,顶点序号,可改进量}
  x              :  Link; {辅助指针}
Begin
  Tot := 0; {流量初始化为0}
```

```
  Repeat
    FillChar(Pth, Sizeof(Pth), 0); {可改进路置空}
    Pth[stt].d := 255; Pth[stt].p := stt; {源点进入可改进路并为之标号}
    cl := 0; op := 1; Q[op] := stt; {源点进入队列}
    While (cl < op) And (Pth[trm].p = 0) Do Begin
    {若队列非空且汇点未标号，则循环}
      Inc(cl); {队首顶点 U 出队}
      u := Q[cl];
      x := Nbs[u];
      While x <> nil Do Begin{分析 U 出发的每一条弧}
        d := x^.c - x^.f;
        If (Pth[x^.k].p = 0) And (d > 0) Then Begin
        {若相邻点未标号且该弧的流量可改进}
          Inc(op); Q[op] := x^.k; {则相邻点进入队列和可改进路并为之标号}
          Pth[x^.k].p := u; Pth[x^.k].w := x;
          If Pth[u].d < d Then Pth[x^.k].d := Pth[u].d
                          Else Pth[x^.k].d := d;
        End; {then}
        x := x^.next{分析 U 出发的下一条弧}
      End{while}
    End; {while}
    If Pth[trm].p <> 0 Then Begin{若汇点已标号}
      u := trm; d := Pth[u].d; {则从汇点出发倒向修正可改进路上的流量}
      Repeat
        Pth[u].w^.f := Pth[u].w^.f + d;
        Pth[u].w^.g^.f := - Pth[u].w^.f;
        u := Pth[u].p
      Until u = stt; {直到倒推至源点为止}
      Inc(Tot){累计流量}
    End{then}
  Until Pth[trm].p = 0{直至不存在可改进路为止}
End; {ford}

Procedure Answer;
Var
  i, u, v        : Integer;
  x              : Link;
Begin
```

```
If Tot <> M Then Writeln('No Answer!'){若流量≠起点个数,则没有逃脱}
  Else Begin
    For i := 1 to M Do Begin{输出N条结点不相交的路径}
      u := Sps[i].x; v := Sps[i].y; {输出当前路径的起点(U, V)}
      Write(i:3,' (', u, ',' , v, ')');
      Repeat
        x := Nbs[N * N + (u - 1) * N + v];
        {求(U, V)在N网中的结点Vu, v的指针}
        While (x <> nil) And (x^.f < 1) Do
        {求Vu, v出发的流量为1的一条弧}
          x := x^.next;
        If x^.k <> trm Then Begin
        {求相邻点Uuv的坐标并输出}
          u := (x^.k - 1) div N + 1;
          v := (x^.k - 1) mod N + 1;
          Write('-->(', u, ',', v, ')')
        End{then}
      Until x^.k = trm; {直到倒推至汇点为止}
      Writeln
    End{for}
  End{else}
End; {answer}

Begin
  Init; {输入数据,构造附加网}
  Ford(stt, trm); {求附加网的最大流}
  Answer; {输出M条结点不相交的路径}
  Writeln('**********')
End. {main}
```

【例4】工程计划

用一个带权的有向无环图表示一个工程的时间安排:其中顶点表示项目，弧表示过程，权 L(e)表示该过程所需时间。由于整个工程只有一个开始点和一个完成点，因此图中有一个入度为零的源点表示工程开始；有一个出度为零的汇点表示工程结束。

例如图 7.4-4 是一个假想的有 11 个过程的工程安排，其中有 9 个项目 $V_1, V_2, V_3, \cdots\cdots, V_9$。每个项目表示在它之前的过程已经结束，在它之后的过程可以开始。如 V_1 表示整个工程开始，V_9 表示整个结束，V_5 表示 A_4 和 A_5 已经完成，A_7 和 A_8 可以开始，每个过程相联系的权是该过程所需的时间。

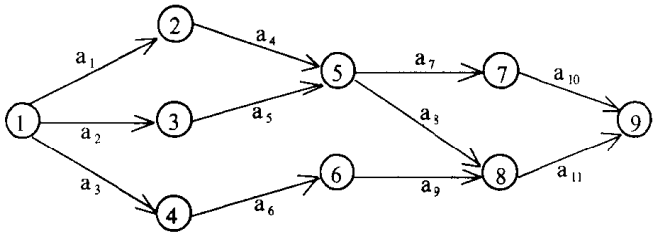


图 7.4-4

问题 1: 完成整个工程至少需要多少时间？哪些活动是影响工程进度的关键?

问题 2: 每一个过程由一台机器（或人）来完成，问需要最少准备多少台机器才能使得对任意给定的 A 值，每一过程都不至于因为机器不够而被延误。

算法分析:

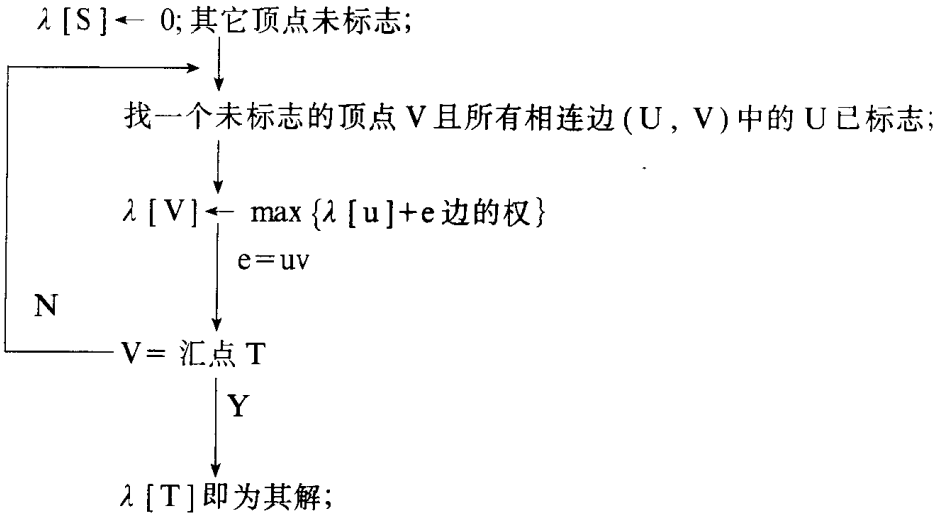
先来分析问题 1:

设 $\alpha[V]$——所有以 V 为头的边集。当 $\alpha[V]$ 中的所有过程完成之后，项目 V 才能开始;

$\beta[V]$——所有以 V 为尾的边集。显然，对于源点 S 来说 $\beta[S]$ 中的所有过程可以马上开始。而对于其顶点 V 来说，$\beta[V]$ 的所有过程必须待 $\alpha[V]$ 中的所有过程完成之后才能开始;

$\lambda[V]$——顶点 V 的标志: $\alpha[V]$ 中的过程全部完成所用的最小时间，即 $\beta[V]$ 中的所有过程的最早开始时间

S——源点; T——汇点;



容易看出，当问题 1 的算法停止时，由 T 向 S 经过确定顶点标的边

$$\lambda[V]=\max_{e=uv}\{\lambda[U]+L[e]\}$$

返回，则得到一条权和最大的路径，这一条路径又称为关键路径。欲缩短工期，必须把每条关键路径上至少一条边的权缩短。关键路径未必唯一。

下面分析问题 2 的算法思想:

该图构成了一个网络。定义每条边容量的下界 b(e)=1, 上界 c(e)=∞。由图中每边皆在从 S 到 T 的路径上以及边容量的定义可以看出,网络上有可行流，所以有从 S 到 T 的最小流。

从 S 到 T 的最小流对应由 T 到 S 的最大流。当求 T 到 S 的最大流时，过程中找不到可改进路，被标志顶点的集合为 S, 则 T∈S, S∈T, 于是截集 (S, T)=φ。不然的话，因 c(e)=∞，仍可由 S 中的某个顶点置向前标志，与算法的终止矛盾，所以 (T, S) 中任两条边都不会在同一条由 T 到 S 的路径上，这些边代表过程同时进行的时刻，所以机器数 ≥|(T, S)|= 由 T 到 S 的最大流量 |F′|

设 F 是从 S 到 T 的最小流量，相应的流为 F, F(e)≥b(e)=1 对应每一条边都成立。F 个单位的总流量分解到从 S 到 T 的 F 条路径上，能使图中的每条边至少在这 F 条路径的一条上。我们把每个机器分配给上述 F 条路径之一，每个机器在分配去的路径上从 S 到 T 依次完成各个过程，于是得知机器数 ≤F

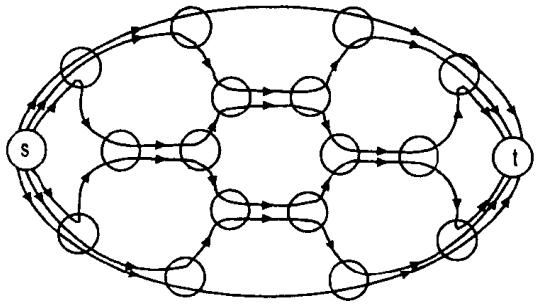


图 7.4-5

由 |F|=|F′| 可知问题 2 的机器数至少为 S 到 T 的最小流量 F′。

下面给出程序题解。为了编程的方便，我们将解题的顺序变换一下；先求解问题 2，再求解问题 1。

最初，我们直接读入工程网，每条弧的下界为工程时间，上界为∞；附加源分别向每一个顶点引出有向弧，下界为 0、上界为 U 顶点的出度和 β[U]; 每一顶点 U 向附加汇引出有向弧，下界为 0、上界为 U 顶点的入度和 α[U]。我们只取这个附加网的上界，使其正好对应求每边容量下界为 1, 上界为∞的最小流的附加网。然后对其使用标号法求出最大流，对每一条的弧的流量 +1 得出一个可行流。在撤去了附加源附加汇及其相连弧后，从工程网的汇点出发，倒向缩小可行流，最终求出最小流。在解出问题 2 后，直接在原工程网上求解问题 1。

```
Program Pert;

Const
  Maxn                    =   1000;{最多顶点数}
```

```
Type
  Link                  = ^Nbour;{邻接表指针类型}
  Nbour                 =  Record {邻接表结点类型}
                             k          :  1..Maxn;{顶点序号}
                             f, c, t :  Integer;
  {流量，下界，上界，(问题1中的过程所需时间)}
                             next, g    :  Link
  {前向弧指针（问题1的邻接指针）前向弧指针)}
                           End;
  WType                 =  Record
                             p, d       :  Integer;
  {前趋指针，当前可改进量（问题1的最早开始时间)}
                             w          :  Link{弧指针}
                           End;

Var
  N, stt, trm           :  Integer;{顶点数，源点，汇点}
  F                     :  Text;
  Nbs                   :  Array [1..Maxn] of Link;
                           {N网（问题1的图）的邻接矩阵}
  Pth                   :  Array [1..Maxn] of WType;
                           {可改进路（问题1的关键路径)}
  a, b                  :  Array [1..Maxn] of Integer;
  {a[V]-β[V]的容量下限和;  b[V]-α[V]的容量下限和}

Procedure Insert(u, v, c, t: Integer);
{在U对应的邻接表中插入下界为T,上界为C,流量为0的前向弧}
Var
  x             :  Link;
  {(U, V);在V对应的邻接表中插入下界为0,上界为0,流量为0,的后向弧(U, V)}
Begin
  New(x);
  x^.k := v; x^.c := c; x^.f := 0; x^.t := t;
  x^.next := Nbs[u]; Nbs[u] := x;
  New(x^.g);
  x^.g^.k := u; x^.g^.c := 0;
  x^.g^.f := 0; x^.g^.g := x; x^.g^.t := 0;
  x^.g^.next := Nbs[v]; Nbs[v] := x^.g
```

```
End ; {insert}

Procedure Init;
Var
  u , v , t        :  Integer ;
Begin
  Assign ( F, 'INPUT . DAT' ) ; { 输入文件名串与文件变量连接 }
  Reset ( F ) ; { 文件读准备 }
  Readln ( F, N, stt , trm) ; { 读入顶点数，源点和汇点 }
  For u := 1 to N + 2 Do Begin
  { N 网的邻接表初始化为空，各顶点的 α [ U ] 和 β [ U ] 的容量下限和初始化为零 }
    Nbs [ u ] := nil ;
    a[u] := 0 ; b[u] := 0
  End ;
  While Not Eof ( F ) Do Begin
    Readln ( F, u, v, t ) ; { 读入弧 (U,V) 及其过程时间 }
    Insert ( u , v , Maxint , t ) ;
    { 设定 ( U, V ) 的上界为∞ 、下界为过程时间，送入邻接表 }
    Inc ( a [ v ] ) ; Inc ( b [ u ] ) { 累计 α [ U ] 和 β [ U ] 的容量下限和 }
  End ; { while }
  Close ( F ) ;
  For u := 1 to N Do Begin { 附加源 }
    Insert ( N + 1, u , a [ u ] , 0 ) ; { 每一顶点 }
    Insert (u , N + 2 , b [ u ] , 0)
  End ;
  Insert ( stt , trm , Maxint , 0 ) ; { 源点和汇点间加两条下界为 0, 上界为∞ 有向弧 }
  Nbs [ trm ]^ . c := Maxint ;
End ; {init}

Procedure Ford ( stt , trm : Integer ) ; { 输入源点和汇点，求该网的最大流 }
Var
  Q                :  Array [1.. Maxn] of 1.. Maxn ;
  op , cl , u , d  :  Integer ;
  x                :  Link ;
Begin
  Repeat
    FillChar ( Pth , Sizeof ( Pth ) , 0 ) ; { 可改进路撤空 }
    Pth [ stt ] . d := Maxint ; Pth [ stt ] . p := stt ;
```

```
    {源点进入可改进路并标号}
    cl := 0; op := 1; Q[op] := stt; {源点进入队列}
    While (cl < op) And (Pth[trm].p = 0) Do Begin
    {若队列非空且汇点未标号，则循环}
      Inc(cl); {移出队首顶点 U}
      u := Q[cl];
      x := Nbs[u];
      While x <> nil Do Begin{分析 U 出发的每一条弧}
        d := x^.c - x^.f;
        If (Pth[x^.k].p = 0) And (d > 0) Then Begin
        {若相邻点未标号且弧的流量可改进}
          Inc(op); Q[op] := x^.k; {相邻点进入队列}
          Pth[x^.k].p := u; Pth[x^.k].w := x; {相邻点标号并进入可改进路}
          If Pth[u].d < d Then Pth[x^.k].d := Pth[u].d
                          Else Pth[x^.k].d := d;
        End; {then}
        x := x^.next{分析 U 出发的下一条弧}
      End{while}
    End; {while}
    If Pth[trm].p <> 0 Then Begin{若汇点已标号}
      u := trm; d := Pth[u].d; {从汇点出发，倒向修正可改进路的流量}
      Repeat
        Pth[u].w^.f := Pth[u].w^.f + d;
        Pth[u].w^.g^.f := - Pth[u].w^.f;
        u := Pth[u].p
      Until u = stt{直到倒推至源点为止}
    End{then}
  Until Pth[trm].p = 0{直至不可存在可改进路}
End; {ford}

Procedure Change;
Var
  u              : Integer;
  x              : Link;
Begin
  Nbs[stt] := Nbs[stt]^.next; {撤去源点与源点间的两条弧}
  Nbs[trm] := Nbs[trm]^.next;
  For u := 1 to N Do Begin{分析初始网的每一个顶点}
```

```
    Nbs[u] := Nbs[u]^.next^.next; {撤去U顶点与附加汇的一条弧}
    x := Nbs[u];
    While x <> nil Do Begin{分析U出发的每一条弧}
      If x^.c > 0 Then Begin{恢复该弧的可行流}
        x^.f := x^.f + 1;
        x^.g^.c := -1{设定后向弧的上界}
      End; {while}
      x := x^.next{分析U出发的下一条弧}
    End{while}
  End{for}
End; {change}

Procedure Answer;
Var
  i, tot        : Integer;
  x             : Link;
Begin
  tot := 0; {机器数初始化为0}
  For i := 1 to N Do Begin{分析每一顶点}
    x := Nbs[i];
    While x <> nil Do Begin {分析顶点i出发的每一条弧}
      If x^.f > 0 {若弧的流量 >0,则输出该弧及其流量}
        Then Writeln(i:4, ' ——→ ', x^.k:3, x^.f:6);
      If (i = 1) And (x^.f > 0) Then Inc(Tot, x^.f); {累计流量}
      x := x^.next{分析顶点i出发的下一条弧}
    End{while}
  End; {for}
  Writeln('********** tot = ', tot){输出机器数}
End;

Procedure Pert_One;

Procedure Print(v : Integer); {递归打印源点至V的关键路径}
Begin
  If v = stt Then Write(stt)
    Else Begin
      Print(Pth[v].p);
      Write(' ——→ ', v)
```

```
    End
End;

Var
  i, top, v      :  Integer; {辅助变量栈指针，顶点序号}
  x              :  Link;
  Q              :  Array [1..Maxn] of Integer;
Begin
  For i := 1 to N Do Begin{各顶点的前趋指针和最早开始时间清零}
    Pth[i].p := 0; Pth[i].d := 0
  End;
  Pth[stt].p := stt; {源点进入关键路径序列和栈}
  top := 1; Q[top] := stt;
  While (top > 0) And (Q[top] <> trm) Do Begin
  {若栈非空且栈顶元素非汇点，则循环}
    v := Q[top]; Dec(top); {栈顶顶点 V 出栈}
    x := Nbs[v];
    While x <> nil Do Begin{分析 V 出发的每一条弧}
      If x^.c > 0 Then Begin{若(V, X)存在}
       Dec(a[x^.k]);{X 的入度减 1}
       If a[x^.k] = 0 Then Begin{若指向 X 的弧已全部检查}
         Inc(top); Q[top] := x^.k{则 X 入栈}
       End; {then}
       If Pth[v].d + x^.t > Pth[x^.k].d Then Begin
       {若经过(V, X)后使得 X 的最早开始时间更晚}
          Pth[x^.k].d := Pth[v].d + x^.t; {则修正 X 的最早开始时间}
          Pth[x^.k].p := v{(V, X)进入关键路径}
       End{then}
      End; {then}
      x := x^.next{分析 V 出发的下一条弧}
    End; {while}
  End; {while}
  Print(trm);{递归打印关键路径上的项目}
  Writeln('PERT'' s time = ', Pth[trm].d);
  {打印完成整个工程至少需要多少时间}
End; {pert_one}

Begin
```

```
Init ; { 输入数据，构造附加网 }
  Ford ( N + 1, N + 2); { 计算附加网的最大流 }
  Change ; { 求可行流，并恢复初始网 }
  Ford ( trm, stt); { 从汇点出发，在原网中倒向缩小可行流 ]
  Answer; { 输出问题 2 的结束 }
  Pert__One ; { 计算和输出问题 1 中的关键路径 }
End . {main}
```

【例 5】 航天实验

NASA 正在为航天飞行计划一系列的太空飞行，在每次飞行中必须决定进行何种商业性实验和配载何种仪器设备。SPOCK 教授正在研究这一问题。对每次飞行，NASA 考察一个实验集合 $E=\{E_1, \cdots E_n\}$，并且实验 E_j 的商业赞助人已同意为该实验的结果向 NASA 支付 P_j 美元。 实验使用的全部仪器为集合为 $I=\{I_1, \cdots, I_n\}$；每个实验所需要用到的仪器为子集 $R_j \in I$。运送仪器 I_k 的费用为 C_k 美元。 教授的任务就是找出一个有效算法以确定在某一次指定飞行中要进行哪些实验并运送哪些仪器才能使净收益最大。 其中净收益指进行实验所获得的全部收入与运送仪器的全部费用的差额。请你帮教授设计这样一个有效算法。

算法分析:

航天实验问题的算法也是以标号法为基础:

1. 构造一个 N 网

① N 网的顶点为 $I_1, I_2, I_3, \cdots I_n$ 和 $E_1, E_2, \cdots, E_m$. 另外增加一个源点 S 和一个汇点 T;

② 从 S 出发，分别向 I_i 引出有向弧 (S, I_i)，该弧的容量为 C_i, $(i = 1..n)$;

每一个 E_j 向 T 引出有向弧 (E_j, t)，该弧的容量为 $P_j (j = 1...m)$;

若 $I_k \in R_j$，则 I_k 向 E_j 引出一条容量为∞的有向弧 (I_k, E_j) $(k = 1..n, j=1..m)$;

2. 求 S 到 T 的最大流。网中最小截量 C(S, T) 就是最大净收益下运送仪器的费用，其中 T 的所有顶点确定了应该进行哪些实验并运送哪些仪器。

下面，我们分析一下算法的由来:

若 $E_j \in T$，则对每一个 $I_k \in R_j$, 有 $I_k \in T$; $S \in V - T$。如若不然 $I_k \in S$, 则 $C(S, T) = \sum_{X \in S} \sum_{Y \in T} c(X, Y)$ 中 $C(I_k, E_j)$ 是一条容量为∞的边，导致截量 C(S, T) 为∞，这是不可能的。对于任一个实验方案 $X \subset E$, 对应于 G 的某个截集 (S, T), 实验的净收益 $= \sum_{j=1}^{N} P_j - C(S, T)$。这个截集由下述方法得到:

$$T = \{T\} \cup \{E_i : E_i \in X\} \cup \{I_k : I_k \in R_j, E_j \in X\}; \quad S = V - T$$

$$\because C(S, T) = \sum_{E} P_j + \sum_{TK \in T} C_K$$

$$\therefore \sum_{J=1}^{N} P_j - C(S, T) = \sum_{J=1}^{N} P_j - \sum_{Ej} P_j - \sum_{TK \in T} C_K$$

$$= \sum_{Ej \in T} P_j - \sum_{TK \in T} C_K$$

即实验方案 X 所获得的全部收入减去运送仪器的全部费用的差额为该方案的净收益。

由上可知，G 的某一个截集(S，T)对应一个实验方案，使得该实验方案净收益 = $\sum_{J=1}^{N} P_j - C(S, T)$。在该方案中，若实验 Ej∈T，则实验 E_j 不进行，实验 E_j 所需的仪器不上飞机；若实验 $E_j \in T$，因此 E_j 所需的仪器都在 T 中。即对每个 $I_k \in R_j$，有 $I_k \in T$，仪器 I_k 带上飞机。

根据最大流最小截量定理

$$|F| = C(S, T)$$

可知，求出了 N 的最大流 F，便有了最小截量 C(S，T)，自然地求得对应的实验方法和最大净收益。

下面给出程序题解。程序中设仪器 1…仪器 M 为顶点 1…顶点 M；实验 1…实验 N 为顶点 M+1…顶点 M+N；顶点 N+M+1 为源点；顶点 N+M+2 为汇点。然后对这个附加网求最大流。每一次找出可改进路后累加汇点的改进量，最终得出最小截量 C(S，T)。附加网的最大流求出后，顶点 M+1…顶点 M+N 中改进量为 0 的那些顶点 M+i，对应需要进行的实验 E_i ($1 \leqslant i \leqslant n$)；顶点 1…顶点 M 中改进量为 0 的那些顶点对应实验所需运送的仪器 I_j ($1 \leqslant J \leqslant M$)。

```
Program NASA;

Const
  Maxn          =  1000;{最多顶点数}

Type
  Link          = ^Nbour;{链指针}
  Nbour         =  Record{链结点}
                     k        :  1..Maxn;{顶点序号}
                     f, c     :  Integer;{流量，容量}
                     next, g  :  Link{前向弧指针，后向弧指针}
                   End;
  WType         =  Record{可改进路的结点类型}
                     p, d     :  Integer;{两个标号}
                     w        :  Link{该边在邻接表中的指针}
                   End;

Var
```

```
  N, M, profit, tot      : Integer;{实验数，仪器数，收入和}
  F                      : Text;{文件变量}
  Nbs                    : Array [1..Maxn] of Link;{N网}
  Pth                    : Array [1..Maxn] of WType;{可改进路}

Procedure Insert(u, v, c : Integer);
{在U对应的邻接表中插入容量为C的前向弧(u, v);在V对应的邻接表中插入容量为0
的后向弧(U, V)}

Var
  x              : Link;
Begin
  x := Nbs[u];
  While (x <> nil) And (x^.k <> v) Do x := x^.next;
  If x <> nil Then x^.c := c
    Else Begin
      New(x);
      x^.k := v; x^.c := c; x^.f := 0;
      x^.next := Nbs[u]; Nbs[u] := x;
      New(x^.g);
      x^.g^.k := u; x^.g^.c := 0; x^.g^.f := 0; x^.g^.g := x;
      x^.g^.next := Nbs[v]; Nbs[v] := x^.g
    End {else}
End; {insert}

Procedure Init; {输入数据，构造N网}
Var
  u, c           : Integer;
Begin
  Assign(F, 'INPUT.DAT'); {输入文件名串与文件变量连接}
  Reset(F); {文件读准备}
  Readln(F, N, M); {读入实验数和仪器数}
  For u := 1 to N + M + 2 Do Nbs[u] := nil; {N网的邻接表初始化为空}
  profit := 0;
  For u := 1 to N Do Begin{依次处理每一个实验}
    Read(F, c);{读入PU}
    Insert(M + u, N + M + 2, c); {当前实验与汇点T间连一条容量为PU的有向弧}
    profit := profit + c; {累计全部收入}
```

```
    While Not Eoln(F) Do Begin
      Read(F, c); {读入当前实验所需的仪器}
      Insert(c, M + u, Maxint){该仪器与当前实验间连一条容量为∞的有向弧}
    End; {while}
    Readln(F)
  End; {for}
  For u := 1 to M Do Begin{依次处理每一个仪器}
    Read(F, c); {读入运送当前仪器的费用 CU}
    Insert(N + M + 1, u, c){源点与当前仪器间连一条容量为 CU 的有向弧}
  End; {for}
  Close(F)
End; {init}

Procedure Ford(stt, trm : Integer);
{输入源点和汇点序号,通过求网的最大流得出其最小截集}
Var
  Q              : Array [1..Maxn] of 1..Maxn;
  op, cl, u, d   : Integer;
  x              : Link;
Begin
  tot := 0;
  Repeat
    FillChar(Pth, Sizeof(Pth), 0); {可改进路置空}
    Pth[stt].d := Maxint; Pth[stt].p := stt; {给源点标号}
    cl := 0; op := 1; Q[op] := stt; {源点进入队列}
    While (cl < op) And (Pth[trm].p = 0) Do Begin
    {若Q队列未空且汇点未标号,则循环}
      Inc(cl); {移出Q的队首结点}
      u := Q[cl];
      x := Nbs[u];
      While x <> nil Do Begin{分析队首结点出发的每一条弧}
        d := x^.c - x^.f;
        If (Pth[x^.k].p = 0) And (d > 0) Then Begin
       {若相邻结点未标号且该弧的流量可改进}
          Inc(op); Q[op] := x^.k; {则该结点进入队列和可改进路并给该结点标号}
          Pth[x^.k].p := u; Pth[x^.k].w := x;
          If Pth[u].d < d Then Pth[x^.k].d := Pth[u].d
                          Else Pth[x^.k].d := d;
```

```
        End ; {then}
          x := x^.next{分析下一个相邻结点}
        End {while}
      End ; {while}
      If Pth[trm].p <> 0 Then Begin{若汇点已标号}
        u := trm; d := Pth[u].d; {则从汇点出发,修正可改进路的流量}
        Repeat
          Pth[u].w^.f := Pth[u].w^.f + d;
          Pth[u].w^.g^.f := - Pth[u].w^.f;
          u := Pth[u].p
        Until u = stt;
        tot := tot + d{累计截量C(S, T)}
      End{then}
    Until Pth[trm].p = 0 {直至不存在可改进路为止}
End ; {ford}

Procedure Answer;
Var
  i                : Integer;
Begin
  Write('EXPERIMENT :');
{若可改进路上 E_i, L_k 的可改进量为0,则确定进行 E_i 实验并运送 1_k 仪器,即(E_i, I_k)∈
(S, T)}
  For i := 1 to N Do
    If Pth[i + M].d = 0 Then Write(' E', i);
  Writeln;
  Write('INSTRUMENT :');
  For i := 1 to M Do
    If Pth[i].d = 0 Then Write(' I', i);
  Writeln;
  Writeln('Profit = ', profit - tot) {输出最大净收益}
End ; {answer}

Begin
  Init ; {输入数据, 构造N网}
  Ford(N + M + 1, N + M + 2); {通过求N网的最大流得出最小截集}
  Answer{输出实验方案和最大净收益}
  End . {main}
```

【例 6】餐厅问题:

一个餐厅在相继的 n 天里，第 i 天需要 r_i 块餐巾 (i=1 ，2 ，…，n)。餐厅可以购买新的，每块餐巾 p 分，或者把旧餐巾送到快洗部，洗一块需 m 天，其费用为 f 分，或者送到慢洗部，洗一块需 n 天 (n>m), 其费用为 s<f 分。每天结束时，餐厅必须决定多少块脏的餐巾送到快洗部，多少块送到慢洗部，以及多少保存起来延期送洗。 但是洗好的餐巾和购买的新餐巾之和，要满足第 i 天的需求量，并使总的花费最小。

算法分析:

为了安排好 N 天里餐巾使用计划，我们将一天看作两个顶点: 即第 i 天拆成顶点 X_i 和 Y_i。X_i 的序号为 i，Y_i 的序号为 i+N+1 (1 ≤ i ≤ N)。另外增加三顶点: 附加顶点 K, 序号为 N+1; 源点 S, 序号为 N*2+2; 汇点 T, 序号为 N*2+3;

按下述规律连接这 N*2+3 个顶点，构造出一个网络 D。D 网中每条弧的容量表示餐中需求量; 单位费用表示购买或送洗每条餐巾的费用; 流量表示餐巾的使用计算:

1. 源点 S 向每个 X_i 引出一条有向弧 (S, X_i), 容量为第 i 天的餐巾需求量 r_i, 费用为 0;

2. 附加顶点 K 向每个 Y_i 引出一条有向弧 (K,Y_i), 容量为∞，费用为餐巾单位价 P, 表示第 i 天允许买餐巾;

3. 每个 X_i 向 X_{i+1} (包括 X_n 向 K) 引出一条有向弧 (X_i , X_{i+1}), 容量为∞ 费用为 0, 表示第 i 天允许保存旧餐巾，延期送洗;

4. 若第 i 天快洗的旧餐巾能在最后一天使用 (i+m ≤ N) 则 X_i 向 Y_{i+m} 引出一条有向弧(X_i , Y_{i+m}) 容量为∞，费用为快洗费用 f, 表示第 i 天允许快洗旧餐巾;

5. 若第 i 天慢洗的旧餐巾能在最后一天使用 (i+n ≤ N) 则 X_i 向 Y_{i+n} 引出一条有向弧(X_i , Y_{i+n}) 容量为 ∞，费用为慢洗费用 s, 表示第 i 天允许慢洗旧餐巾;

6. 每个 Y_i 向汇点 T 引出一条有向弧 (Y_i , T), 容量为 r_i, 费用为 0; (1 ≤ i ≤ N)

网中所有弧的流量设为 0, 表示最初的餐巾使用计划，我们从这个可行流出发，求网的最大费用最大流。

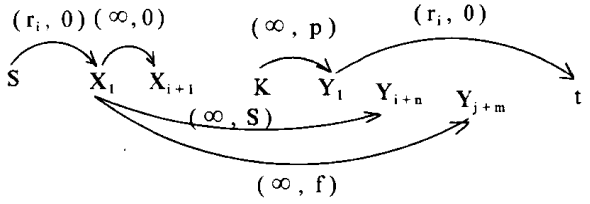


图 7.4-6

由于 X_i 与 X_{i+1}、X_i 与 Y_j 之间的弧的容量为∞，而源点 S 通向每个 X_i 的弧的容量和每个 Y_j 通向汇点 T 的弧的容量都限制在 r_i，因此最终求出的最大流使得 S 出发的所有弧满载，即

$F = \sum_{r=1}^{N} r_i$，满足了每天的需求量，同时使总的花费最小。

我们可以从弧流量中找出 N 天里使用餐巾的计划:

对于第 i (1≤ i ≤ N) 天来说，(K , Y_i) 的弧流量表示当天买餐巾的数量; (X_i , X_{i+1})

的弧流量表示当天保存的餐巾数量；若 $i+m \leqslant N$，则 (X_i, Y_{i+m}) 的弧流量表示当天快洗的餐巾数量；若 $i+n \leqslant N$，则 (X_i, Y_{i+n}) 的弧流量表示当天慢洗的餐巾数量。

下面给出程序题解：

```
Program Resturant;

Const
  Maxn              =  100; {最多顶点数}

Type
  Link              = ^Nbour; {N网邻接表的指针类型}
  Nbour             =  Record{邻接表的结点类型}
                         k          :  1..Maxn; {顶点序号}
                         f, c, w    :  Integer; {流量,容量,费用}
                         next, g    :  Link {前向弧指针,后向弧指针}
                       End;
  WType             =  Record {可改进路的结点类型}
                         p, d, ct   :  Integer;
                  {标号(前趋指针,当前可改进量的费用和可改进路的费用和}
                         w          :  Link {弧指针}
                       End;

Var
  D, p, f, s, m, n,
{天数,买餐巾的费用,快洗费用,慢洗费用,快洗天数,慢洗天数}
  stt, trm          :  Integer; {源点,汇点}
  Fl                :  Text; {文件变量}
  Nbs               :  Array [1..Maxn] of Link; {N网的邻接表}
  Pth               :  Array [1..Maxn] of WType; {可改进路}
  R                 :  Array [1..Maxn] of Integer;
  {每天所需餐巾数的序列}

Procedure Insert(u, v, c, w : Integer);
Var
{在U对应的邻接表中插入前向弧(U, V),容量为C,流量为0,费用为W;在V对应的
邻接表中插入后向弧(U, V),容量为0,流量为0,费用为-W}
  x                 :  Link;
Begin
  New(x);
```

```
  x^.k := v; x^.c := c; x^.f := 0; x^.w := w;
  x^.next := Nbs[u]; Nbs[u] := x;
  New(x^.g);
  x^.g^.k := u; x^.g^.c := 0; x^.g^.f := 0;
  x^.g^.w := - w; x^.g^.g := x;
  x^.g^.next := Nbs[v]; Nbs[v] := x^.g
End; {insert}

Procedure Init;
Var
  u                : Integer;
Begin
  Assign(Fl, 'INPUT.DAT');
  Reset(Fl);
  Readln(Fl, D, p, m, f, n, s);
  {读入天数,买餐巾的费用,快洗天数,快洗费用,慢洗天数,慢洗费用}

  For u := 1 to D Do Readln(Fl, R[u]); {读入每天所需的餐巾数}
  Close(Fl);
  stt := D * 2 + 2; trm := stt + 1; {设置源点和汇点}
  For u := 1 to trm Do Nbs[u] := nil; {网的邻接表初始化为空}
  For u := 1 to D Do Begin
    Insert(stt, u, R[u], 0); {源点向 X_U 引出一条弧,容量为 R_U,费用为 0}
    Insert(D + 1, D + 1 + u, Maxint, p);
    {K 向 Y_U 引出一条弧,容量为∞,费用为 P}
    Insert(u, u + 1, Maxint, 0);
    {X_U 向 X_U+1 引出一条弧,容量为∞,费用为 0}
    If u + m <= D Then Insert(u, D + u + m + 1, Maxint, f);
    {若允许快洗,则 X_U 向 Y_U+M 引出一条弧,容量为∞,费用为 F}
    If u + n <= D Then Insert(u, D + u + n + 1, Maxint, s);
    {若允许慢洗,则 X_U 向 Y_U+N 引出一条弧,容量为∞,费用为 S}
    Insert(D + u + 1, trm, R[u], 0)
    {Y_U 向汇点引出一条弧,容量为 R_U,费用为 0}
  End {for}
End; {init}

Function Solve(stt, trm : Integer): Integer;
Var
```

```
  u, d, cost      :  Integer;
  x               :  Link;
  break           :  Boolean;
Begin
  cost := 0;
  Repeat
    For u := 1 to trm  Do Begin {可改进路撤空}
      Pth[u].p := 0; Pth[u].d := Maxint; Pth[u].ct := Maxint
    End;
    Pth[stt].p := stt; Pth[stt].ct := 0; {源点进入可改进路}
    Repeat
      Break := False;
      For u := 1 to trm  Do Begin {分析每一个顶点}
        x := Nbs[u]; {分析U顶点出发的每一条弧}
        While x <> nil Do Begin
          d := x^.c - x^.f;
          If (d > 0) And (Pth[u].ct < Maxint) And
{若该弧的流量可改进且源点至U点有通路且源点经U至相邻点的费用和目前最小}
             (Pth[u].ct+x^.w < Pth[x^.k].ct) Then Begin
            Break := True;
            Pth[x^.k].ct := Pth[u].ct + x^.w;
{修正相邻点的费用和,相邻点进入可改进路并为之标号}
            Pth[x^.k].w := x; Pth[x^.k].p := u;
            If d < Pth[u].d Then Pth[x^.k].d := d
                            Else Pth[x^.k].d := Pth[u].d
          End; {then}
          x := x^.next {分析U顶点出发的下一条弧}
        End {while}
      End; {for}
    Until Not Break; {直到最短路无法延伸为止}
    If Pth[trm].p <> 0 Then Begin
  {若汇点已标号,则从汇点出发倒向修正可改进路的流量,并累计总的输送费用}
      u := trm; d := Pth[u].d;
      Repeat
        cost := cost + d * Pth[u].w^.w;
        Pth[u].w^.f := Pth[u].w^.f + d;
        Pth[u].w^.g^.f := - Pth[u].w^.f;
        u := Pth[u].p
```

```
      Until u = stt{直到倒推至源点}
    End{then}
  Until Pth[trm].p = 0;{直至汇点已标号}
  Solve := cost{返回总的运输费用}
End;

Procedure Answer;
Var
  i              : Integer;
  x              : Link;
  list           : Array[1..Maxn, 1..4] of Integer;
Begin
  FillChar(List, Sizeof(List), 0);
  For i := 1 to D Do Begin
    x := Nbs[i];
    While x <> nil Do Begin
      If (i + m <= D) And (x^.k = D + i + m + 1) Then List[i, 2] := x^.f;
{若第i到允许快洗且(Xi, Yi+m)存在,则List[i, 2]← F(Xi, Yi+m)}
      If (i + n <= D) And (x^.k = D + i + n + 1) Then List[i, 3] := x^.f;
{若第i到允许慢洗且(Xi, Yi+n)存在,则List[i, 3]← F(Xi, Yi+n)}
      If x^.k = i + 1 Then List[i, 4] := x^.f;
{若(Xi, Xi+1)存在,则List[i, 4]← F(Xi, Xi+1)}
      x := x^.next{分析Xi出发的下一条弧}
    End{while}
  End;{for}
  x := Nbs[D + 1]^.next;{分析K出发的每一条弧(K, YU)}
  While x <> nil Do Begin
    List[x^.k - D - 1, 1] := x^.f;{List[u, 1]← F(K, Yu);}
    x := x^.next;{分析K出发的下一条弧}
  End;{while}
  Writeln('DAY':5, 'BUY':5, 'FAST':5, 'SLOW':5, 'SAVE':5);
{输出每天的餐巾计划}
  For i := 1 to D Do
    Writeln(i:5, List[i, 1]:5, List[i, 2]:5, List[i, 3]:5, List[i, 4]:5);
End;{answer}

Begin
  Init;{输入数据,构造附加网}
 Writeln('Total cost = ', Solve(stt, trm));{计算和输出总的花费}
  Answer{输出每天的餐巾计划}
End.{main}
```

第八章　动态程序设计

我们曾在第一章基础算法中分别介绍过贪心法（§1.2 章节）和分治算法（§1.4 章节）。它们都是将问题实例归纳为更小的、相似的子问题，并通过求解子问题产生一个全局最优解。其中贪心法的当前选择可能要依赖已经作出的所有选择，但不依赖于有待于做出的选择和子问题。因此贪心法自顶向下，一步一步地作出贪心选择；而分治法中的各个子问题是独立的（即不包含公共的子子问题），因此一旦递归地求出各子问题的解后，便可自下而上地将子问题的解合并成问题的解。但不足的是，如果当前选择可能要依赖子问题的解时，则难以通过局部的贪心策略达到全局最优解；如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题。

针对上述问题，我们提出了动态程序设计的方法。该方法主要应用于最优化问题，这类问题会有多种可能的解，每个解都有一个值，而动态程序设计找出其中最优（最大或最小）值的解。若存在若干个取最优值的解的话，它只取其中的一个。在求解过程中，该方法也是通过求解局部子问题的解达到全局最优解，但与分治法和贪心法不同的是，动态程序设计允许这些子问题不独立，（亦即各子问题可包含公共的子子问题）也允许其通过自身子问题的解作出选择，该方法对每一个子问题只解一次，并将结果保存起来，避免每次碰到时都要重复计算。

动态程序设计的过程可分为四个步骤

① 刻划最优解的结构；

② 递归定义最优解的值；

③ 按自底向上的方式或自顶向下记忆化的方式计算最优解的值并纪录下求解的途径；

④ 按照求解途径给出最优解的形成过程；

下面各章节将结合一些实例来分析动态程序设计的思想和方法。

§8.1　矩阵链乘法

【例 1】 矩阵链的最佳乘法

给定 N 个矩阵构成一个链 $A_1, A_2, \cdots, A_n$，矩阵 A_i 的维数为 $P_{i-1} * P_i$ $(1 \leqslant i \leqslant n)$，以一种最小化标量乘法次数的方式进行完全括号化。

算法分析：

对矩阵加括号的顺序对求乘积运算的代价有很大影响。先来看看两个矩阵 A 和 B 相乘的代价。设 $A=(a_{ij})$ 是一个 $m \times s$ 矩阵，$B=(b_{ij})$ 是一个 $s \times n$ 矩阵，那末矩阵 A 与矩阵 B 的乘积是一个 $m \times n$ 矩阵。

$$C=(C_{ij}) = (a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{is}b_{sj})$$

$$= (\sum_{k=1}^{s} a_{ik}b_{kj})$$

$$=AB \qquad (1 \leqslant i \leqslant m,\ 1 \leqslant j \leqslant n)$$

显然标量乘法的次数为 m*s*n。若 A 的列数不等于 B 的行数，则不可相乘。两个矩阵相乘的算法如下：

```
proceudre matrix__multiply ( A , B );
begin
  if 矩阵 A 的列数 <> 矩阵 B 的行数
    then begin 输出无解信息; exit end
    else for i:=1 to 矩阵 A 的行数 do
         for j:=1 to 矩阵 B 的列数 do begin
           c[i, j]← 0;
           for k:=1 to 矩阵 A 的列数 do c[i,j]← c[i,j]+A[i, k]*B[k, j];
         end ; {for}
  for i:=1 to 矩阵 A 的行数 do
  for j:=1 to 矩阵 B 的列数 do 输出 c[i, j];
end ; { matrix__multiply }
```

矩阵乘法适合结合律，因此无论对一个矩阵链怎样加括号都产生出相同的结果，只是运算代价不同而已。为说明由不同的括号化顺序所带来的矩阵乘积的不同代价，考虑三个矩阵链 $<A_1, A_2, A_3>$ 的乘法问题。其中

A_1 的维数为 10 * 100

A_2 的维数为 100 * 5

A_3 的维数为 5 * 50

如果按 $((A_1, A_2)A_3)$ 规定的次序做乘法，则为求 $A_1 * A_2$ 要做 10*100*5=5000 次的标量乘法运算，再乘上 A3 时还要做 10*5*5 的 2500 次运算，总共 7500 次标量乘法运算；

如果按 $(A_1(, A_2, A_3))$ 次序计算，则为求 $A_2 * A_3$ 要做 100*5*50=25000 次标量乘法运算，再乘上 A_1 时还要做 10*100*50=50000 次运算，总共 75000 次运算。显然 $((A_1, A_2)A_3)$ 运算顺序比 $(A_1, (A_2, A_3))$ 就要快到 10 倍。

计算最优化括号的一种最简单的方法是枚举所有可能的括号化方案并计算出每一种方案的标量乘法次数，从中找出运算次数最少的括号方案。显然，这种靠穷尽所有可能的括号方案是不会产生出一种有效算法的，原因是完全括号化的方案太多：

设 P(n)← n 个矩阵可能的括号化方案数。n 个矩阵可加入 n−1 对括号。我们将 n 个矩阵从第 k 和第 (k+1) 个处隔开，然后对两个子序列分别加括号，则可得出递归式

$$P[n] = \begin{cases} 1 & n=1 \\ \sum_{K=1}^{n-1} P(k)P(n-k) & n \geqslant 2 \end{cases}$$

这个递归式的解为 catalan 序数

$$P[n] = c(2n-2, n-1)/n$$

例如(A_1, A_2, A_3, A_4)就有 5 种括号化方案

$(A_1(A_2(A_3, A_4)))$

$(A_1((A_2, A_3)A_4))$

$((A_1, A_2)(A_3, A_4))$

$((A_1(A_2, A_3))A_4)$

$(((A_1, A_2)A_3)A_4)$

下面我们来看看如何通过动态程序设计。求矩阵链乘法中的最优化括号方案。 按动态程序设计的四个求解步骤分析:

一、最优括号化的结构

用动态程序设计方法求解问题的第一步是刻划一个最优解的结构。 如果我们能分析出一个问题的最优解包含了子问题的最优解,则可以说该问题具备了最优子结构。此时,动态程序可能就是一个合适的候选方法。(候选方法也包括贪心法和分治法)

设 $A_{i..j}$——对乘积 $A_i A_{i+1}, \cdots, A_j$ 求值的结果

对乘积 $A_1, A_2, \cdots, A_n$ 的一种最优括号化将乘积在 A_k 与 A_{k+1} 间隔开($1 \leqslant k \leqslant n-1$)。这就是说,对某个 k 值,我们先计算 $A_{1..k}$ 和 $A_{k+1..n}$ 的代价之积,再加上两者相乘的代价。

要注意的关键一点就是,$A_1, \cdots, A_n$ 最优括号化的"前缀"子链 $A_1..A_k$ 的括号化必须是 $A_1, \ldots, A_k$ 的最优化括号。为什么? 如果对 $A_1, \cdots, A_k$ 有一个代价更小的括号化,那未把它替换到 $A_1, \cdots, A_n$ 的最优括号化中就会产生另一种括号化,其代价小于最优代价,则出现矛盾。同样地,对后缀子链 $A_{k+1}A_{k+2}, \cdots, A_n$ 的括号化也要求是最优化的。通常,我们是通过反证法来验证最优子结构的合理性:先假设子问题有更好一个解,再证明这个假设与原的最优性的矛盾。

一个问题的最优子结构常常暗示了可应用动态程序设计方法的子问题空间。 典型的情况是,某一个特定问题可有几类"自然"的子问题。例如矩阵链乘法的子问题空间包含了输入链的所有子链。如果我们放弃对链的有序性要求,而选择由输入链中的任意矩阵序列构成的子问题空间,那未基于这样一个子问题空间的动态程序设计则要解许多不必要的问题,使得算法效率骤然下降。

找出合适的动态程序设计的子问题空间的一个好方法是通过对子问题的迭代来考察一个问题的最优子结构。例如,在看了矩阵问题的一个最优解的结构后,我们可以不断迭代以考察子问题,子子问题…… 直到两个矩阵相乘.我们发现,所有问题都被有序链 $<A_1, A_n>$ 所包含。这样,所有形如 $<A_i, A_{i+1}, \cdots, A_j>$ 的子链构成的集合就是一个自然和合理的子问题空间。

二、递归定义最优解的值

问题最优解的值的递归定义是依据各子问题的最优解。对矩阵链乘法问题,子问题就是确定 $A_i A_{i+1}, \cdots, A_j$ 括号化的最小代价问题。

设 m[i, j]——计算矩阵 $A_i \cdots j$ 所需的标量乘法次数的最小值。显然矩阵链乘法问题

的最优解的值为 m[1, n]。($1 \leqslant i \leqslant j \leqslant n$)

我们是这样来递归定义 m[i, j]的:

如果 i=j,则矩阵链只包含一个矩阵 $A_{i\cdots j}=A_i$,故无需任何标量乘法,当然有 m[i, i]=0(i=1…n);

如果 i<j,为计算 m[i, j],我们可利用步骤 1 中所刻划的最优解的结构。假设最优括号化将乘积 $A_iA_{i+1}\cdots A_kA_{k+1}\cdots A_j$ 从 A_k 与 A_{k+1} 处隔开($i \leqslant k \leqslant j-1$),则 m[i, j]就等于计算子链乘积 $A_{i\cdots k}$ 与 $A_{k+1\cdots j}$ 的代价之和加上这两个矩阵相乘的代价的最小值。因为计算 $A_{i\cdots k}A_{k+1\cdots j}$ 要化 $P_{i-1}P_kP_j$ 次标量乘法,因此有

$$m[i,j]=m[i,k]+m[k+1,j]+P_{i-1}P_kP_j$$

这个递归方程假设了我们已知 k 的值,但实际上我们并未确定其值,只知道 k 有 j−i 种可能值,即 k=i, i+1…j−1,而最优括号化必然要用到其中之一的 k 值。故我们可逐个检查这些值,以求出最优的一个。

综上分析,得出关于对乘积 $A_iA_{i+1}\cdots A_j$ 的括号化的最小代价的递归定义:

$$m[i,j]=\begin{cases}0 & i=j\\ \min\{m[i,k]+m[k+1,j]+P_{i-1}P_kP_j\} & i<j\\ i\leqslant k\leqslant j-1\end{cases}$$

各 m[i, j]值给出了子链乘积的最小代价。为了记录下 m[i, j]的求解过程,我们定义 S[i, j]为这样一个 k 的值:在 k 处隔开乘积 $A_iA_{i+1}A_kA_{k+1}\cdots A_j$ 后可得出一种最优括号化,即 s[i, j]为一个使得 $m[i,j]=m[i,k]+m[k+1,j]+P_{i-1}P_kP_jk$ 的值。

有了上述递归式,我们可以直接写出对应的程序。下述的 recursive_matrix_chain 函数输入矩阵链的维数序列 $P=\{P_0P_1\cdots P_n\}$(其中 $P_{i-1}P_i$ 为矩阵 A_i 的维数)和 i、j,返回 m[i, j],并计算出所有 $A_{i1\cdots j1}$ 的最小乘积代价 $m[i_1, j_1]$ 和对应的 $s[i_1, j_1]$($1 \leqslant i_1 \leqslant i$, $1 \leqslant j_1 \leqslant j$, $i_1 \leqslant j_1$)。

```
function recursive_matrix_chain (p, i, j) integer
begin
  if i = j then begin
     m[i, j] ← 0;
     recursive_matrix_chain ← 0;
   end; {then}
   m[i, j] ← ∞;
   for k:=i to j−1 do begin
    q ← recursive_matrix_chain (p, i, k)+recursive_matrix_chain (p, k+1, j)+P_{i-1}P_kP_j;
    if q<m[i, j] then begin
          m[i, j] ← q;
          s[i, j] ← k;
```

```
        end ; { then }
    end ; { for }
    recursive_matrix_chain ← m[i, j];
end ; { recursive_matrix_chain }
```

上述递归算法在求解过程中反复出现了同一个子问题，且对每次重复出现的子问题都要重新解一次。假如图 8.1 - 1 示出了由调用 recursive_matrix_chain(p,1,4) 所产生的递归树，每一个结点上标有参数 i, j 的值。请注意某些数对出现了多次。

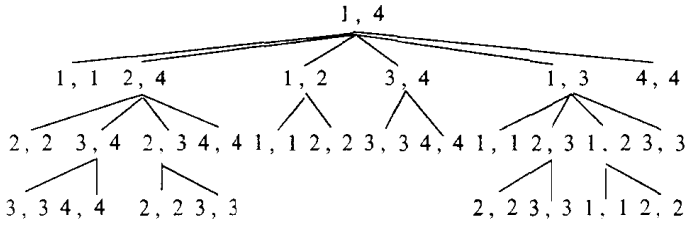


图 8.1 - 1

例如，表项 m[3, 4]被引用了 2 次：在计算 m[2, 4] 和 m[1, 4] 中分别被引用。如果 m[3, 4]每次都要被重新计算，则增加的运行时间相当多。下面，我们来看看动态程序设计方法是如何解决这个问题的。

三、按自上而下记忆化方式或自底向上的方式求最优解

一般地，如果一个解最优化问题的递归算法经常反复地解同样的子问题，而不是总在产生新的子问题时，我们说该最优化问题包含重迭子问题。这类问题不宜用分治法和贪心法求解，因为分治法递归的每一步要求产生相异的子问题，而贪心法也不允许对同一个子问题做重复选择。在这种情况下，采用动态程序设计的方法是很合适的，因为该方法对每一个子问题只解一次，然后把解放在一个表中，以便解同样的子问题时查阅，充分利用了重迭子问题。一般来说，解决重迭子问题的方式有二种。

1. 自上而下的记忆化方式

该方式的程序流程基本按照原问题的递归定义，不同的是，它专门设置了一张表，以记忆在求解过程中得出的所有子问题的解。一个记忆的递归算法为每个子问题的解在表中记录一个表项。初始时，每个表项都包含一个特殊值，以示该表项的解有待填入。例如乘法链问题中

$$m[i, j]=\infty;\ s[i, j]=\infty;\ (1 \leqslant i, j \leqslant n, i \leqslant j)$$

当在递归算法的执行中第一次遇到一个子问题时 (即 $m[i, j]=\infty$)，计算其解并填入表中，以后每遇到该子问题，只要查看表中先前填入的值即可。

下面，我们按照自上而下的记忆化方式，写出求最优括号化代价的递归算法。函数 memorized_matrix_chain(p) 输入维数序列 P, 返回 m[1,n]。m 表和 s 表应设为全局变量，使得函数执行后，传出所有子问题的解 m[i, j] 和 s[i, j] (i=1 … n, j=i … n).

```
function memorized_matrix_chain(p): integer;
    function lookup_chain ( p , i , j ): integer;
```

```
    begin
     if m[i, j]< ∞  then lookup__chain ← m[i, j];
      if i=j then m[i, j]← 0
            else for k:=1 to j−1 do begin
        q ← lookup__chain(p, i, k)+lookup__chain(p, k+1, j)+P_{i-1}P_kP_j;
                if q<m[i, j] then begin
                   m[i, j]← q;
                   s[i, j]← k;
                   end; {then}
                end; {for}
     lookup__chain ← m[i, j];
  end; {lookup__chain}

begin
  for i:=1 to n do
   for j:=1 to n do m[i, j]← ∞;
  memorized__matrix__chain ← lookup__chain(p, 1, n);
end; {memorized__matrix__chain}
```

显然这种自顶向下记忆化的算法效率比重复解重迭子问题的 recursive__matrix__chain 算法要强一些。另外，程序中可以方便地加入哪些子问题需要解的判别的语句，有着只解那些肯定要解的子问题的优点。

2. 自底向上的方式

自顶向下的记忆化方式只不过是动态程序设计的一种变形。动态程序设计的执行方式是自底向上，所有子问题计算一次，充分利用重迭子问题。另外，它无需递归代价，且维护记忆表的开销也要小些，因此其效率好于自顶向下的记忆法。

我们知道，计算 $A_{i\cdots j}$ 的最优代价 m[i, j] 仅依赖于计算包含少于 j−i+1 个矩阵相乘的代价，也就是说，对 k=i, i+1, …, j−1, 而矩阵 $A_{i\cdots k}$ 是 k−i+1(<j−i+1) 个矩阵的乘积，而矩阵 $A_{k+1\cdots j}$ 是 j−k(<j−i+1) 个矩阵的乘积，因此填 m 表和 s 表的方式与解决按长度递增的矩阵链上的括号化问题相对应:

最初设定长度为 1 的所有矩阵链的最优代价 m[i, i]=1(1≤i≤n)。然后依次考虑长度为 2, 3, …, n 的所有矩阵链的最优代价。在考虑当前长度为 L 的矩阵链时，分别求出 m[1, L], m[2, L+1], …, m[n−L+1, n] 和 s[1, L], s[2, L+1], …, s[n−L+1, n]. 每填一个 m[i, j] 和 s[i, j](j−i+1=L, 1≤i≤n−l+1, L≤j≤n) 时，充分利用所有重迭子问题的解: m[i, k] 和 m[k+1, j](i≤k≤j−1)。

下面是按照自底向上方式计算最优括号化方式的程序流程。过程 matrix__chain__order (p) 输入维数序列 P. m 表和 s 表设为全局变量。过程执行后所有子问题的解通过 m 表和 s 表传给主程序。

```
procedure matrix__chain__order(p);
```

```
begin
    for i:=1 to n do m[i, i]← 0;
    for l:=2 to n do
        for i:=1 to n-l+1 do begin
            j← i+l-1;
            m[i, j]← ∞;
            for k:=i to j-1 do begin
                q← m[i, k]+m[k+1, j]+P_{i-1}P_kP_j;
                if q<m[i, j]then begin m[i, j]← q; s[i, k]← k; end;
            end; {for}
        end; {for}
end; {matrix__chain__order}
```

四、构造最优解的方法

动态程序设计的第三步骤确定了计算矩阵链乘积所需的最少标量乘法次数，但它没有直接说明如何对这些矩阵相乘。动态程序设计的第四步骤将按照上一步骤记录下的求解途径，构造最优解的形成过程。

在矩阵链乘法问题中，我们利用表S[1..n,1..n]来确定矩阵相乘的最佳方式。每一表项S[i, j]记录了对乘积$A_iA_{i+1}\cdots A_j$在A_kA_{k+1}之间划分以取得最优括号化时的K值。由此可知，按最佳方式计算$A_{1\cdots n}$的最佳次序为$A_{1\cdots S.[1,n]}$和$AS.[1,n]+1\cdots n$。该式中在前面做的矩阵乘法可递归地进行，因为s[1..s[1..n]]确定了计算$A_{1\cdots S.[1\cdots n]}$中最后的矩阵乘法，S[S[1, n]+1, n]则确定了计算$A_{s[1,n]+1\cdots n}$中最后的矩阵运算。

下面的递归过程matrix__chain__multiply(a, s, i, j, x)中有四个参数，其中三个输入参数(值参)

(1)矩阵$A=<A_1\cdots A_n>$;

(2)由第三步骤计算出的S表;

(3)下标i, j;

一个输出参数(变量参数)X,表示矩阵$A_{i\cdots j}$;

该过程依次输出矩阵链乘积$A_{i\cdots j}$的形成过程。初始调用为matrix__chain__multiply(a, s, 1, n, x)。

```
procedure matrix__chain__multiply(a, s, i, j, x);
begin
    if j>i then begin
        matrix__chain__multiply(a, s, i, s[i, j], x);
        matrix__chain__multiply(a, s, s[i, j]+1,j, y);
        matrix__mutiply(x, y); {计算和输出c=xy}
    end{then}
    else x← Ai;
```

end;{matrix__chain__multiply}

从矩阵乘法问题可以引伸出一个同类的问题——对凸多边形做最优三角剖分。虽然从表面上看这个几何问题与矩阵乘法毫不相干，实际上两者却存在着一一对应关系。

【例 2】最优多边形三角剖分

由 n 个顶点组成一个凸 n 边形。通过不相交于多边形内 P 的对角线，可把凸 n 边形拆分成 n－2 个三角形，每一种三角剖分都有 n－3 根弦。例如图 8.1-2 为七边形的二种三角剖分

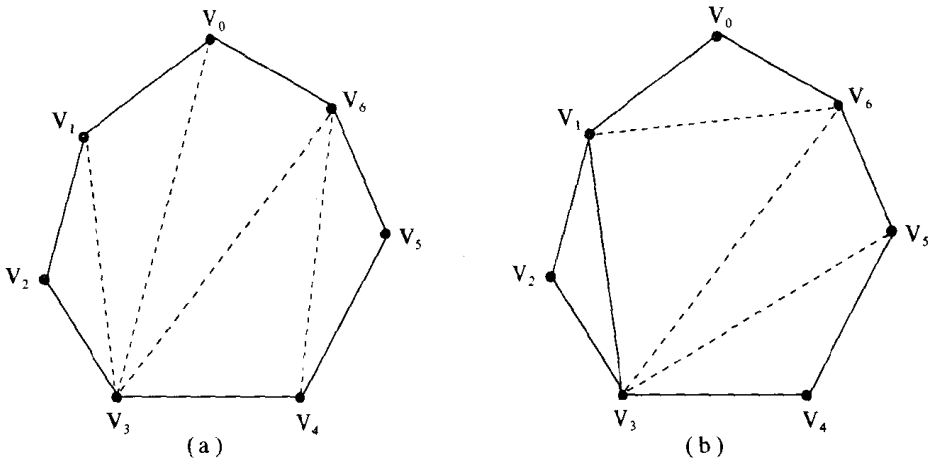


图 8.1-2

已知一个凸 n 边形 $P=<V_0, V_1, \cdots, V_{n-1}>$ 和一个定义在由边和 P 的弦构成三角形上的权函数 W，要求得出一个三角剖分，使得所产生的所有三角形的权之和最小。

算法分析：

首先我们来分析一下矩阵链括号化与多边形的三角剖分之间的对应关系：一个矩阵链乘积的括号化可用一棵二叉分析树表示。例如 $((A_1(A_2A_3))(A_4(A_5A_6)))$

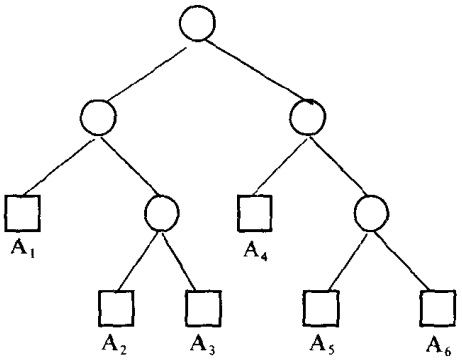


图 8.1-3

树中每个叶结点为矩阵链中的一个矩阵。如果某棵子树的根有左子树表示一个子矩阵链 E_L，又有右子树表示一个子矩阵链 E_R，则该子树本身就表示两个子矩阵链的乘积 (E_L, E_R)。类似的分析树和含 n 个矩阵的完全括号化的矩阵链之间有着一一对应关系。

一个凸多边形 $<V_0, V_1, \cdots V_{n-1}>$ 的一种三角剖分也可由一棵二叉分析树表示。例

如与图 8.1-3 中的二叉分析树对应的凸多边形的三角剖分如下:

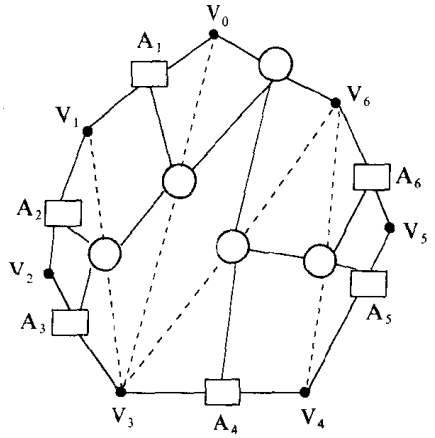


图 8.1-4

在凸多边形的边中，一条边 V_0V_6 对应树的根，其它边对应树的叶结点；三角剖分的弦对应树的内结点。V_0V_6 所在的三角形$\triangle V_0V_3V_6$把原多边形分成了三个部分:

1. $\triangle V_0V_3V_6$本身;

2. 子多边形 $<V_0, V_1, V_2, V_3>$;

3. 子多边形 $<V_3, V_4, V_5, V_6>$;

两个子多边形除了由弦 V_0V_3 和 V_3V_6 表示根之外，其它边完全由原多边形的边构成。子多边形 $<V_0, \cdots, V_3>$ 又递归地包含分析树根的左子树，子多边形 $<V_3, \cdots, V_6>$ 包含右子树。

因为 n 个矩阵乘积的一个完全括号化与具有 n 个叶结点的二叉分析树对应，所以它又与包含 (n+1) 个顶点的多边形的一个三角剖分相对应。图 8.1-3 和图 8.1-4 说明了这种对应关系: 乘积 $A_1A_2\cdots A_n$ 中的每个 A_i 与有 (n+1) 个顶点的多边形的一条边 $V_{i-1}V_i$ 相对应; 弦 $V_iV_j(i<j)$ 与计算矩阵乘积过程中的某个矩阵 $A_{i+1\cdots j}$ 相对应。

现在，我们来定义最优三角剖分的权的递归解，就像我们递归定义计算矩阵链乘积 $A_iA_i+1\cdots A_j$ 的最小代价 m[i, j] 一样:

设 t[i, j]——子多边形 $<V_{i-1}, V_i, \cdots, V_j>$ 的一个最优三角剖分的权($1\leqslant i\leqslant j\leqslant n$)。显然，凸 n 边形的一个最优三角剖分的权由 t[1, n] 给出。

为了方便起见，我们认为含两个顶点的退化多边形 $<V_{i-1}, V_i>$ 的权为 0, 即 t[i, i]=0 ($i=1\cdots n$)。当 j>i 时，有至少包含三个顶点的多边形 $<V_{i-1}, V_i, \cdots, V_j>$。我们希望对所有顶点 $V_K(k=i, i+1, \cdots, j-1)$ $\triangle V_{i-1}V_kV_j$ 的权与子多边形 $<V_{i-1}, V_i, \cdots, V_k>$ 和 $<V_k, V_{k+1}, \cdots, V_j>$ 的最优三角剖分的权之和最小。由此得出递归式为

$$t[i,j]=\begin{cases}0 & i=j\\ \min\{t[i,k]+t[k+1,j]+w(\triangle V_{i-1}V_kV_j)\} & i<j\\ (i\leqslant k\leqslant j-1)\end{cases}$$

将上述递归式与计算 $A_iA_{i+1}\cdots A_j$ 所需的最少标量乘法次数 m[i, j]的递归式相比较，除了权函数 W 和记忆表的名称稍有不同外，其它都相同。所以，我们只要将矩阵的维数序列 $P=<P_0, P_1, \cdots, P_n>$ 替换成顶点序列 $V=<V_0, V_1, \cdots, V_n>$，将 m[i, j] 定义中对 P 的引用改为对 V 的引用，则计算矩阵乘积最优括号化的代价的算法就可用来计算最优三角剖分的权了。 至于记忆表的名称是 m 还是 t，则无关紧要。

§8.2 最长公共子序列

【例 1】最长公共子序列

一个给定序列的某个子序列，即给定的序列再去掉几个元素（可能一个也不去掉）。形式化地，给定序列 $X=<X_1, X_2, \cdots, X_m>$ 和另一序列 $z=<z_1, z_2, \cdots, z_K>$，如果存在 X 的一个严格递增的下标序列 $<i_1, i_2, \cdots, i_K>$，使得对所有的 J=1, 2,…k, 有 $X_{ij}=Z_j$ 则 Z 是 X 的子序列。例如 Z=<B, C, D, B> 就是 X=<A, B, C, B, D, A, B> 的一个子序列，相应的下标序列为 <2, 3, 5, 7>

给定三个序列 X, Y 和 Z, 如果 Z 既是 X 的一个子序列又是 Y 的一个子序列，则称 Z 是 X 和 Y 的公共子序列。例如 X=<A, B, C, B, D, A, B>, Y=<B, D, C, A, B, A>，则序列 <B, C, A> 即为 X 和 Y 的一个公共子序列，但不是 X 和 Y 的最长公共子序列（LCS），因为还有比它更长的公共子序列 <B, C, B, A>。<B, C, B, A> 是 X 和 Y 的一个 LCS，序列 <B, D, A, B>也是。

现给定两个序列 $X=<X_1, X_2, \cdots, X_m>$ 和 $Y=<Y_1, Y_2, \cdots, Y_n>$，要求出 X 和 Y 的 LCS。

算法分析:

求解 LCS 的一个最简单的方法是穷举 X 的所有子序列，一一检查其是否为 Y 的子序列，并随时记录下所发现的最长子序列。显然这种算法对长序列来说是不实际的。因为一个长度为 m 的序列有 2^m 种子序列，这一算法需要指数时间代价。

下面，我们按照动态程序设计的方法对 LCS 问题进行分析。

一、刻划 LCS 问题的最优解的结构

首先，我们给出一个序列“前缀”的概念:

给定一个序列 $X=<X_1, X_2, \cdots, X_m>$, 对 i=0, 1, …, m, 定义 X 的第 i 个前缀为 $X_i'=<X_1, X_2, \cdots, X_i>$。

例如，如果 X=<A, B, C, B, D, A, B>, 则 $X_4'=<A, B, C, B>$。而 X_0' 则为空序列。

两个输入序列 X 和 Y 的所有前缀组成了 LCS 问题的子问题空间。由此我们发现 LCS 问题的最优子结构的三个性质:

设 $X=<X_1, \cdots, X_m>$ 和 $Y=<Y_1, \cdots, Y_n>$ 为两个输入序列，并设 X 和 Y 的 LCS 为 $Z=<Z_1, \cdots, Z_k>$

性质 1: 如果 $X_m=Y_n$, 则 $Z_k=X_m=Y_n$ 且 Z_{k-1}' 是 X_{m-1}' 和 Y_{n-1}' 的一个 LCS。

证: 我们用反证法来验证这一最优子结构的合理性。假设 $Z_k \neq X_m$。我们将 $X_m=Y_n$ 拼接到 Z 上，得出 X 和 Y 的一个长度为 K+1 的公共子序列，这与 Z 是 X 和 Y 的最长公共子序列的假设相矛盾。因此 $Z_k=X_m=Y_n$。此时前缀 Z_{k-1}' 为 X_{m-1}' 和 Y_{n-1}' 的长度为

(K−1)的公共子序列。 为了用反证法证实它是一个 LCS，假设 X'_{m-1} 和 Y'_{n-1} 有一个长度大于(K−1)的公共子序列 W。我们将 $X_m=Y_n$ 拼接到 W 上产生出 X 和 Y 的一个大于 K 的公共子序列，这又与 Z 为 LCS 的假设矛盾。由此证实 Z'_{k-1} 是 $X'_{m-1}Y'_{n-1}$ 的一个 LCS。

性质 2: 如果 $X_m \neq Y_n$，则 $Z_k \neq X_m$ 蕴含 Z 是 X'_{m-1} 和 Y 的一个 LCS。

证: 在 $Z_k \neq X_m$ 的情况下，假设 Z 不是 X'_{m-1} 和 Y 的一个 LCS，即 X'_{m-1} 和 Y 有一个长度大于 K 的公共子序列 W，则得出 W 是 X'_m 和 Y 的一个公共子序列，与 Z 为 X 和 Y 的 LCS 的假设矛盾。 由此证实 Z 是 X'_{m-1} 和 Y 的一个 LCS。

性质 3. 如果 $X_m \neq Y_n$，则 $Z_k \neq X_m$ 蕴含 Z 是 X 和 Y'_{n-1} 的一个 LCS。

性质 3 的证明与性质 2 的证明是对称的。

由此可见，序列 X 和序列 Y 的一个 LCS 也是包含了两个序列的前缀的一个 LCS，这就说明了 LCS 问题具备最优子结构性质。

二、递归定义 LCS 的长度值

由 LCS 问题的最优子结构的三个性质可知，在找 $X=<X_1,\cdots,X_m>$ 和 $Y=<Y_1,\cdots,Y_n>$ 的一个 LCS 时，可能要检查如下子问题:

1. 若 $X_m=Y_n$，我们就要找出 X'_{m-1} 和 Y'_{n-1} 的一个 LCS。将 $X_m=Y_n$ 拼接到这个 LCS 上就产生 X 和 Y 的一个 LCS;

2. 如果 Xm ≠ Yn, 我们就要解决两个问题:

找出 X'_{m-1} 和 Y 的一个 LCS 和找出 X 和 Y'_{n-1} 的一个 LCS。取两者中较长者作为 X 和 Y 的 LCS。由此我们很容易看出重迭子问题的性质: 为找出 X 和 Y 的一个 LCS，我们可能需要找 X 和 Y'_{n-1} 的一个 LCS 与 X'_{m-1} 和 Y 的一个 LCS，这两个子问题都包含着找 X'_{m-1} 和 Y'_{n-1} 的 LCS 的子子问题。还有许多其它子问题都具有公共子子问题。

像矩阵链乘法问题，需要对 LCS 长度值定义一个递归式:

设 c[i, j] 为序列 X 的前缀 X_i 和序列 Y 的前缀 Y_j 的 LCS 的长度。若 i=0 或者 j=0，亦即两个前缀之一的长度为 0，当前 LCS 的长度值也必为 0。由 LCS 问题的最优子结构可得递归式

$$c[i,j]=\begin{cases}0 & i=0\text{ 或 }j=0\\ c[i-1,j-1]+1 & i,j>0\text{ 且 }X_i=Y_j\\ \max(c[i,j-1],c[i-1,j]) & i,j>0\text{ 且 }X_i\neq Y_j\end{cases}$$

三、按自底向上方式计算 LCS 的长度

过程 LCS_Length 以两个序列 $X=<X_1,\cdots X_m>$ 和 $Y=<Y_1,\cdots Y_n>$ 作为输入，计算 X 的所有前缀与 Y 的所有前缀的 LCS 长度，共产生 m*n 个前缀对的 LCS 长度，分别填入 C 表中。C 表的行位置为 X 的元素序号，列位置为 Y 的元素序号，c[m, n] 为 X 和 Y 的一个 LCS 的长度。

最初设 c[1…m, 0] 为 0，即 X 的所有前缀与空序列的 LCS 长度为 0; c[0, 1…n] 为

0, 即空序列与 Y 的所有前缀的 LCS 长度为 0。由于计算 c[i, j]仅依赖于 c[i−1, j−1], c[i, j−1]、c[i−1, j]由此填 C 表的方式与解决按 X 前缀长度递增的 LCS 问题相对应:

按由左至右的顺序取出 X 序列的每一个元素。在取出 $X_i(1 \leqslant i \leqslant m)$ 时，依次比较 Y 中的每个元素 $Y_j(J=1 \cdots n)$:

若 $X_i=Y_j$, 则 X_i' 与 Y_j' 的 LCS 长度应为 X_{i-1}' 与 Y_{j-1}' 的 LCS 长度 +1, 即 c[i, j]: ← c[i−1, j−1]+1;

若 $X_i \neq Y_j$，则在 X_{i-1}' 与 Y_j 的 LCS 长度和 X_i' 与 Y_{j-1}' 的 LCS 长度之间取一个较大者，作为 X_i' 与 Y_j' 的 LCS 长度，即 c[i, j]← max(c[i, j−1], c[i−1, j])

按先行后列，由左至右的顺序填 C 表，直到将 X 和 Y 的 LCS 长度填入 c[m, n]为止。

```
procedure LCS__length (x, y);
begin
  m← Length (X); n← Length (Y);
  for i:=1 to m do c[i, 0]← 0;
  for j:=1 to n do c[0, j]← 0;
  for i:=1 to m do
  for j:=1 to n do
  begin
    if Xi =Yj then c[i, j]← c[i−1, j−1]+1
              else if c[i−1, j] ≥ c[i, j−1]
                      then c[i, j]← c[i−1, j]
                      else c[i, j]← c[i, j−1];
  end; {for}
end ; {LCS__Length}
```

四、构造一个最长公共子序列

由 LCS__Length 过程计算出的 C 表可被用来快速构造 $X=<X_1, \cdots, X_m>$ 和 $Y=<Y_1, \cdots, Y_n>$ 的一个 LCS，并按正常次序输出这个 LCS。方法是从 c[m, n]出发，进行反序的递归搜索:

当在 C 表中遇到 c[i, j]=c[i−1, j−1]+1 时，它意味着 $X_i=Y_j$ 是 LCS, 的一个元素，因此我们在递归搜索出则 X′i 与 Y′j−1 的 LCS 后输出 Xi;

当在 C 表中遇到 c[i,j] ≥ c[i,j−1]时，则递归搜索 X′i−1 与 Y′j−1 的一个 LCS。否则递归搜索 X_i' 与 Y_{j-1}' 的一个 LCS。

递归程序 Print__LCS 输入序列 X 和下标 i, j, 按正常次序输出 X 的前缀 X_i' 和 Y 的前缀 Y′的一个 LCS。初始调用为 Print__LCS (x, Length (X), Length (Y));

```
Procedure print__LCS (x, i, j);
begin
  if (i=0)or(j=0) then exit;
  if c[i, j]=c[i−1, j−1]+1
```

```
    then begin print_LCS(x, i-1, j-1); 输出 X_i; end {then}
     else if c[i, j]>c[i, j-1]
            then print_LCS(x, i-1, j)
            else print_LCS(x, i, j-1)
end; {print_LCS}
```

§8.3 应用举例

由前二节可以看出，在求解具有最优子结构且含重迭子问题的最优化问题时，动态程序设计只不过是一种途径，一种方法，而不是一种特殊算法。不像算法那样，具有一个标准的数学表达式或者一组明确定义的规则，因此读者在学习时，除了要对基本概念和方法正确理解外，必须具体问题具体分析，以丰富的想像力去建立模型，用创造性的技巧去求解。

为了帮助读者积累动态程序设计的经验，我们再例举几个题解。限于篇幅，对这些例题的算法分析，不再严格地按动态程序设计的四个步骤详尽地展开。

【例 1】欧几里德货郎担问题

平面上有 n 个点，任两点的 X 坐标都不相同。确定一条连结点的、闭合的游历路线，这条游历路线即从最左点开始，严格地由左至右直到最右点，然后严格地由右向左直至出发点。在所有这样的游历路线中，这条游历路线最短。

算法分析:

首先将 n 个点按 X 坐标递增的顺序排列成一个序列 L=< 点 1, 点 2,…, 点 n>。显然 L[n] 为最右点，L[1] 为最左点。由于点 1 往返经过二次 (出发一次，返回一次)，因此点 1 拆成两个点: L[0]=L[1]= 点 1。

设:

$W_{i,j}$——边 <i, j> 的距离;

$W_{i\cdots j}$——j−i 条连续边<i, i+1>,<i+1, i+2>…, <j−2, j−1>,<j−1, j>的距离和 $\sum_{i\leqslant k\leqslant j-1} W_{k,k+1}$。由点 i 至点 j 的连续边组成的路径称为连续路径;

d[i]——从点 i 出发由左至右直到 n 点后再由右至左到达点 i−1 的最短距离。d[i] 的递归式如下:

$$d[i]=\min_{i\leqslant j\leqslant N-1}\{W_{i\cdots j}+d[j+1]+W_{j+1,i-1}\} \quad (1\leqslant i\leqslant N)$$

我们专门设置了一张记忆表 k[i](1≤ i ≤ n), 记下使得 d[i] 最小的 J 值。显然递归式的边界 $d[n]=W_{n,n-1}$, k[n]=n。

由 d[i] 的递归式和记忆表 k[i] 的定义可以看出，在由点 i−1 至点 i 的最短路上，点 i 至 k[i] 的子路径上的点序号是逐一递增的，为由左至右方向上的连续子路径。按照递归定义，若 k[i]≠ N 且 k[k[i]+1]≠ N, 则点 k[k[i]+1]+1 至点 K[k[k[i]+1]+1] 也是由左至右方向上的连续子路径；否则 k[k[i]+1] 至 k[i]+1 为由右至左方向上的连续子路径，该子路径上的点序号是逐一递减的。

显然要使 d[i] 最小，必须分别使 d[i+1]…d[n-1], d[n] 最小，d[i] 包含了 d[i+1]…d[n] 最小的子问题，因此该问题具有最优子结构和重迭子问题的性质。为了提高算法效率，我们从 d[n] 出发，运用动态程序设计方法依次求 d[n-1], d[n-2]…d[1], 充分利用重迭子问题。最后求得的 d[1] 便是最短游历路线的距离；从点 1 出发，顺着记忆表 K 的指示可以递归输出这条游历路线。

下面是程序题解:

```
Program Bitonit_problem;
Const
  Maxn                          =100; {最多点数}

Type
  PosType                       =Record{坐标类型}
                                  x, y:real
                                End;
  Node                          =Record
                                  d:Real; {路线代价}
                                  k:Integer; {记忆表}
                                End;
Var
  N, i                          : Integer; {实际点数，辅助变量}
  F                             : Text; {文件变量}
  L                             : Array [0…Maxn] of PosType; {坐标序列}
  B                             : Array [0…Maxn] of Node;
{B[i].d——从点 i 出发，由左至右 n 点后，再由右至左到达 i-1 点的最短路长。B[i].k
——记忆表，记下使得 B[i].d=min{W(i…j)+B[j+1].d+W(j+1,i-1)} (i≤j≤N-1) 的 J 值}

Procedure Swap(Var a, b:PosType); {平面上的 A 点和 B 点交换}
Var
  t:PosType;
Begin
  t:=a; a:=b; b:=t;
End; {Swap}
Procedure Sort(p, r:Integer); {对L[P…R] 按 X 坐标递增的顺序进行堆排序}
Var
  i, q:Integer;
  s  :PosType;
Begin
  If p<r Then Begin
```

```
    s:=L[p+Random(r-p+1)];
    i:=p-1; q:=r+1;
    Repeat
      Repeat Inc(i) Until L[i].x>=s.x;
      Repeat Dec(q) Until L[q].x<=s.x;
      If i<q Then Swap(L[i], L[q]);
    Until i>q;
    Sort (p, q);
    Sort (q+1, r);
  End {then}
End; {sort}

Function Dist(i, j:Integer):Real; {计算平面上点 i 与点 j 之间的距离}
Begin
  Dist:=Sqrt(Sqr(L[i].x-L[j].x)+Sqr(L[i].y-L[j].y))
End; {Dist}

Procedure Bitonic;
Var
  i, j: Integer;
  d   : Real;
Begin
  B[n].d:=Dist(N, N-1); B[N].k:=N; {求递归式的边界 B[n]}
  For i:=N-1 Downto 1 Do Begin{依次求 B[n-1]…B[1]}
    B[1].d:=B[j+1].d+Dist(i-1, i+1);
   {求 B[i].d=min{W_{i…j}+B[j+1].d+W_{j+1,i-1}},
    并将 B[i].d 最小时的 J 值送入记忆表 B[i].k (i≤j≤n-1)}
    B[i].k:=i;
    d:=0;
    For j:=i+1 to N-1 Do Begin
      d:=d+Dist(j, j-1); {累计 W_{i…j}}
      If d+B[j+1].d+Dist(i-1, j+1)<B[i].d Then Begin
        B[i].d:=d+B[j+1].d+Dist(i-1, j+1);
        B[i].k:=j
      End {then}
    End {for}
  End {for}
End; {Bitonic}
```

```
Procedure Print (i:Integer);
Var
  k : Integer;
Begin
  For k:=i to B[i].k do{打印由左向右方向上的连续子路径}
    Write(' (', L[k].x:6:2,',', L[k].y:6:2,')⟶ ');
  If (B[i].k<N) And (B[B[i]. k+1].k<N)
    Then Print(B[B[i].k+1].k+1); {递归延伸游历路线}
  For k:=B[B[i].k+1].k Downto B[i].k+1 Do
  {打印由右向左方向上的连续子路径}
    Write(' (', L[k].x:6:2,',', L[k].y:6:2,')⟶ ')
End; {Print}
Begin
  Assign(F,' INPUT DAT'); {输入文件名串与文件变量连接}
  Reset(F); {文件读准备}
  Readln (F, N); {读入点数}
  For i:=1 to N Do Readln(F, L[i].x, L[i].y); {读入各点坐标}
  Sort(1, N); {将各点按X坐标递增的顺序排列}
  L[0]:=L[1]; {点1拆分成点0和点1}
  Bitonic; {计算最短游历路线}
  Print(1); {输出游历路线}
  Writeln(' (', L[1].x:6:2,',', L[1].y:6:2,')'); {最后输出返回点}
  Writeln(' Tot=', B[i].d :8:2); {输出该游历路线的距离}
End. {main}
```

【例2】优美打印

考虑在一台打印机上优美地打印一段文章的问题。输入的正文是长度为 $I_1, I_2, \cdots, I_n$ 的n个单词构成的序列。我们希望将这段文章在几行上打印出来，每行的最大长度为m，且"优美"度的标准如下：

如果某一行包含从i到j的单词，且每两个单词间留一空，则在行末多余的空格为 $m-j+i-\sum_{k=i}^{j} I_k$。我们希望除最后一行的所有行中，行末多余空格的总和最小。请给出一个动态程序设计算法来优美地打印出一段有n个单词的文章。

算法分析：

由于必须打印完n个单词且每行打印的单词是连续的，因此，我们从第n个单词开始，依次考虑填一个单词(单词n)，填两个单词(单词n-1，单词n)…，填n个单词(单词1，单词2，…单词n)的打印方案。

设当前行填入单词 i … 单词 j (i ≤ j) 每两个单词间留一空，单词间的空格数为 j−i，从第 i 个单词到第 j 个单词的长度和为 $\sum_{k=i}^{j} I_k$. 因此行末多余的空格为 $m - j + i - \sum_{k=i}^{j} I_k$ 。由于单词填入的方式是按单词序号递减的顺序进行的，因此填入单词 i … 单词 n 后的行末空格数的总和应为当前行的行末空格数 + 前面行的行末空格数的和。我们的目标是使它最小。

设

r[i]—填入单词 i … 单词 n 后，所有被填行的行末空格数总和的最小值。显然，r[i] 的递归式如下:

$$r[i]:=\min_{i\leq j\leq n}\{m-((j-i)+\sum_{k=i}^{j} I_k)+r[j+1]\} \qquad (1\leq i\leq n+1)$$

另外，我们专门设置了一张记忆表 k[i] (1 ≤ i ≤ n+1), 记下使得 r[i] 最小的 J 值，表示填单词 i … 单词 n 的最佳方案中，最后一行应填单词 i … 单词 j (=k[i])。

r 的递归的边界为

r[n+1]=0； k[n+1]=n+1;

表示不填任何单词时的行末空格数为 0。

我们从 r[n+1] 出发，依次求 r[n], r[n−1] … r[1]. 由 r[i] 的递归式可以看出，求 r[i] 最小值的子问题，包含了求 r[i+1] … r[n+1] 的子子问题。要使 r[i] 最小，必须使这些子子问题的值最小，因此符合动态程序设计要求的″最优子结构″和″重迭子问题″两个要素。我们按自下而上的方式求解，充分利用了重迭子问题。最后求出的 r[1] 为优美打印方案中行末空格数的总和；从单词 1 出发，顺着记忆表 K 的指示，可顺序打印出文章的各行。

下面给出程序题解:

```
Program Print_Beautifully ;

Const
  Maxn                    = 1500 ; { 最多单词数 }
Type
  Word                    = String [ 20 ] ; { 单词类型 }
  Node                    = Record
                              r, k: Integer ;
  { 目前行末多余的最小空格数，记忆表 }
                            End ;

Var
  N, M, i, j, r           : Integer ;
  { 单词数，行长，辅助变量，行末空格数 }
  F                       : Text ; { 文件变量 }
```

```
  L                              : Array[1…Maxn] of Word;{单词序列}
  P                              : Array[1…Maxn] of Node;
{P[i].r——单词i…单词n填入正文后行末多余的最少空格数}
 {P[i].r——当前行填入单词i…单词P[i].k后，可使P[i].r最小}
Procedure Init;
Var
  c                              : Char;{输入字符}
  w                              : Word;{单词}
Begin
  Assign(F,' INPUT DAT');{输入文件名串与文件变量连接}
  Reset(F);{文件读准备}
  Readln(F, M);{读入行长}
  N:=0; W:=' ';{单词数清零，单词初始化为空}
  While Not Eof(F) Do Begin{依次读入每个单词并赋给L序列，累计单词数}
    Read (F, c);
    If c in [' a' ..' z',' A' ..' Z' ]
       Then W:=W+C
       Else If W<>' '  Then Begin
         Inc(N);
         L[N]:=W;
         W:=' ';
       End{then}
  End;{while}
  Close(F)
End;{init}

Procedure Main ;
Var
  i, j, r                        : Integer;
Begin
  P[N+1].k:=N+1; P[N+1].r:=0;{设置递归边界}
  For i:=N Downto 1 Do Begin{依次倒推p[n], p[n-1],…p[1]}

    P[i].r:=Maxint;

{求p[i].r=min(m-(j-i)-∑(k=1 to j) I_k+P[j+1].r),并将P[i].r最小时的j值送入P[i].k}

    j:=i;  r:=M+1;
```

```
    Repeat
      r:=r-Length(L[j])-1;
      If (r>=0) And (r+P[j+1].r<=P[i].r) Then Begin
        P[i].r:=r+P[j+1].r;
        P[i].k:=j;
      End;{then}
      Inc(j);
    Until (r<=0)Or(j>N);
  End;{for}
End;{main}

Begin
  Init;{输入单词}
  Main;{计算行末多余空格数最少的方案}
  i:=1;{从第1个单词开始打印}
  While i<N Do Begin{若未打印完N个单词，则循环}
    Write (L[i]); r:=M-Length(L[i]);{当前行打印从i到p[i].k的单词}
    For j:=i+1 to P[i].k Do Begin
      Write (' ', L[j]);
      r:=r-Length(L[j])-1
    End;{for}
    Writeln (' ': r+1);{打印多余空格}
    i:=P[i].k+1{打印单词p[i].k所在的行}
  End;{while}
Writeln ('Total rest=', P[1].r);{打印所有行中行末多余的空格数}
End.{main}
```

【例3】编辑距离

当一个智能终端将一行正文更新，并用新的目标串 y[1…n]来替换现存的源串 X[1…m]时，可有几种方式来做这种变换：源串中的单个字符可被删除(delete)；被替换(replace)；或被复制到目标串中去(copy)；字符也可被插入(insert)；源串中的两个相邻字符可进行交换并复制到目标串中去(twiddle)；在完成其它所有操作之后，源串中余下的全部后缀就可用删至行末的操作删除(kill)。

例如，将源“algrorithm”转换成目标串“altruistic”的一种方法是采取下面的操作序列：

操作	目标串	源串
copy a	a	lgorithm
copy l	al	gorithm

replace g by t	alt	orithm
delete o	alt	rithm
copy r	altr	ithm
insert u	altru	ithm
insert i	altrui	ithm
insert s	altruis	ithm
twiddle it into ti	altruisti	hm
insert c	altruitic	hm
kill hm	altruitic	

要达到这个结果还可有其它一些操作序列。操作 delete，replace，copy，insert，twiddle 和 kill 中每一个都有一个相联系的代价 cost。例如

cost(delete)=3； cost(replace)=6; cost(copy)=5； cost(insert)=4;

cost(twiddle)=4; cost(kill)=被删串长*cost(delete)−1;

一个给定的操作序列的代价为序列中各操作代价之和。例如上述操作序列的代价为

(3*cost(copy))+cost(replace)+cost(delete)+(3*cost(insert))

+cost(twiddle)+cost(kill)

=3*5+6+3+3*4+4+2*3−1

=45

给定两个序列 x[1…m]，y[1…n]和一些操作代价集合，X 到 Y 的编辑距离为将 X 转化为 Y 的最“便宜”的转换序列的代价。请给出一个动态程序设计方法来找出 x[1…m]至 y[1…n]的编辑距离，并输出一个最优转换序列。

算法分析:

我们按目标串长递增的顺序进行变换：先变换 Y_n，再变换 $Y_{n-1}Y_n$…… 直到变换至 $Y_1Y_2\cdots Y_n$ 为止。

设 C_1——delete 操作的代价； C_2——replace 操作的代价；

C_3——copy 操作的代价； C_4——insert 操作的代价；

C_5——twiddle 操作的代价； C_6——kill 操作的代价，其值为当前源串长$*c_1-1$;

c[i, j]——将 $X_j\cdots X_m$ 变换至 $Y_i\cdots Y_n$ 的编辑距离。变换的可能方式有五种:

1. delete 操作：删除源串中的 X_j

2. replace 操作：源串中的 X_j 被替换到目标串的 Y_i 位置;

3. copy 操作：源串中的 X_j 被拷贝到目标串的 Y_i 位置，使得 $Y_i=X_j$;

4. insert 操作：插入一个字符至目标串的 Y_i 位置;

5. twiddle 操作：X_jX_{j+1} 交换并复制到目标串的 Y_iY_{i+1} 位置，使得 $Y_i=X_{j+1}$，$Y_{i+1}=X_j$;

要使 c[i, j]最小，必须使得 $X_j\cdots X_m$ 与 $Y_i\cdots Y_n$ 中除参与变换的字符外的子串间的变换代价最小：即 delete 操作前 $X_{j+1}\cdots X_m$ 变换至 $Y_i\cdots Y_n$ 的代价和要最小；replace 或 copy 操作前 $X_{j+1}\cdots X_m$ 变换至 $Y_{i+1}\cdots Y_n$ 的代价和最小；insert 操作前 $X_j\cdots X_m$ 变换至 $Y_{i+1}\cdots$

Y_n的代价和要最小；twiddle操作前$X_{j+2}\cdots X_m$变换至$Y_{i+2}\cdots Y_n$的代价和要最小。我们还不可能知道究竟使用哪一种操作可得出$X_j\cdots X_m$至$Y_i\cdots Y_n$的编辑距离，因此必须比较五种操用的代价和，从中找出代价和最小的一种。

由此得出c[i, j]的递归式:

$$c[i,j]=\min\{c[i,j+1]+c_1,\ c[i+1,j+1]+c_2,\ c[i+1,j+1]+c_3\ y_i=x_j,\ c[i+1,j]+c_4,\ c[i+2,j+2]+c_5 \mid (i<n)\wedge(j<m)\wedge(y_i=x_{j+1})\wedge(y_{i+1}=x_j)\}$$

我们专门设置了一个记忆表k[i, j],以记下c[i, j]最小时的操作序号$C_i(1\leqslant i\leqslant 5)$。

显然c[i, j]的递归边界有三个:

1. $c[i, m+1]=(N-i+1)*C_4$;　　$k[i, M+1]=4$　$(1\leqslant i\leqslant N)$

即源串空时，要变换目标串$Y_i\cdots Y_m$必须进行N−i+1次insert操作。

2. $c[n+1, i]=(M-i+1)*C1-1$;　$k[n+1, i]=6$;

即目标串空时，进行一次kill操作删除源串中余下的全部后缀。kill的代价为删除后缀$X_i\cdots X_m$(M−i+1个字符)的delete代价减1。

3. $c[n+1, m+1]=0$;

即源串和目标串空时的编辑距离为0。

由c[i, j]的递归式可以看出，求c[i,j]最优解的子问题，包含了求$c[i\cdots n, j+1\cdots M]$的子子问题，要使c[i, j]的值最优，必须保证这些子子问题的值最优，因此具有"最优子结构"和"重迭子问题"两个要素。我们从变换Y_n字符出发，按目标串长度递增的顺序自下而上地依次计算$c[n,1,\cdots m]\rightarrow c[n-1,\ 1\cdots m]\rightarrow\cdots\rightarrow c[1,\ 1\cdots m]$，充分利用了重迭子问题，使得算法效率显著提高。

下面是程序题解

```
Program Editment_Distance;

Const
  Maxn                    = 100;{最大串长}
  Cost                    : Array[1··· 5] of Shortint
  {前五种操作的代价}
                          =(3, 6, 5, 4, 9);
Type
Node                      = Record
                              c, k:Integer;{操作代价和记忆表}
                            End;

Var
  N, M                    : Integer;{目标串和源串的长度}
  X, Y                    : String[Maxn];{源串和目标串}
  T                       : Array [1··· Maxn, 1··· Maxn] of Node;
```

```
{T[i,j].c——将源串中的x[j]…x[m]变换成目标串的y[i]…y[n]所需的最少操作
代价和; T[i,j].k——记忆表．使得T[i,j].c最小的最后一次操作序号}

rocedure Main;
Var
  i,j                              : Integer;
Begin
  For i:=N Downto 1 Do Begin {源串空时，通过insert操作变换至目标串}
    T[i,M+1].c:=(N-i+1)*cost[4];
    T[i,M+1].k:=4;
  End;{for}
  For i:=1 to M Do Begin {目标串空时通过kill操作将源串删除．kill的代价为
                          源串的目前串长*delete的代价-1}
    T[N+1,i].c:=Cost[1]*(M-i+1)-1;
    T[N+1,i].k:=6;
  End;{for}
  T[N+1,M+1].c:=0;
  For i:=N Downto 1 Do {按目标串长递增的方向求解T[i,j].c和T[i,j].k}
    For j:=M Downto 1 Do Begin
      T[i,j].c:=T[i,j+1].c+Cost[1]; T[i,j].k:=1;
      If T[i+1,j+1].c+Cost[2]< T[i,j].c Then Begin
        T[i,j].c:=T[i+1,j+1].c+Cost[2];
        T[i,j].k:=2;
      End;{then}
      If (Y[i]=X[j]) And (T[i+1,j+1].c+Cost[3]< T[i,j].c) Then Begin
        T[i,j].c:=T[i+1,j+1].c +Cost[3];
        T[i,j].k:=3;
      End;{then}
      If T[i+1,j].c+Cost[4]< T[i,j].c Then Begin
        T[i,j].c:=T[i+1,j].c+Cost[4];
        T[i,j].k:=4;
      End;{then}
      If (i<N) And (j<M) And (Y[i]=X[j+1]) And (Y[i+1]=X[j]) And
         (T[i+2,j+2].c+Cost[5]<T[i,j].c) Then Begin
        T[i,j].c:=T[i+2,j+2].c+Cost[5];
        T[i,j].k:=5;
      End;{then}
    End;{for}
```

```
End ; { main }

Procedure Print ; { 输出 x [ 1… m ] 至 Y [ 1… n ] 的最优转换序列和编辑距离 }
Var
  i , j                                : Integer;
Begin
  i := 1; j:= 1; { 从两串的首字符开始转换 }
  While ( i <= N ) Or ( j <= M ) Do Begin
    Case T [ i , j ] . k of
      1: Begin
           Writeln ( ' delete' , X [ j ] );
           Inc ( j );
         End ; { 1 }
      2: Begin
           Writeln ( ' replace' , X [ j ],' by' , Y [ i ] );
           Inc ( i ); Inc ( j );
         End; { 2 }
      3: Begin
           Writeln ( ' copy ' , Y [ i ] );
           Inc ( i ); Inc ( j );
         End; { 3 }
      4: Begin
           Writeln ( ' insert' , Y [ i ] );
           Inc ( i );
         End ; { 4 }
      5: Begin
           Writeln ( ' twiddle' , X [ j ],'  ' , X [ j+1 ],' to' , Y [ i ],'  ' , Y [ i+1 ] );
           Inc ( i, 2 ); Inc ( j , 2 )
         End ; { 5 }
      6: Begin
           Writeln ( ' kill' , copy ( X , j , Length ( X )-j+i ) );
           j := Length( X )+1
         End ; { 6 }
    End ; { Case }
    Writeln ( ' Y=' , copy ( y , 1 , i-1 ) , ' X=' , Copy ( X , j , Length ( X )-j+1 ) );
    Readln;
  End ; { while }
```

```
End ; { print }

Begin
  Write ('Y=' ); Readln ( Y ); { 输入目标串 }
  Write ('X=' ); Readln ( X ); { 输入源串 }
  M:=Length ( X ); N:=Length ( Y ); { 计算源串和目标串的串长 }
  Main ; { 求 X 至 Y 的编辑距离 }
  Print ; { 输出最优转换序列和编辑距离 }
End . { main }
```

【例 4】vitebi 算法

我们可用一有向图 G=(V, E) 上的动态程序设计来做语音识别。每条边 (U, V)∈E 上标以选自有限的声音集∑ 中的一种声音 &(U, V)。另外，还有一相联系的非负概率 P(U, V)，表示从顶点 U 开始遍历边 <U, V> 并产生相应声音的可能性。这种图是一个人说一种有限语言的形式模型。

现指定一个开始顶点 V_0，规定从 V_0 出发的边的概率之和为 1，另外指定声音集 $S=<\&_1, \&_2, \cdots, \&_K>$。请描述一个有效算法，求一条开始于 V_0 的具有标记 S 的一条最有可能的路径，即路径上的边依次标记 $\&_1, \cdots, \&_K$，且其上所有边的概率积最大。如果不存在这样的路径，返回“No__Such__Path”信息。

算法分析:

设 $L_i(0 \leqslant i \leqslant K)$ 链——存储所有标以声音 $\&_i$ 的边 <U,V> 的终点 V，且 V_0 通向每个 V 有一条路径，这条路径上的边顺序标记 $\&_1, \&_2, \cdots, \&_i$ 且这 i 条边的概率之积在所有这样的路径中是最大的。

我们首先将 V_0 放入 L_0 链，设 V_0 出发的边的概率之积为 1。然后按下述方法依次扩展 L_1 链，L_2 链，$\cdots$，L_K 链:

扩展 L_i 链的当前顶点时检查 L_{i-1} 链中的所有顶点。若与 L_{i-1} 链中某顶点 U 相连的边 <U, V> 上的声音为 $\&_i$，则再依次检查 L_i 链的每一顶点: 若 V 未在 L_i 链中，则 V 进入 L_i 链和 V_0 至 U 的路径; 若 V 在 $\&_i$ 链中且 V_0 出发、经过 <U, V> 的路径概率为目前最大，则修正 L_i 链中 V 的概率积，使得 V_0 至 V 的路径经过 <U, V>。

显然这条路径具备最优子结构: 即 V_0 至路径上每顶点的概率积最大; 同时具有重迭子问题的性质: 扩展 L_i 链的子问题包含了扩展 L_{i-1}，$L_{i-2}, \cdots, L_1$ 链的子子问题。因此，我们可以使用动态程序设计的方法，自下而上地顺序扩展 L_1，$L_2, \cdots, L_K$ 链。在扩展出 L_k 链后，V_0 与 L_K 链中每一顶点 V_k 间都有一条标记 S 的、最可能（相对其它通往它的路径而言）的路径。L_K 链中概率积最大的一个顶点所对应的一条路径即为问题的解。

下面给出程序题解:

```
Program Viterbi;

Const
  Maxn                    = 80;{最多顶点数}

Type
  Link                    = Node;{图的邻接表指针}
  Node                    = Record{邻接表顶点类型}
                              v,d: Integer;{相邻点序号，声音}
                              a   : Real;{边的概率}
                              next: Link;{后继指针}
                            End;
  LLink                   =^LType;{链表指针}
  LType                     = Record{链结点类型}
                              v   : Integer;{顶点，序号}
                              a   : Real;{V0至该顶点的概率积}
                              next,p:LLink;{后继指针，前趋指针}
                            End;
Var
  N,K,v0                    : Integer;{顶点数，声音数，起始顶点}
  G                         : Array[1…Maxn]of Link;{图的邻接表}
  L                         : Array[0…Maxn]of LLink;{L[i]——&i链}
  S                         : Array[1…Maxn]of Integer;{声音链}
x,y                       : LLink;{辅助指针}

Procedure Init;
Var
  F                         : Text;
  i                         : Integer;
  x                         : Link;
Begin
  Assign(F,'INPUT.DAT');{输入文件名串与文件变量连接}
  Reset(F);{文件读准备}
  Readln(F,N,K,v0);{读入顶点数，声音数，起始顶点}
  For i:=1 to K Do Readln(F,S[i]);{读入声音集}
  For i:=1 to N Do G[i]:=nil;{图的邻接表置空}
  While Not Eof(F) Do Begin{依次读入每边的两个端点、声音、概率，建立图的邻接表}
```

```
    New(x);
    Readln (F, i, x^.v, x^.d, x^.a);
    x^.next:=G[i];G[i]:=x
  End; {while}
  Close(F);
End; {Init}

Procedure Main;
Var
  i                              : Integer;
  x, z                           : LLink;
  y                              : Link;
Begin
  New(L[0]); {为 &0 链申请内存}
  L[0]^.v:=v0; L[0]^.a:=1; {V0 进入 &0 链}
  L[0]^.p:=nil; L[0]^.next:=nil;
  For i:=1 to K Do Begin{依次搜索具有 &1, &2,… &K 声音的边}
    x:=L[i-1]; L[i]:=nil; {设 &i-1 链的首指针为 X, &i 链初始化为空}
    While x<>nil Do Begin{若 &i-1 链未搜索完}
      y:=G[x^.v]; {求 X 在图的邻接表中的指针}
      While y<>nil Do Begin {分析每一条与 X 相边的边 <X, Y>}
        If y^.d=S[i] Then Begin {若相连边 <X, Y> 的声音为 &i}
          z:=L[i];
          While (z <> nil) And (z^.v<>y^.v) Do z:=z^.next;
          If z=nil Then Begin{若 &i 链中未有指向相邻点 Y 的边，则 Y 进入 &i 链和路
径中}
            New(z);
            z^.v:=y^.v; z^.a:=x^.a*y^.a; z^.p:=x;
            z^.next:=L[i]; L[i]:=z
          End {then}
          Else If x^.a*y^.a>z^.a Then Begin
{若 Y 在 &i 中且 V0 经 <X, Y> 至 Y 的概率积目前最大，则修正 &i 中 Y 的概率积，Y
进入路径}
            z^.a:=x^.a*y^.a;
            z^.v:=y^.v; z^.p:=x;
          End {then}
        End; {then}
        y:=y^.next; {分析 X 的下一个相邻点}
```

```
      End ; { while }
       X := X^ . next { 分析 &i-1 链的下一个顶点 }
    End ; { while }
    End ; { for }
End ; { main }

Procedure Print ( X : LLink ); { 递归打印 V0 至 X 的一条路径 }
Begin
  If x^ . v = v0 Then Write ( v0 )
    Else Begin
      Print ( x^ . p );
      Write ( ' - - > ' , x^ . v );
    End ; { else }
End; { Print }

Begin
  Init ; { 输入数据建立邻接表 }
  Main; { 求 V0 至其它顶点的概率积最大的路径 }
  If L [ k ] = nil Then Writeln ( ' No Path' )
  { 若未搜索到具有 &K 声音的边，则返回无解信息 }
    Else Begin
      x := L [ k ] ; y : = x ; { 求 &K 链中概率积最大的一个顶点 Y }
      While x <> nil Do Begin
        If x^ . a > y^ . a Then y := x ;
        x := x^ . next ;
      End ; { while }
      Print ( y ); { 递归打印 V0 至 Y 的一条路径 }
      Writeln ;
      writeln ( ' p = ' , y^ . a : 5 : 3 ); { 输出该路径的概率 }
    End ; { else }
End . { main }
```

第九章　题　库

一、神秘的大陆

一张地图是一个 48 * 16 个坐标格组成的矩形。当两个坐标格在上、下、左或右方向上有公共边时，我们称它们是相邻的。开始我们只知道每个坐标格是水（W）或地（G）。

有 4 种地形（GT）：G、M、P、C。

同时又有 4 种水形（WT）：W、O、B、L。

并假设地图外是海洋（O）。

现可通过一种“地理规定”改变坐标格的类型。坐标格可能成为：

- 山形（Mountain）：如果一个 GT 与 4 个 GT 相邻。
- 半岛（Peninsula）：如果一个 GT 与 3个 WT，或与 2 个 WT、至少 1 个 P、或与 1 个 WT、至少 2 个 P 相邻。
- 海岸（Coastline）：如果一个 GT 非 M 且非 P。
- 海洋（Ocean）：如果一个 WT 与至少 1 个 O 相邻。
- 海湾（Bay）：如果一个 O 与至少 2 个 B、至多 1 个 O，或与 1 个 B、至少 2 个 GT，或与至少 2 个 GT、至少 1 个 O 相邻。
- 湖形（Lake）：如果一个 W 在其它类型转换结束后，类型仍未改变。

一张地图上的每一格的类型都有可能被多次改变，因为当某格的类型被改变时，它邻格的类型可能也被改变。

当一张地图不再变动后，我们称“地图已确定”。

问题描述

编程执行以下步骤：

1. 从一个 ASC11 文件中读入一张地图，并将地图与地图参数显示在屏幕上。
2. 通过“地理规定”转换坐标格的类型，直到“地图已确定”。
3. 在屏幕上显示确定后的地图与地图参数。
4. 将地图与地图参数存入 ASC11 文件中。

输入输出举例

输入文件：

```
WWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWGGGGGGWWWWWWWWWWWWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWGGWWGGWWWWWWWWWWWWWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWGGGWGGWWWWWWWWWWWWWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWGGWWGGWWWGGGWGWWWWWWWWWWWWWWWWWWW
```

WWWWWWWWWWWWWWWWGGGGGGGGGGGGGWWWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWGGGWWWGGWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWWGGGWWWGGWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWWWGGGGWWGGWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWWWGGWWWGGWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWWWWGWWWGGWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWWWWGWWWWWWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWWWGGGWWWWWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWW
WWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWW

屏幕同时显示:

MYSTERIOUS : G = 61 W = 707 ALL = 768

输出文件:

000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000
000000000000000CCCCCCC0000000000000000000000000
000000000000000CCLLCCC0000000000000000000000000
000000000000000CMPLCCC0000000000000000000000000
000000000000000CCLLCCBBBCCCBP0000000000000
000000000000000BCCCCCCCCCMCCCCB0000000000
000000000000000000000000BCMCBBBCC0000000000
000000000000000000000000CMCB00CC0000000000
000000000000000000000000BCCB00CC0000000000
000000000000000000000000BPB0000000000000000
000000000000000000000000BPB0000000000000000
000000000000000000000000PPP0000000000000000
000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000

EXPLORED: P = 8 C = 47 M = 6 O = 685 B= 17 L = 5 ALL = 768

二、迷宫车间

一个迷宫完成覆盖一个 N * M (2 < = N, M < = 20) 的矩形区域。它由许多“墙”与“空格”组成，后者包含一个“入口”与一个“财宝格”。

一条道路由一系列以“墙”为界的相邻“空格”组成，它开始于“入口”结束于“末点”(即死胡同)。路径长度为路所含“空格”个数（包括“入口”与“末点”)。

迷宫中的所有道路能分叉，但不能相连（比如任两条道路不会有同一“末点”)。“入口”位于迷宫的第一行。“财宝格”位于最长路径的“末点”。

迷宫必须可能多得被道路覆盖。我们希望能高兴地看到当迷宫构造完毕时、道路已盖

满整个矩形区域。由于屏幕显示太快，因此需要一段延时（Delay Time）。

问题描述

编程实现以下任务（菜单形式）：

1. 以人机对话的形式设定 N，M。
2. 以人机对话的形式设定 Delay Time。
3. 基本上随机地构造迷宫，并显示构造过程。
4. 将构造好的迷宫及迷宫参数存入 ASC11 文件。
5. 从 ASC11 文件中读入一个迷宫，并以高亮度字符显示“入口”到“财宝格”的道路。

输出规则

输出格子的形式为：

“墙”：“□□□”（#219）

“路径”，“入口”：“－－－”（#32）

“财宝格”：“－T－”

高亮度路径：“－.－”（句点）

三、Hamilton 机器人

某平面上有 n 个 Inteqer 型坐标 $(x_1, y_1), (x_2, y_2), \cdots, (x_n, y_n)$ 的点 $P_1, P_2, \cdots P_n$。

一个机器人必须从 P_1 开始走遍所有点最后回到 P_1，且每个点仅经过一次。

我们对机器人的行动加以约束：它只能沿直线行进。开始在 P_1 时，它能面向任何方向。到达任一 P_i 后，必须向左或向右转 90 度，然后才能继续走向下一个点。

一个机器人程序包含以下指令：

1. “Orientation xk yk”：此指令只能用于开头。

   命令机器人面向 $P_k(2 <= k <= n)$。

2. “MoveTo xj yj”：仅当可能时，

   命令机器人朝着当前方向移至 $P_j(1 <= j <= n)$。

3. “TurnLeft”：命令机器人向左转 90 度。
4. “TurnRight”：命令机器人向右转 90 度。
5. “Stop”：终止机器人的行动。此指令只能用于程序末尾。

问题描述

编程实现以下各项任务：

1. 从 INPUT.TXT 文件中读入数据（n，xi，yj，$1 <= i <= n$），并显示在屏幕上。
2. 编写一段机器人程序遍历所有点。
3. 若无可行方案，则程序仅含“Stop”。
4. 显示机器人程序与路线长度（保留 2 位小数）。
5. 将机器人程序写入 OUTPUT.TXT 文件。

输入输出举例

输入文件

输入文件第一行为 n。以下 n 行为点的坐标(x,y)。

例如: 4

```
 2 -2
 0  2
-1 -1
 3  1
```

输出文件

相应的输出文件为:

```
Orientation  3  1
MoveTo       3  1
TurnLeft
MoveTo       0  2
TurnLeft
MoveTo      -1 -1
TurnLeft
MoveTo       2  2
Stop
```

四、魔方工具包

一个魔方是一个由 3*3*3 个小立方体组成的立方体。最初立方体的 6 个面分别涂上不同颜色，我们称之为“最初魔方”。魔方的每一面上的 3*3 个小立方体组成它的一层。

魔方所能见到的每一层（6 个面）都能旋转 90，180，270 或 360 度。所有层的旋转轴都垂直于面且通过其中心。旋转的结果是另一个魔方，它的所有面的颜色都改变了。

现在我们用字符来代替颜色: U = 上，D = 下，F = 前，B = 后，L = 左，R = 右。任何一个序列的旋转都能表示成{U，R，F，B，L，D}中一些字符组成的字符串，其中每个字符表示它所指定的面顺时针旋转 90 度。

问题描述

编程完成以下 3 个任务（菜单形式），你可以假设任何输入的字串长度都 < = 35。

1. 输入一个字串，并将其缩减至没有任何一种旋转次数超过 3 次。

你的算法须能处理非法输入的情况。例如:

输入	输出
L	L
LL	LL
LLL	LLL
LLLL	""(空串)
LLLLL	L
LLRRRFFFFRLB	LLLB
HELLO	" error "

2. 判断输入的 2 个字串的旋转结果是否相同。例如:

输入一	输入二	输出
RU	UR	no
RRFFRRFFRRFFRRFF	FFRRFFRR	yes
RRFFRRFFRRFFRRFF	RRFFRRFF	no

3. 求出输入字串至少须使用几次才能将魔方转回到“最初魔方”(一定大于 0)

输入	输出
L	4
DD	2
BULB	36
RUF	80
BLUFF	180

五、Tom, Jerry 和奶酪

猫 Tom 和鼠 Jerry 同住在一矩形地窖中。猫要吃鼠，鼠要吃奶酪。地窖中有 2 种地砖: 有洞砖与无洞砖。一个洞足以让鼠钻入，但猫不能。

问题描述

以菜单形式完成以下任务:

- 随机地生成一个地窖，并给猫、鼠和奶酪安排一个位置。如:

```
f f f f f f f f f f f f f f f
f p p p p p p p p p p p p C f
f h f f f f f f f f f f p f
f p p p J h p p p p p p p p f
f p f f f f f f p f f f f f f
f p p p p p p p p p p T p p f
f f f f f f f f f f f f f f f
```

将地窖写入 Cellar.DAT 文件。

- 从 Cellar.DAT 文件读入地窖，以任何适当的形式显示在屏幕上。
- 鼠先行，猫后行。两者皆须满足以下规定:
  - 必须上、下、左或右移动。
  - 鼠必须走 1 步 (穿过 p 或 h)。
  - 猫必须走 1 或 2 步 (穿过 p)。
- 当鼠吃到奶酪或猫抓到鼠时，游戏结束。

· 游戏结束后，你可以任意奖励或惩罚猫或鼠。

六、债务

近来经济形势越来越坏。借款者不守信用。许多工厂（< = 1000）陷入三角债中。所有工厂既有借入，又有借出。

假设：工厂 A 须还工厂 B$100; 工厂 B 须还工厂 C$50; 工厂 C 须还工厂 A$75。则所有欠款总额为 $ 225。工厂 A 以他的威信请求工厂 C 还工厂 B$ 75。结果：

工厂 A 须还工厂 B $ 25;
工厂 B 须还工厂 C $ 50;
工厂 C 须还工厂 B $ 75。

用这种方法，欠款总额将会最小化。最后结果为：

工厂 A 须还工厂 B $ 25;
工厂 C 须还工厂 B $ 25。

编程处理债务。

1. 从 Debt . DAT 文件中读入债务表，将表写入 Debt . SOL 文件。文件的每行包含 2 工厂号与它们间的欠款数额。每 2 组数据间以空行分隔。
2. 求出使欠款总额最小化的方案，写入 Debt . SOL 文件。
3. 求出最小化的欠款总额，写入 Debt . SOL 文件。每 2 组结果间以空行分隔。

```
Debt . DAT
A B 100 . 00
B C 50 . 00
C A 75 . 00

A B 1500 . 00
B C 1100 . 00
D A 1400 . 00

Debt . SOL
A B 100 . 00
B C 50 . 00
C A 75 . 00
Total = $ 225 . 00
A B 25 . 00
C B 25 . 00
Total = $ 50 . 00

A B 1500 . 00
```

B C 1100 . 00
D A 1400 . 00
Total = $ 4000 . 00
A B 100 . 00
D B 300 . 00
D C 1100 . 00
Total = $ 1500 . 00

七、Sinistra 的城市

在 Sinistra 的城市 TownCouncil，是禁止右转弯的。为了执行这一规定，所有的车上都装有计算机。这种计算机能记录车在每一时刻的转弯情况。每次旅行结束后，计算机能统计出司机右转弯的次数，并给出罚款总额（右转一次罚 $ 50）。

问题描述

一次旅行是由 n(n >= 2) 个点 p1,p2,…,pn 的坐标定义的。其中 p1 是起点，pn 是终点。车只在 n 个点上改变方向。

1. 从 Sinistra . DAT 文件中读入 n 和 n 个坐标，并写入 Sinistra . SOL 文件。文件中的坐标以旅行顺序给出。每 2 组数据间以空行分隔。
2. 计算出罚款总额。
3. 将罚款总额写入 Sinistra . SOL 文件。每 2 组结果间以空行分隔。

Sinistra . DAT
3
4　　1
5 . 5　4
5　　2

4
0　　1
－1　0
0　－1
1　　0

Sinistra . SOL
3
4　　1
5 . 5　4
5　　2
Traffic ticket : $ 50

4

0 1

−1 0

0 −1

1 0

Traffic ticket: $0

八、游戏

计算机与孩子做游戏:

孩子想1个由4种颜色组成的序列（4种颜色不一定完全不同)。每种颜色只能是6种颜色之一。方便起见，我们用数字1到6表示6种颜色。

计算工机必须根据孩子的回答找出孩子所想的颜色序列。计算机在屏幕上显示一个序列，孩子用键盘回答以下两个问题:

1) 猜对的颜色中位置不对的有几个？

2) 猜对的颜色中位置对的有几个?

输入输出举例

假设孩子想的是4655。游戏过程是:

Computer	Your answer
1234	a)1 b)0
5156	a)2 b)1
6165	a)1 b)1
5625	a)1 b)2
5653	a)1 b)2
4655	a)0 b)4

编程使至多6次问答后猜出序列。

如果你办不到的话，编程使至多10次问答后猜出序列。

九、加法链

若一个整数序列

a1,…,an

中对与每个 k(1 < k < = n)，总存在 i 和 j(1 < = i < = j < = n) 使得:

ak = ai + aj

则此序列称为加法链。

例如: 1 2 3 5

1 2 3 4 5

1 2 4 5

都是加法链。

问题描述

编程对给定的整数 a，构造一个最短的加法链，且 a1 = 1，an = a。

1. 从 Chain.DAT 文件读入 a。每 2 组数据间以空行分隔。

2. 构造一个最短的加法链，使 a1 = 1 且 an = a。

3. 将结果写入 Chain.SOL 文件，每 2 组结果间以空行分隔。

Chain.DAT

3

Chain.SOL

1 2 3

1 2 3 5

十、黑白棋

你将得到一个 n*n(n < = 10) 的棋盘。白格和黑格的排布满足下列条件:

1. 每列至少一个白格。

2. 至少有一列全是白格。

请在棋盘上放置棋子，满足:

1. 能放在白格上。

2. 每列每行至多一个棋子。

3. 每个不放棋子且被同行中的某棋子控制的白格必须被同列上的某棋子控制。

问题描述

1. 从 Chess.DAT 文件读入 n 与 n*n 的棋盘。棋盘每格以 1 或 0 表示: 1 表示白格; 0 表示黑格。1 与 0 之间无空格。在读入的同时，将数据写入 Chess.SOL 文件。每 2 组数据间以空行分隔。

2. 在满足条件的情况下，放置尽量多的棋子。

3. 将放置的棋子个数与放置方案写入Chess.SOL 文件。每个棋子用“X”表示。每 2 组数据间以空行分隔。

Chess.DAT

3

010

110

011

4

1111
0001
0001
0001

Chess . SOL
3
010
110
011
3
0X0
X10
01X

4
1111
0001
0001
0001
1
1111
000X
0001
0001

十一、化妆品

新妆有限公司是生产 N 种不同化妆品的重要公司。新妆公司每个月都要将产品运往它在 Mallorca 的支店。每种化妆品的价值与体积都不相同。

你的任务是设计一个运货方案（即确定每种化妆品运的件数，可以不运），使得所运货物的总体积不大于货船容积的条件下，所运货物的总价值最大。

问题描述

1. 从 Makeup . DAT 文件中读入数据。格式为:

N	（产品数）
M	（货船容积）
P_1 V_1	（第 1 种化妆品的价值与体积）
P_2 V_2	（第 2 种化妆品的价值与体积）
……	

P_n V_n　（第 n 种化妆品的价值与体积）

2. 确定运货方案。

3. 将方案写入 Makeup . SOL 文件。格式为：

每种货物所运件数；

运输的货物的总价值；

运输的货物的总体积。

输入输出文件中每 2 组数据间以空行分隔。

Makeup . DAT		Makeup . SOL	
2	5	Prod . 1 = 2	Prod . 1 = 1
6	20	Prod . 2 = 0	Prod . 2 = 0
43	23	Value = $ 8	Prod . 3 = 0
74	34	Volume = 6	Prod . 4 = 1
	12 10		Prod . 5 = 1
	17 11		Value = $ 26
	76		Value = 20

十二、花园

从前有一个花匠，它的工作是将花园划分成若干三角形，并在不同三角形内种上不同颜色的花。一个百万富翁要求这个花匠将他的花园布置成如下形状：

- 花园中的 n 股泉水必须处在三角形的顶点上，而不在三角形的边上。
- 一只蜜蜂能从任一股泉水处沿直线连续飞过泉水或花到达其它泉水出。
- 每 2 个相邻三角形（即有一条公共边）必须种上不同颜色的花。
- 这个花匠必须确定划分方案与种花方案，使所用颜色数最少。

编程从 Garden . DAT 文件中读入 n 股泉水的坐标。格式为：

N　　（泉水股数）

x_1 y_1　　（泉水 1 的坐标）

……

x_n y_n　　（泉水 n 的坐标）

其中 n , xi , yj , (1 < = i < = n < = 100) 都是整数。每 2 组数据间以空行分隔。

结果写入 Garden . SOL 文件。格式为：

N　　（泉水股数）

m　　（划分的三角形个数）

p　　（种花所用的颜色数）

r_1 s_1 t_1 c_1　　（第 1 个三角形的 3 个顶点号及此三角形中种的花色）

……

r_m s_m t_m c_m　　（第 n 个三角形的 3 个顶点号及此三角形中种的花色）

输入数据不需判错。每 2 组结果间以空行分隔。

Garden . DAT		Garden . SOL	
7	9	7	9
−2 2	−1 1	8	8
2 2	0 1	3	2
−2 −2	1 1	1 3 7 1	5 1 2 2
2 −2	−1 1	1 7 5 2	5 2 3 1
0 1	−1 0	1 5 2 1	5 3 6 2
1 0	0 0	2 5 6 2	5 6 9 1
−1 0	1 0	2 6 4 1	5 9 8 2
	−1 −1	4 6 3 2	5 8 7 1
	0 −1	3 6 7 3	5 7 4 2
	1 −1	7 5 6 1	5 4 1 1

十三、晚会

在一次晚会上，餐厅中有许多餐桌。所有客人必须按照这个规定就坐：每个不是单独坐一张餐桌的客人至少认识同桌的某一位客人。

假设若某位客人认识另一位客人，则另一位客人也认识这位客人，没人会向其它客人做自我价绍。也就是说：互相不认识的客人，在晚会上始终不会认识。

1）确定需要使用多少餐桌，以及各客人所坐的餐桌号。

每桌只有一位客人与服务员谈话，这位客人被称为主桌者。每位客人只能将他点的菜的菜名告诉同桌的他所认识的客人。每位客人的这个过程所用的时间相等。

2）找出每桌最合适的主桌者，使每桌的主桌者尽早知道同桌的每位客人点的菜。

此后，晚会主人想要合并餐桌。为此，他叫了些朋友。每位朋友来到后，就向某两桌的主桌者作自我介绍，将两桌合并成一桌，并在新桌上就坐，成为新桌的主桌者。

3）以怎样的次序合并餐桌，使最终并成的唯一一桌的主桌者尽早知道每位客人点的菜。并求出主桌者得知每位客人点的菜的时间。

合并结束后，晚会主人的所有朋友将离开，所有的餐桌将回到原来的状态。晚餐结束后，客人们陆续离开餐桌。

4）确定每桌至少需离开几位客人，才能保证每桌剩下的客人不认识同桌剩下的其它任何一位客人。

每 2 组数据间以空行分隔。

Meeting . DAT		Meeting . SOL	
8	7	Table 1 : 1 2 3 4	Table 1 : 1 2 3
1 2	1 2	Table 2 : 5 6 7	Table 2 : 4 5 6 7
1 3	1 3	Table 3 :8	Leader table 1 :
2 4	2 3	Leader table 1 : 2	Leader table 2 : 5
7 6	4 5	Leader table 2 : 6	15 New leadetr 8

4 3	5 6	Leader table 3:	Period of time: 3
5 6	6 7	68 New leader: 9	Persons leaving table 1: 1 2
0 0	0 0	29 New leader: 10	Persons leaving table 2: 5 6
		Period of time: 3	
		Persons leaving table 1: 2 3	
		Persons leaving table 2: 6	
		Persons leaving table 3:	

十四、最短路径

找一条从(0,0)到(100,100)的最短路径。在(0,0)到(100,100)的矩形区域中放有 n(n < = 30)个正方形。

- 每个正方形都是 5 * 5 大小。
- 正方形的边平行于坐标轴。
- 正方形角的坐标都是 Integer 型。
- 任两个正方形间距至少 1 个单位。

输入数据

输入文件为 INPUT.TXT。

文件含有正方形的个数及每个正方形左下角的(0 < = x < = 95; 0 < = y < = 95)坐标。

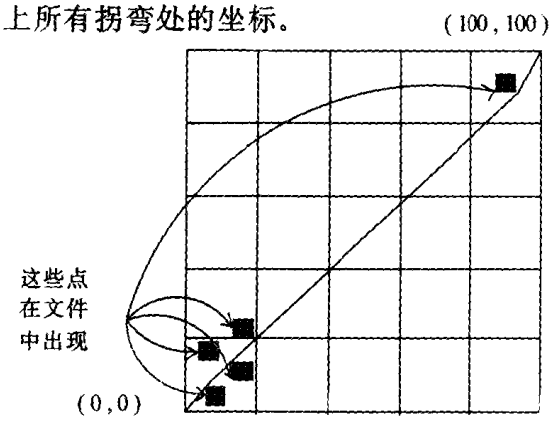
```
5
5    5
5    15
15   10
15   20
90   90
```

输出数据

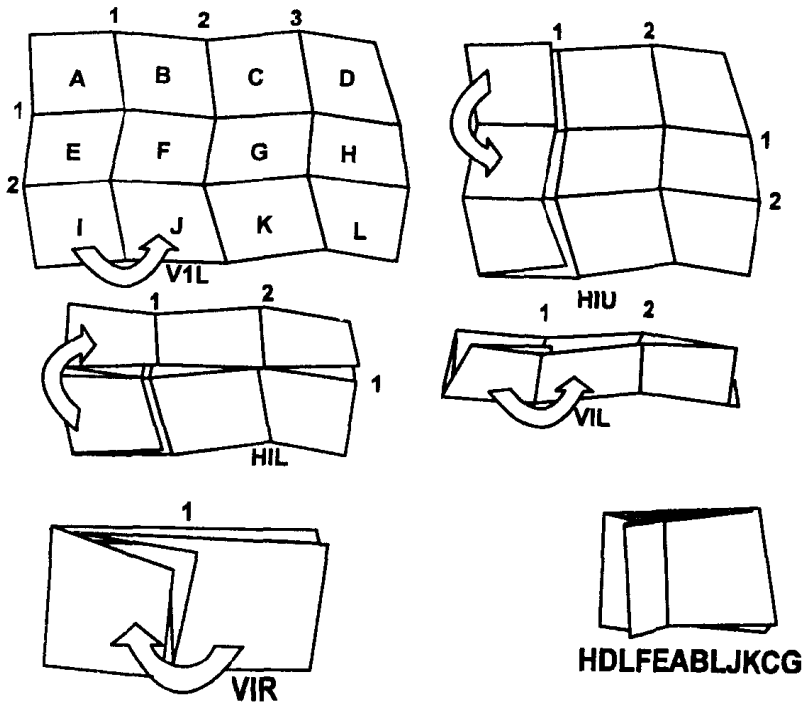
把结果写入 OUTPUT.TXT 文件。

文件包含最短路上所有拐弯处的坐标。

```
0    0
5    10
20   20
90   85
```



十五、车用地图



一张车用地图是一个 n(n < = 8) 行、m(m < = 4，但 m * n < = 16) 列的矩形。它能通过各种方式折叠。编程求出用什么方式折叠，能使折出的小矩形从外到里的编号所组成的序列为指定序列。

- 地图的每一部分都已按行编号 (A,B,C,…,P)。
- 折叠用 3 个字符描述:

  V(垂直)或 H(水平)；折线号；所折部分 (L = 左，R = 右，U = 上，L = 下)。
- 上图中第一张地图可能的折叠方式有:

  V1L, V1R, V2L, V2R, V3L, V3R, H1U, H1L, H2U, H2L。
- 每次折叠后，所有的折线必须重新编号。
- 折叠时，折叠的部分必须全部折，不能仅折最前面的几张。

输入数据

输入文件为 INPUT.TXT。

第一行为 n，即行数；下一行为 m，即列数；再下一行为指定的编号序列。例如:

3

4

HDLFEABIJKCG

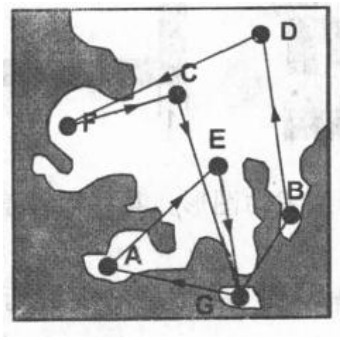
输出数据

输出文件为 OUTPUT . TXT 。

我们的例子是:

V1L H1U H1L V1L V1R

十六、运输地图



某运输公司位于 A 镇，它拥有一架可往返于 7 个镇 (A , B , C , … , G) 之间的运输机。

7 个镇之间的距离表如下:

	A	B	C	D	E	F	G
A	0	56	43	71	35	41	36
B	56	0	54	58	36	79	31
C	43	54	0	30	20	31	58
D	71	58	30	0	38	59	75
E	35	36	20	38	0	44	70
F	41	79	31	59	44	0	72
G	36	31	58	75	70	72	0

每天早晨，公司必须决定当天的运货路线，并使得路线长度尽量短。

- 运输机总是从 A 镇出发且在工作结束时返回 A 镇。
- 运输机在每一时刻只能装一件货物。

编程输入所有任务，求一条起始于 A 且结束于 A 的能完成所有任务的最短路径。

输入数据

输入文件为 INPUT . TXT 。

文件开始于数 n (1 < = n < = 20), 即任务数。以后的 n 行为任务的有序对:

5

F C

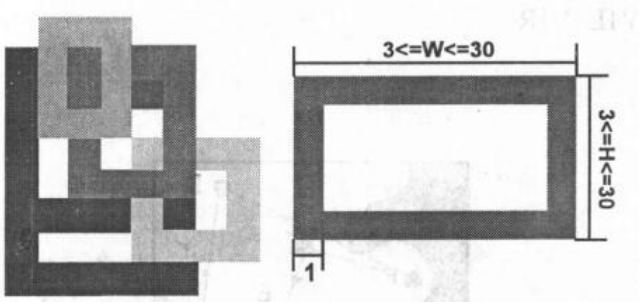
G B

B D

A E

G A

十七、覆盖框



图中显示了一个由一些框所组成的图形。某些框覆盖在另一些框上。

编程求出框的覆盖顺序。

图形由 INPUT . TXT 文件输入：框由 A，…，Z 表示，背景由“.”表示。

- 框宽 1，边长 > = 3。
- 所给图形至少露出每个框的每条边的一部分，注意露出一个角相当于露出了两条边。

输入数据

输入文件为 INPUT . TXT。

文件包含输入图形的高 h(h < = 30) 和宽 W(W < = 30)。

图形由 h 个 W 长的字串组成。

我们的例子是:

```
9
8
.CCC....
ECBCBB..
DCBCDB..
DCCC.B..
D.B.ABAA
D.BBBB.A
DDDDAD.A
E...AAAA
EEEEEE..
```

输出数据

输出文件为 OUTPUT . TXT。

文件包含经字符表示框的叠置顺序。

所有字符在同一行中，字符间无空格。

我们的例子是:

```
CBADE
```

十八、错连

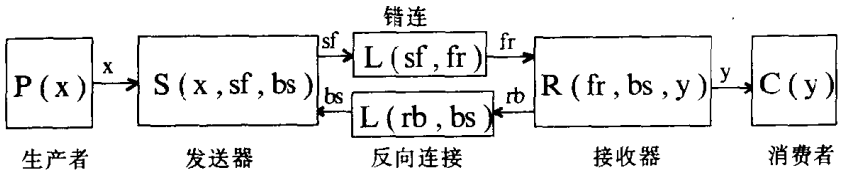


图 1：通过 2 个 L 与 S、R 将 P 与 C 连接

这是一个关于从生产者到消费者信息通讯的问题（图 1）。一位信息是 { 0,1 } 中的某个位，或一个出错指示符 E。

生产者 P 通过它的输出通道 x 送出一系列信息（图 2）。消费者 C 通过它的输入通道 y 接收一系列信息(图 3)。我们把 P 送出的信息序列表示成 P_i ($P_i \in \{0, 1\}$, $0 <= i$)、而 C 接收的信息表示成 C_i ($0 <= i$)。

```
P = proc(s!Msg).
begin i:var Nat & m:var Msg
| i:=0
; forever
  do m:=p_i;xlm
  ; i:=i+1
  od
end
```

图 2：生成器 P 的程序

```
C = proc(y?Msg).
begin i:var Nat & m:var Msg
| i:=0
; forever
  do y?m;c_j:=m
  ; i:=i+1
  od
end
```

图 3：消费器 C 的程序

我们希望保证 C 能正确地收到 P 送出的信息，即 $P_i = C_i$ ($0 <= i$)。注意，由 P 送出的每一位必须在有限步内被 C 正确地接收到。一种方法是将 P 与 C 直接连接（图 4）。但现在，由于 P 与 C 距离太远，我们只能通过一个会出错的通讯连接将其相连。

一个会出错的通讯连接 L(从现在起简称为错连) 有一个输入通道 x 与一个输出通道 y(图 5)。它将复制从 x 输入的信息到 y，但有可能将信息毁坏。毁坏是能检测的：每个毁坏的信息都是一个出错符 E，它不同于没被毁坏的信息：$E \notin \{0,1\}$。

一个错连可能在一段时间内毁坏所有的信息，使得 C 无法收到正确的信息。因此我们必须假设：一个错连仅能有限次毁坏连续信息（图 5），即我们能通过对某信息的连续多次发送，使信息最终被 C 正确地收到。

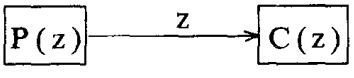


图 4：将 P 与 C 直接连接

```
L = proc(x?Msg & ylMsg).
begin m:var Msg & n:var Nat
| forever
  do n:∈ NAt
  ; for n do x?m; ylE od
  ; x?m; ylm
  od
end
```

图 5：错连 L 的程序

现把 P 的输出通道与 L 的输入通道连接，L 的输出通道与 C 的输入通道连接（图 6）。这种连接并不完善，因为信息可能被毁坏。例如：$P_0 = 0$，而 $C_0 = E$。

重发方案

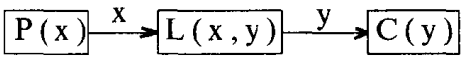


图 6：将 P 与 C 通过 L 连接

我们能通过信息的重发以挽救丢失的信息。所以，我们需要一种反向错连。重发方案是建立在发送器 S 与接收器 R 上的（图 1）。

P 将信息输入 S（图 7），S 通过错连将信息输入 R。然后，S 等待 R 的应答：1 表示得到正确信息；0 表示得到毁坏信息。S 将重发同一信息，直到 R 的应答为 1 为止。然后继续用同样的方法处理 P 输出的下一信息。

错连将信息输入 R（图 8）。若 R 收到 E，就通过反向连接将 0 发送入 S。若 R 收到正确信息，就将信息发给 C，同时通过反向错连将 1 发关给 S。然后继续用同样的方法处理错连输出的下一信息。

```
S = proc(x?Msg & sflMsg & bs?Msg).
begin m,a:var Msg
| x?m
; forever
  do sflm; bs?a
  ; if a = 1 then x?m fi
  od
end
```

图7：发送器S的程序1

```
R = proc(fr?Msg & rblMsg & ylMsg).
begin m:var Msg
| forever
  do fr?m
  ; if m = E then rblo
   else ylm; rbl1
   fi
  od
end
```

图8：接收器R的程序1

图 9 给出了一个实例：$P_0 = 0, P_1 = 1$。其中发送 P_0 时出现错误。表中第一列为时间，其余列均为各通道的信息情况。

任务 A

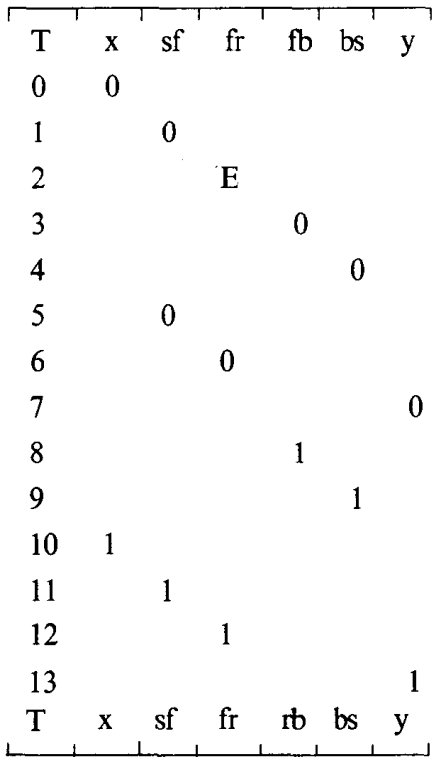
以上方法并不总能奏效。请给出一个实例证明。将过程写在答纸 A 上，你并不一定，要填满所有时间行。

任务 B

我们将 S 与 R 进行改进。S（图 10）将从 P 输出的每一位信息交替地以 0 或 1 标号。其它工作都像往常那样进行。一位信息 $m \in TMsg, m \neq E$ 增加一个标记位：m.v 为位值，m.t 为标记位。

R（图 11）有一个局部变量 $u \in \{0,1\}$ 来表示期望标记。通过 L 接收信息后，R 将重复以下步骤：当信息被毁坏时，R 发回 0。否则，当标记不同与 u 时，R 发回 1、但并不向 C 发送 m。仅当一个拥有期望标记且没被毁坏的信息被接收时，R 向 C 发送 m 并更新期望标记值。此后，R 将等待下一信息。

P, L, C 的程序不变（除了 L 中传送的信息多了一个标记位（以下 TMsg 代替 Msg）。



T	x	sf	fr	fb	bs	y
0	0					
1		0				
2			E			
3				0		
4					0	
5		0				
6			0			
7						0
8				1		
9					1	
10	1					
11		1				
12			1			
13						1
T	x	sf	fr	rb	bs	y

图 9: $P_0 = 0$ $P_1 = 14$ 的实例

给出一个实例证明改进后的方案并不总能奏效。将过程写在答纸 B 上，你并不一定要填满所有时间行。用一个箭头表示事件的重复。

任务 C

改进 R 与 S 的程序，使之能正确运行。写出你的答案的程序形式。你并不一定要填满所有行。

你只能修改程序“|”与“and”之间的部分。你不能增加任何变量，改变 P，L 或 C 的程序，或改变它们的连接。你也不能创造任何新的语言格式（语句或表达式）。

```
S = proc(x?Msg & sflTMsg & bs?Msg).
begin m:var TMsg & a:var Msg
| m.t:=0; x?m.v
; forever
  do sflm; vs?a
  ; if a = 1
    then m.t:=1-m.t; x?m.v
    fi
  od
end
```

图 10: 发送器 S 的程序2

```
R = proc(fr?TMsg & rblMsg & ylMsg).
begin m:var TMsg & u:var Nat
| u:=0
; forever
  do fr?m
  ; if m = E then rblo
    else if m.t = u
      then ylm.v; u:=1-u
      fi
      ; rbl1
    fi
  od
end
```

图 11: 接收器 R 的程序2

答纸 A

T	x	sf	fr	fb	bs	y
0						
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
T	x	sf	fr	rb	bs	y

答纸 B

T	x	sf, v	sf. t	fr. v	fr. t	fb	bs	y
0								
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
T	x	sf, v	sf. t	fr. v	fr. t	fb	bs	y

答纸 C

发送器程序

```
.R = proc(fr?TMsg & rblMsg & ylMsg).
.begin m:var TMsg & u:var Nat .
.|
.
.
.
.
.
.
.
.
.
.
.
.
.end .
```

接收器程序

```
.S = proc(x?Msg & sflTMsg & bslMsg).
.begin m:var TMsg & a:var Msg .
.|
.
.
.
.
.
.
.
.
.
.
.
.
.end .
```

参赛者编号: _____

十九、高科技计划

一个高科技计划需要 100 名科学家。每位科学家都为自己高超的心算能力而骄傲。在一间嘈杂的机房中，数 0 已被写在黑板上。在做其它工作的同时，每位科学家执行下列步骤 100 次:

1. 进入机房
2. 记住黑板上的数字。
3. 离开机房
4. 心算出记下的数字与 37 的和。
5. 进入机房。
6. 擦去黑板上的数字。
7. 将计算出的和写在黑板上。
8. 离开机房。

每位科学家可以自由地以任何速度工作。但机房太小，每一时刻机房最多只能容纳一位科学家。

黑板上最终留下的最大可能数是 370,000 。可能的最小数是几?

二十、排序序列

在一个排序序列中寻找一个指定元素是一种简单操作。现有两种算法，要求你通过一些尝试比较两个算法的效率。

设 S 是一个 N 个元素组成的递增排序序列。每个元素可都用 S[i](0 < = i < n) 来表示。两个算法的输入都是 S 中的某个元素 x，输出都是一个数 i(0 < = i < n,A[j] = x)。两者都使用了两个变量 j,h。下面是算法 A 的描述:

1. i ← 0; j ← N;
2. While i < > j − 1 do 3(a), 3(b);
3. (a)If i + j 是偶数 then h ← (+)/2 浊 else h ← (i + j − 1)/2;
   (b)If x < S[h] then j ← h else i ← h 。

算法 A 的关键在于它始终保证 S[i] < = x < = S[j − 1]，即 x 在 S[i] 到 S[j − 1] 的区间中。若此区间仅含一个元素 (i = j − 1)，则算法结束。否则，将区间划为两部分: 依 x 的位置选择下一步寻找的区间。

算法 B 在算法 A 的基础上将 3(b) 扩展为:

3(b)If S[h] = x then i ← h; j ← h + 1
else if x < S[h] then j ← h else i ← h;

不同的在于，当 S[h] = x 时，算法 B 立即结束。现请研究以下两个问题:

1. 两个算法第三步的执行次数的平均数的不同。
2. 两个算法的平均寻找次数的不同 (用基本操作的次数表示)。

你必须考虑不同长度的序列。你可假设每个元素被寻找的概率相同。赋值和比较是基本操作。由于许多机器都有一个专门指令 (移位)，3(a) 可看作一个简单的基本操作。因此，算法 A 中: 第 1 步有 2 个基本操作 (2 次赋值); 第 2 步有 1 个基本操作 (1 次比较); 第 3(a) 步有 1 个基本操作 (1 次特殊操作); 第 3(b) 步有 2 个基本操作 (1 次比较 1 次赋值)。